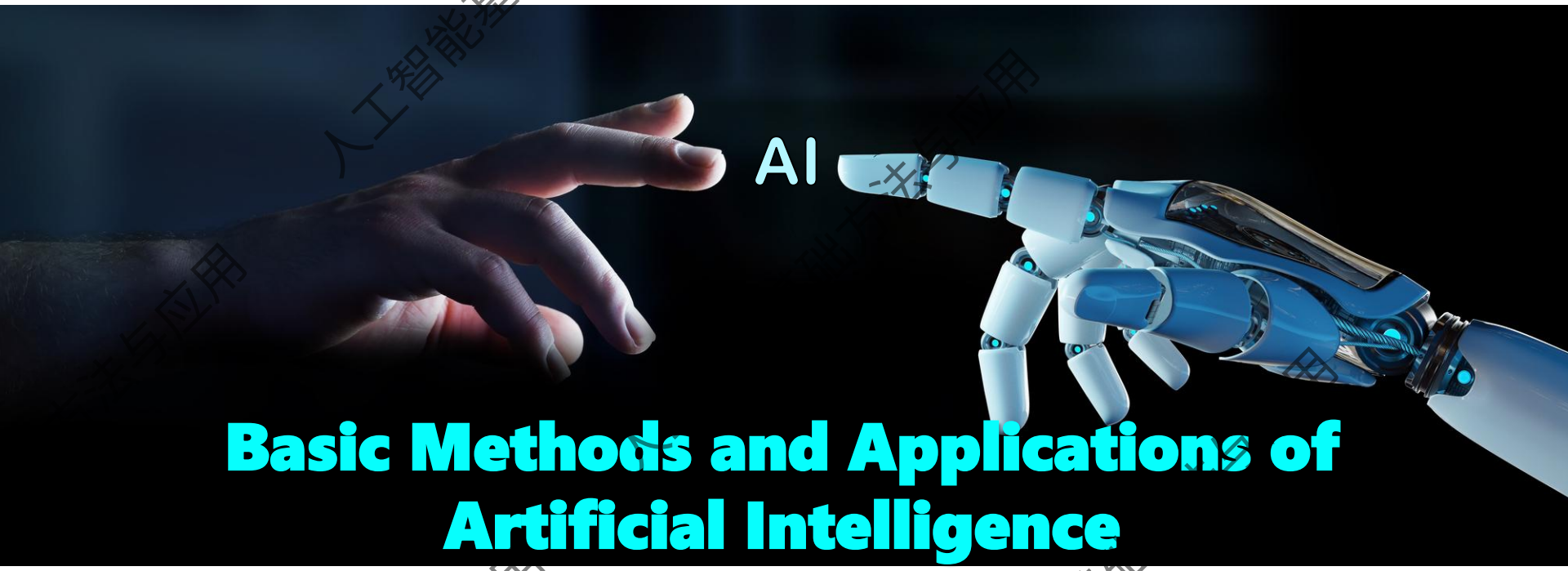


Basic Methods and Applications of Artificial Intelligence

——深度学习

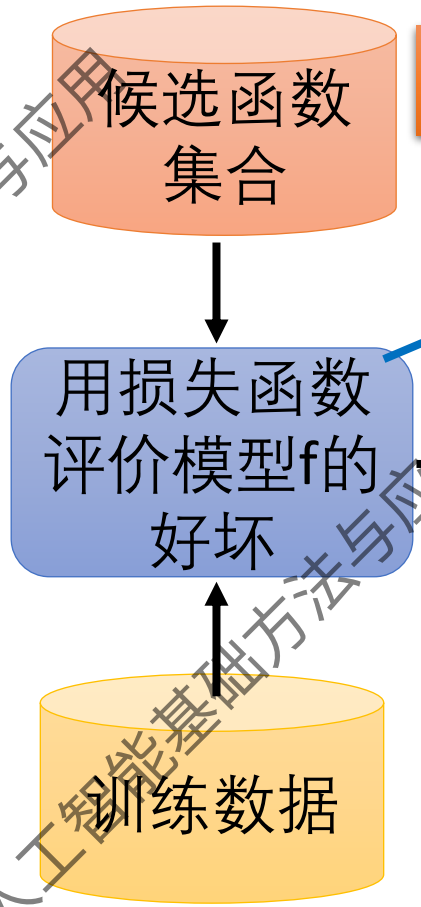
人工智能学院 / 人工智能与机器人研究所
人机混合增强智能全国重点实验室
陈炜煌

IAIR Est. 1986
Institute of Artificial Intelligence and Robotics, XJTU

IAIR Est. 1986

人工智能基础方法与应用

线性回归



模型

$f_1, f_2 \cdots$

$$L(w,b) = \sum_{n=1}^{10} \left(\hat{y}^n - \left(b + w \cdot x_{cp}^n\right)\right)^2$$

选择“最佳”函数

通过梯度下降寻找最优参数

$$f^* = arg\min_f L(f)$$

$$w^*, b^* = arg\min_{w,b} L(w,b)$$

$$= arg\min_{w,b} \sum_{n=1}^{10} \left(\hat{y}^n - \left(b + w \cdot x_{cp}^n\right)\right)^2$$

人工智能基础方法与应用

分类

$$P(x \mid C_i) = \mathcal{N}(x; \mu_i, \Sigma_i)$$

$$f_{\mu,\Sigma}(x) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma|^{1/2}} exp\left\{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right\}$$

候选函数集合

用似然函数评价模型f的好坏

训练数据

选择“最佳”概率模型

$$P(C_1|x) = \frac{P(x|C_1)P(C_1)}{P(x|C_1)P(C_1) + P(x|C_2)P(C_2)}$$

分类规则：如果$P(C_1|x) > 0.5$, 预测类别为1

否则，预测类别为2

通过最大似然估计，无需梯度下降，直接得到最优参数

$$\mu^*, \Sigma^* = arg \max_{\mu,\Sigma} L(\mu, \Sigma)$$

$$\mu^* = \frac{1}{79}\sum_{n=1}^{79} x^n \quad \Sigma^* = \frac{1}{79}\sum_{n=1}^{79} (x^n - \mu^*)(x^n - \mu^*)^T$$

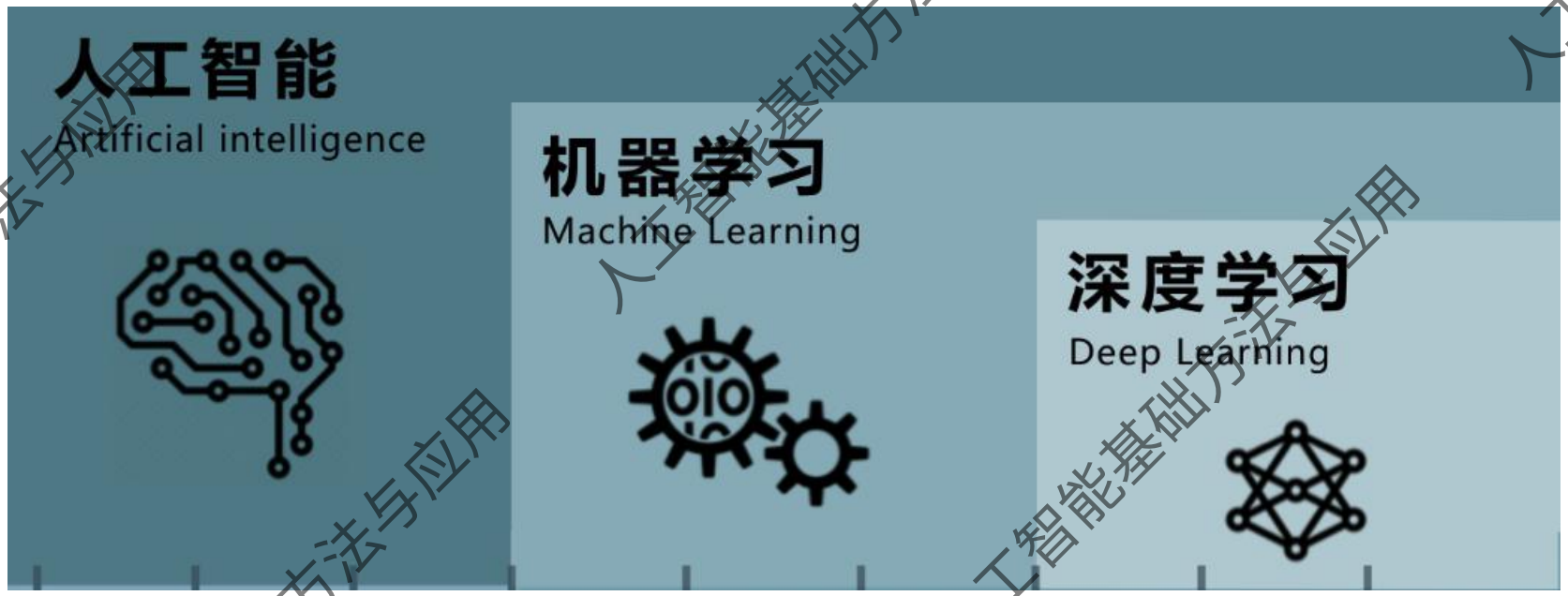
人工智能基础方法与应用

课堂作业

- 请利用**线性回归**方法，根据温度、湿度、风速、降雨量、降雪量、季节和节假日等特征，预测共享单车租赁数量。数据来源于真实的 Seoul Bike Sharing Demand 数据集
- **数据说明（微信群内提供：共享单车回归作业数据.zip）**
  本作业提供训练集：100 条数据，测试集：60 条数据
- **作业要求**

1. 建立线性回归模型：$y = b + \sum w_i x_i$
2. 使用 100 条训练数据训练线性回归模型，说明数据处理和模型求解过程，并给出模型学习得到的参数b和w_i
3. 分析线性回归模型在训练集和测试集上的表现
4. 提交时限：两周之内
5. 提交PDF格式电子文档，以***姓名-学号-共享单车回归作业***命名，发送至助教邮箱：Email：liushuaiyu@stu.xjtu.edu.cn

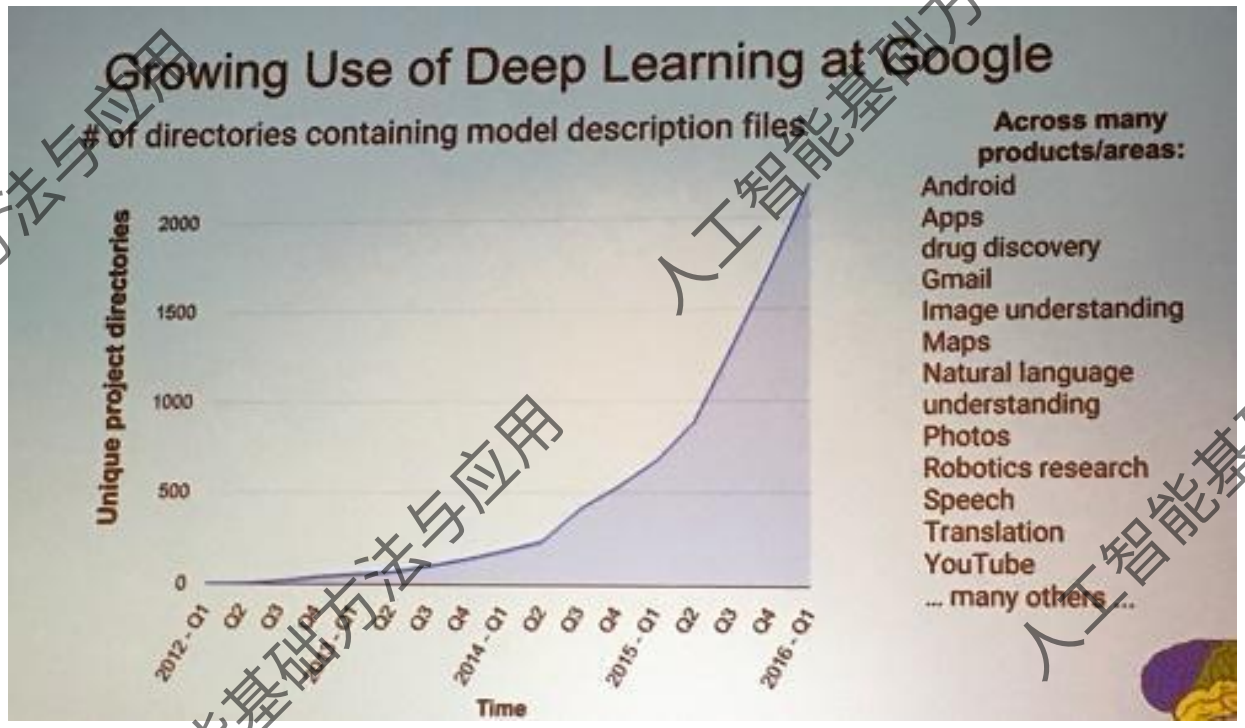
完整数据集：https://archive.ics.uci.edu/dataset/560/seoul+bike+sharing+demand



人工智能基础方法与应用

深度学习为何受到关注

- 深度学习在图像、语音、自然语言等领域不断取得突破，并迅速走向大规模实际应用

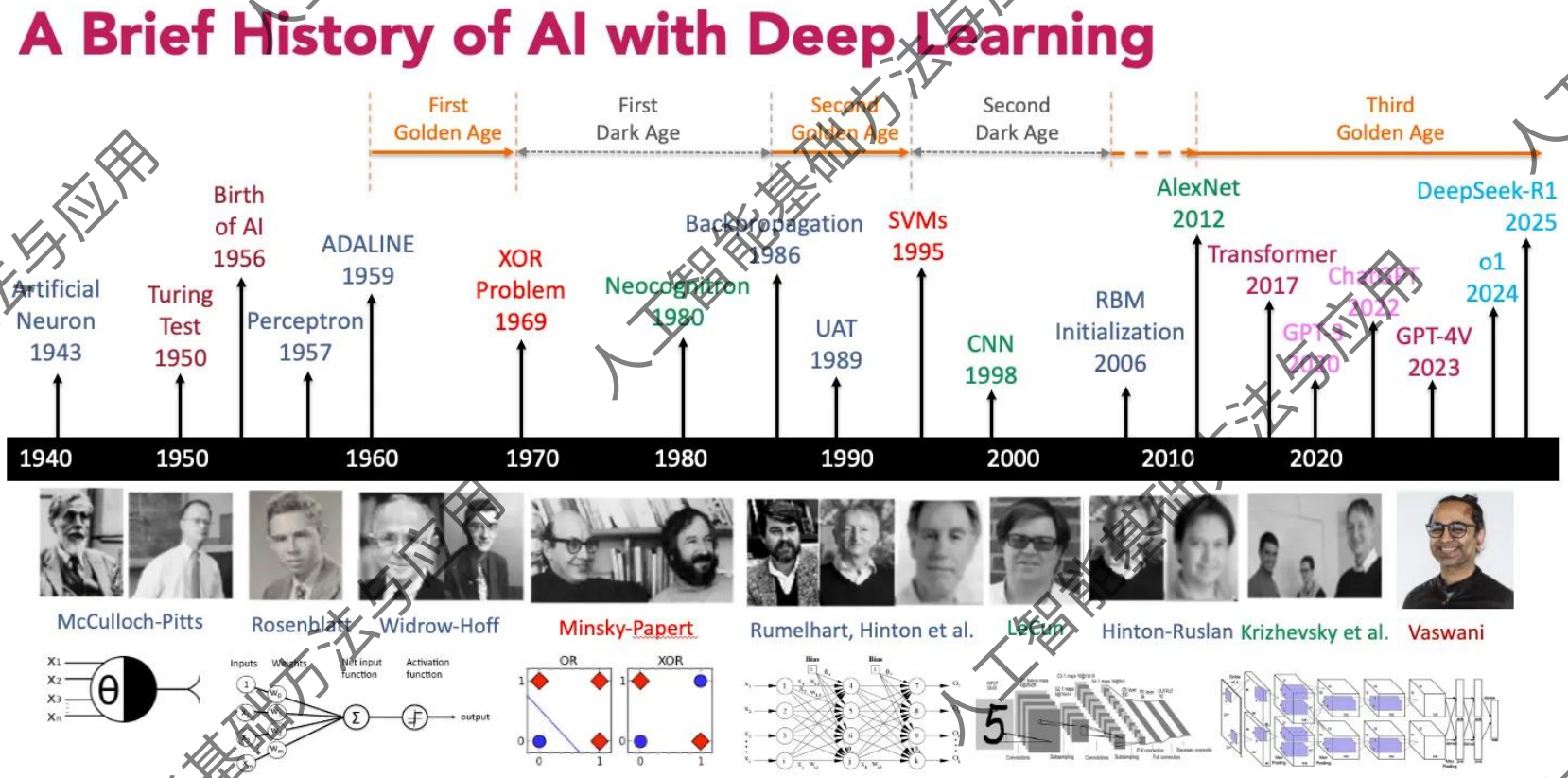


Deep learning trends at Google. Source: SIGMOD/Jeff Dean

- 安卓系统
- 应用程序
- 药物发现
- Gmail 邮箱
- 图像理解
- 地图
- 自然语言理解
- 照片/图像管理
- 机器人研究
- 语音
- 机器翻译
- YouTube 视频平台

人工智能基础方法与应用

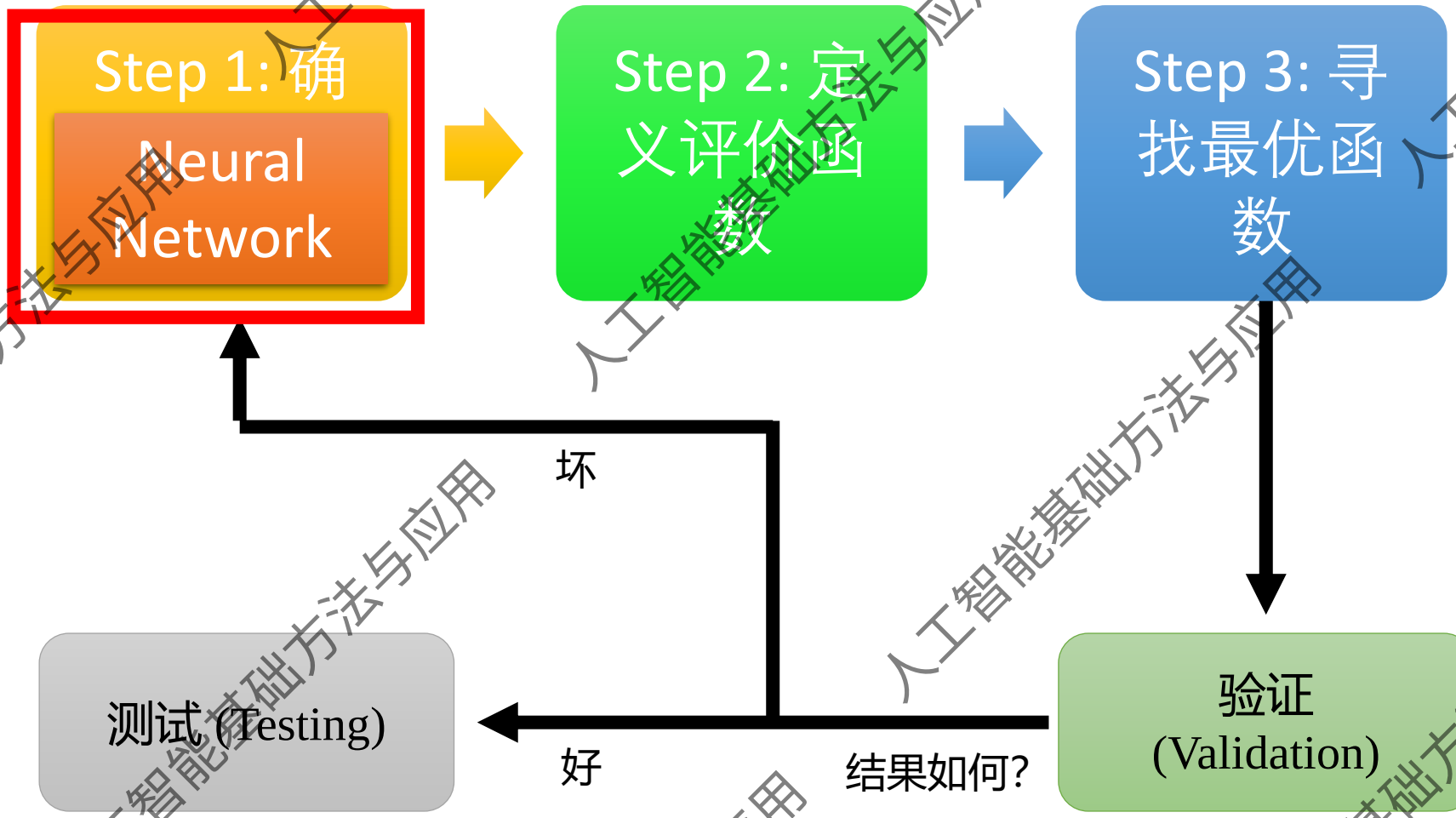
深度学习发展史


A Brief History of AI with Deep Learning
First Golden Age
First Dark Age
Second Golden Age
Second Dark Age
Third Golden Age
Artificial Neuron 1943
Turing Test 1950
Birth of AI 1956
Perceptron 1957
ADALINE 1959
XOR Problem 1969
Neocognitron 1980
Backpropagation 1986
UAT 1989
SVMs 1995
CNN 1998
RBM Initialization 2006
AlexNet 2012
Transformer 2017
ChatGPT 2022
GPT-4V 2023
o1 2024
DeepSeek-R1 2025
1940
1950
1960
1970
1980
1990
2000
2010
2020
McCulloch-Pitts
Rosenblatt
Widrow-Hoff
Minsky-Papert
Rumelhart, Hinton et al.
LeCun
Hinton-Ruslan
Krizhevsky et al.
Vaswani
OR
XOR

一、深度学习的三个步骤

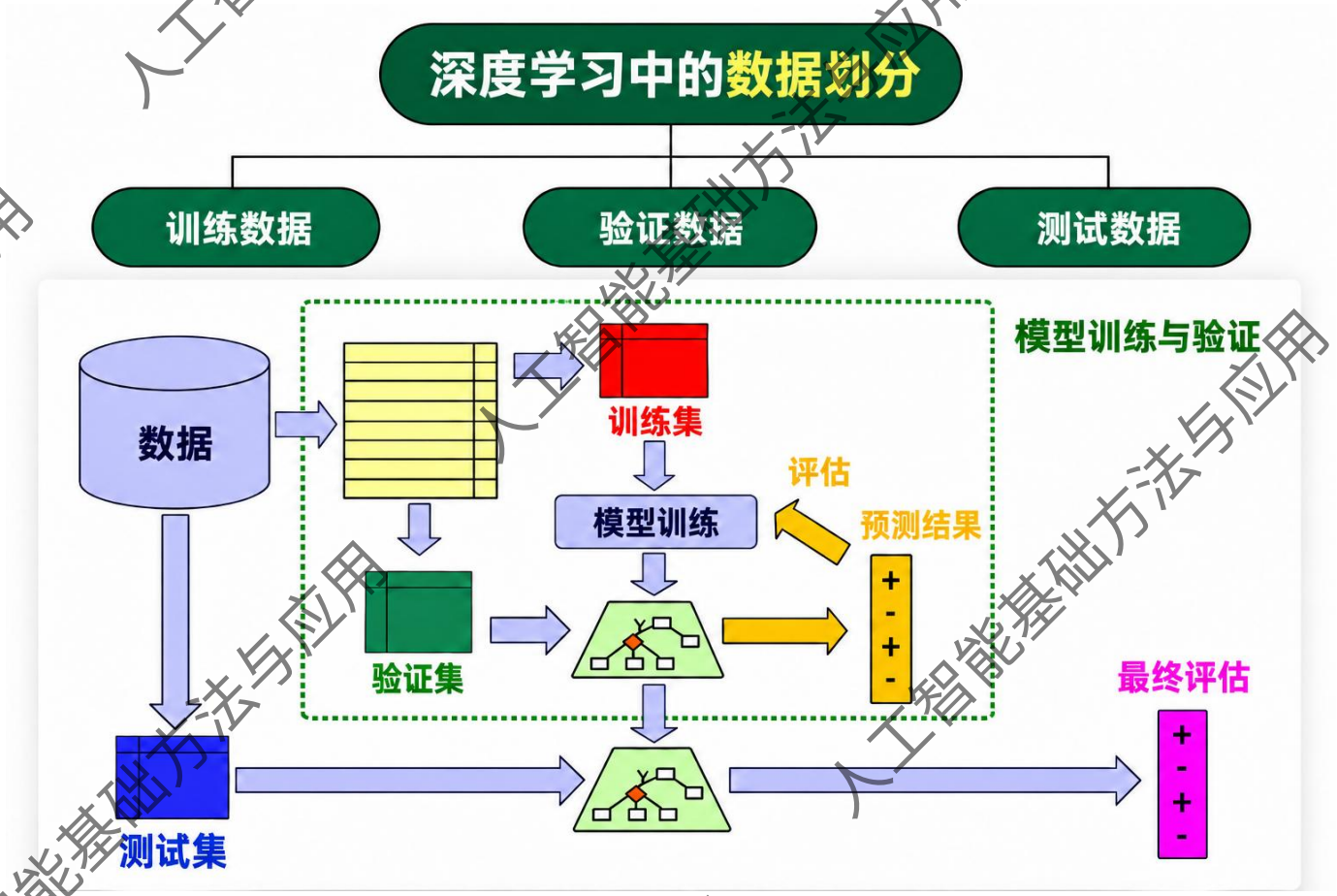
人工智能基础方法与应用

深度学习的三个步骤



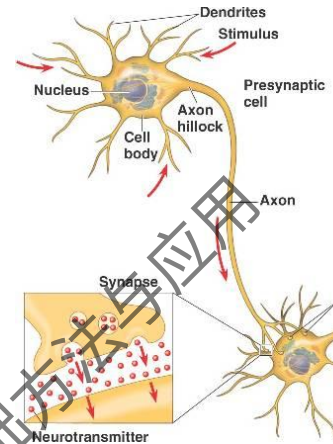
人工智能基础方法与应用

数据集拆分

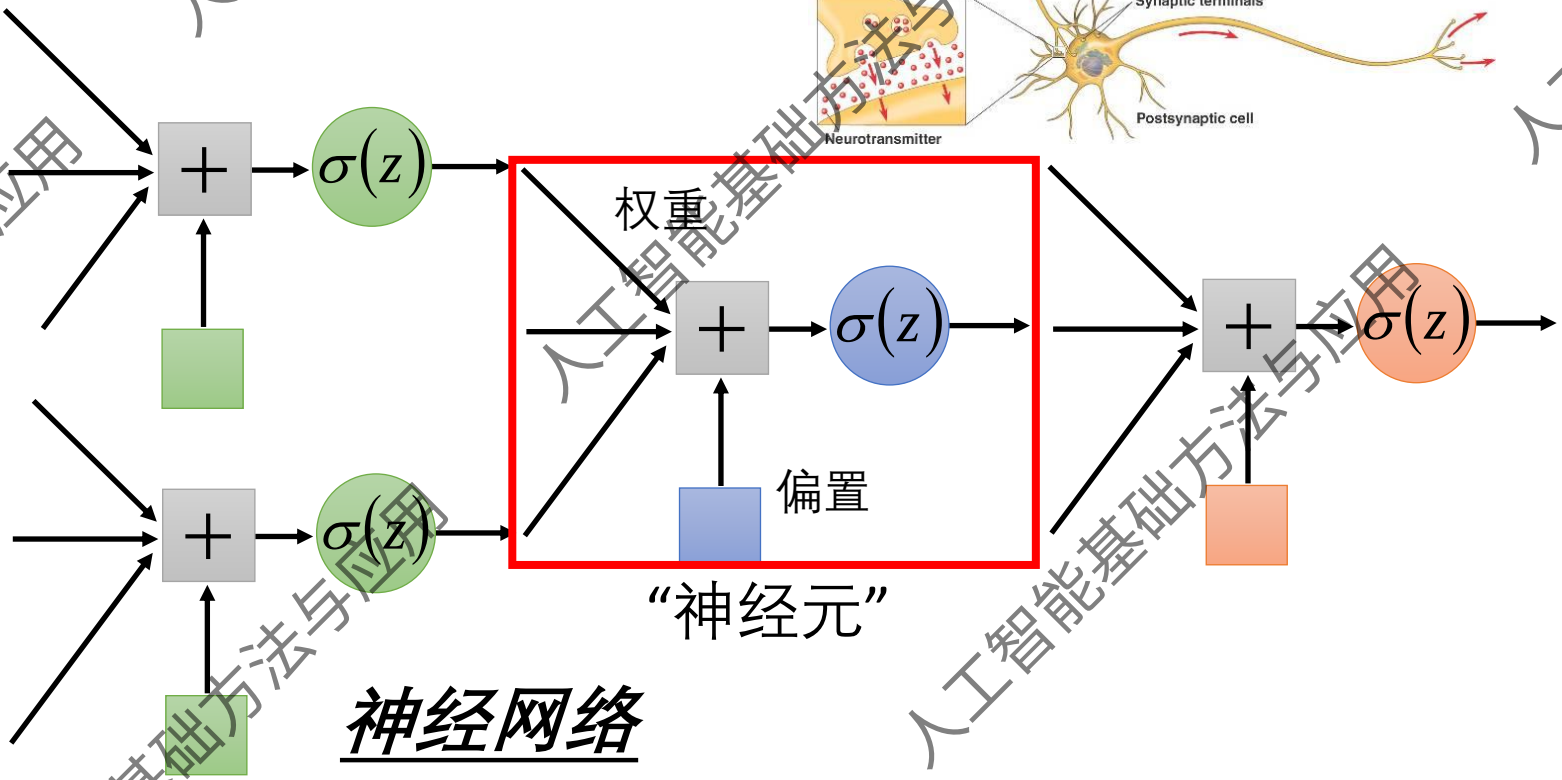


人工智能基础方法与应用

神经网络







神经网络

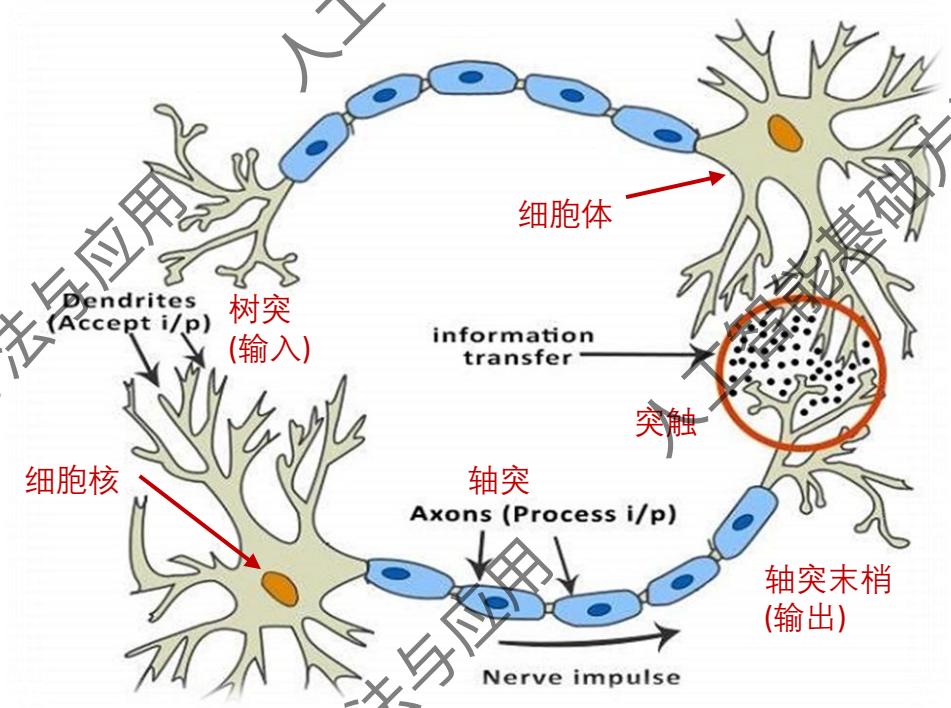
不同的神经元连接形成的**网络**结构

网络参数θ: 网络中所有权重和偏置的集合

人工智能基础方法与应用

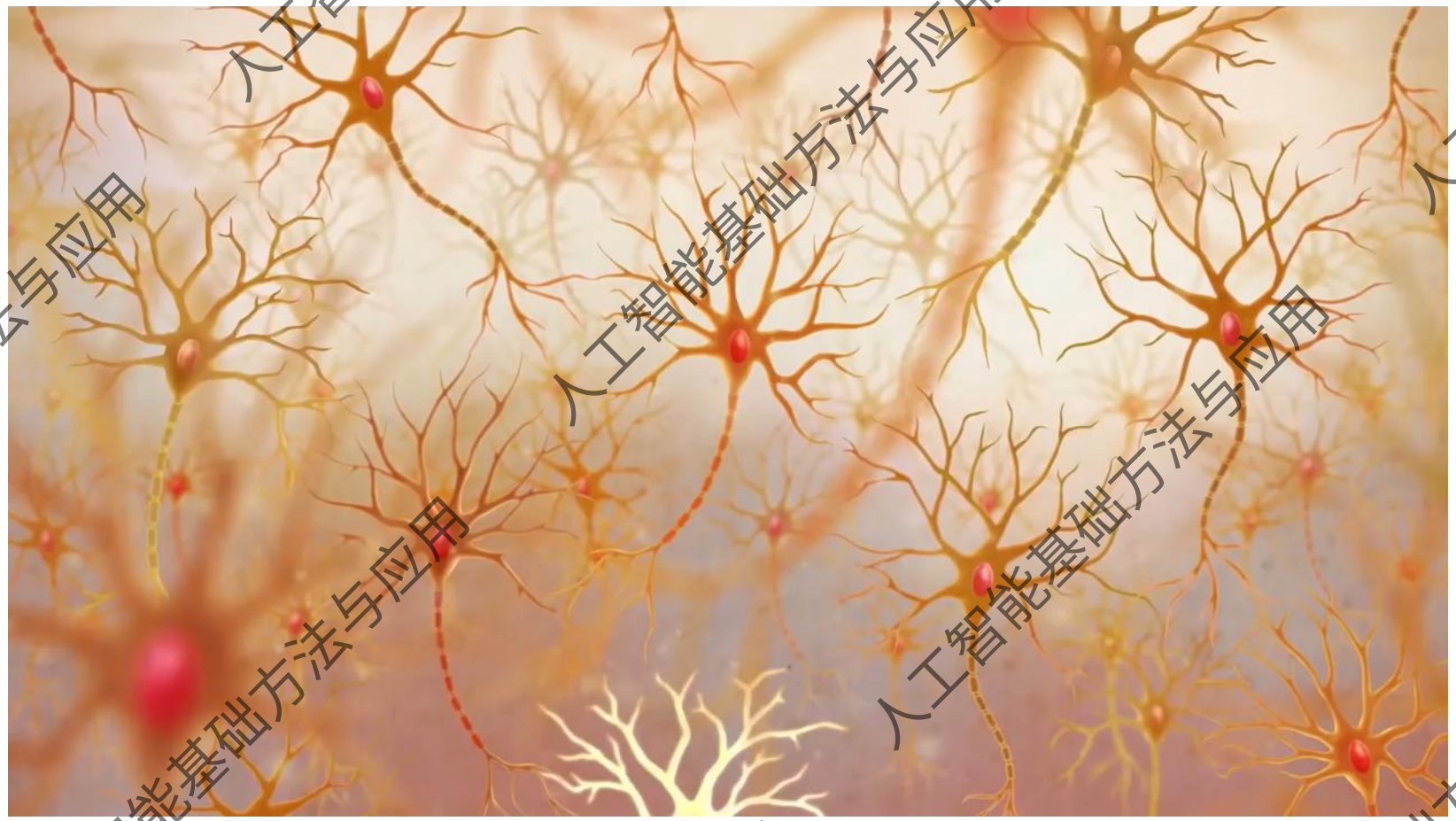
神经元（Neuron）

- 神经元的基本结构：
  - **细胞体**：神经细胞的主体部分，由细胞核、细胞质、细胞膜等组成
  - **树突**：细胞体向外伸出的众多较短的神经纤维分枝，用以**接受来自其他神经元的信号**
  - **轴突**：细胞体向外伸出的最长的一条神经纤维分枝，用以**输出信号**



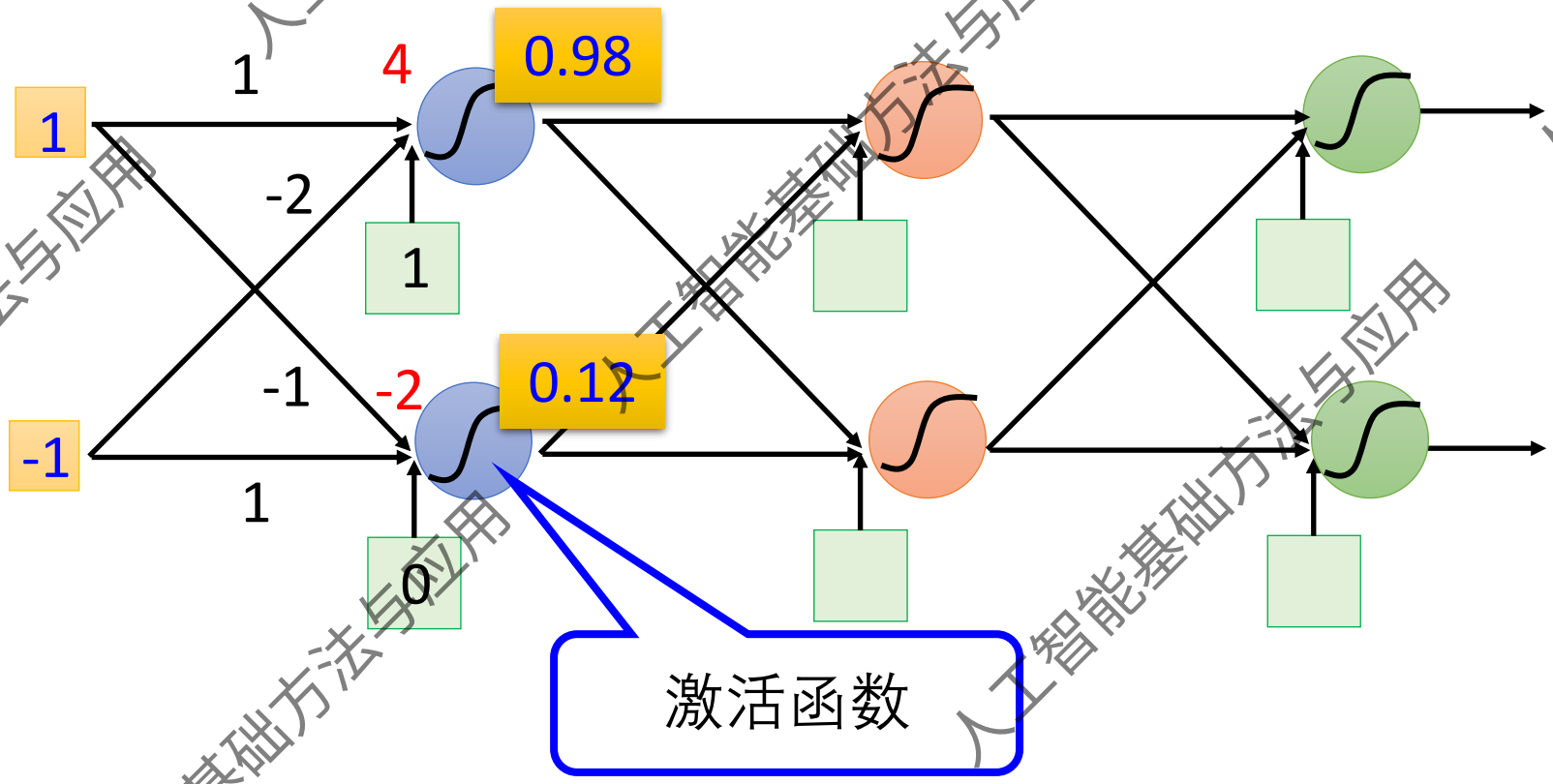
人工智能基础方法与应用

神经元及其网络



人工智能基础方法与应用

全连接前馈网络：第一层计算



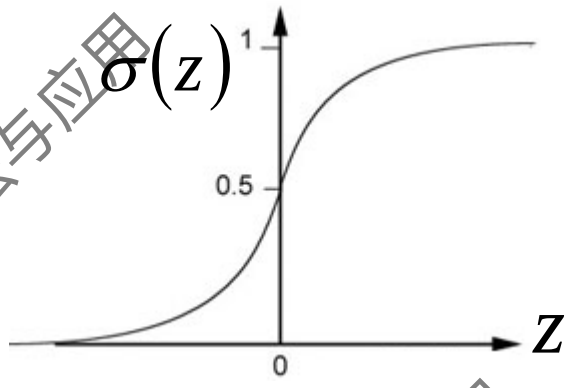
全连接”（Fully Connected）：描述神经网络**层与层之间的连接方式**，如果前一层有m个神经元，后一层有n个神经元，那么全连接表示**前一层的每一个神经元，都和后一层的每一个神经元相连接**

激活函数

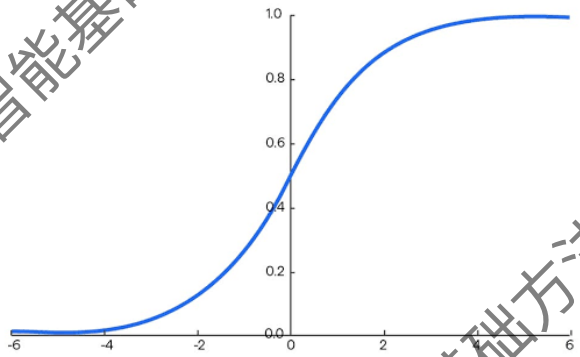
- 作用于神经元线性组合结果上的非线性映射函数，用于产生该神经元的输出
- 核心作用：引入非线性，帮助神经网络学习数据中的复杂模式
- 激活函数没有引入额外的参数

Sigmoid函数

$$\sigma(z)=\frac{1}{1+e^{-z}}$$

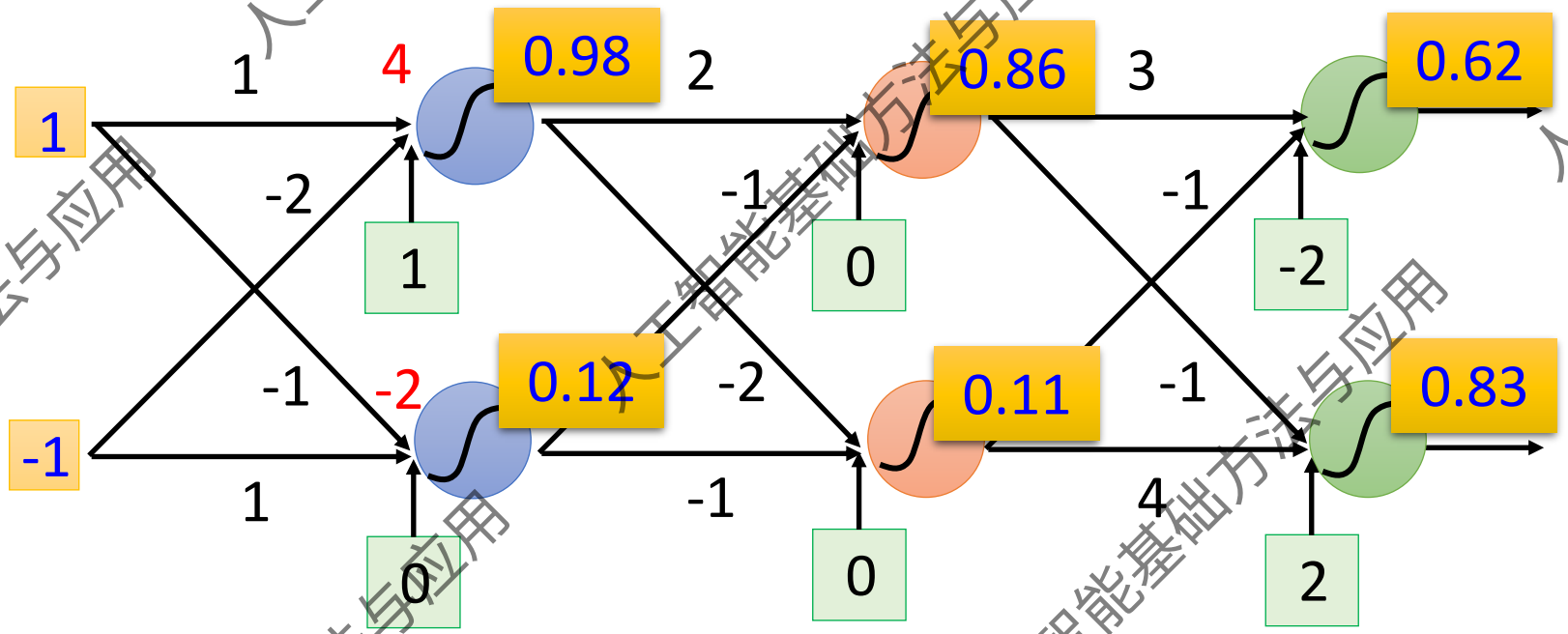


Softmax Function



$$\alpha_i' = exp(\alpha_i)/\sum_j exp(\alpha_j)$$

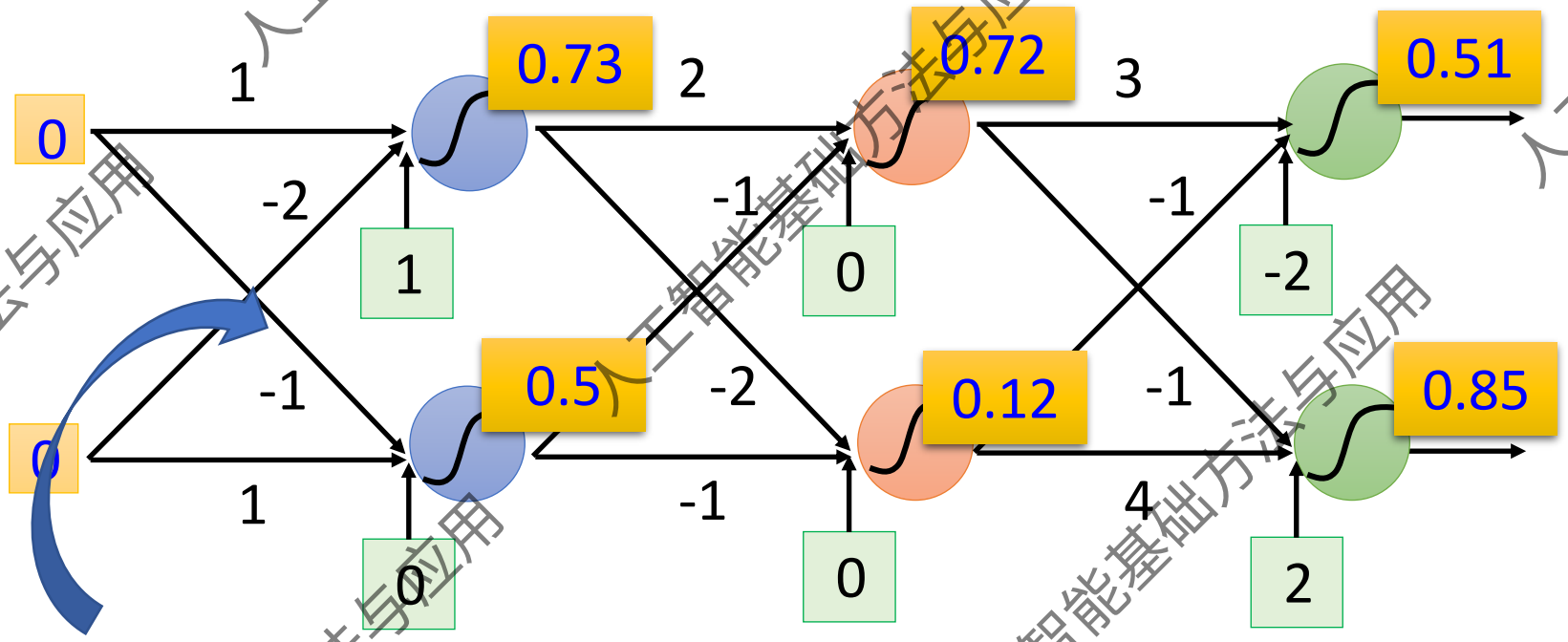
神经网络的前向传播



在全连接网络中，每一层的输出都可以看作一个新的**特征向量**；随着网络逐层向前传播，原始输入被不断转换成更适合当前任务的**特征表示**

人工智能基础方法与应用

神经网络就是函数集合



整个网络就是一个函数集合

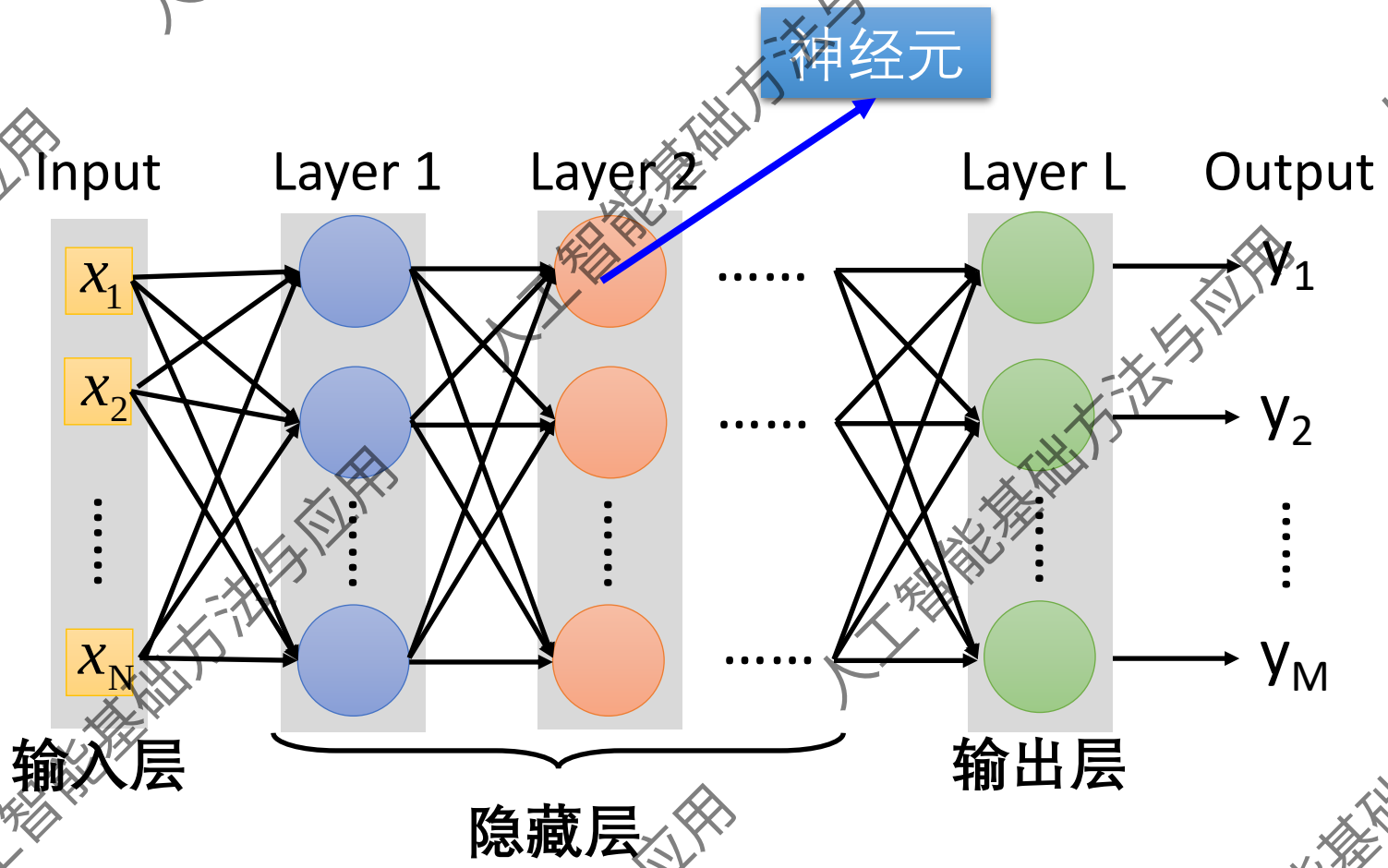
输入向量，输出向量

$$f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right)=\begin{bmatrix}0.62\\0.83\end{bmatrix} \quad f\left(\begin{bmatrix}0\\0\end{bmatrix}\right)=\begin{bmatrix}0.51\\0.85\end{bmatrix}$$

给定神经网络结构，就确定了*一个函数集合*

人工智能基础方法与应用

输入层、隐藏层与输出层



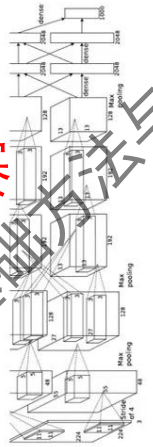
既不是直接观测到的输入，也不是任务要求的最终输出，而是网络在计算过程中形成的内部表示

Deep = 隐藏层很多

8 layers

19 layers

22 layers



7.3%

6.7%

ImageNet竞赛
错误率
16.4%





AlexNet (2012)

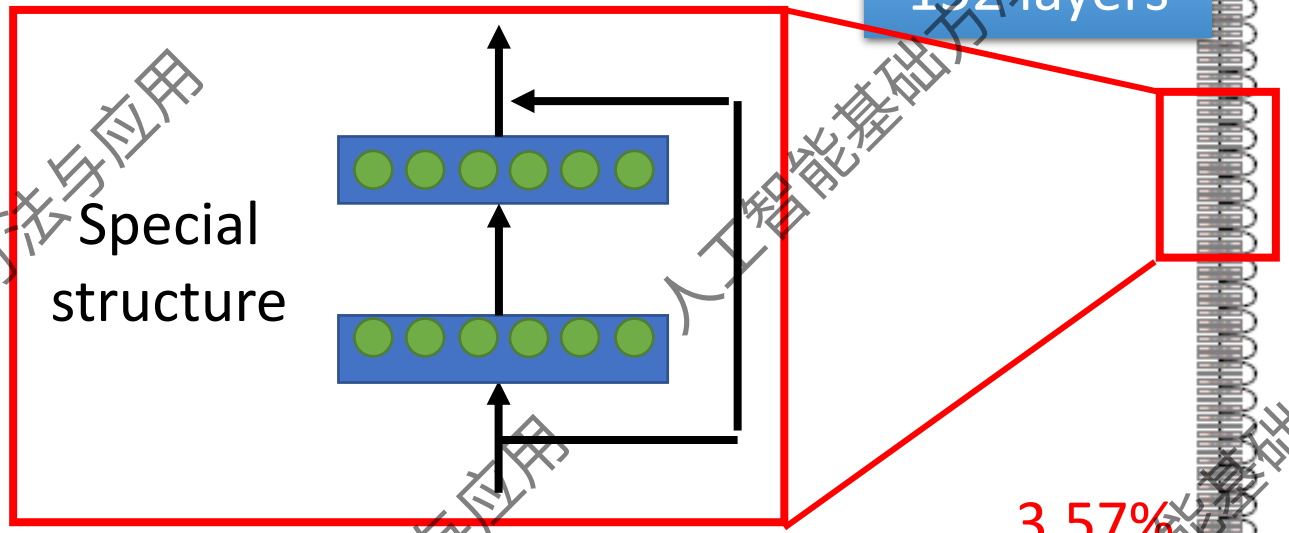
VGG (2014)

GoogleNet (2014)

人工智能基础方法与应用

Deep = 隐藏层很多

ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-152



152 layers

127 layers



16.4%

AlexNet
(2012)

7.3%



VGG
(2014)

6.7%



GoogleNet
(2014)

3.57%

Residual Net
(2015)

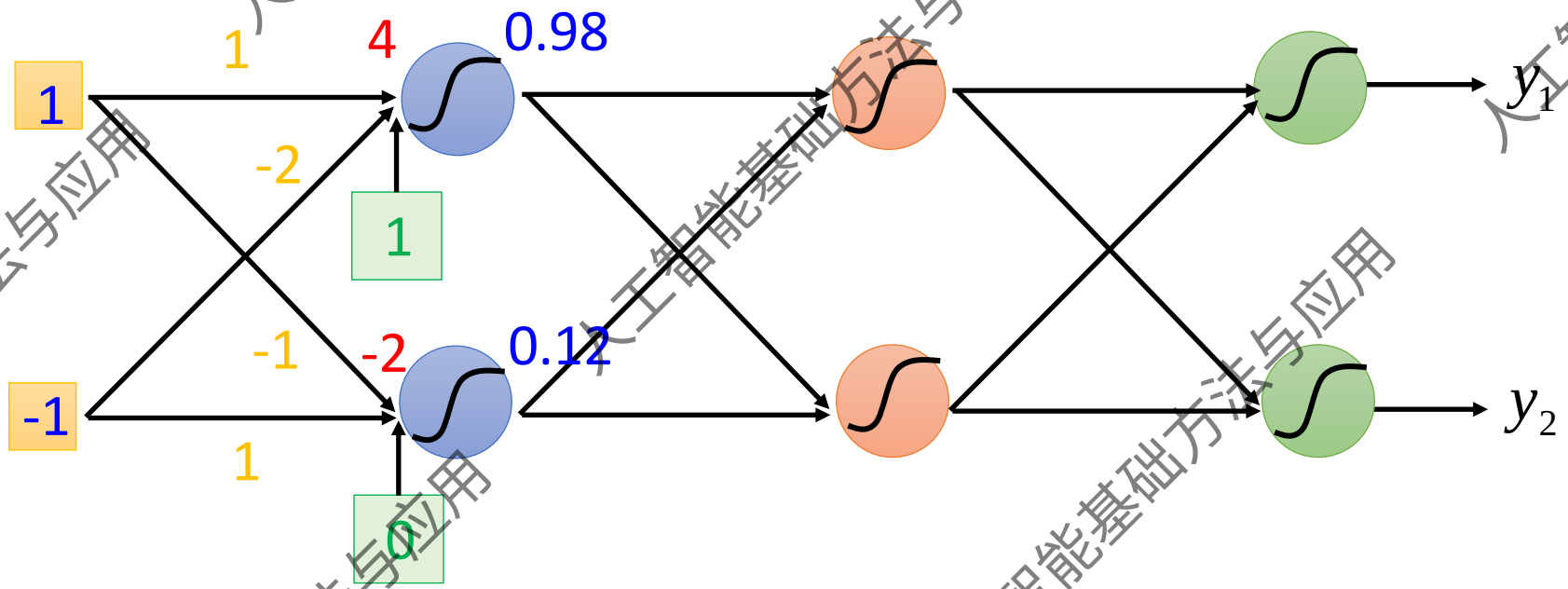
上海中心
大厦

人工智能基础方法与应用

ImageNet大规模视觉识别挑战



用矩阵形式表示神经元运算

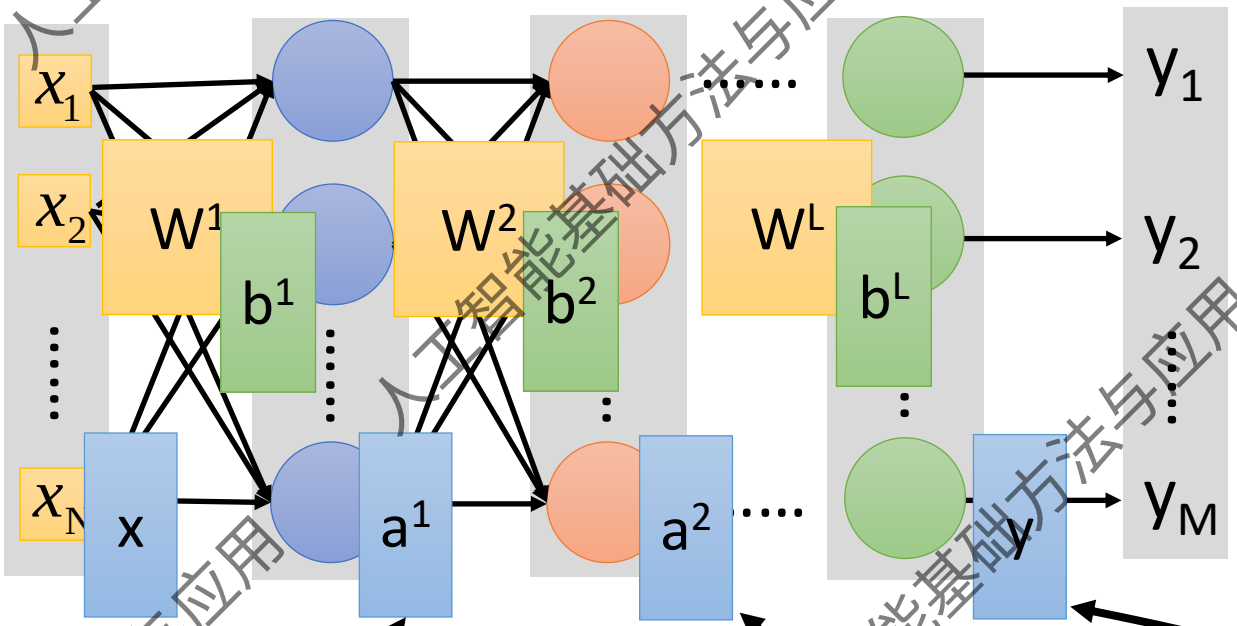


$$\sigma\left(\begin{bmatrix}1 & -2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\ -1\end{bmatrix}+\begin{bmatrix}1\\ 0\end{bmatrix}\right)=\begin{bmatrix}0.98\\ 0.12\end{bmatrix}$$

$$\begin{bmatrix}4\\ -2\end{bmatrix}$$

人工智能基础方法与应用

多层网络的矩阵递推



$$\sigma\left(W^1 x + b^1\right)$$

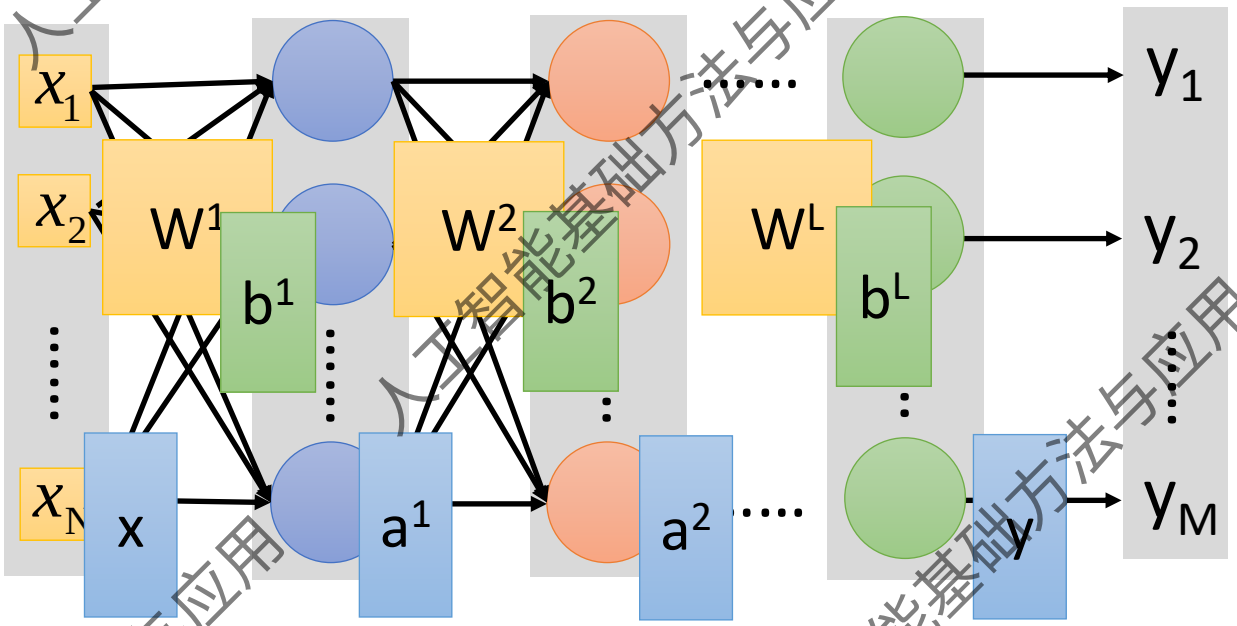
$$\sigma\left(W^2 a^1 + b^2\right)$$

$$\sigma\left(W^L a^{L-1} + b^L\right)$$

矩阵W的行数等于本层神经元数，列数等于输入维度

人工智能基础方法与应用

复合函数与并行计算



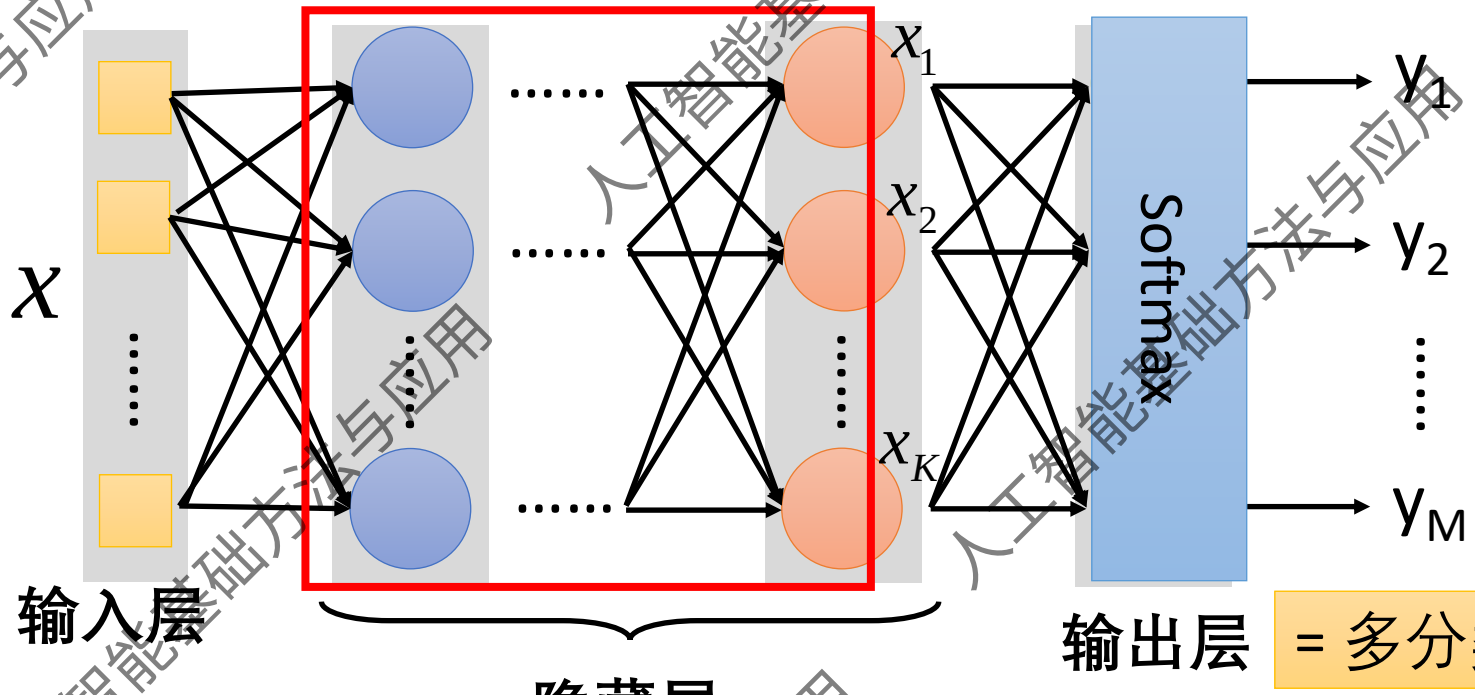
$$y = f(x)$$

使用并行计算技术加速矩阵运算

$$= \sigma(W^L \cdots \sigma(W^2 \sigma(W^1 x + b^1) + b^2) \cdots + b^L)$$

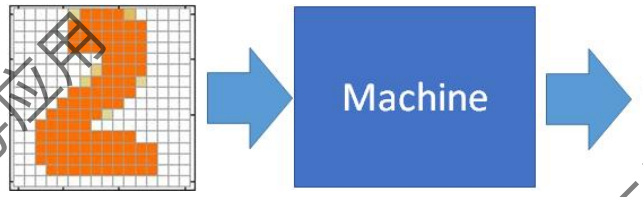
隐藏层提取表示，输出层完成分类

自动特征提取，
替代人工特征工程

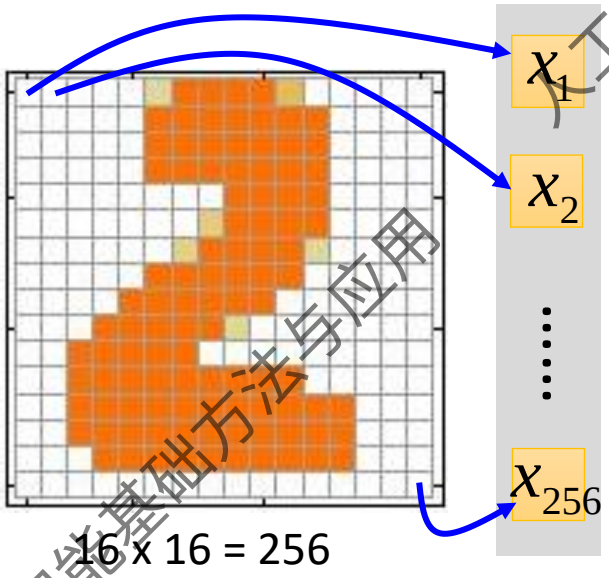


人工智能基础方法与应用

手写数字识别：从图像到概率

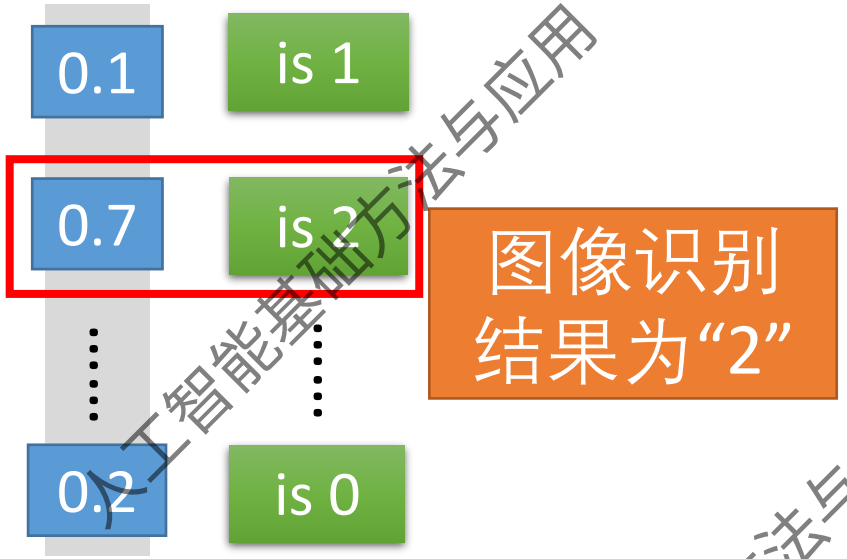


输入



有笔迹 → 1
无笔迹 → 0

输出

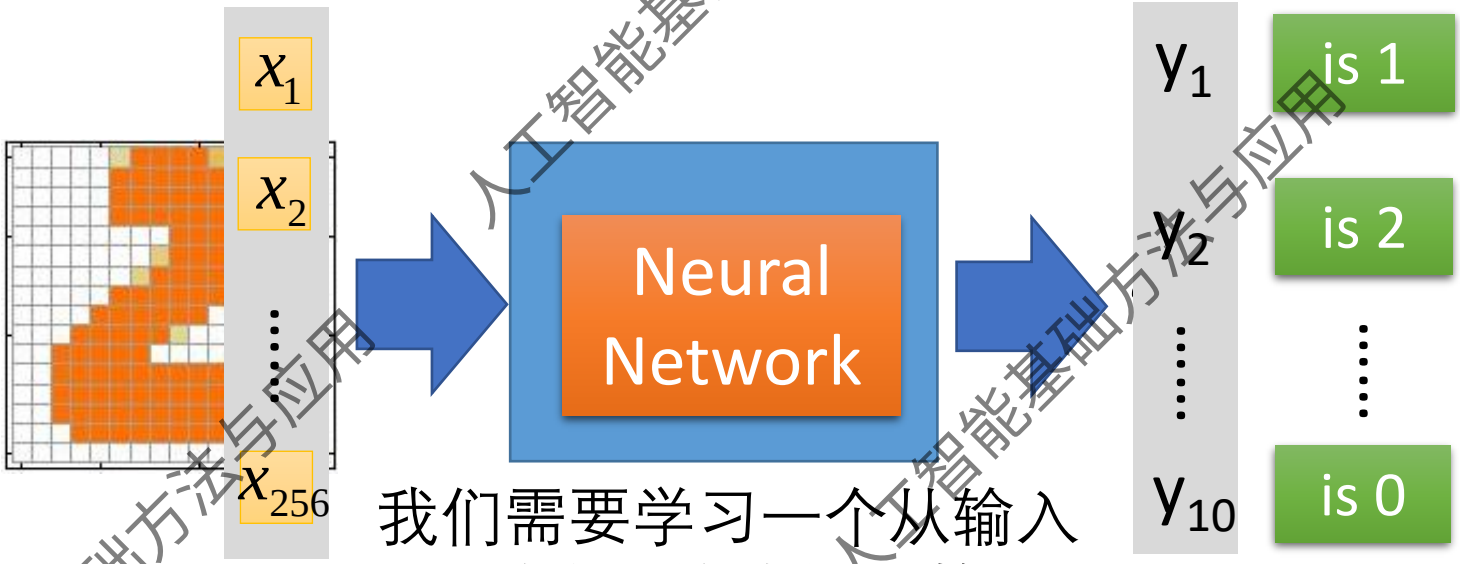


每个输出值表示图像属于对应数字类别的置信度

人工智能基础方法与应用

需要学习一个怎样的函数

- 手写数字识别



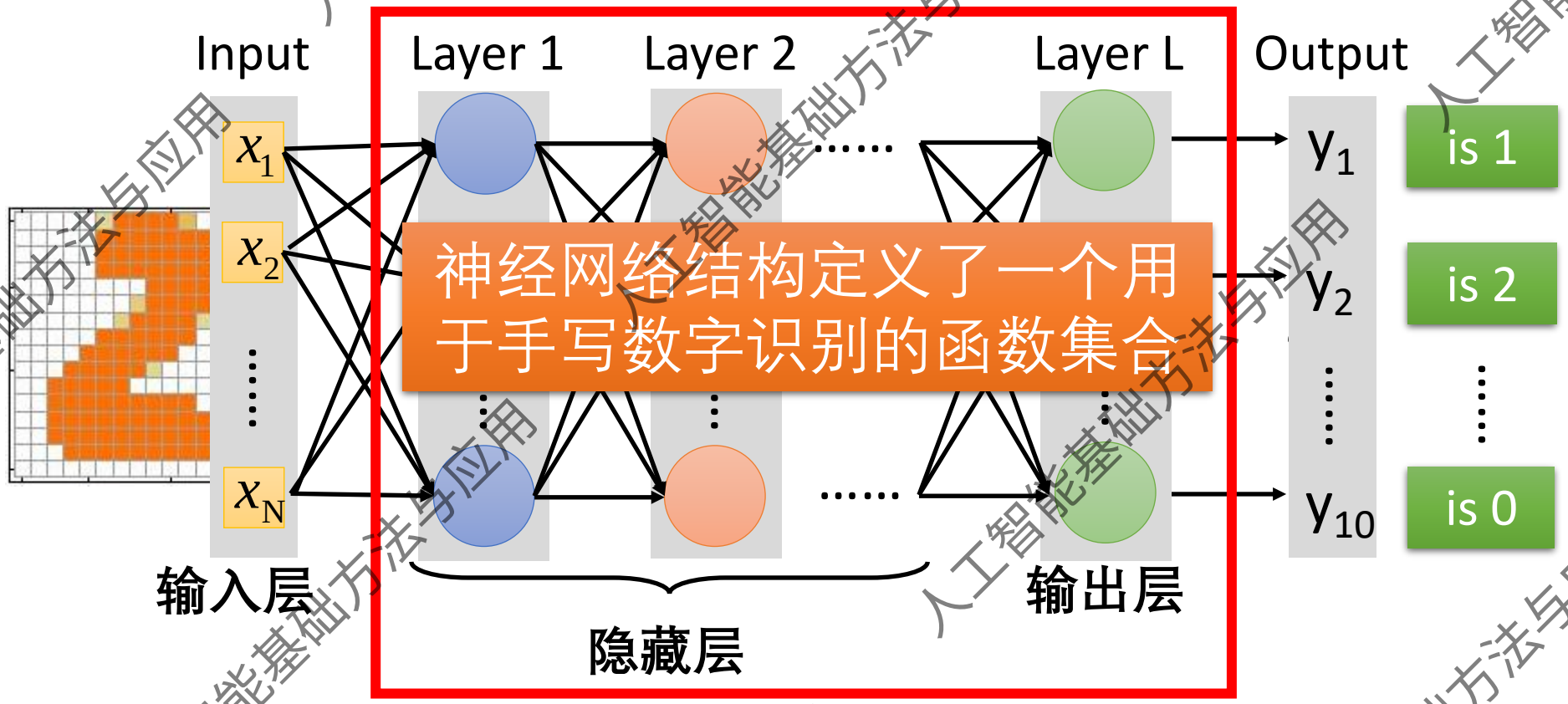
我们需要学习一个从输入到输出的映射函数

输入:
256维向量

输出:
10维向量

人工智能基础方法与应用

网络结构决定候选函数集合



网络结构决定函数集合；好的网络结构更有可能包含适合任务的函数

人工智能基础方法与应用

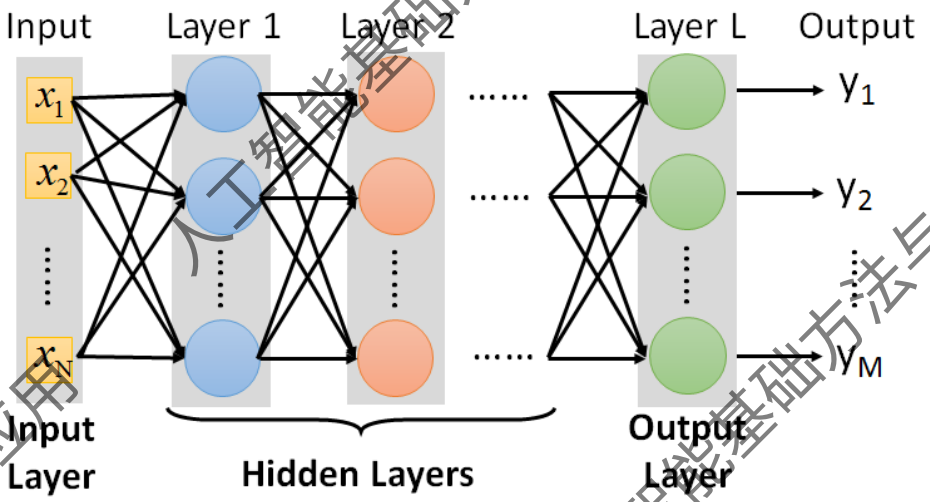
网络结构常见问题

- Q: 网络需要多少层? 每层多少个神经元?

反复试验 + 经验直觉

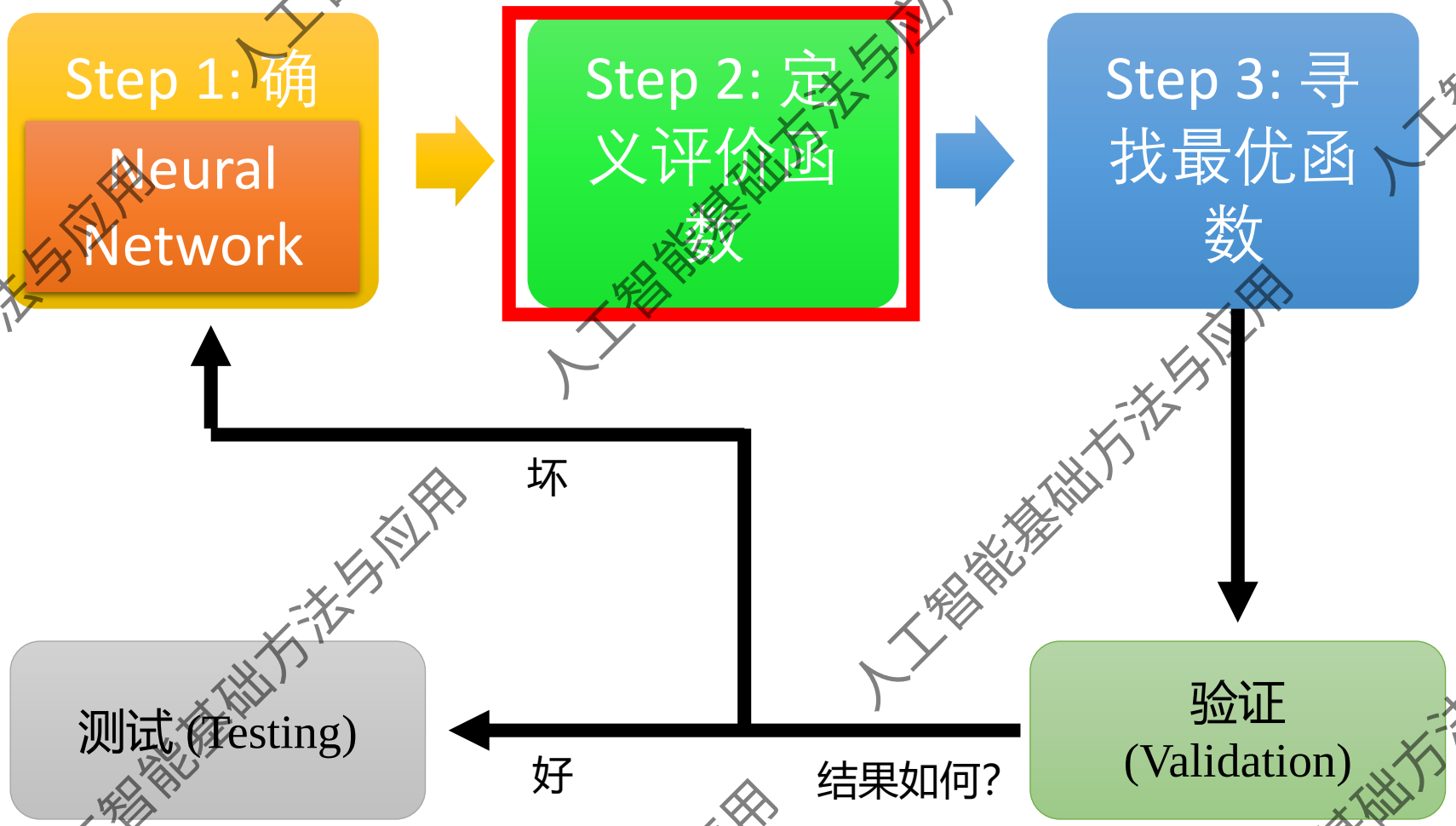
- Q: 网络结构能否自动确定?
  - E.g. 进化人工神经网络
- Q: 我们能否针对任务专门设计网络结构?

卷积神经网络(CNN)



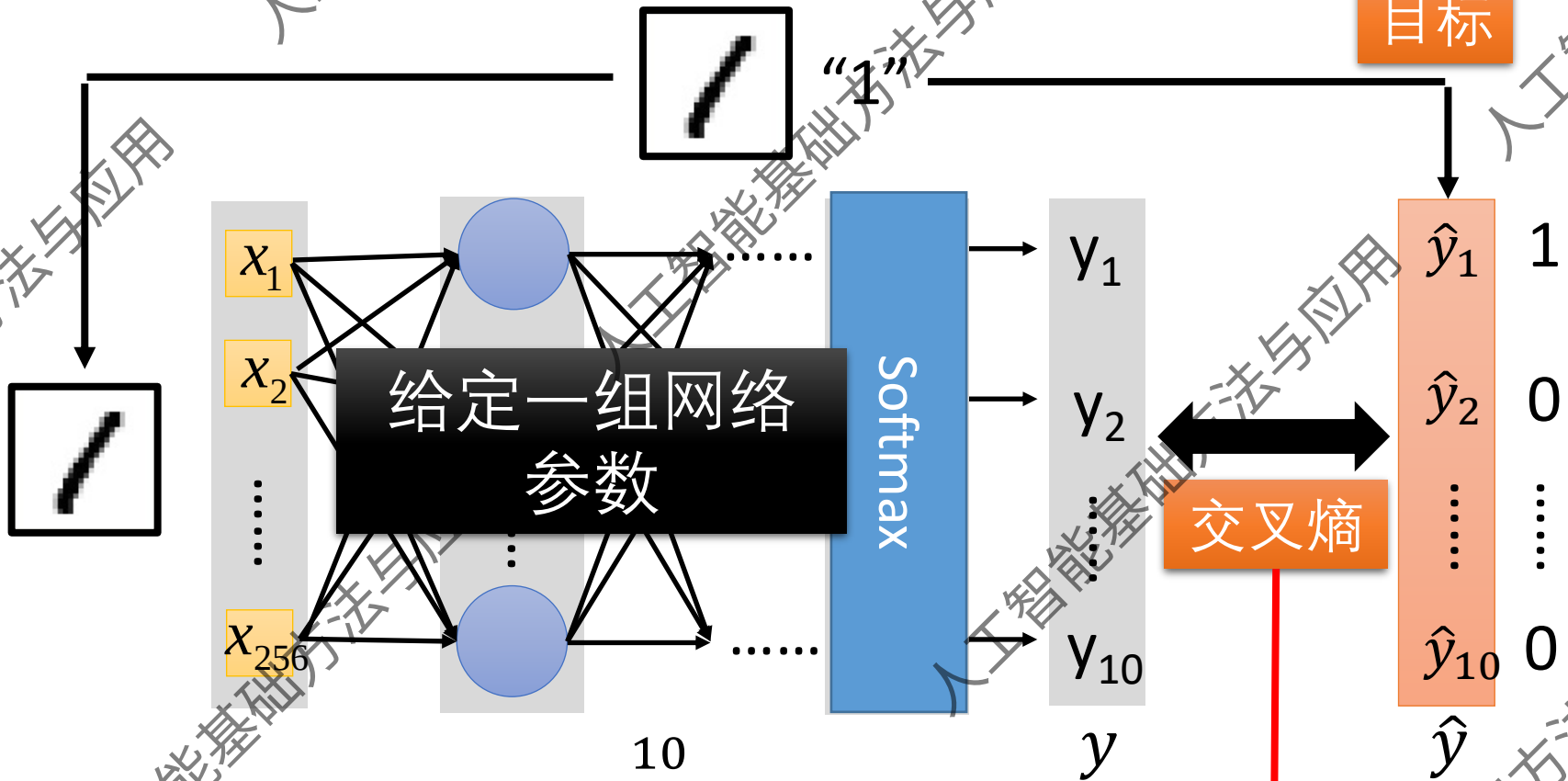
人工智能基础方法与应用

深度学习的三个步骤



人工智能基础方法与应用

单个样本的Softmax交叉熵



$$C(y,\hat{y}) = -\sum_{i=1}^{10} \hat{y}_i ln y_i$$

什么是交叉熵

$$C(y,\hat{y}) = -\sum_{i=1}^{10} \hat{y}_i ln y_i$$

- 假设输入的手写数字真实答案是“1”，那么真实标签采用 one-hot 表示：

$$\hat{y} = [1, 0, 0, \cdots, 0]$$

- 而神经网络经过 Softmax 后可能输出：

$$y = [0.8, 0.05, 0.03, \cdots]$$

- 也就是说，模型认为“是数字1”的概率是 0.8。代入交叉熵：

$$C = -(1 \ln 0.8 + 0 \ln 0.05 + 0 \ln 0.03 + \cdots)$$

- 后面所有乘以 0 的项都消失了，于是：

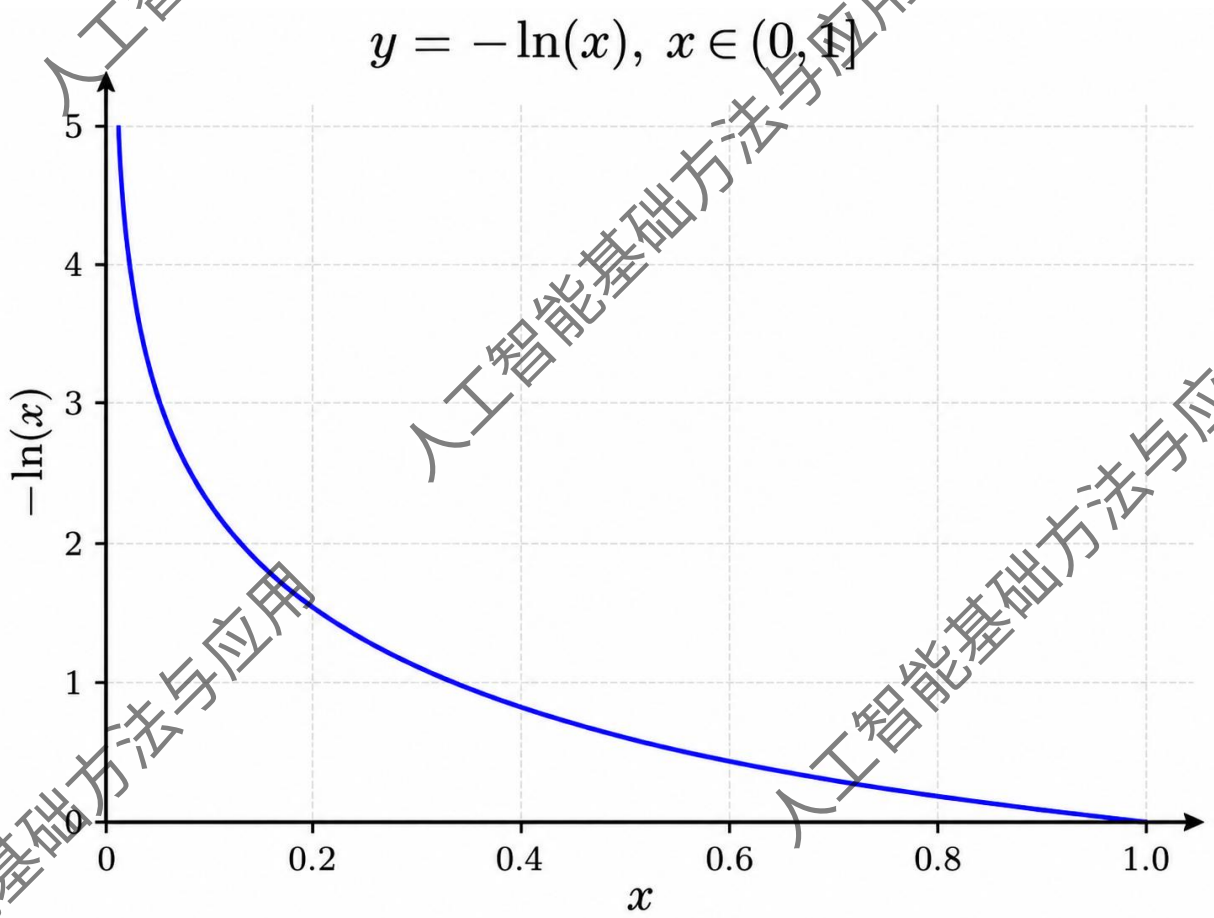
$$\boxed{C = -\ln 0.8}$$

- 所以对于 one-hot 标签，复杂的交叉熵公式实际上可以简单理解成：

$$\boxed{\text{交叉熵损失} = -\ln(\text{正确类别的预测概率})}$$

人工智能基础方法与应用

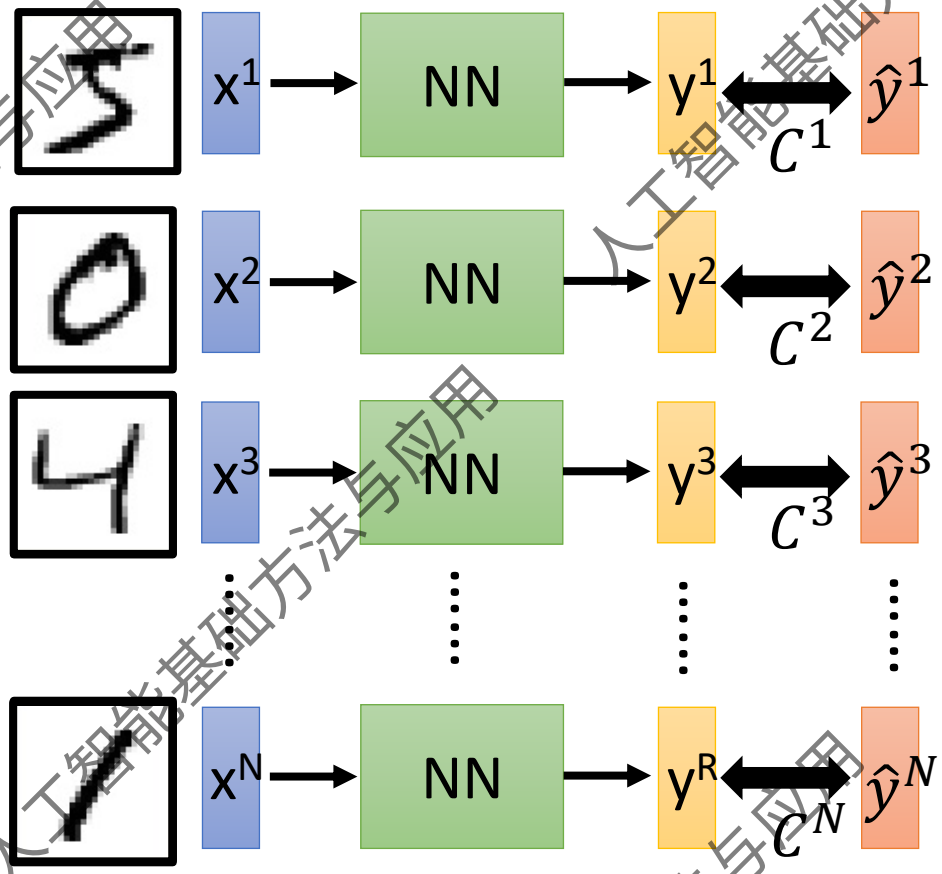
什么是交叉熵



人工智能基础方法与应用

训练集上的整体损失

对于所有的训练数据



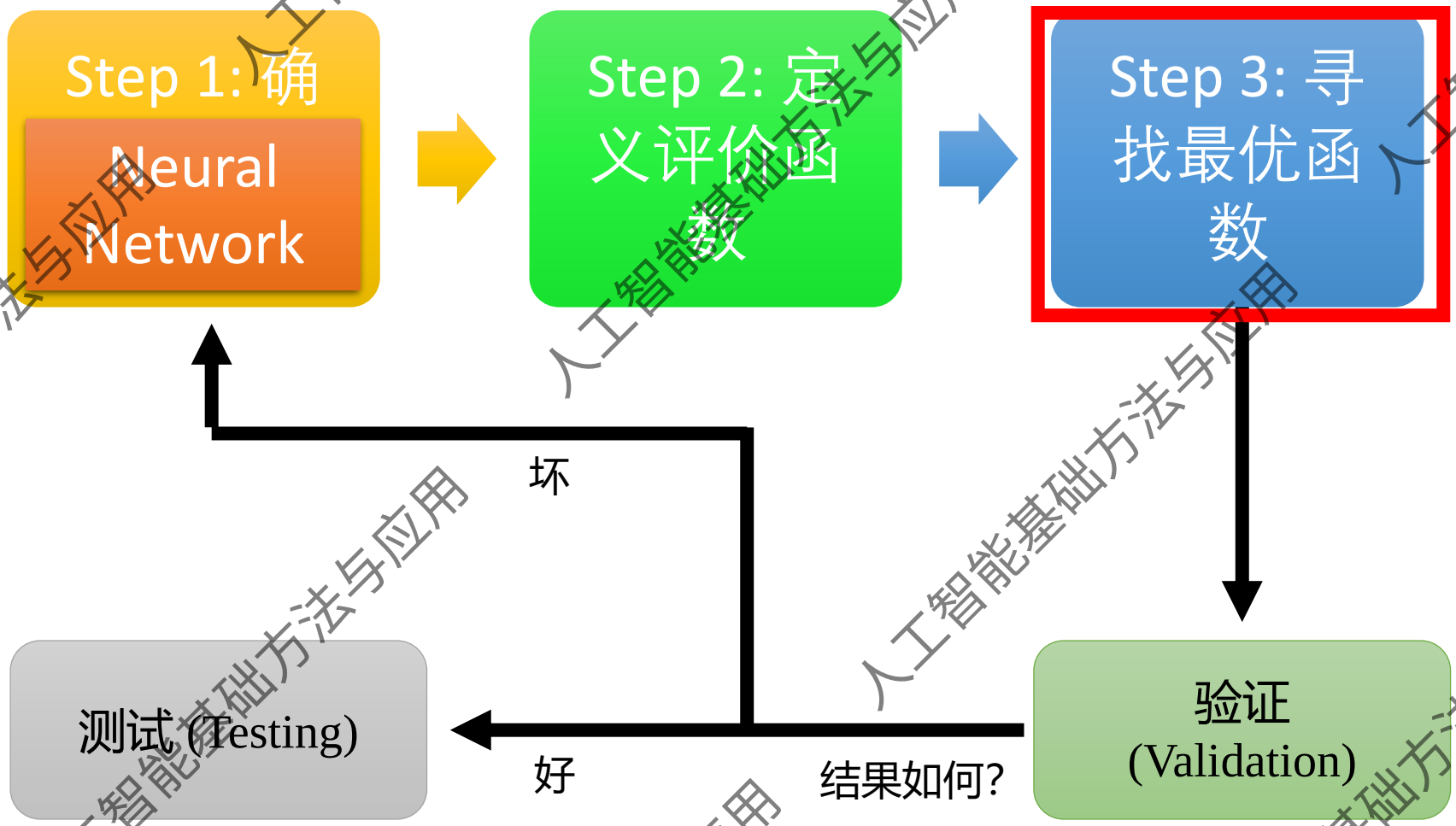
Total Loss:

$$L = \sum_{n=1}^{N} C^n$$

在 ***候选函数集合中寻找一个最优函数*** 能够最小化总损失L

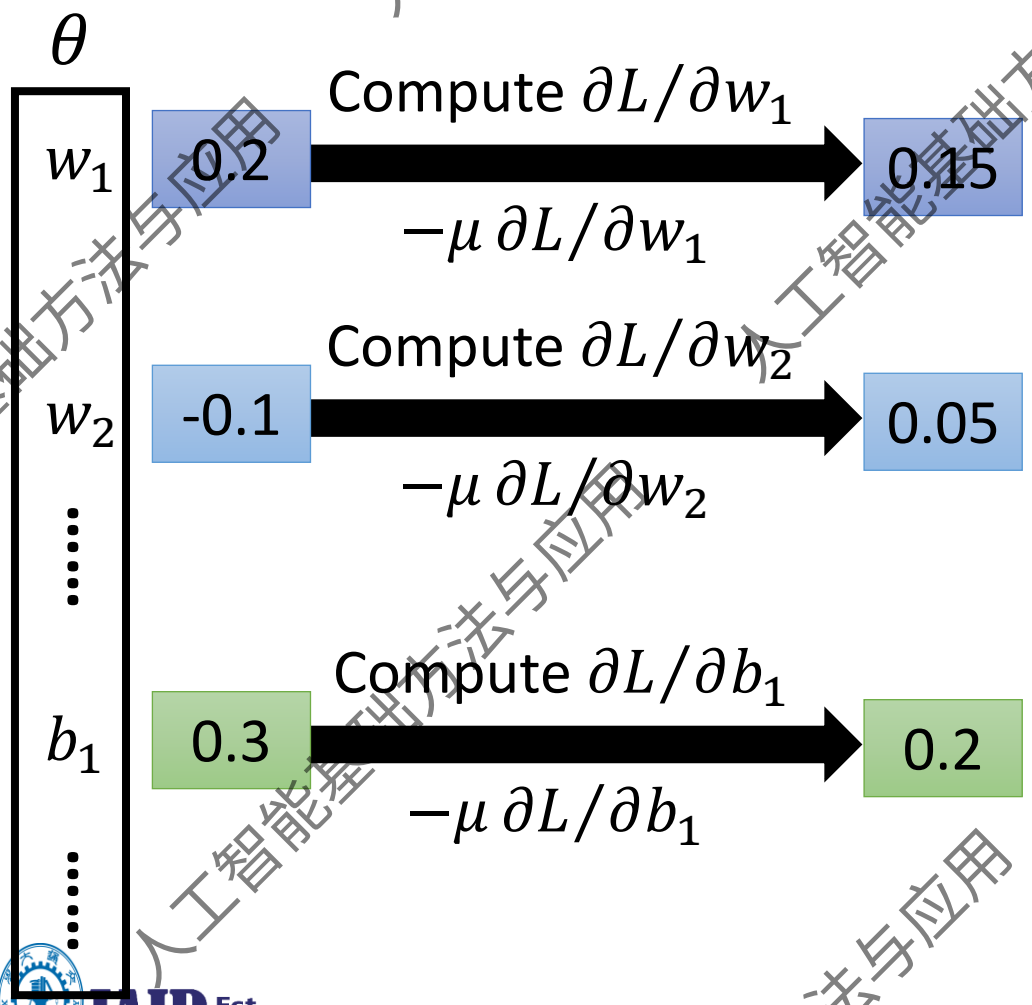
寻找 ***最优神经网络参数θ^**** 能够最小化总损失L

深度学习的三个步骤



人工智能基础方法与应用

梯度下降：完成一次参数更新

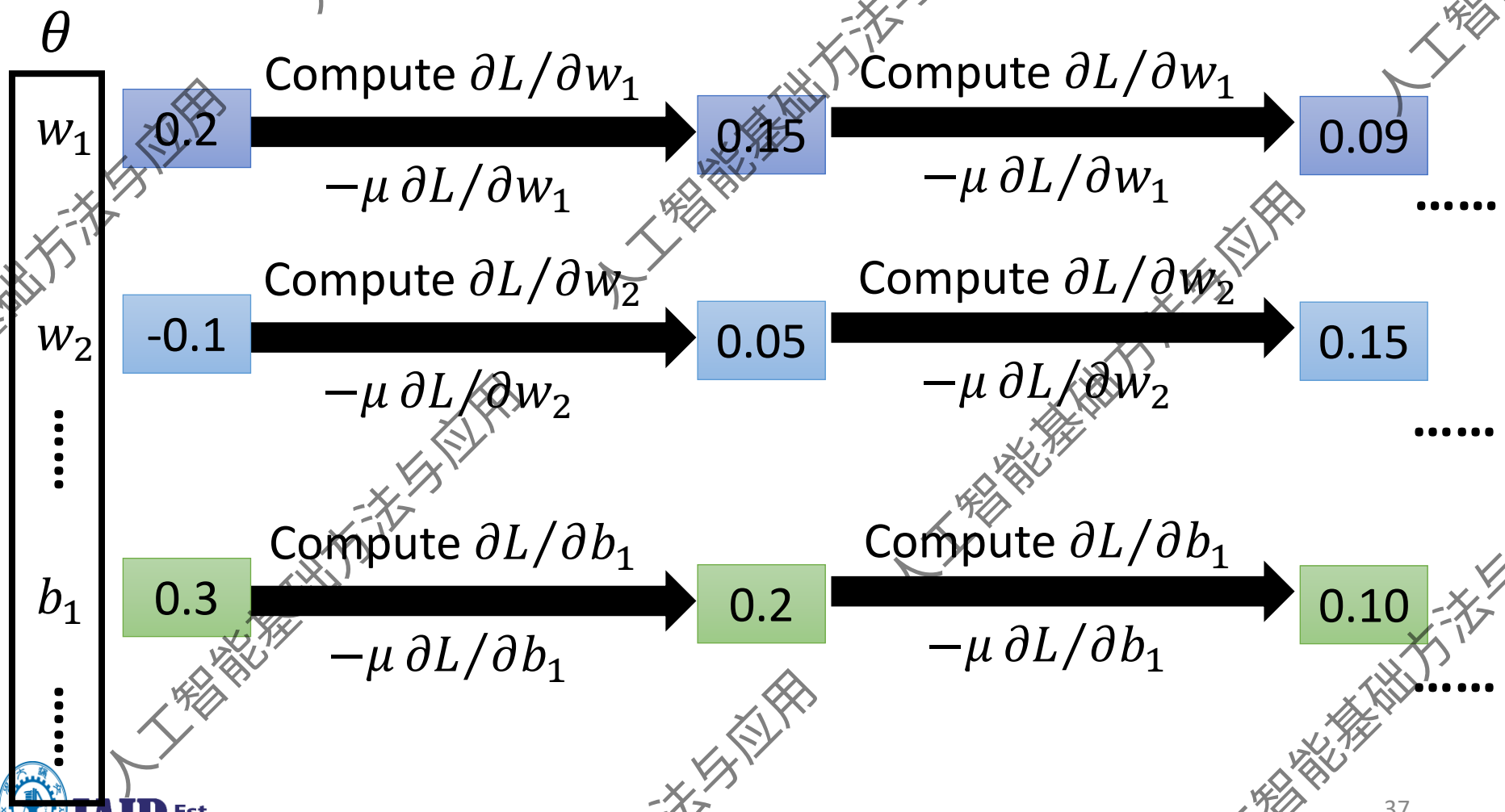


$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_1} \\ \vdots \end{bmatrix}$$

gradient

人工智能基础方法与应用

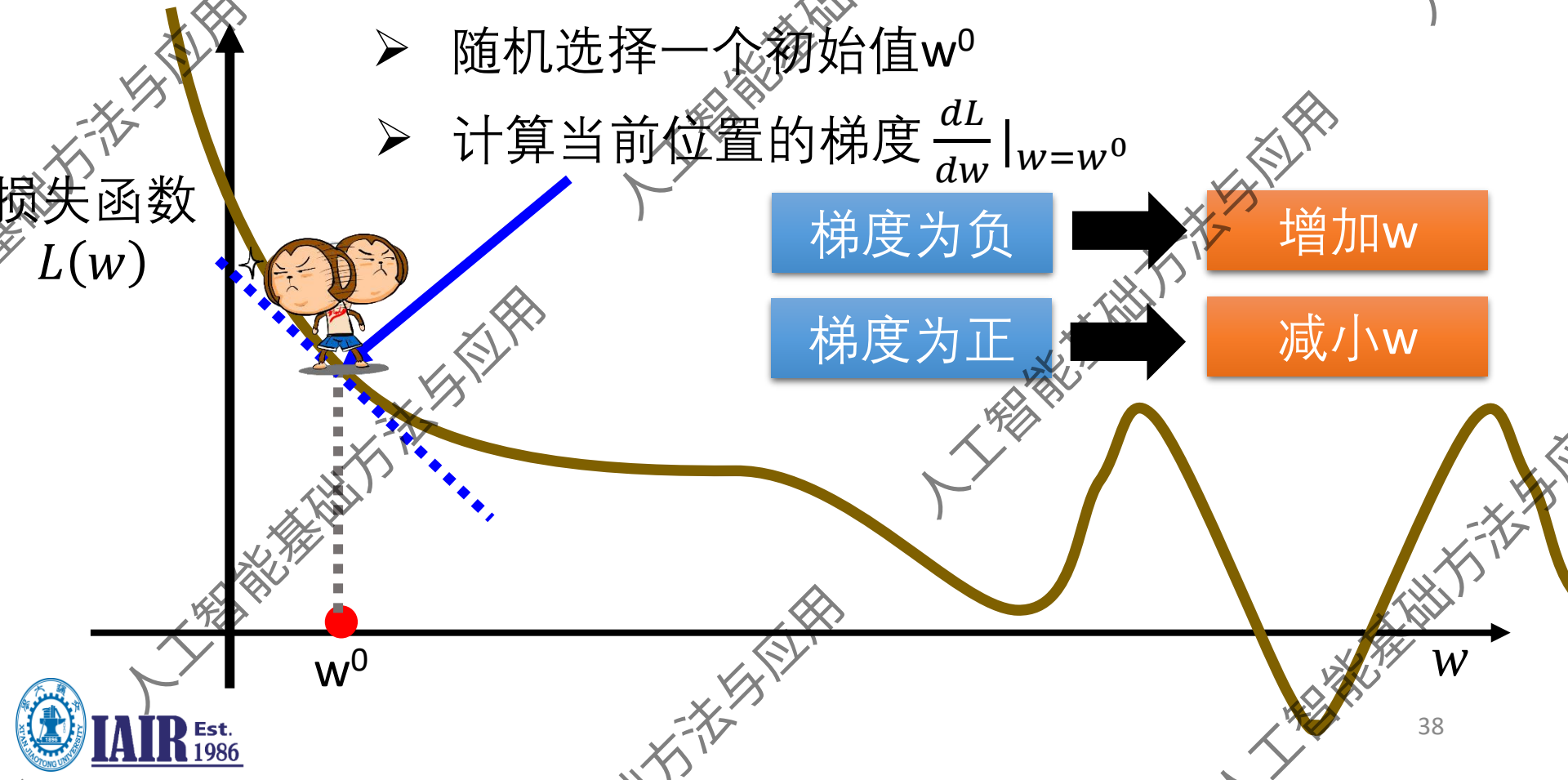
梯度下降：重复迭代



人工智能基础方法与应用

梯度下降：先判断往哪边走

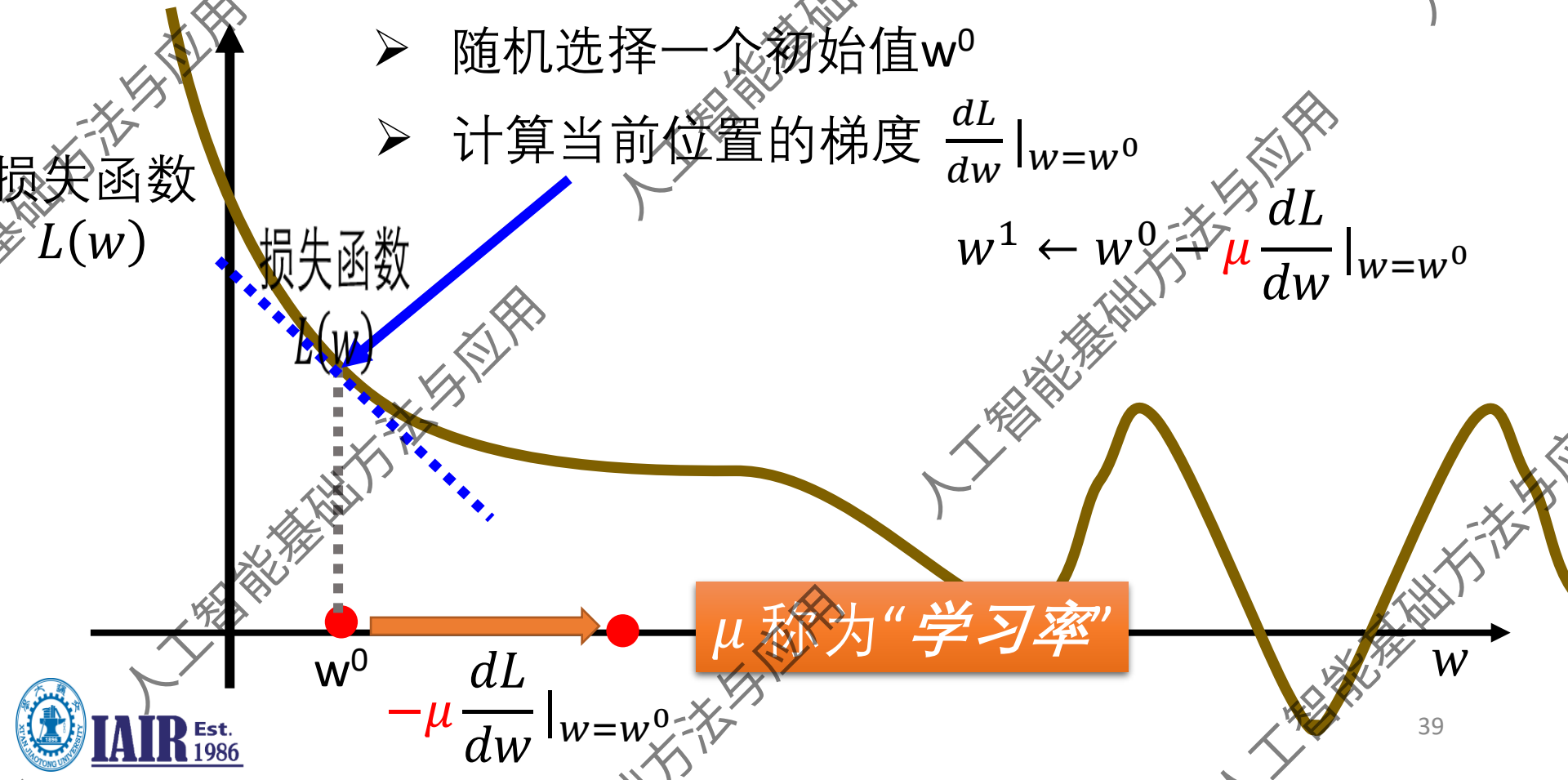
- 考虑只有一个参数w的损失函数$L(w)$：　$w^* = arg\min_w L(w)$



人工智能基础方法与应用

学习率决定每一步的大小

- 考虑只有一个参数w的损失函数$L(w)$：　$w^* = arg\min_w L(w)$



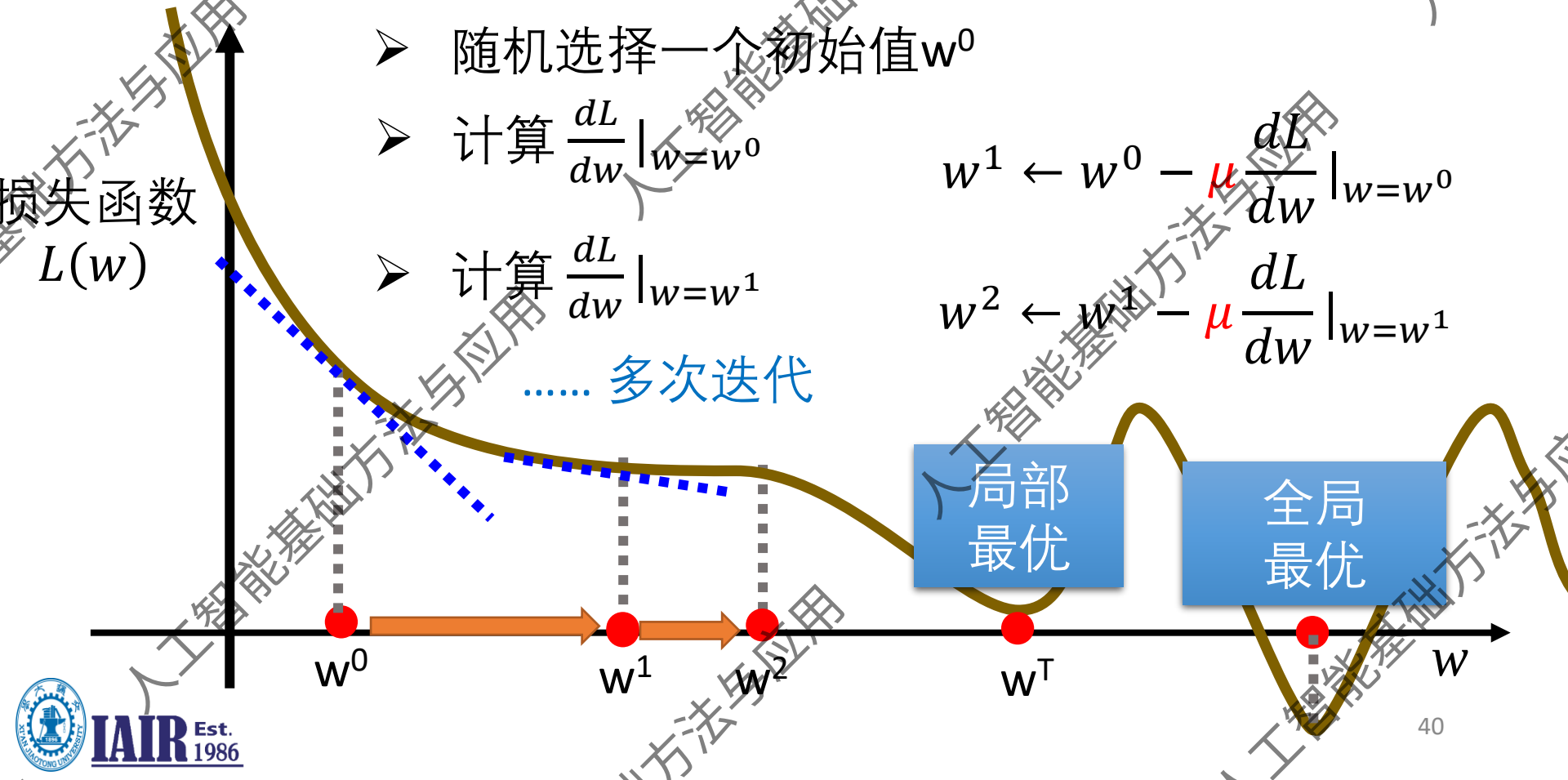
- 随机选择一个初始值w^0
- 计算当前位置的梯度 $\frac{dL}{dw}|_{w=w^0}$

$$w^1 \leftarrow w^0 - \mu \frac{dL}{dw}|_{w=w^0}$$

人工智能基础方法与应用

反复更新直到损失不再明显下降

- 考虑只有一个参数w的损失函数$L(w)$：　$w^* = arg\min_w L(w)$



人工智能基础方法与应用

梯度

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$ gradient

- 两个参数如何计算梯度?

$$w^*, b^* = arg \min_{w,b} L(w, b)$$

- 随机选择初始值w^0, b^0
- 计算 $\frac{\partial L}{\partial w}|_{w=w^0,b=b^0}, \frac{\partial L}{\partial b}|_{w=w^0,b=b^0}$

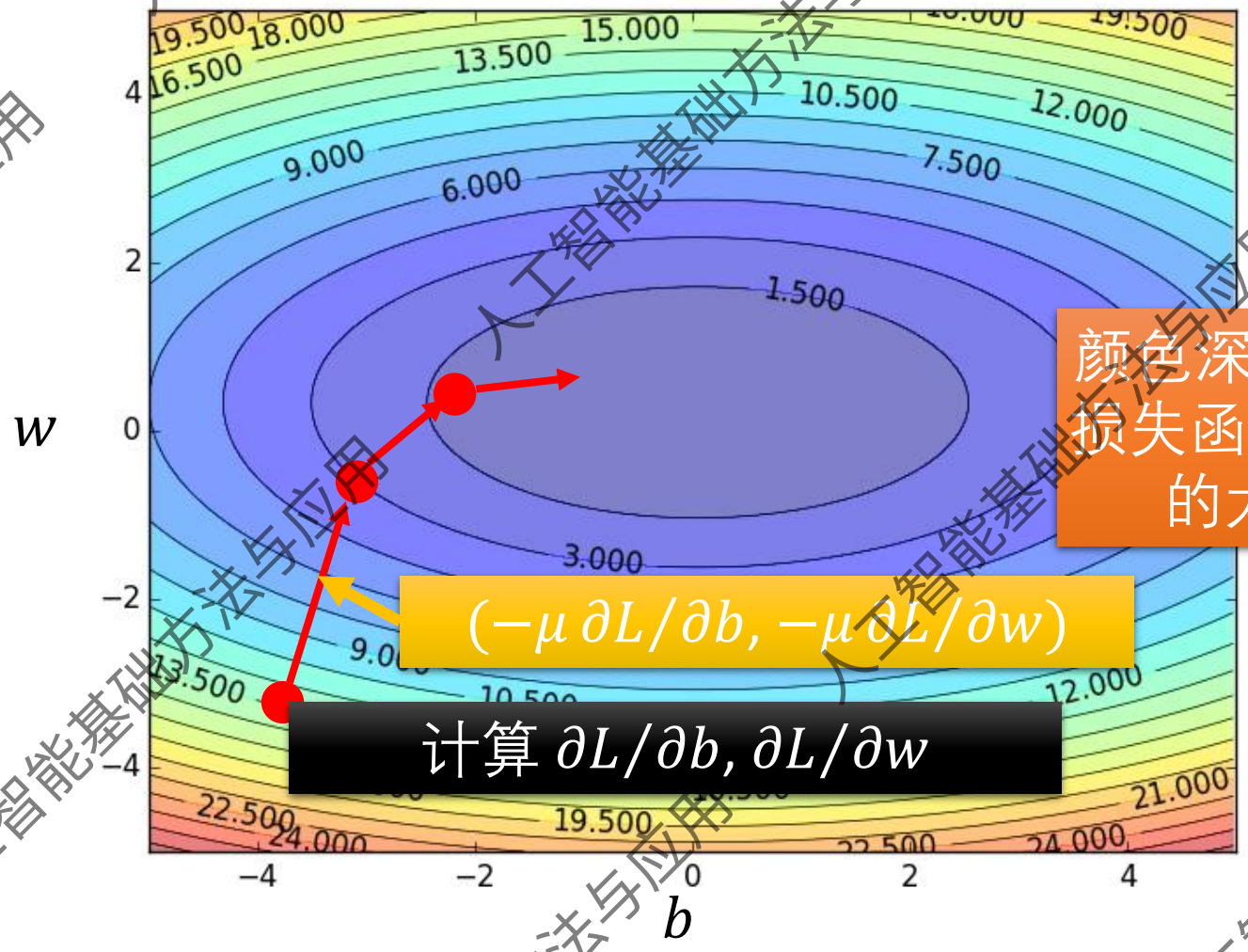
$$w^1 \leftarrow w^0 - \mu \frac{\partial L}{\partial w}|_{w=w^0,b=b^0} \qquad b^1 \leftarrow b^0 - \mu \frac{\partial L}{\partial b}|_{w=w^0,b=b^0}$$

- 计算 $\frac{\partial L}{\partial w}|_{w=w^1,b=b^1}, \frac{\partial L}{\partial b}|_{w=w^1,b=b^1}$

$$w^2 \leftarrow w^1 - \mu \frac{\partial L}{\partial w}|_{w=w^1,b=b^1} \qquad b^2 \leftarrow b^1 - \mu \frac{\partial L}{\partial b}|_{w=w^1,b=b^1}$$

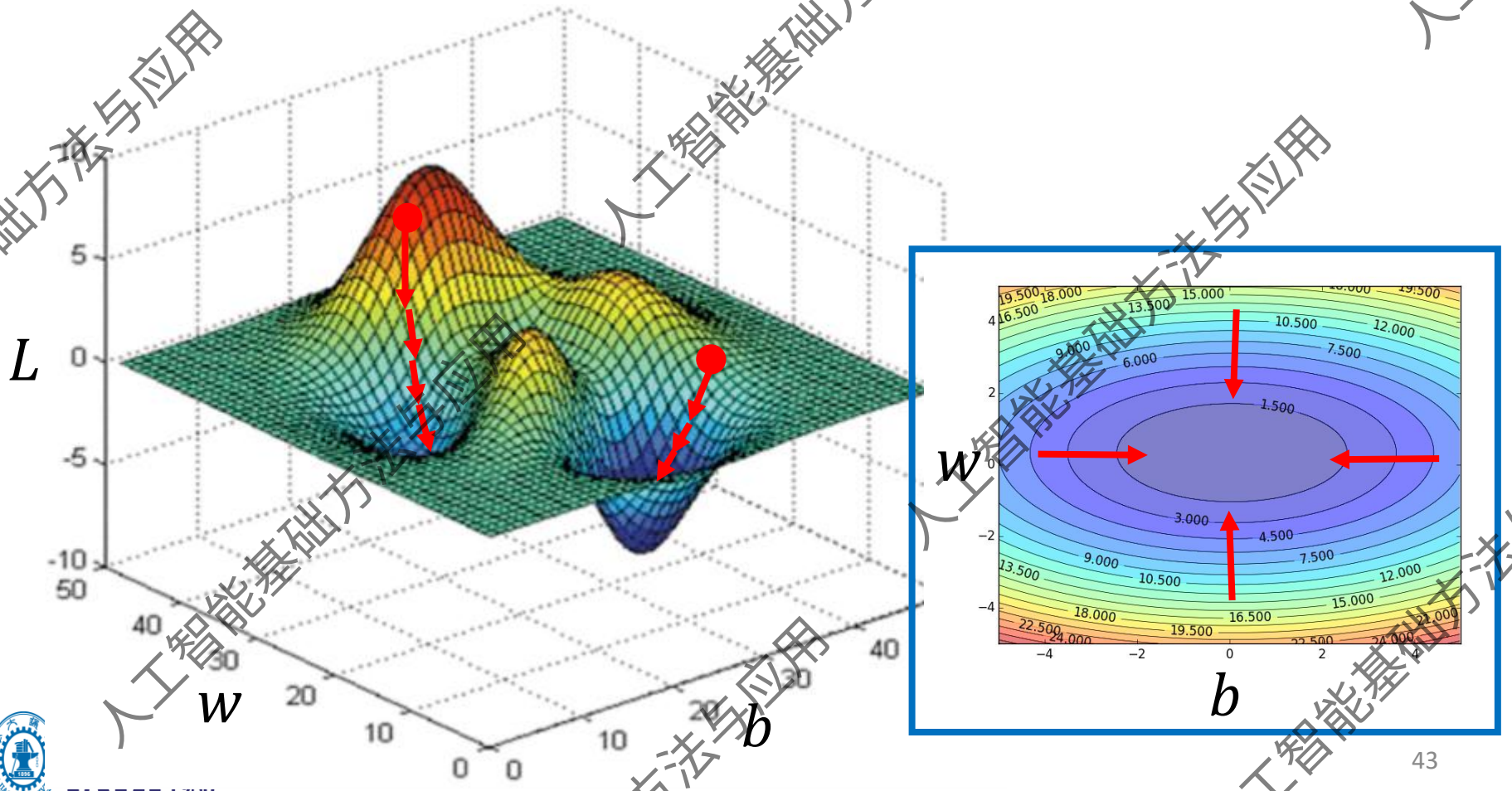
人工智能基础方法与应用

在二维损失地形下降


19.500
18.000
16.500
15.000
13.500
10.500
12.000
9.000
7.500
6.000
1.500
3.000
22.500
24.000
19.500
21.000
4
2
0
−2
−4
−4
−2
0
2
4
w
b
颜色深浅表示损失函数L(w,b)的大小
$(-\mu\,\partial L/\partial b, -\mu\,\partial L/\partial w)$
计算 $\partial L/\partial b, \partial L/\partial w$

神经网络的优化

- 深度神经网络训练通常面对非凸损失地形，因此一般无法保证梯度下降找到全局最优



人工智能基础方法与应用

机器的“学习”没有那么神秘

这就是深度学习中机器“学习”的基本过程……

➡ 甚至 AlphaGo 也使用了类似的方法

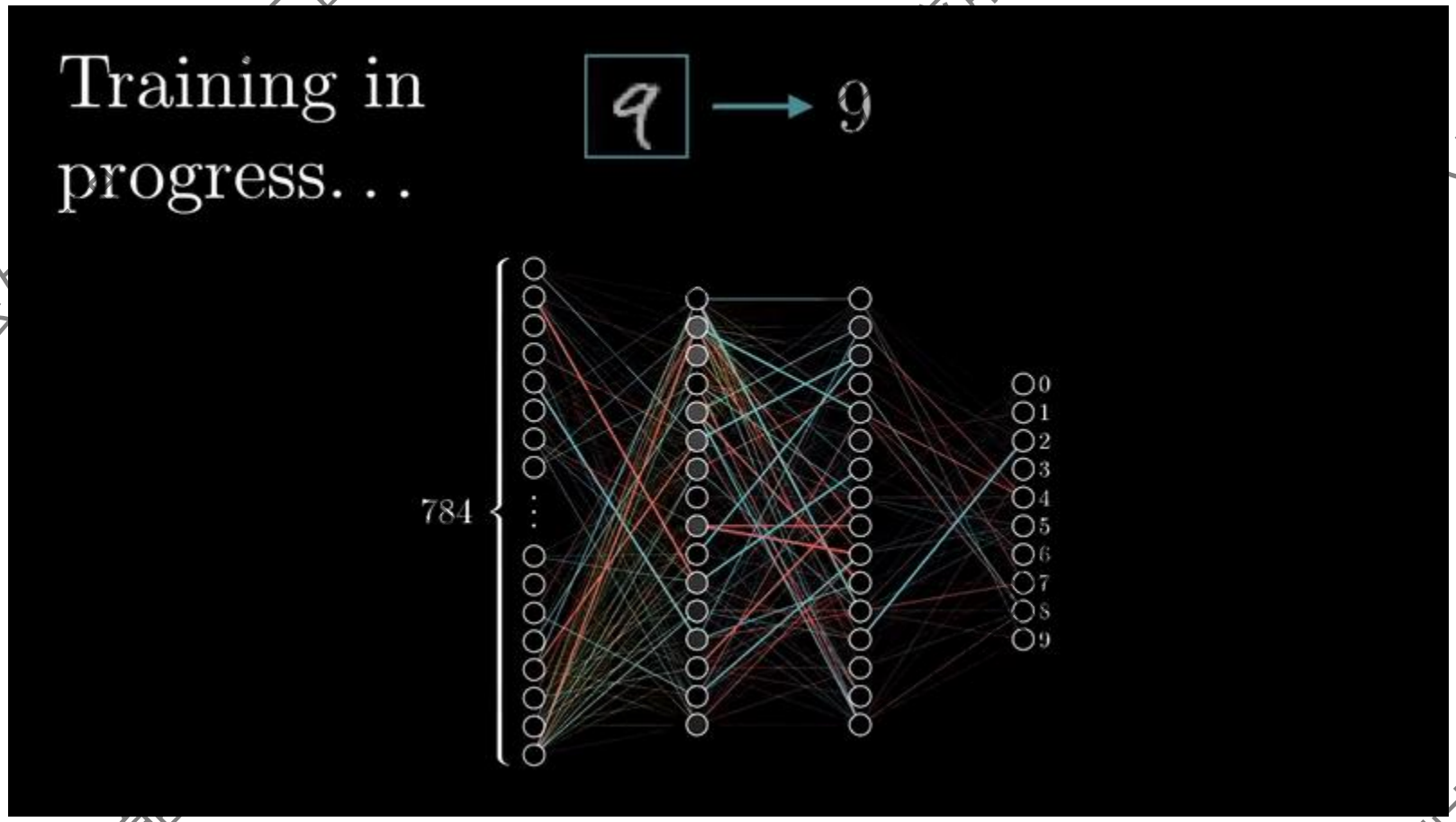
人们想象中的 AI……



实际上的训练过程 ……



机器学习的“学习”本质上就是不断计算、比较误差并调整参数


Training in progress...
9
784
0
1
2
3
4
5
6
7
8
9

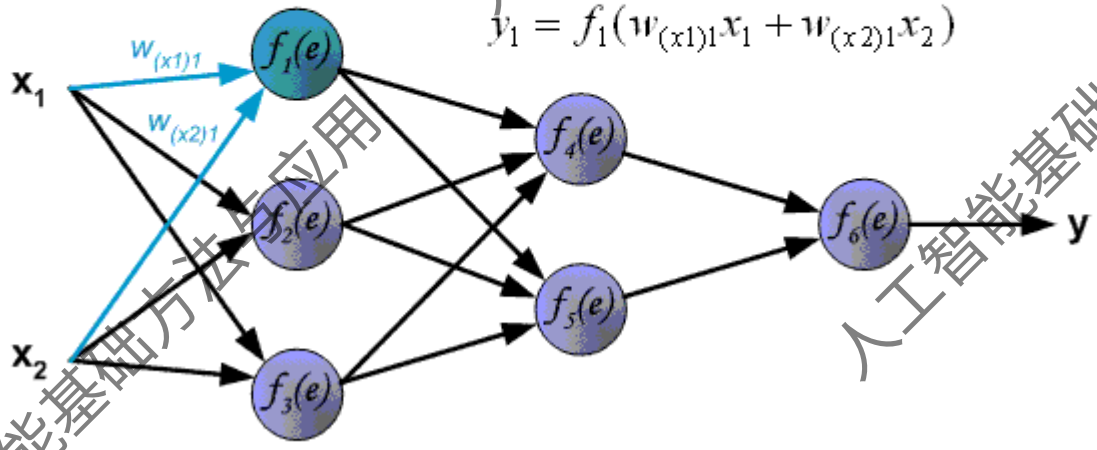
反向传播

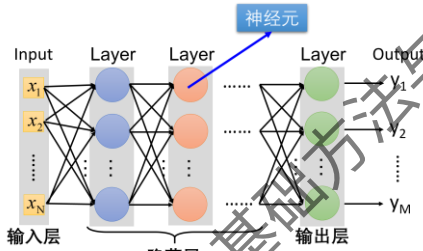
$$\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

链式法则

- 利用**链式法则**，从输出层向输入层逐层传播，计算损失函数对各个参数的梯度

FP

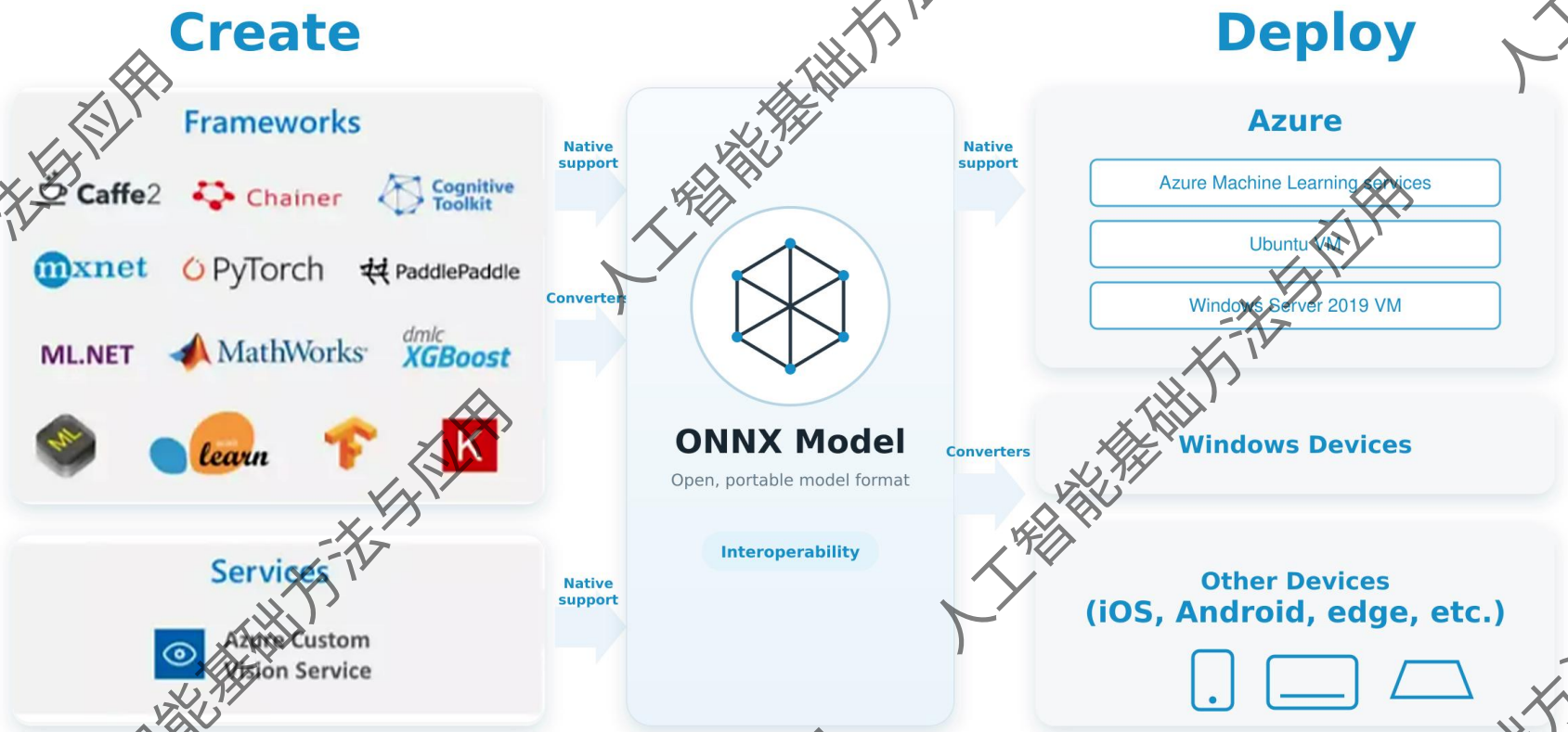




全连接前馈神经网络

反向传播

- 在神经网络中高效计算$\partial L/\partial w$的方式



人工智能基础方法与应用

为什么需要深层结构



人工智能基础方法与应用

越深越好?

语音识别实验

隐藏层数	词错误率(%)
1 X 2k	24.2
2 X 2k	20.4
3 X 2k	18.4
4 X 2k	17.8
5 X 2k	17.2
7 X 2k	17.1

随着网络规模增大、参数增多，词错误率逐渐下降

人工智能基础方法与应用

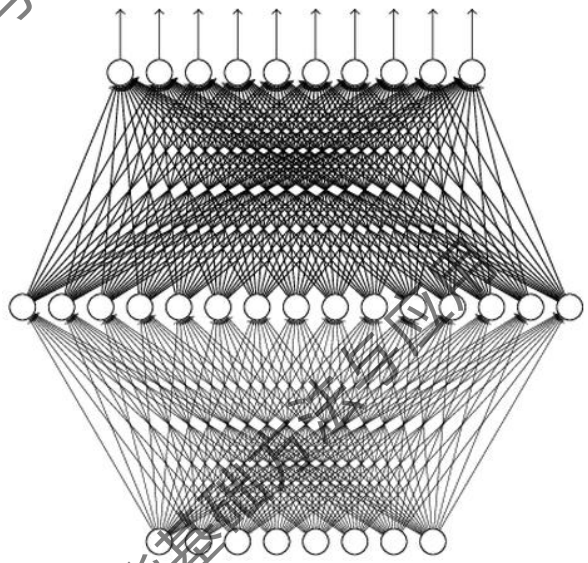
万能逼近定理

任意连续函数f

$$f: R^N \rightarrow R^M$$

都可以由一个只有一层隐藏层的神经网络进行逼近

(前提是：隐藏层中有足够多的神经元)



既然一层隐藏层就可以逼近任意连续函数，为什么还要使用“深”神经网络，而不是“宽”神经网络？

万能逼近定理

隐藏层数	词错误率(%)	隐藏层数	词错误率(%)
1 X 2k	24.2		
2 X 2k	20.4		
3 X 2k	18.4		
4 X 2k	17.8		
5 X 2k	17.2	1 X 3772	22.5
7 X 2k	17.1	1 X 4634	22.6
		1 X 16k	22.1

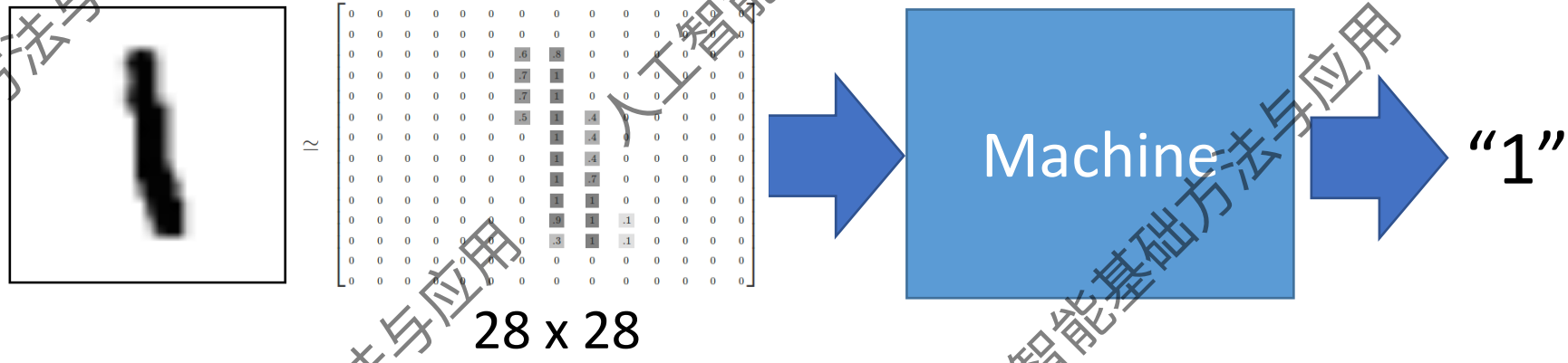
人工智能基础方法与应用

二、完整应用示例

人工智能基础方法与应用

手写数字识别

- MNIST 是最经典的手写数字识别数据集之一，包含 **60,000 张训练图像和 10,000 张测试图像**，每张图像大小为 **28×28 灰度图**，类别为数字 0–9



“Hello world” for deep learning

数据集下载地址：http://yann.lecun.com/exdb/mnist/

PyTorch：深度学习开发框架



人工智能基础方法与应用

PyTorch：深度学习开发框架

- 一个开源深度学习框架，广泛用于人工智能研究和工程开发。它提供了从**张量计算、神经网络构建、自动求导，到模型训练与部署**的一整套工具
- 核心功能
  - **Tensor 张量计算：**类似 NumPy，但可以利用 GPU 加速大规模矩阵运算
  - **自动求导 Autograd：**为反向传播和参数更新提供基础
  - **神经网络模块 torch.nn：**可以方便地搭建全连接网络、CNN、Transformer 等模型
  - **优化器 torch.optim：**提供 SGD、Adam 等常用优化算法
  - **数据处理Dataset/DataLoader：**支持训练数据的读取、批处理和随机打乱

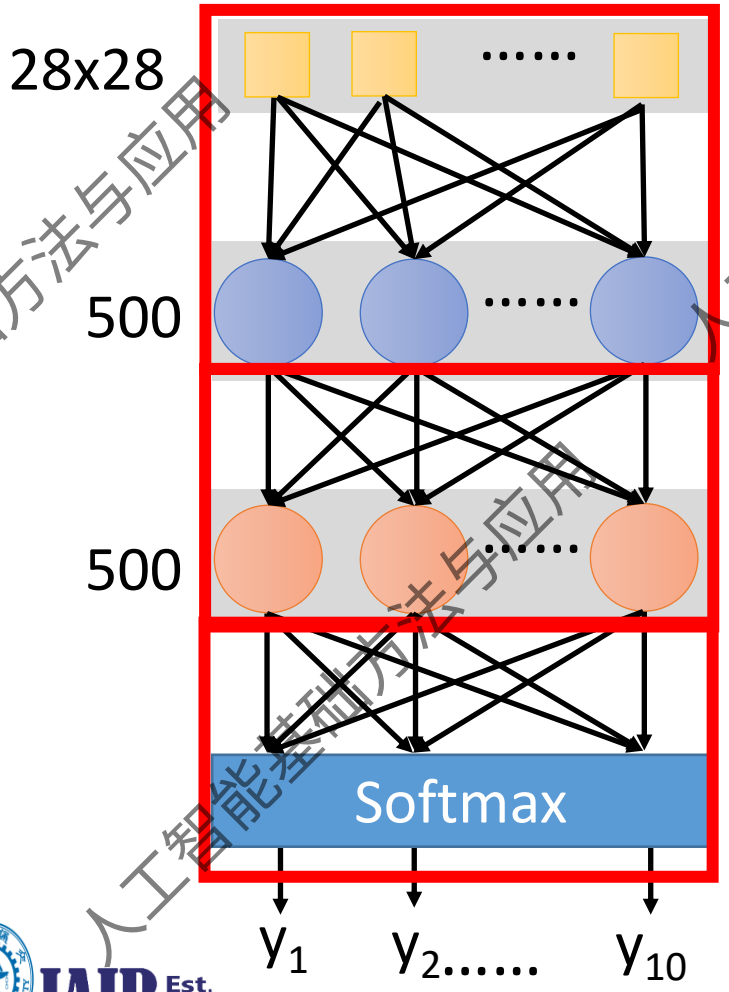
人工智能基础方法与应用

PyTorch

Step 1: define a set of function

Step 2: goodness of function

Step 3: pick the best function

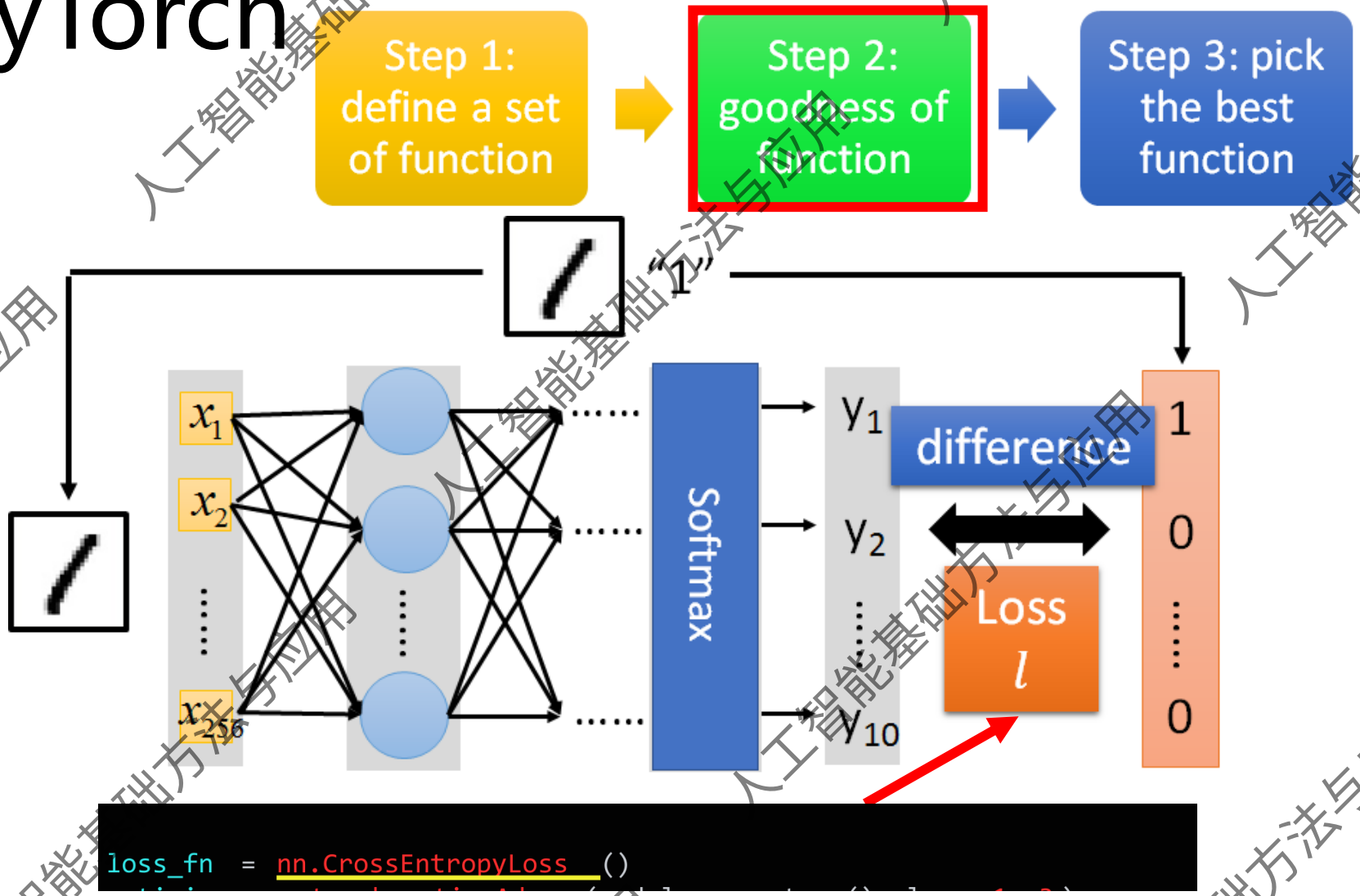



```
model = nn.Sequential (
    nn.Flatten (),
    nn.Linear (28 * 28, 500),
    nn.Sigmoid (),
```

softplus, softsign, relu, tanh, hard_sigmoid, linear

```
    nn.Linear (500, 500),
    nn.Sigmoid (),
    nn.Linear (500, 10)
)
```

PyTorch



人工智能基础方法与应用

PyTorch



Step 3.1: 设定配置

```
loss_fn   = nn.CrossEntropyLoss ()
optimizer   = torch.optim.Adam  (model.parameters(), lr=   1e-3 )
```

SGD, RMSprop, Adagrad, Adadelta, Adam, Adamax, Nadam

Step 3.2: 找到最优的网络参数

```
ds = TensorDataset (x_train ,        y_train ); loader  = DataLoader (ds,  batch size=100  , shuffle= True )
```

训练数据集 (Images)

真实标签 (digits)

数据加载方式

PyTorch

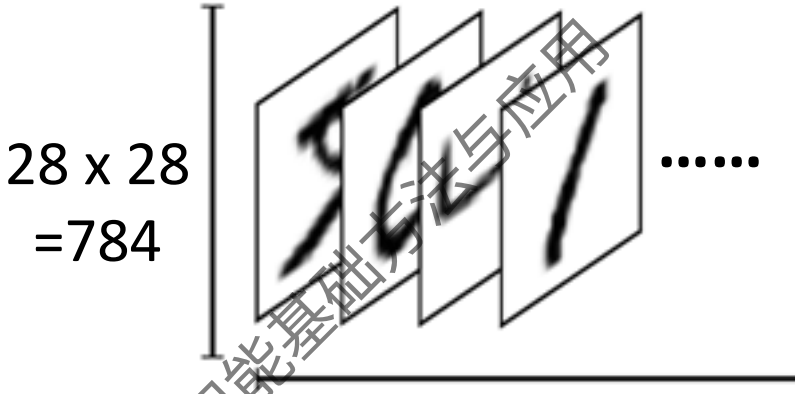


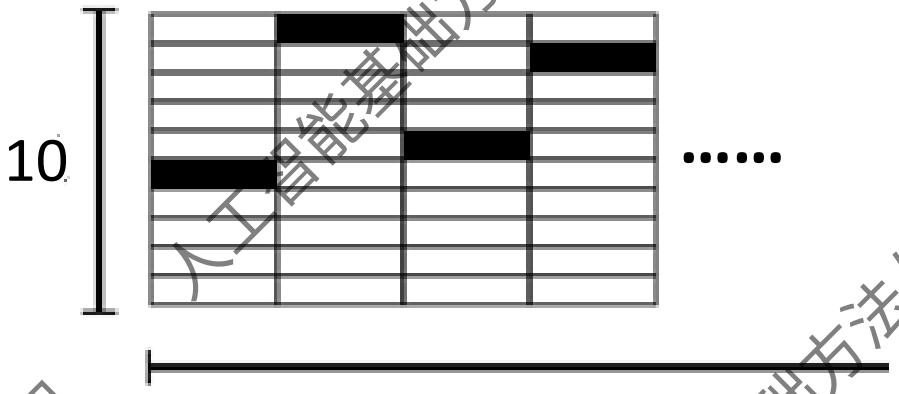
Step 3.2: 找到最优的网络参数

```
ds = TensorDataset (x_train ,        y_train ); loader = DataLoader (ds, batch size=100 , shuffle= True )
```

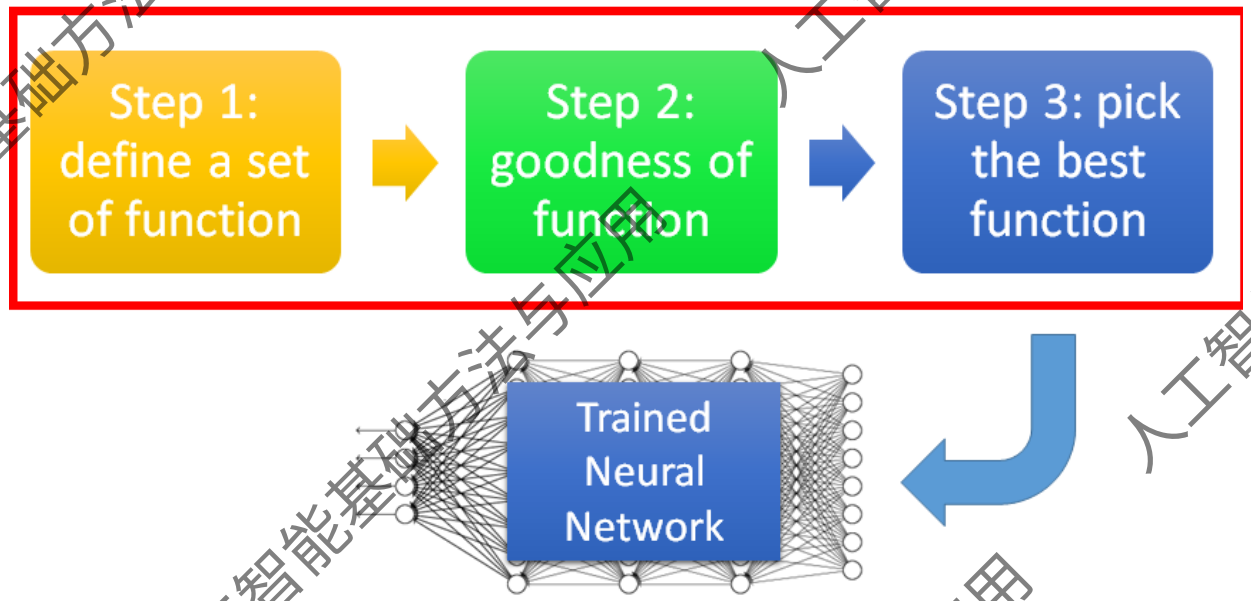
image tensor

label tensor





PyTorch



保存和加载模型

pytorch.org/tutorials/beginner/saving_loading_models.html

如何使用深度学习网络进行推理：

case 1:

```
model.eval ()
with  torch.inference_mode():
    test_loss, accuracy = test_loop(model, test_loader, loss_fn)
```

case 2:

```
result  = model(x_test).argmax(dim=   1)
```

PyTorch

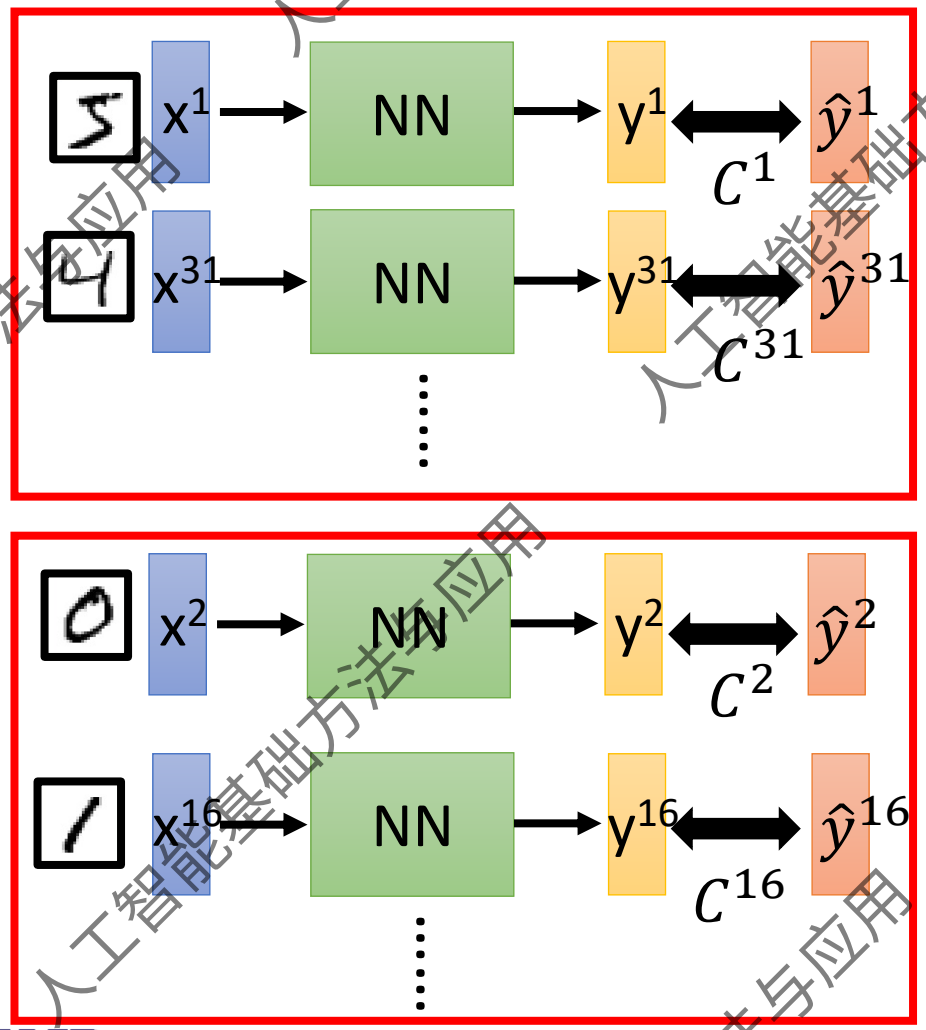
- 使用 GPU 加速训练
- 方法 1：在 Shell / 命令行中指定 GPU
  - CUDA_VISIBLE_DEVICES=0 python YourCode.py
- 方法 2：在代码中指定 GPU
  - device = torch.device("cuda:0")

    model, x, y = model.to(device), x.to(device), y.to(device)

人工智能基础方法与应用

Mini-batch（小批量训练）

我们实际上并不是每次都最小化整个训练集的总损失



- 随机初始化网络参数
- 取1st batch

  $L' = C^1 + C^{31} + \cdots$

  更新一次神经网络参数
- 取2nd batch

  $L'' = C^2 + C^{16} + \cdots$

  更新一次神经网络参数

  ⋮
- 直到所有 mini-batch 都被使用一遍

完成 1 个 Epoch
（训练集完整遍历一遍）

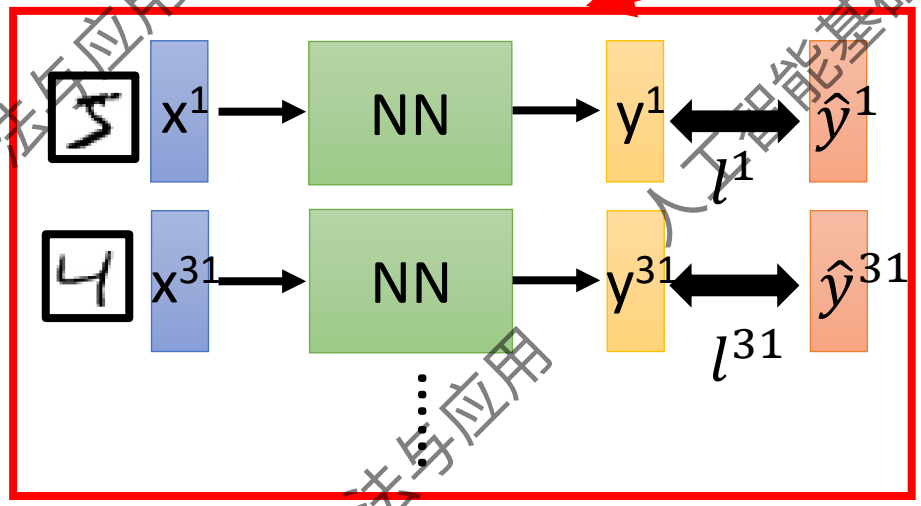
重复上述过程

人工智能基础方法与应用

Mini-batch

Batch size会同时影响训练速度和模型性能，需要根据任务进行调节

```
loader =DataLoader (train_ds,  batch_size=100 ); epochs=20
```



每个 mini-batch 包含 100 个样本

Batch size = 1

随机梯度下降

- 取1st batch

  $L' = C^1 + C^{31} + \cdots$

  更新一次神经网络参数
- 取2nd batch

  $L'' = C^2 + C^{16} + \cdots$

  更新一次神经网络参数

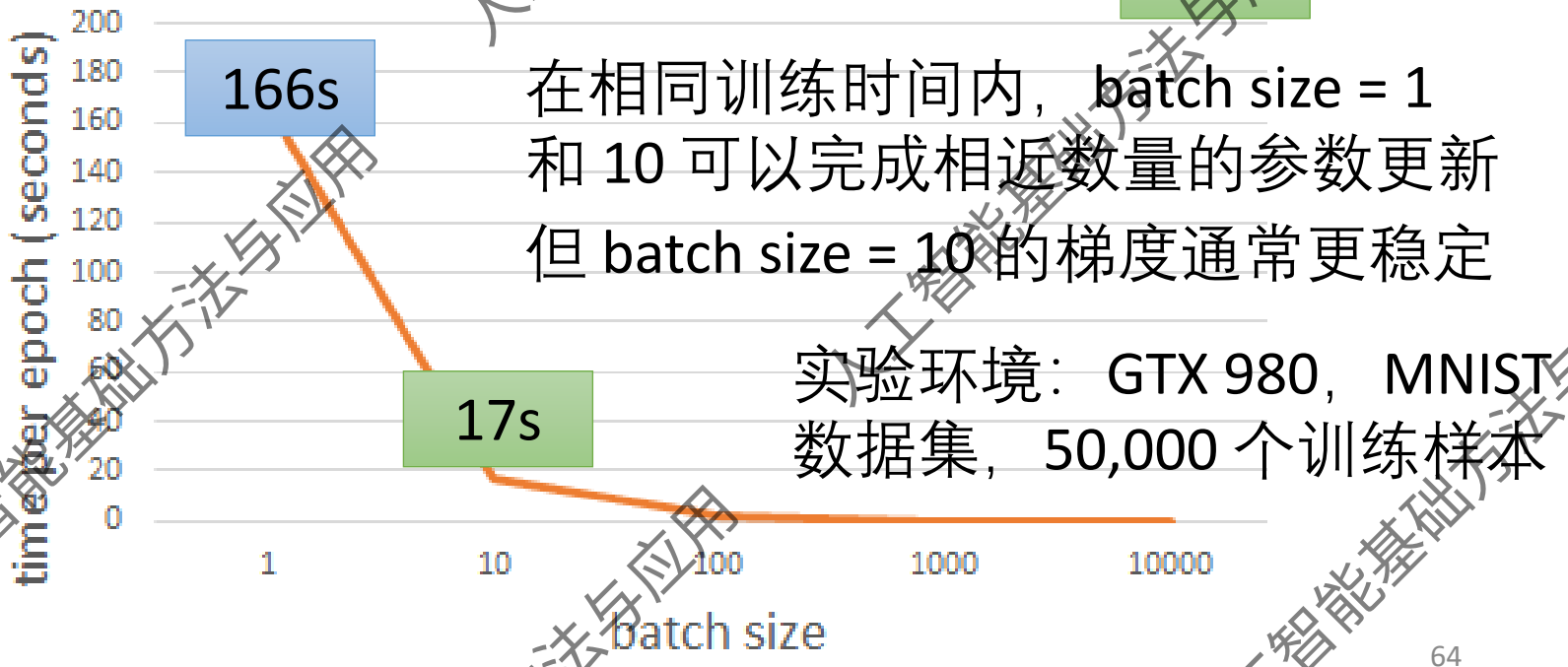
  ⋮
- 直到所有 mini-batch 都被使用一遍

重复20次

one epoch

训练速度

- Batch size 越小，一个 epoch 内参数更新次数越多
  - E.g. 50000 examples
  - batch size = 1, 一个 epoch 更新 **50,000 次**
  - batch size = 10, 一个 epoch 更新 **5,000 次**

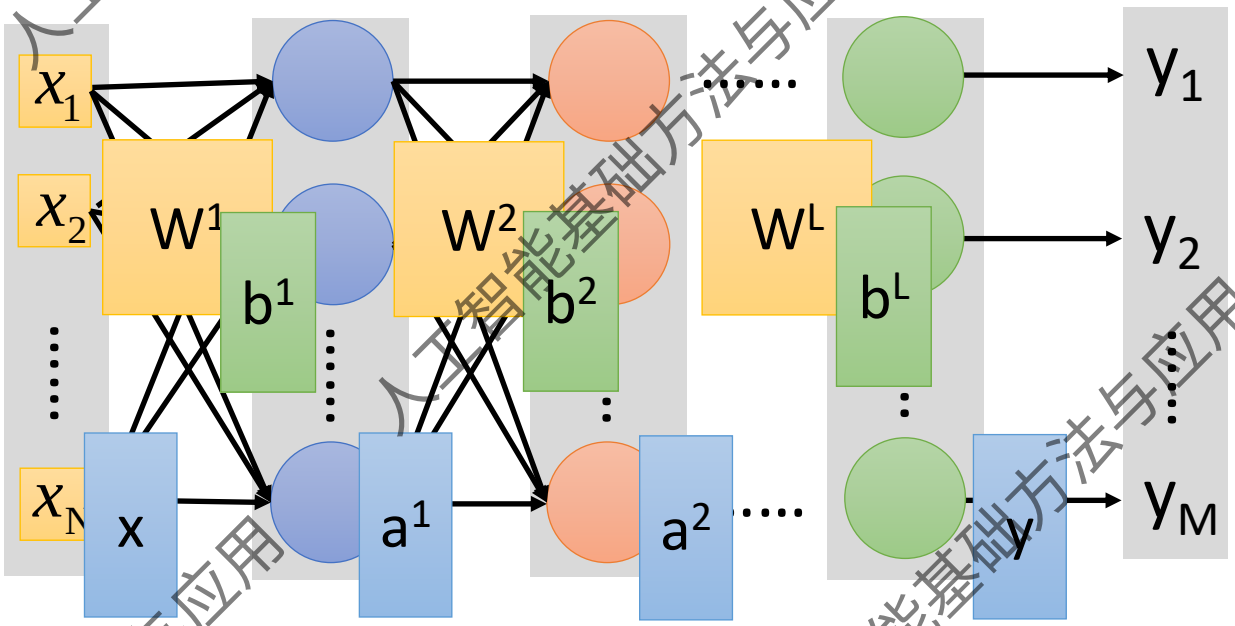


在相同训练时间内，batch size = 1 和 10 可以完成相近数量的参数更新

但 batch size = 10 的梯度通常更稳定

实验环境：GTX 980，MNIST 数据集，50,000 个训练样本

矩阵运算能够加快训练速度



$$y = f(x)$$ 前向传播 （后向传播是相似的）

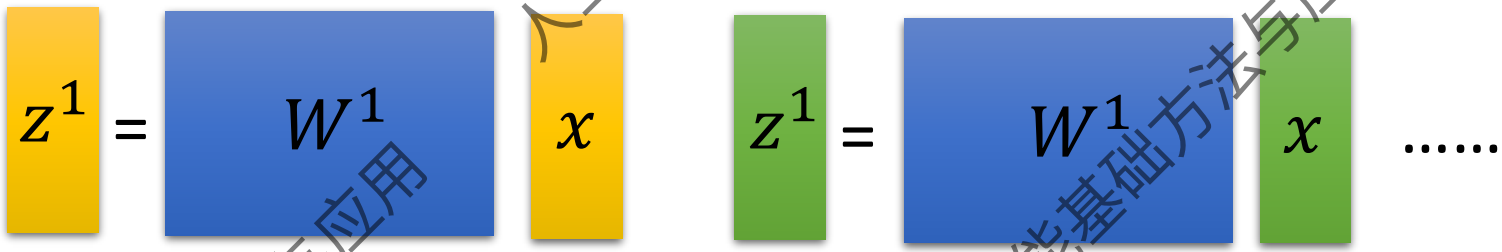
$$= \sigma(W^L \cdots \sigma(W^2 \sigma(W^1 x + b^1) + b^2) \cdots + b^L)$$

人工智能基础方法与应用

训练速度：矩阵运算

- 为什么 Mini-batch 通常比逐样本随机梯度下降更快?

逐样本随机梯度下降



Mini-batch：把很多次小规模计算，合并成一次大规模矩阵计算



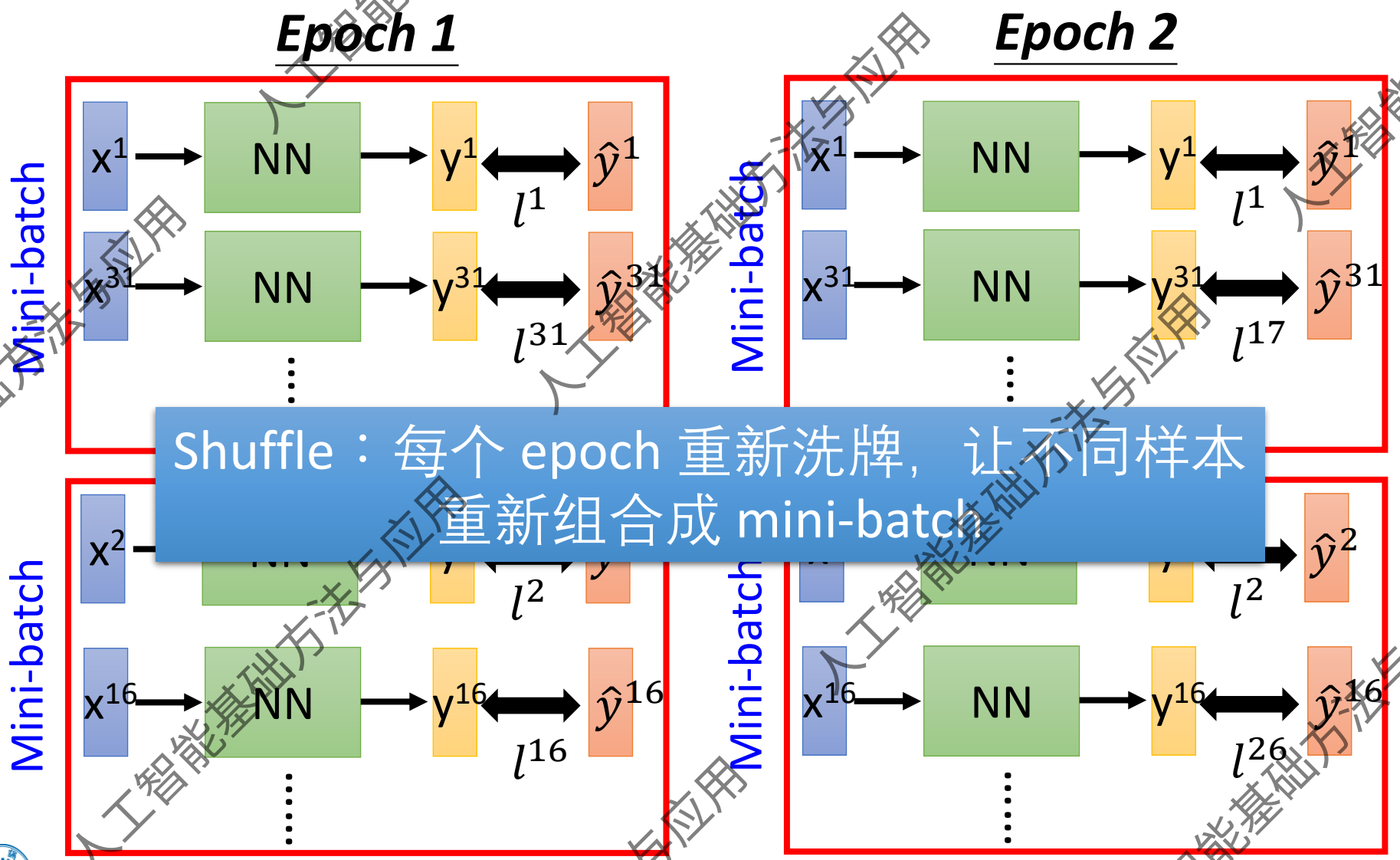
实际计算中，哪一种更快?

Batch Size对模型性能的影响

- Batch Size 越大，计算通常越高效
- 但过大的 Batch Size 可能降低模型的泛化性能

人工智能基础方法与应用

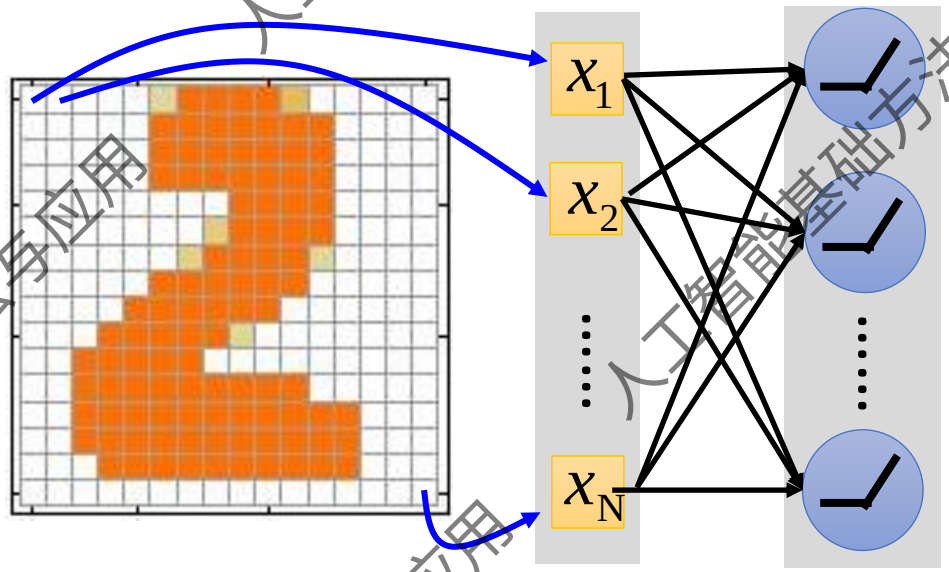
每个 Epoch 都重新打乱（Shuffle）训练样本



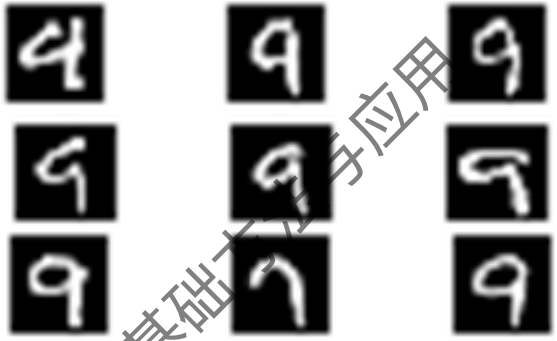
Shuffle：每个 epoch 重新洗牌，让不同样本重新组合成 mini-batch

人工智能基础方法与应用

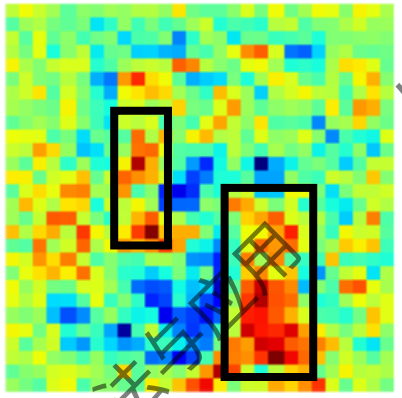
分析神经元学到了什么特征



什么样的输入会让某个神经元产生最大响应？



将该神经元的权重按对应像素位置重新排列，可以观察它关注的图像区域。

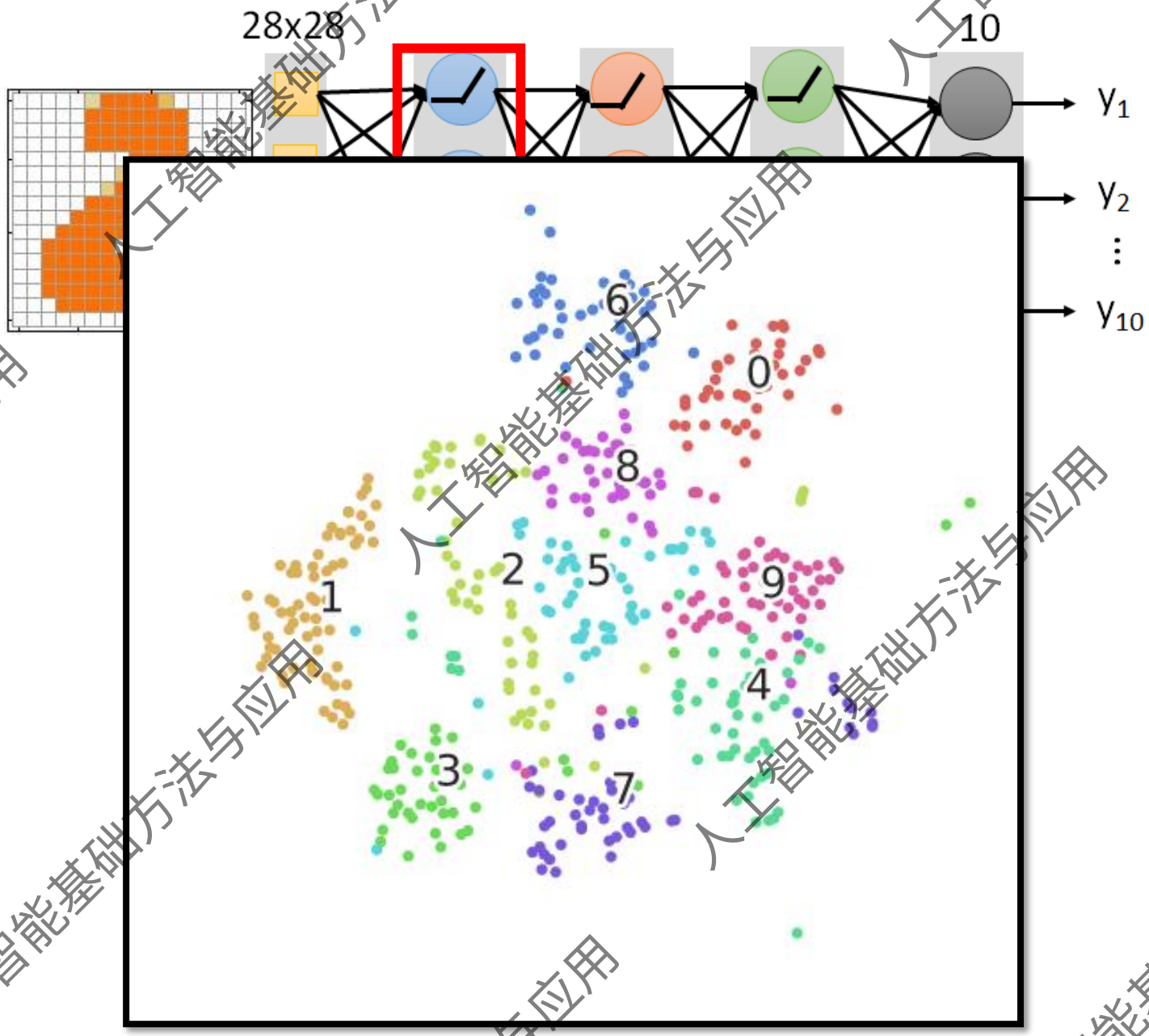


红色：正权重

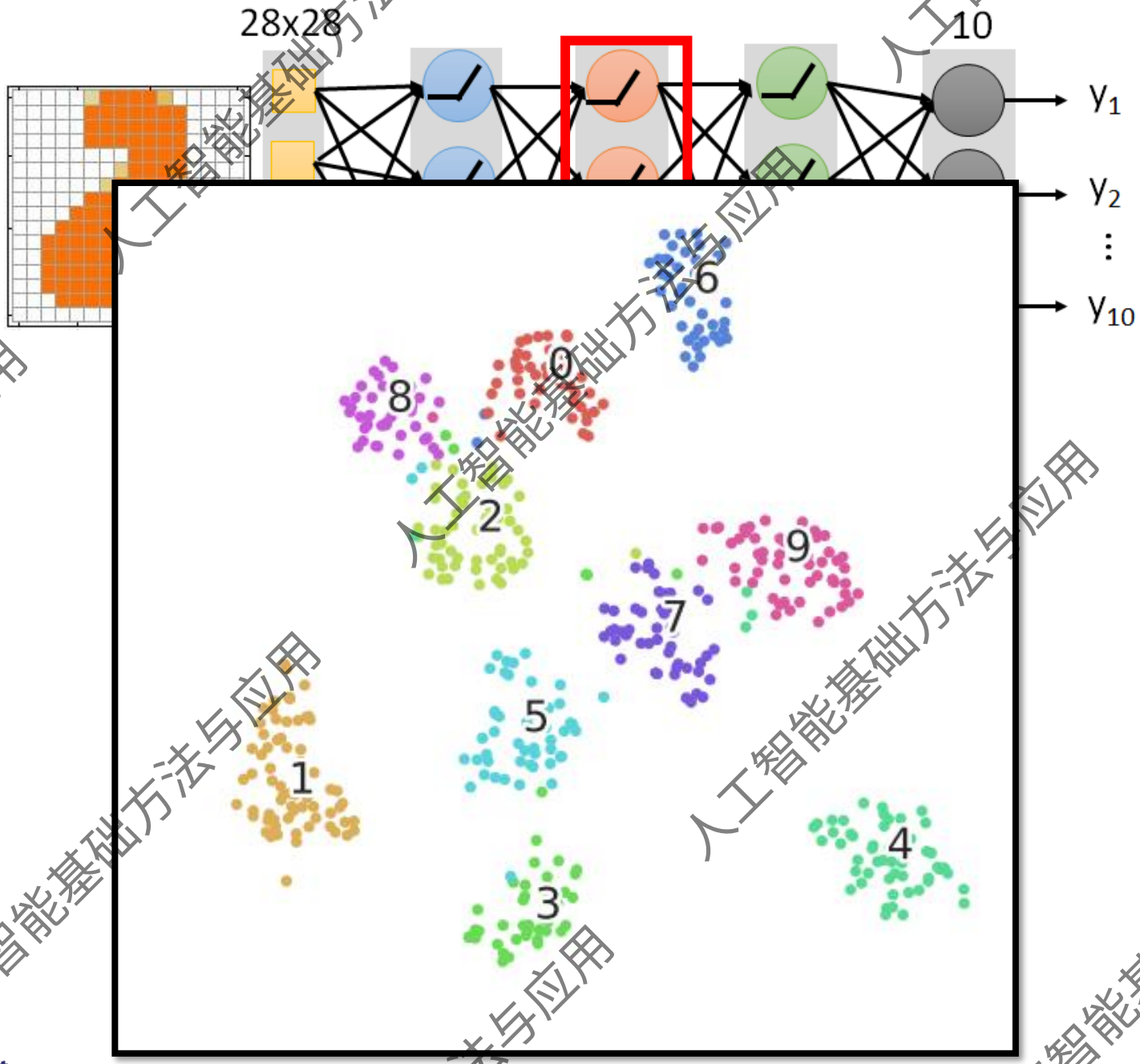
蓝色：负权重

第一层神经元通常学习的是数字的**局部笔画、边缘或形状特征**

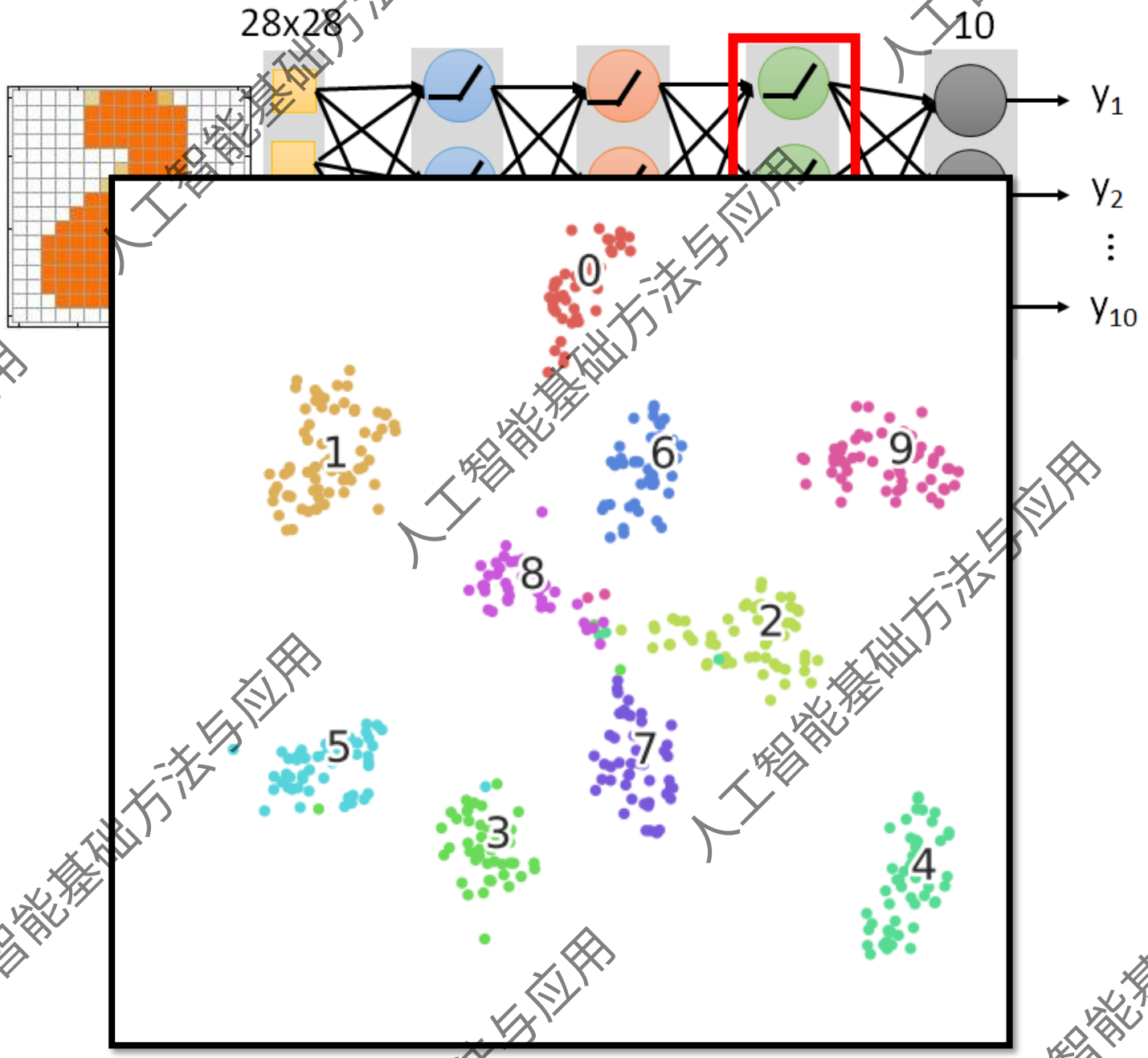
IAIR Est. 1986



人工智能基础方法与应用



人工智能基础方法与应用



人工智能基础方法与应用

三、深度学习的技巧

人工智能基础方法与应用

读懂训练技巧：它改了哪一步

- 当你学到一种新的神经网络训练方法时，先问自己：它到底改了训练流程中的哪一步？

方法名	改哪一步	好在哪里
……	……	……



人工智能基础方法与应用

读懂训练技巧：带来什么好处

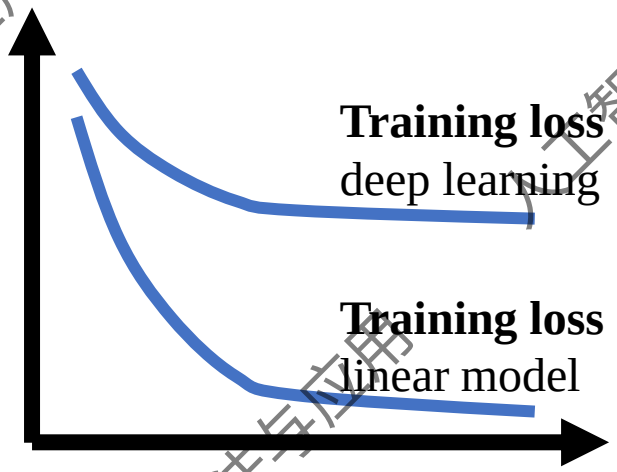
- 当你学到一种新的神经网络训练方法时，先问自己：它到底改了训练流程中的哪一步？

方法名	改哪一步	好在哪里
……	……	……

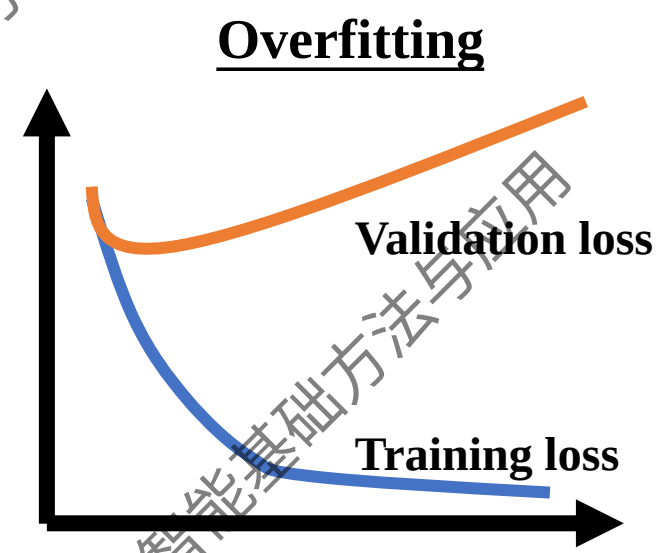
- Better Optimization：更低的 Training Loss
- Better Generalization：更低的 Validation Loss
- ……

人工智能基础方法与应用

先诊断，再选择技巧



Better Optimization!



Better Generalization!

人工智能基础方法与应用

3.1、从步骤三开始：优化



Optimization

普通梯度下降

$$\boldsymbol{\theta}^* = arg\min_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$$

- 随机选择初始化参数$\boldsymbol{\theta^0}$
- 计算梯度$\boldsymbol{g^0} = \nabla L(\boldsymbol{\theta^0})$

$$\boldsymbol{\theta^1} \leftarrow \boldsymbol{\theta^0} - \mu \boldsymbol{g^0}$$

- 计算梯度$\boldsymbol{g^1} = \nabla L(\boldsymbol{\theta^1})$

$$\boldsymbol{\theta^2} \leftarrow \boldsymbol{\theta^1} - \mu \boldsymbol{g^1}$$

- 计算梯度$\boldsymbol{g^2} = \nabla L(\boldsymbol{\theta^2})$

$$\boldsymbol{\theta^3} \leftarrow \boldsymbol{\theta^2} - \mu \boldsymbol{g^2}$$

人工智能基础方法与应用

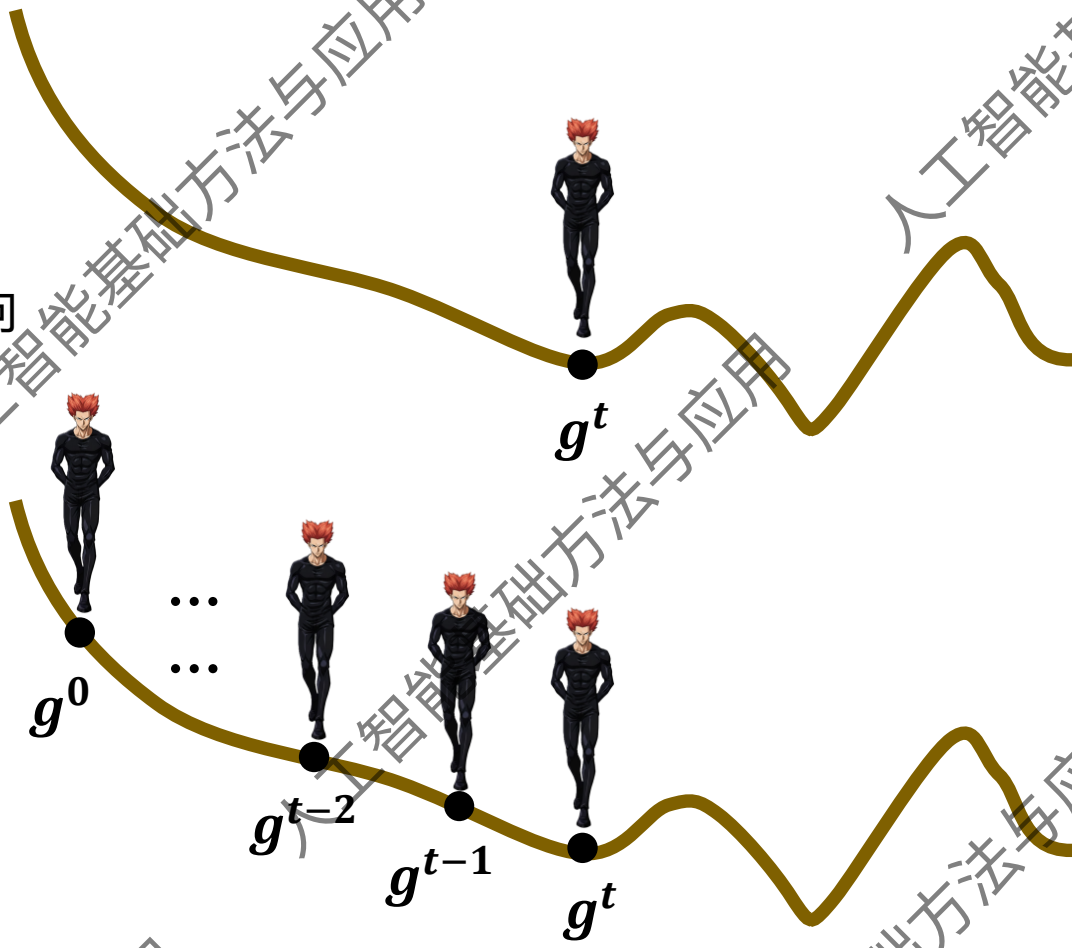
优化器：利用历史梯度

普通梯度下降

根据当下算出来的 g^t 来决定方向

梯度下降 + 优化器

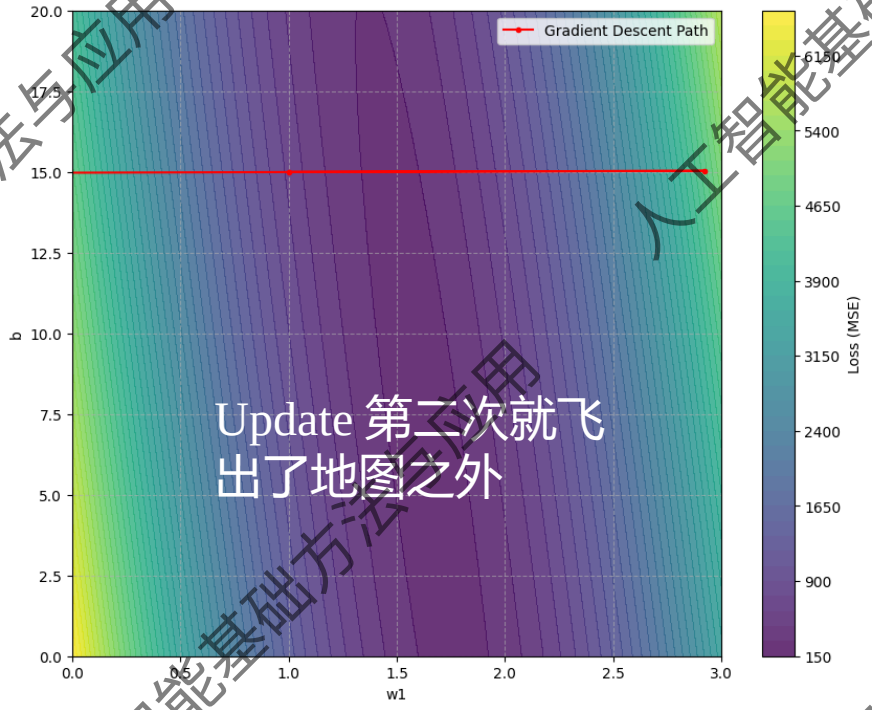
根据 $g^0, g^1, g^2, \ldots, g^t$ 一起来决定方向



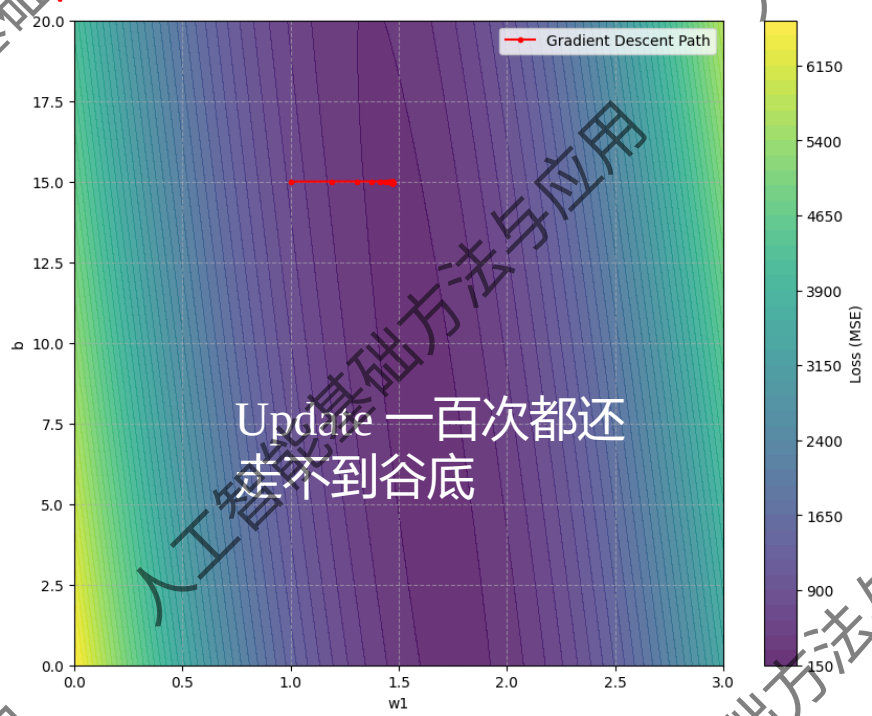
人工智能基础方法与应用

学习率为什么难调

$\mu = 0.001$

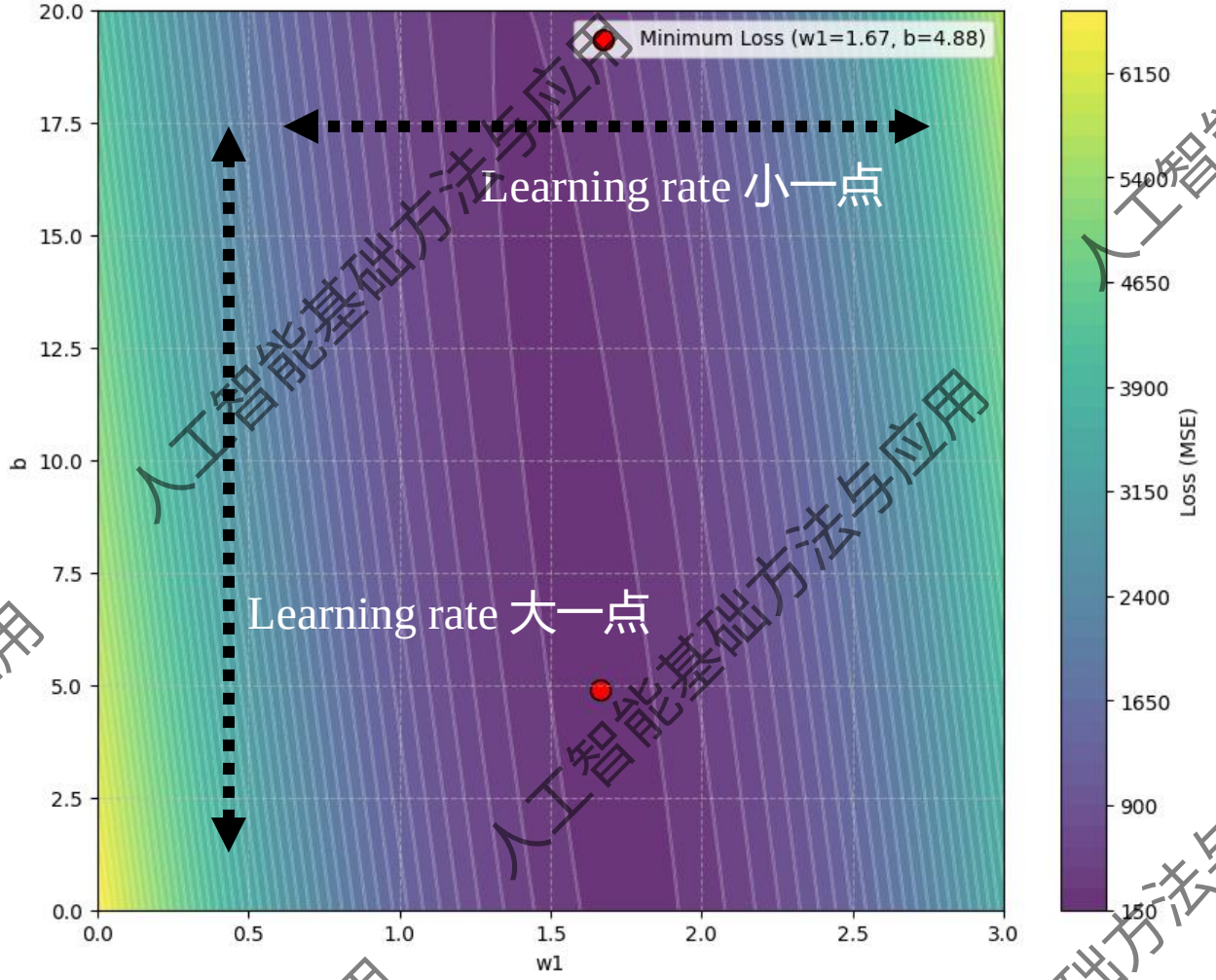


$\mu = 0.0001$



人工智能基础方法与应用

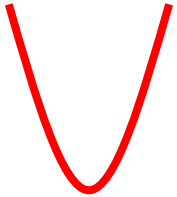
不同参数需要不同学习率



怎么知道哪一个方向的 gradient 大、那一个小?

人工智能基础方法与应用

用历史梯度调节学习率



Learning rate 小一点

$$\boldsymbol{g^0} = \begin{bmatrix} 500 \\ 0.4 \end{bmatrix} \quad \boldsymbol{g^1} = \begin{bmatrix} 432 \\ -0.3 \end{bmatrix} \quad \boldsymbol{g^2} = \begin{bmatrix} -211 \\ 0.2 \end{bmatrix} \quad \boldsymbol{g^3} = \begin{bmatrix} 139 \\ 0.1 \end{bmatrix}$$



Learning rate 大一点

人工智能基础方法与应用

Adagrad：累计平方梯度

- 计算梯度 $\boldsymbol{g^0} = \nabla L(\boldsymbol{\theta^0})$

  对于每个参数维度i: $\sigma_i^0 = \sqrt{\left(g_i^0\right)^2} = \left|g_i^0\right|$

- 计算梯度$g^1 = \nabla L(\boldsymbol{\theta}^1)$

  对于每个参数维度i: $\sigma_i^1 = \sqrt{\left[\left(g_i^0\right)^2 + \left(g_i^1\right)^2\right]}$ Average?

- 计算梯度 $g^2 = \nabla L(\boldsymbol{\theta}^2)$

  对于每个参数维度i: $\sigma_i^2 = \sqrt{\left[\left(g_i^0\right)^2 + \left(g_i^1\right)^2 + \left(g_i^2\right)^2\right]}$

⋮ ⋮

- 计算梯度$\boldsymbol{g^t} = \nabla L(\boldsymbol{\theta}^t)$

  对于每个参数维度i: $\sigma_i^t = \sqrt{\sum_{i=0}^{t}\left(g_i^t\right)^2}$

Adagrad

$$\theta_i^1 \leftarrow \theta_i^0 - \frac{\mu}{\sigma_i^0} g_i^0$$

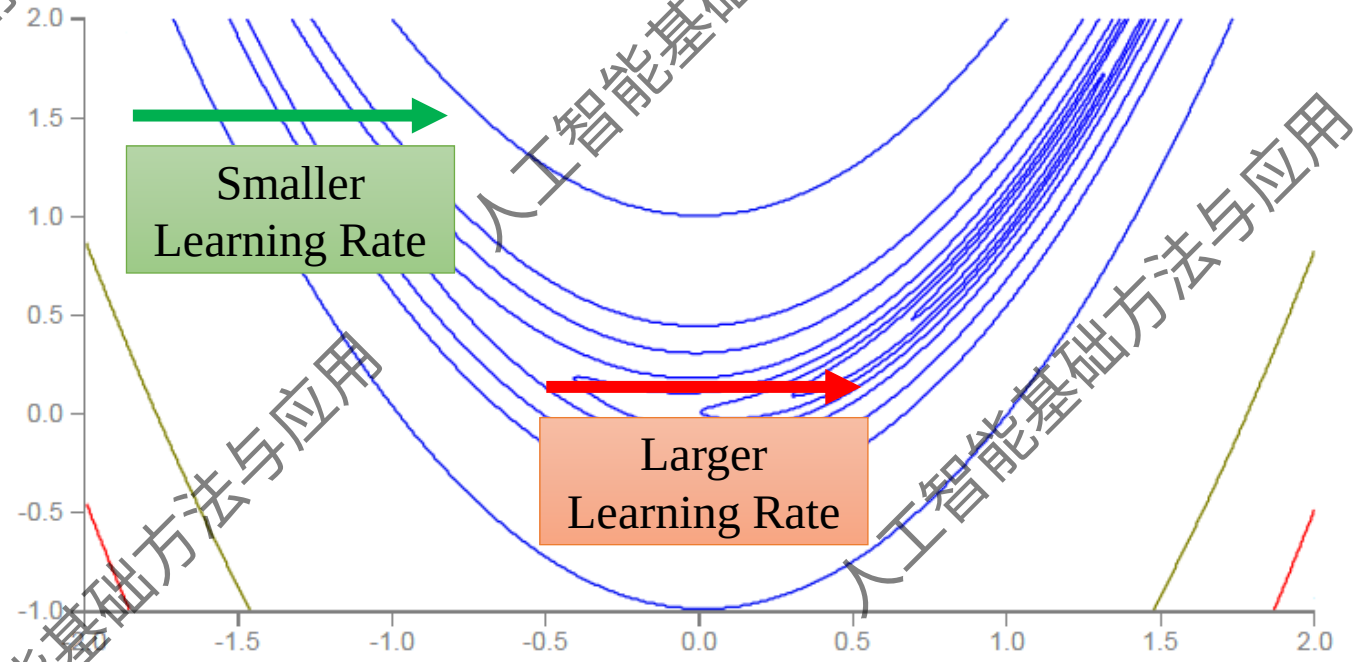
$$\theta_i^2 \leftarrow \theta_i^1 - \frac{\mu}{\sigma_i^1} g_i^1$$

$$\theta_i^3 \leftarrow \theta_i^2 - \frac{\mu}{\sigma_i^2} g_i^2$$

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\mu}{\sigma_i^t} g_i^t$$

人工智能基础方法与应用

同一参数的梯度也会变化



人工智能基础方法与应用

从Adagrad到RMSProp

$$g^0 = \begin{bmatrix} 500 \\ 0.4 \end{bmatrix} \quad g^1 = \begin{bmatrix} 432 \\ -0.3 \end{bmatrix} \quad \cdots\cdots \quad g^{t-1} = \begin{bmatrix} -0.1 \\ 229 \end{bmatrix} \quad g^t = \begin{bmatrix} 0.1 \\ 100 \end{bmatrix}$$

变大　　变大



Adagrad: 全部平方加起来

RMSProp: 最近算出来的 gradient 给比较大的影响

RMSProp：重视近期梯度

RMSProp

- 计算梯度 $g^0 = \nabla L(\theta^0)$

  对于每个参数维度i: $\sigma_i^0 = \sqrt{(g_i^0)^2}$ $\quad \theta_i^1 \leftarrow \theta_i^0 - \frac{\mu}{\sigma_i^0} g_i^0$

- 计算梯度 $g^1 = \nabla L(\theta^1)$ $\quad 0 < \alpha < 1$

  对于每个参数维度i: $\sigma_i^1 = \sqrt{\alpha(\sigma_i^0)^2 + (1-\alpha)(g_i^1)^2}$ $\quad \theta_i^2 \leftarrow \theta_i^1 - \frac{\mu}{\sigma_i^1} g_i^1$

- 计算梯度 $g^2 = \nabla L(\theta^2)$

  对于每个参数维度i: $\sigma_i^2 = \sqrt{\alpha(\sigma_i^1)^2 + (1-\alpha)(g_i^2)^2}$ $\quad \theta_i^3 \leftarrow \theta_i^2 - \frac{\mu}{\sigma_i^2} g_i^2$

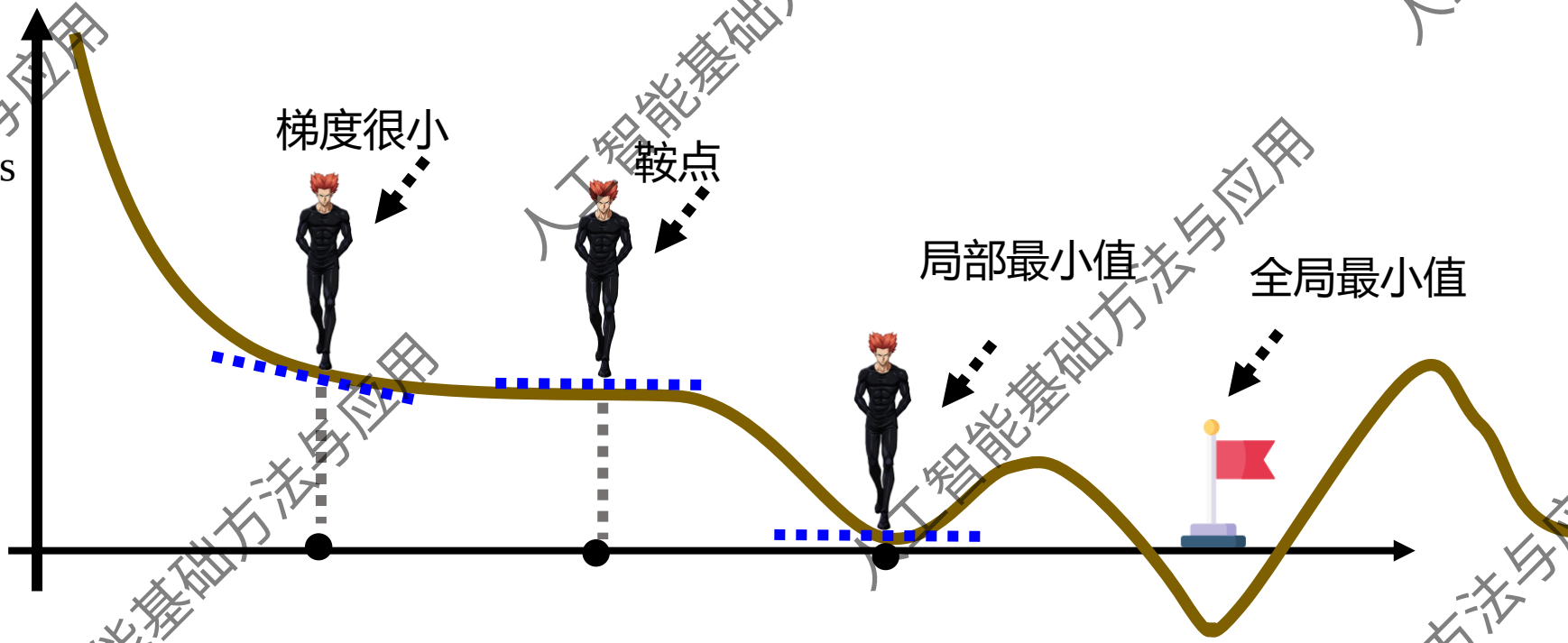
⋮ ⋮

- 计算梯度 $g^t = \nabla L(\theta^t)$

  对于每个参数维度i: $\sigma_i^t = \sqrt{\alpha(\sigma_i^{t-1})^2 + (1-\alpha)(g_i^t)^2}$ $\quad \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\mu}{\sigma_i^t} g_i^t$

人工智能基础方法与应用

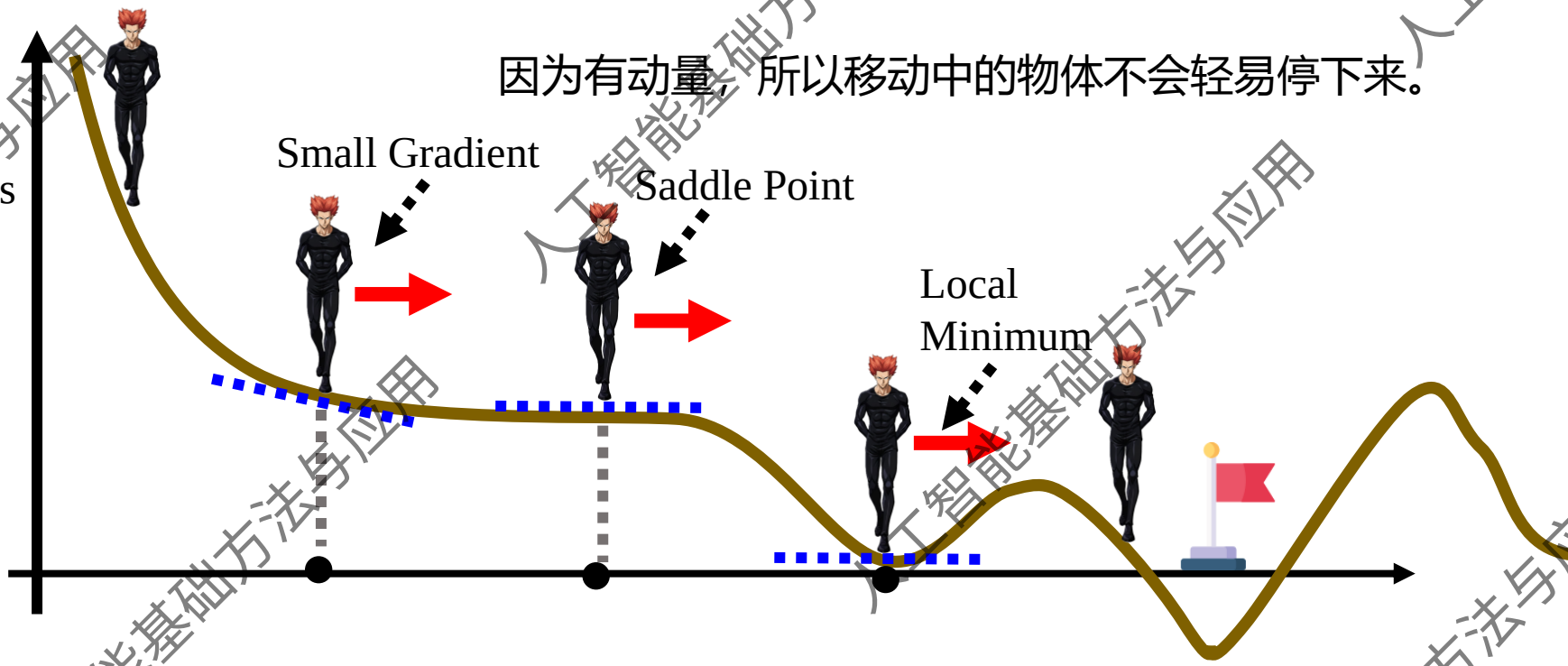
小梯度为何会让优化停下



人工智能基础方法与应用

考虑动量(Momentum)

因为有动量，所以移动中的物体不会轻易停下来。



人工智能基础方法与应用

动量的基本计算

Momentum

- 计算梯度 $\boldsymbol{g^0 = \nabla L(\theta^0)}$

  对于每个参数维度i: $m_i^0 = g_i^0$ $\theta_i^1 \leftarrow \theta_i^0 - \mu m_i^0$

- 计算梯度 $\boldsymbol{g^1 = \nabla L(\theta^1)}$

  对于每个参数维度i: $m_i^1 = g_i^0 + g_i^1$ $\theta_i^2 \leftarrow \theta_i^1 - \mu m_i^1$

- 计算梯度 $\boldsymbol{g^2 = \nabla L(\theta^2)}$

  对于每个参数维度i: $m_i^2 = g_i^0 + g_i^1 + g_i^2$ $\theta_i^3 \leftarrow \theta_i^2 - \mu m_i^2$

⋮ ⋮

- 计算梯度 $\boldsymbol{g^t = \nabla L(\theta^t)}$

  对于每个参数维度i: $m_i^t = g_i^0 + g_i^1 + g_i^2 + \cdots + g_i^t$ $\theta_i^{t+1} \leftarrow \theta_i^t - \mu m_i^t$

人工智能基础方法与应用

指数加权的动量

Momentum (这不是经典的 Momentum)

- 计算梯度 $\boldsymbol{g}^0 = \nabla L(\boldsymbol{\theta}^0)$

  对于每个参数维度i: $m_i^0 = g_i^0$ $\theta_i^1 \leftarrow \theta_i^0 - \mu m_i^0$

- 计算梯度 $\boldsymbol{g}^1 = \nabla L(\boldsymbol{\theta}^1)$

  $0 < \beta < 1$

  对于每个参数维度i: $m_i^1 = \beta m_i^0 + (1-\beta) g_i^1$ $\theta_i^2 \leftarrow \theta_i^1 - \mu m_i^1$

- 计算梯度 $\boldsymbol{g}^2 = \nabla L(\boldsymbol{\theta}^2)$

  对于每个参数维度i: $m_i^2 = \beta m_i^1 + (1-\beta) g_i^2$ $\theta_i^3 \leftarrow \theta_i^2 - \mu m_i^2$

⋮ ⋮

- 计算梯度 $\boldsymbol{g}^t = \nabla L(\boldsymbol{\theta}^t)$

  对于每个参数维度i: $m_i^t = \beta m_i^{t-1} + (1-\beta) g_i^t$ $\theta_i^{t+1} \leftarrow \theta_i^t - \mu m_i^t$

人工智能基础方法与应用

Adam: RMSProp + Momentum

- 计算梯度 $\boldsymbol{g}^t = \nabla L(\boldsymbol{\theta}^t)$

Momentum

对于每个参数维度i: $m_i^t = \beta m_i^{t-1} + (1-\beta) g_i^t$ $\theta_i^{t+1} \leftarrow \theta_i^t - \mu m_i^t$

RMSProp

对于每个参数维度i: $\sigma_i^t = \sqrt{\alpha\left(\sigma_i^{t-1}\right)^2 + (1-\alpha)\left(g_i^t\right)^2}$ $\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\mu}{\sigma_i^t} g_i^t$

Adam

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\mu}{\sigma_i^t} m_i^t$$

Adam 还包含偏差修正项，这里为简化表达暂时省略

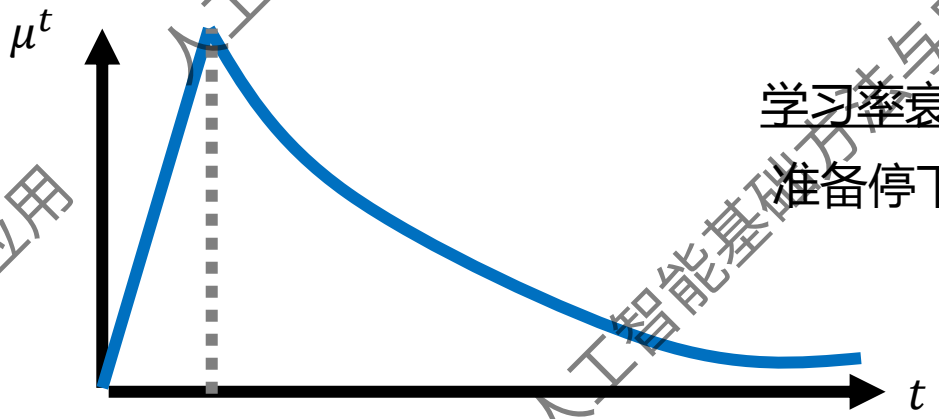
人工智能基础方法与应用

学习率调度

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\mu}{\sigma_i^t} m_i^t \quad \Rightarrow \quad \theta_i^{t+1} \leftarrow \theta_i^t - \frac{\mu^t}{\sigma_i^t} m_i^t$$

学习率预热（Warm Up）

探索地形



学习率衰减

准备停下来

人工智能基础方法与应用

优化器总结

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\mu^t}{\sigma_i^t} m_i^t$$

学习率调度

动量：综合历史梯度信息

考虑梯度方向

梯度平方

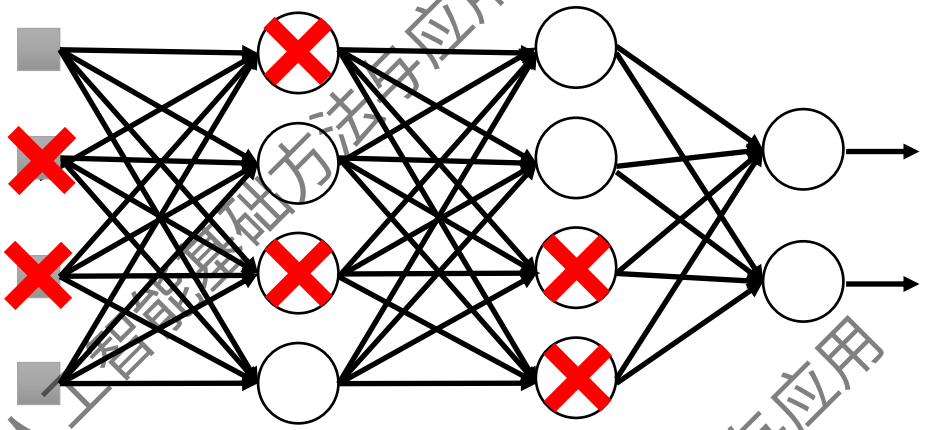
只考虑梯度大小

方法名	改哪一步	好在哪里
Adagrad, RMSProp, Momentum, Adam, etc.	找最优函数	Better Optimization (not for Generalization)

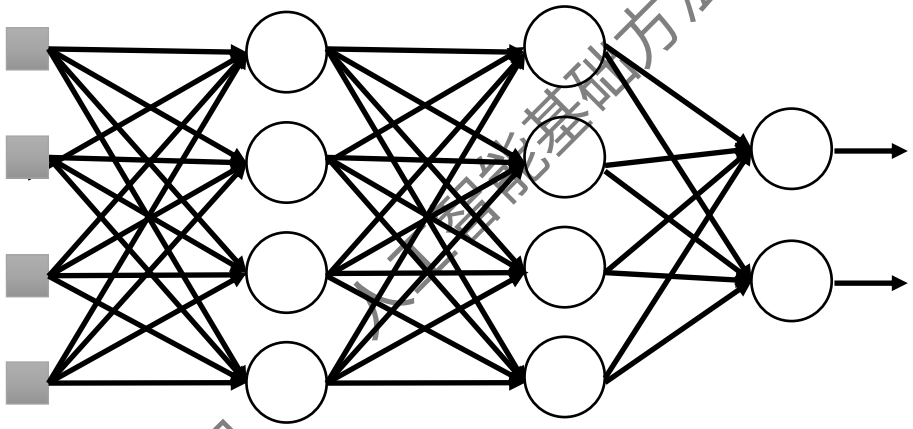
人工智能基础方法与应用

Dropout

训练时随机丢掉一些神经元



验证和测试时火力全开



人工智能基础方法与应用

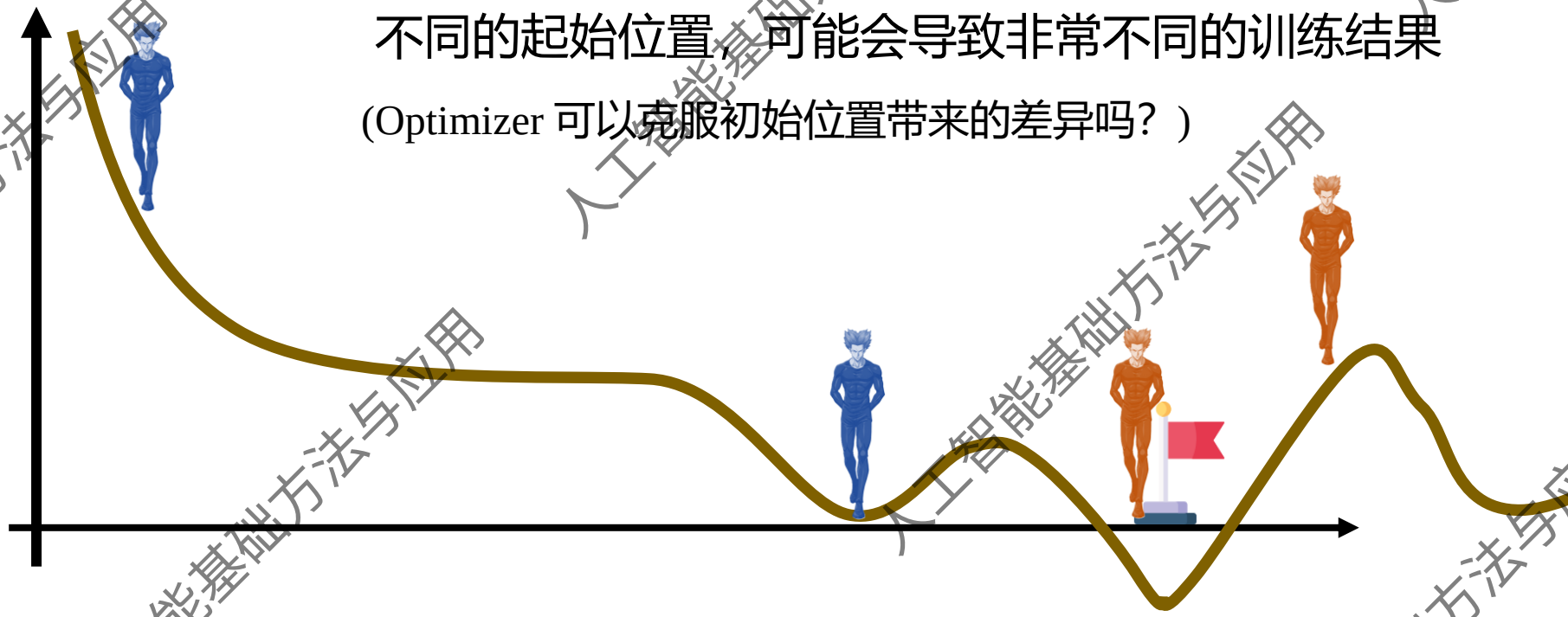
Dropout

方法名	改哪一步	好在哪里
Dropout	找最优函数	Better Generalization (Worse Optimization)

注意 Dropout 的选择时机!

人工智能基础方法与应用

初始化



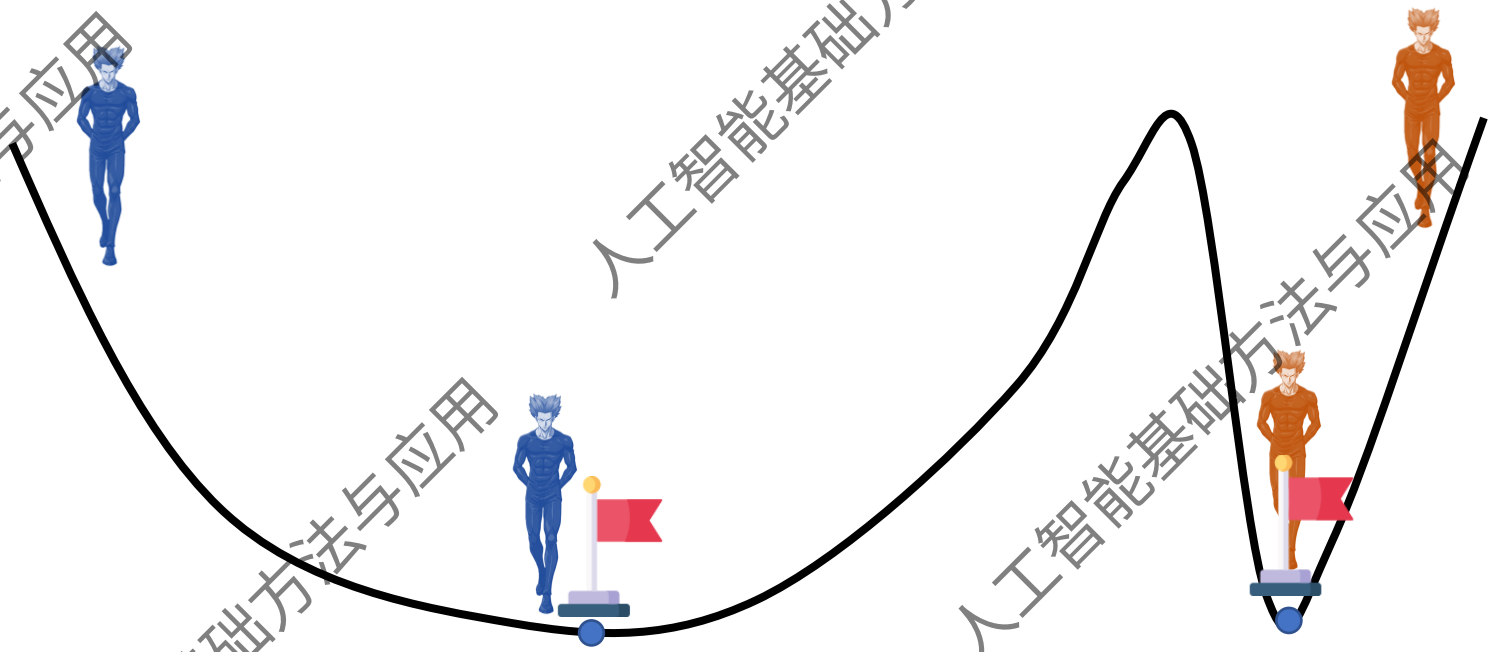
不同的起始位置，可能会导致非常不同的训练结果

(Optimizer 可以克服初始位置带来的差异吗？)

人工智能基础方法与应用

初始化

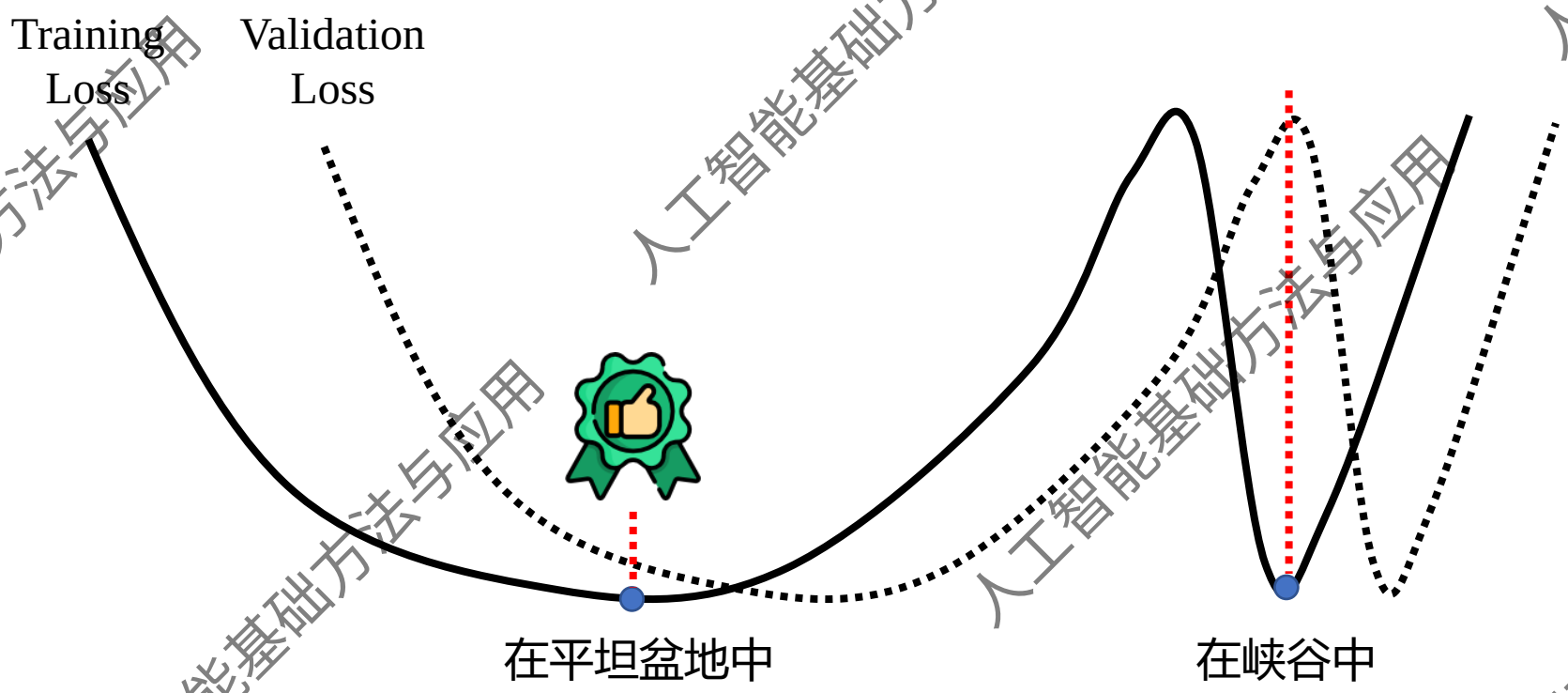
当有多个 Training Loss 最低点的时候，
起始位置决定我们走到哪一个最低点



人工智能基础方法与应用

初始化

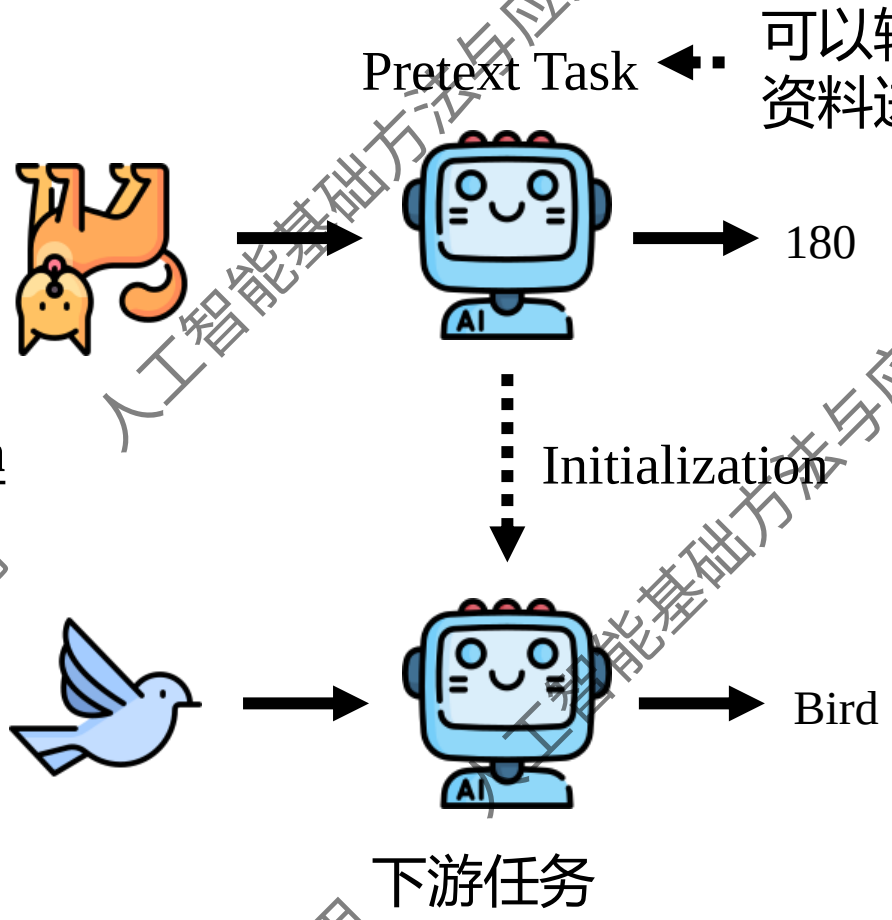
当有多个 Training Loss 最低点的时候，
起始位置决定我们走到哪一个最低点



人工智能基础方法与应用

初始化

增加预训练任务Pre-train



人工智能基础方法与应用

初始化

Pre-train 对于 Optimization 和 Generalization 都有帮助

方法名	改哪一步	好在哪里
初始化 (e.g., pre-train)	找最优函数	Better Optimization, Better Generalization

人工智能基础方法与应用

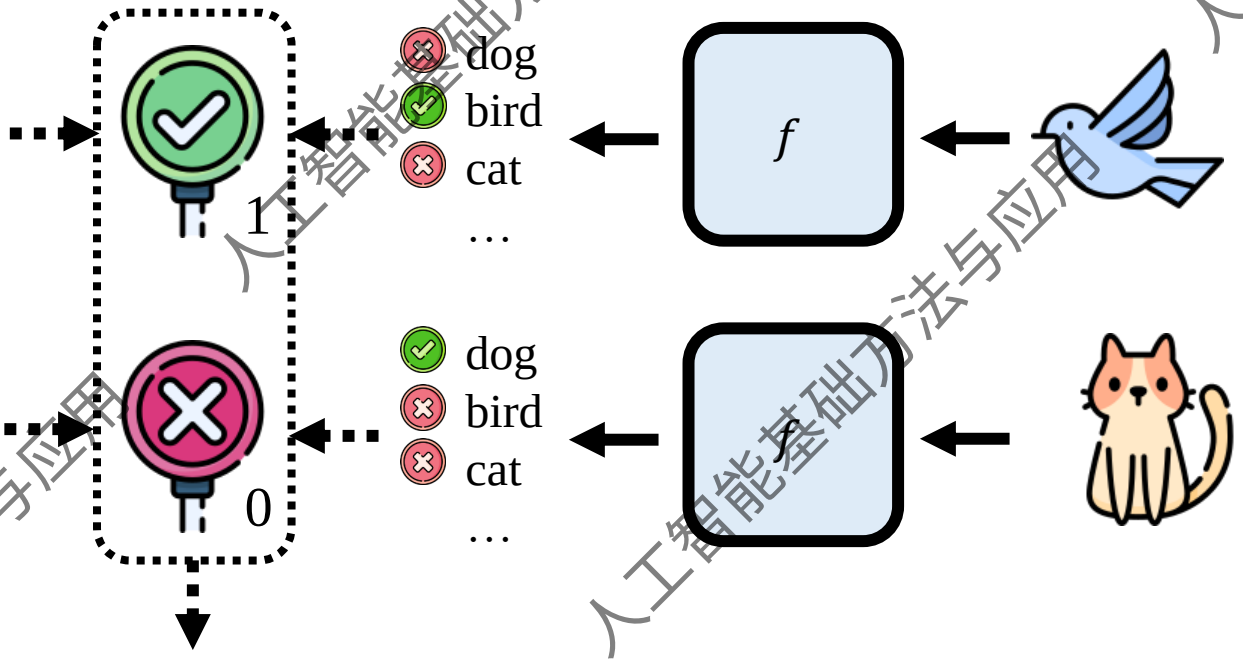
3.2、损失函数优化



人工智能基础方法与应用

损失函数≠评估标准?

分类任务



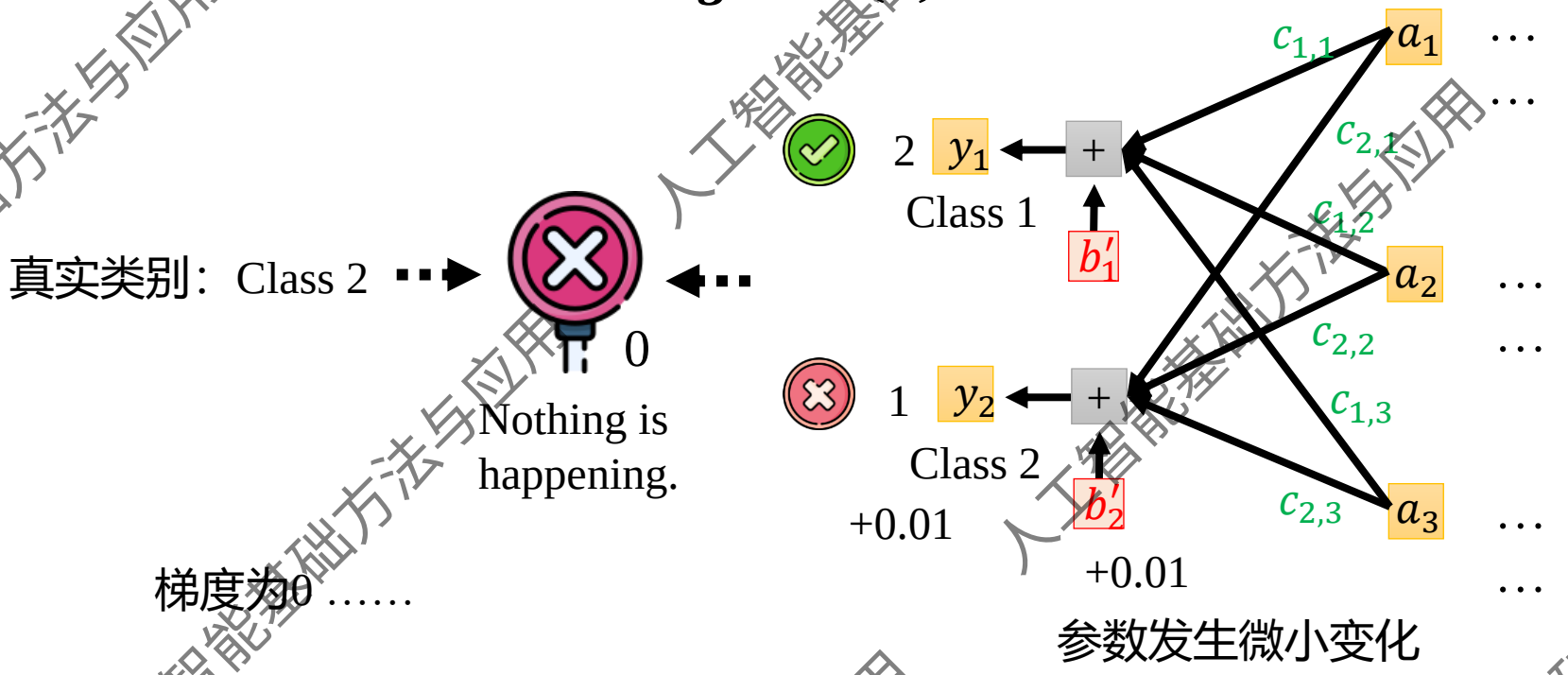
准确率 = 0.5

准确率能直接作为训练时的损失函数吗?

损失函数= 评估标准?

分类任务

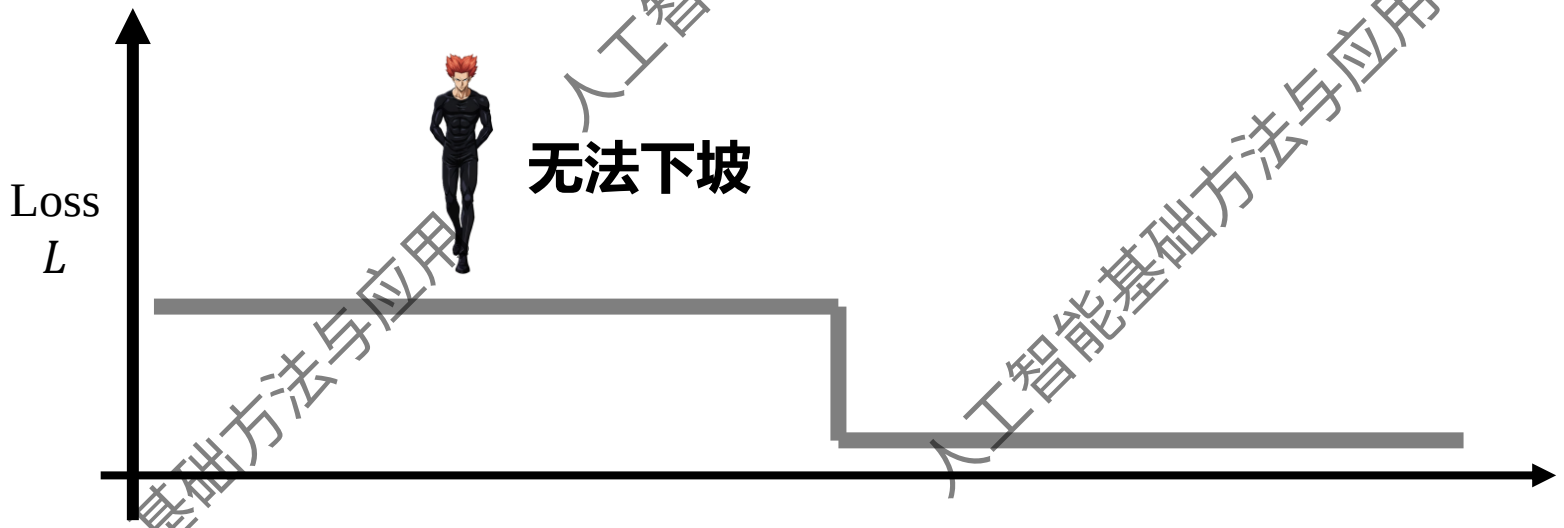
$$g = \nabla L(\theta)$$



人工智能基础方法与应用

损失函数= 评估标准?

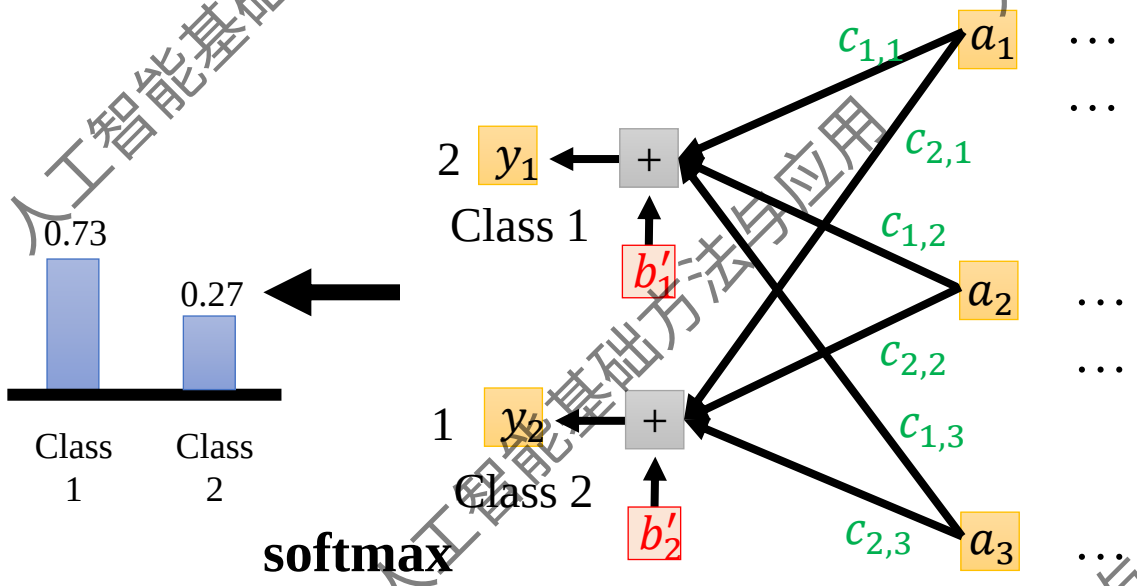
- 在分类问题中，如果直接把准确率作为损失函数……



人工智能基础方法与应用

交叉熵损失函数

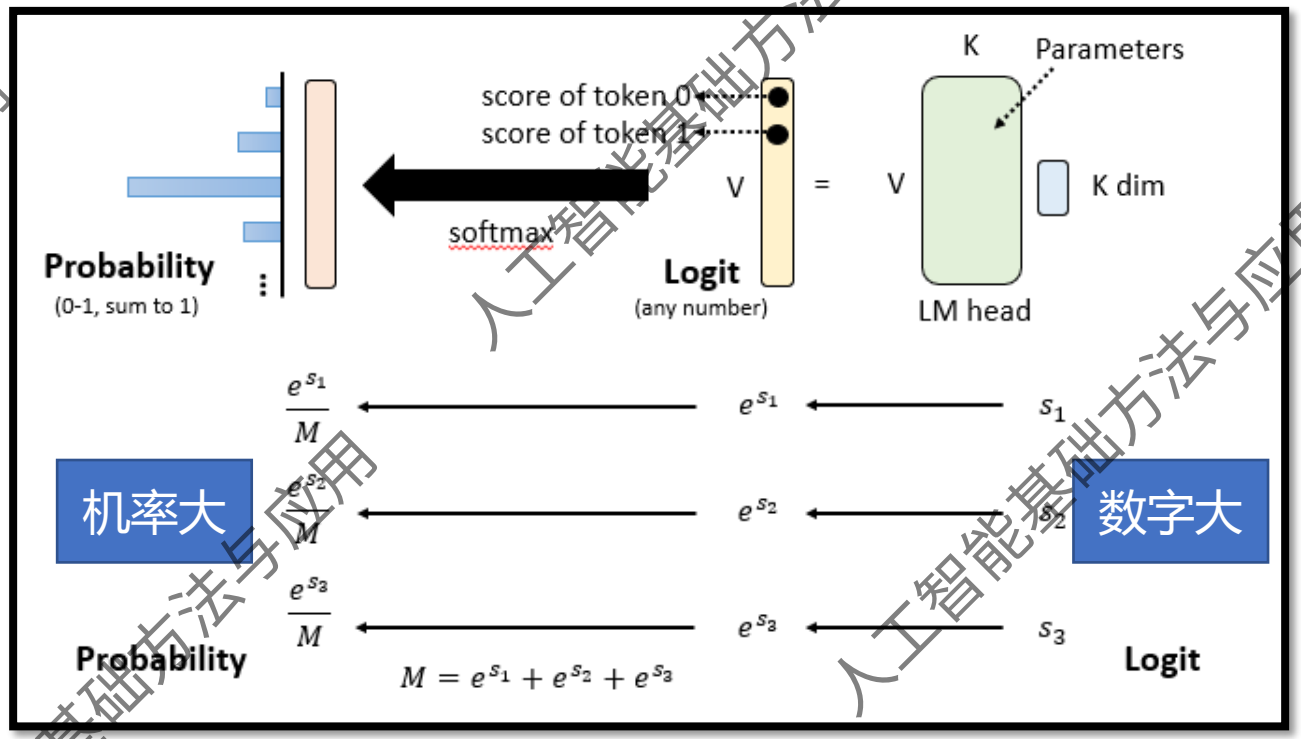
Ground truth: Class 2



人工智能基础方法与应用

什么是 Softmax?

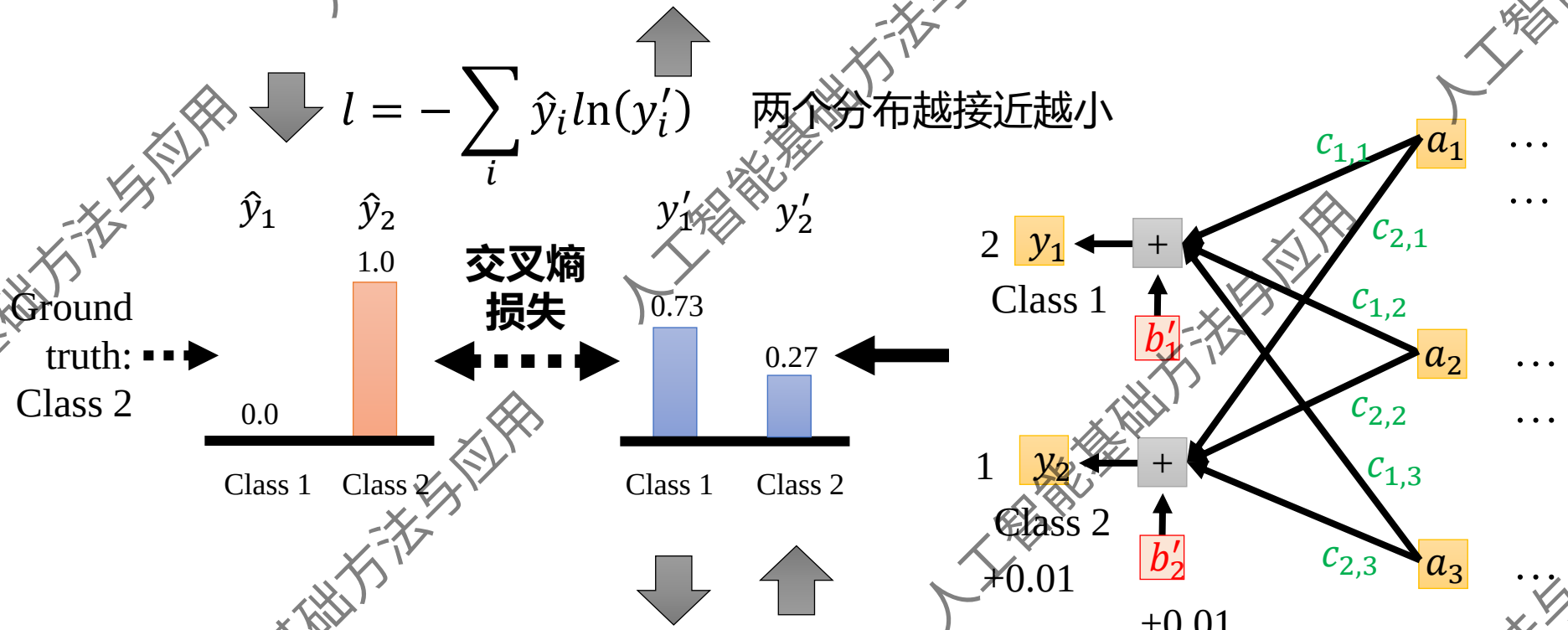
- Softmax 就是把一组“原始分数”，转换成一组“概率”



交叉熵损失

$$l = -\sum_i \hat{y}_i \ln(y_i')$$

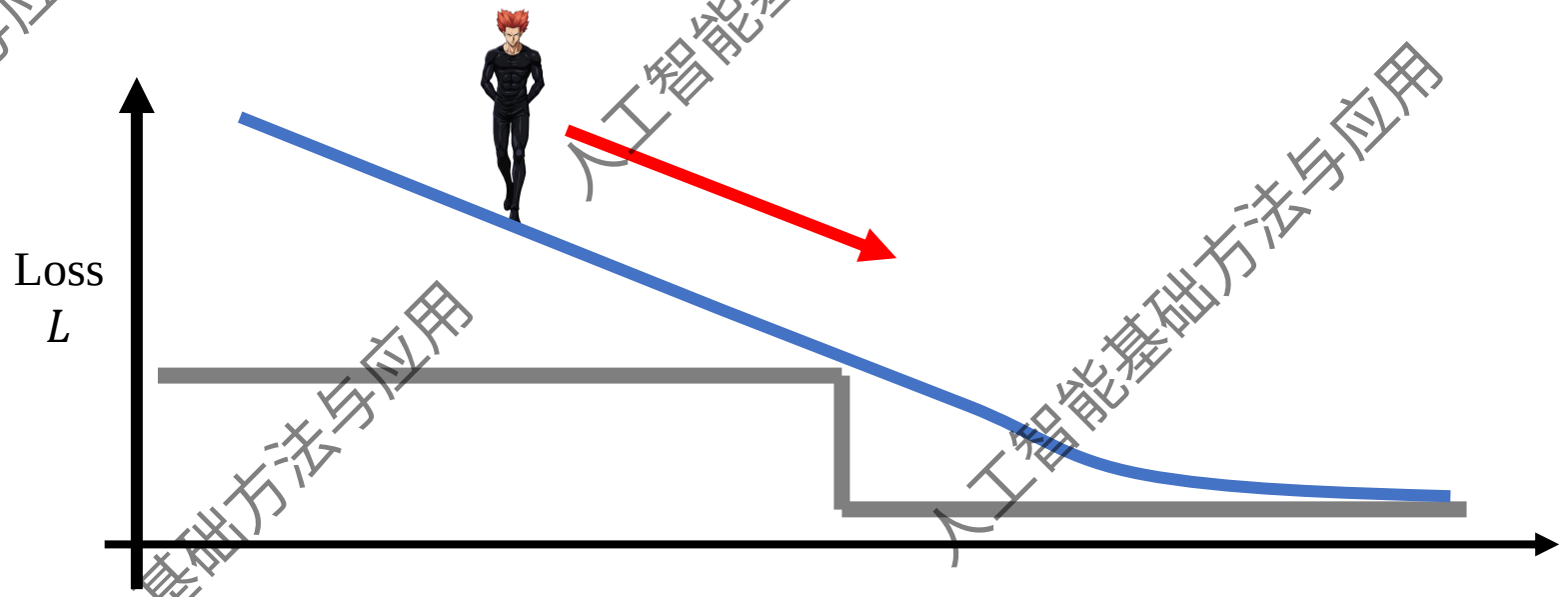
两个分布越接近越小



人工智能基础方法与应用

损失函数=评估标准?

- 在分类问题中，如果采用交叉熵作为损失函数……



人工智能基础方法与应用



方法名	改哪一步	好在哪里
Do not use accuracy as loss	我要找什么	Better Optimization (Not for Generalization)

人工智能基础方法与应用

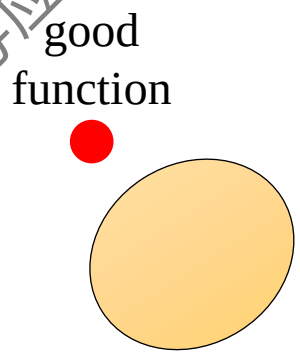
3.3、网络结构优化



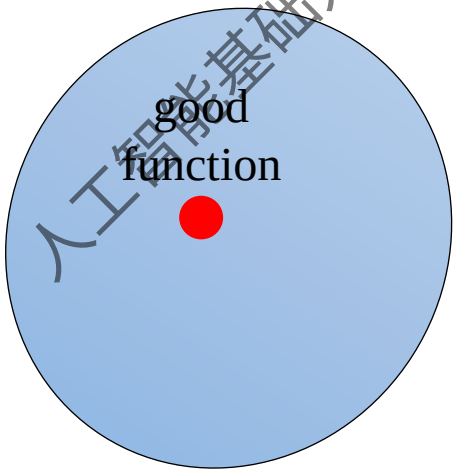
更好的网络结构

人工智能基础方法与应用

划定合适的函数范围

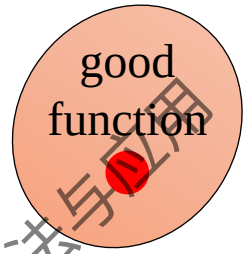


函数集合太小，可能会没有包含好的函数



函数集合大一点，比较可能包到好的函数

(容易 overfitting)



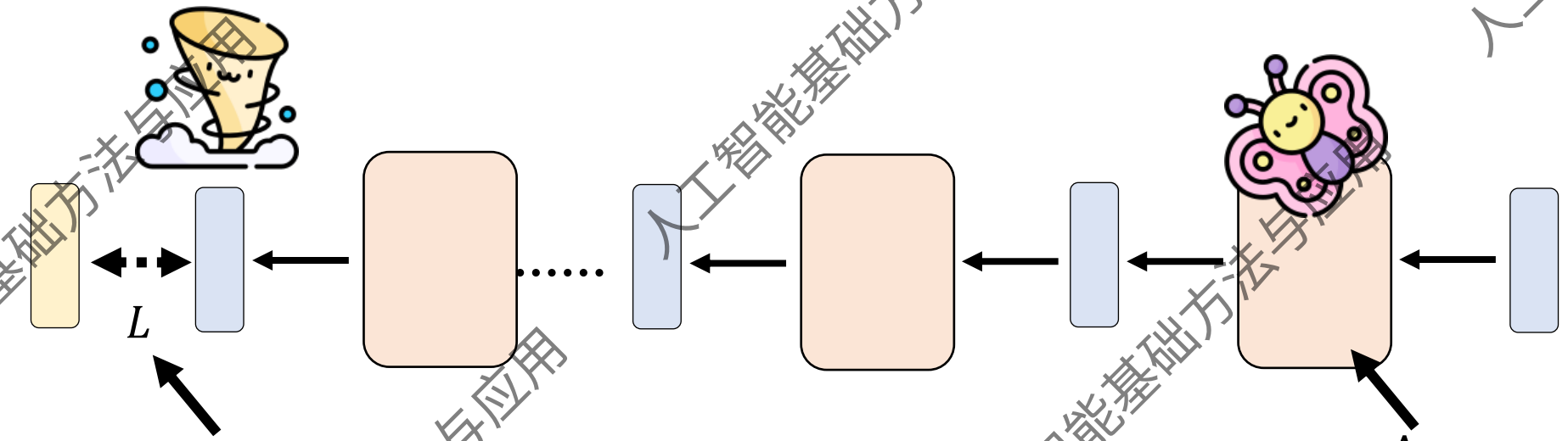
选一个小范围但刚好包含好的函数

(根据 domain knowledge)

人工智能基础方法与应用

梯度消失与梯度爆炸

(为了简化，此处没有画出 activation 和 bias)



ΔL ⋯⋯▶ 往往很小 (梯度消失 gradient vanishing)

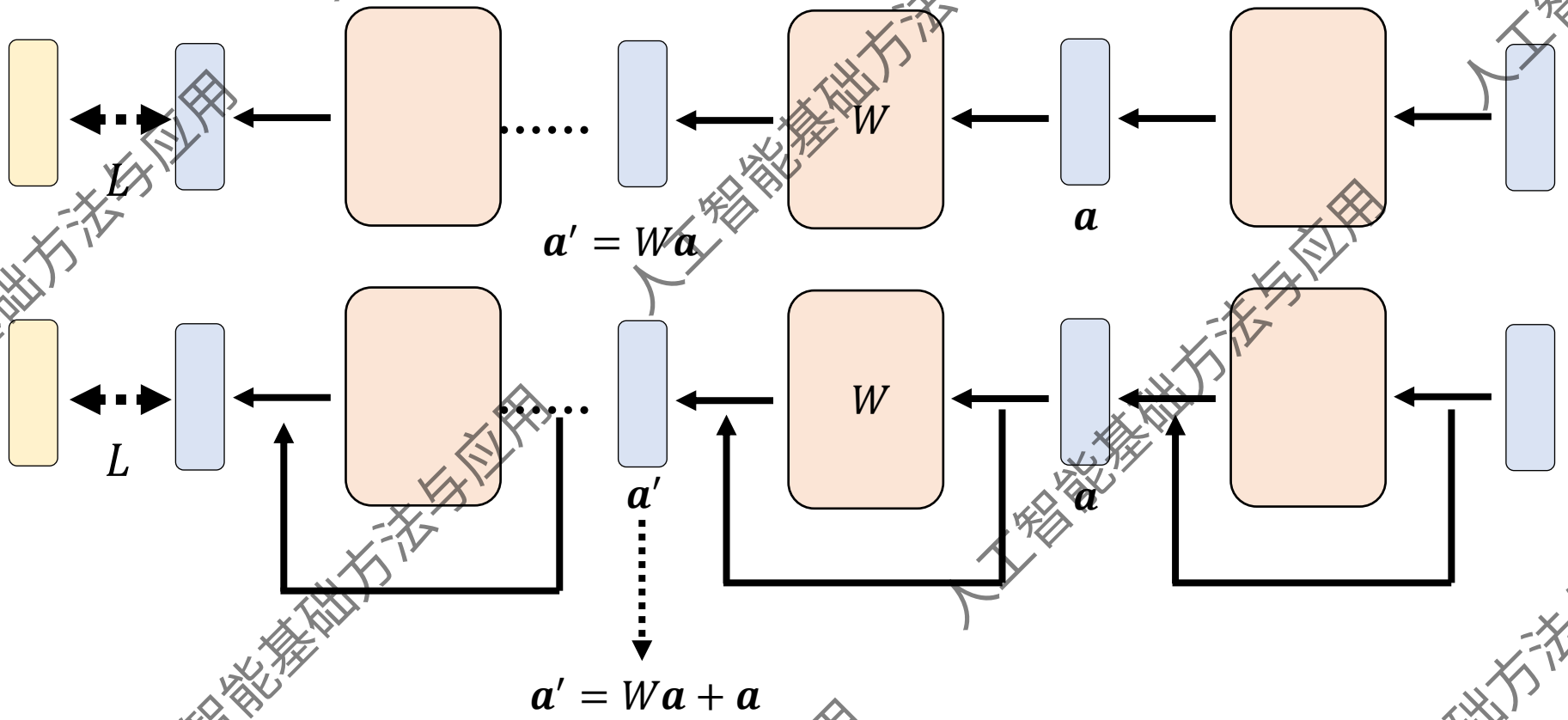
$$\frac{\partial L}{\partial w} \approx \frac{\Delta L}{\Delta w}$$

有时候又会突然变很大 (梯度爆炸 gradient exploding)

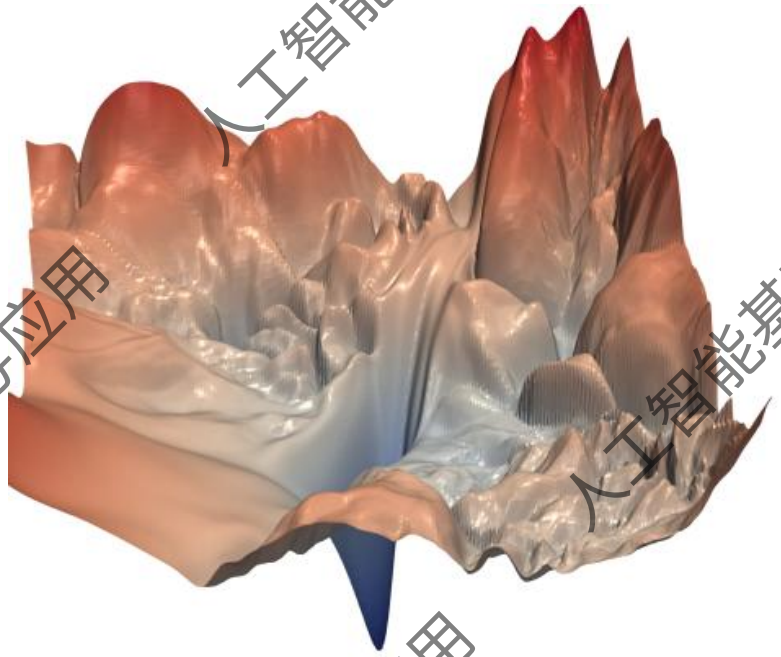
人工智能基础方法与应用

残差连接的高速通道

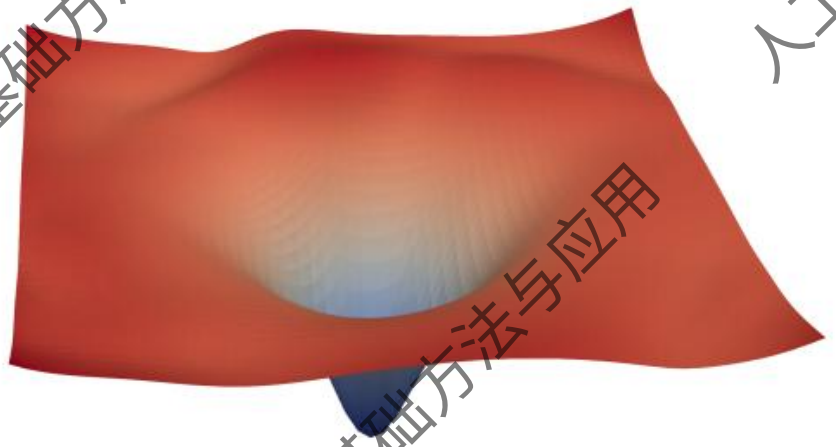
(为了简化，此处没有画出 activation 和 bias)



人工智能基础方法与应用



没有残差连接

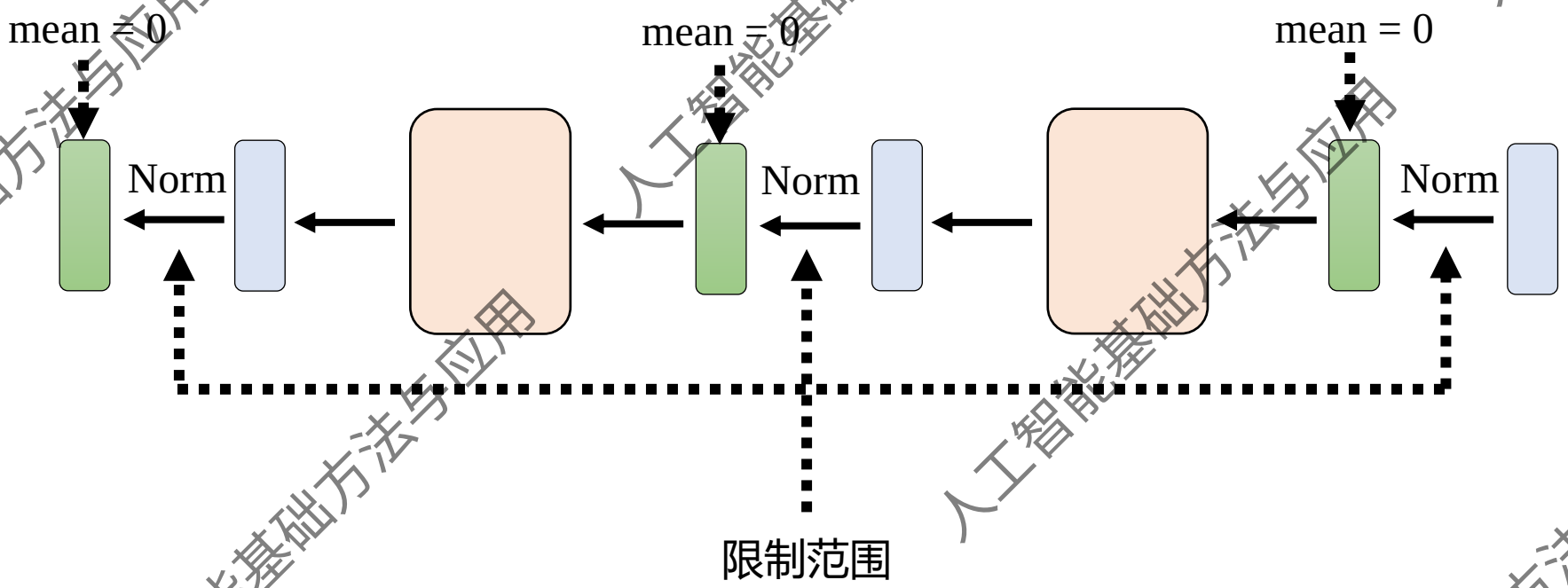


有残差连接

方法名	改哪一步	好在哪里
残差连接	改变函数搜寻范围	Better Optimization

人工智能基础方法与应用

各种 Normalization



人工智能基础方法与应用

Normalization

- Batch Renormalization
- Layer Normalization

方法名	改哪一步	好在哪里
Normalization	改变函数搜寻范围	Better Optimization (Sometimes Better Generalization)

人工智能基础方法与应用

3.4、数据增强

人工智能基础方法与应用

需要更多训练数据...

过拟合

$$L = \sum_{x \sim Train} l(f(x), \hat{y}) \quad \cdots\cdots\blacktriangleright \quad L = \sum_{x \sim Test} l(f(x), \hat{y})$$

收集更多训练数据

(注意训练数据要具有代表性)

方法名	改哪一步	好在哪里
More training data	我要找什么	Better Generalization Not for Optimization

人工智能基础方法与应用

数据增强

过拟合

$$L = \sum_{x \sim Train} l(f(x), \hat{y}) \quad \cdots\cdots\blacktriangleright \quad L = \sum_{x \sim Test} l(f(x), \hat{y})$$

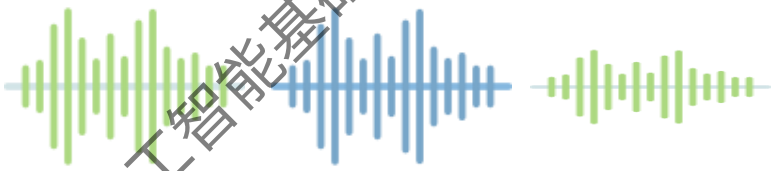
收集更多训练数据

翻转　　模糊化



bird　　bird　　bird

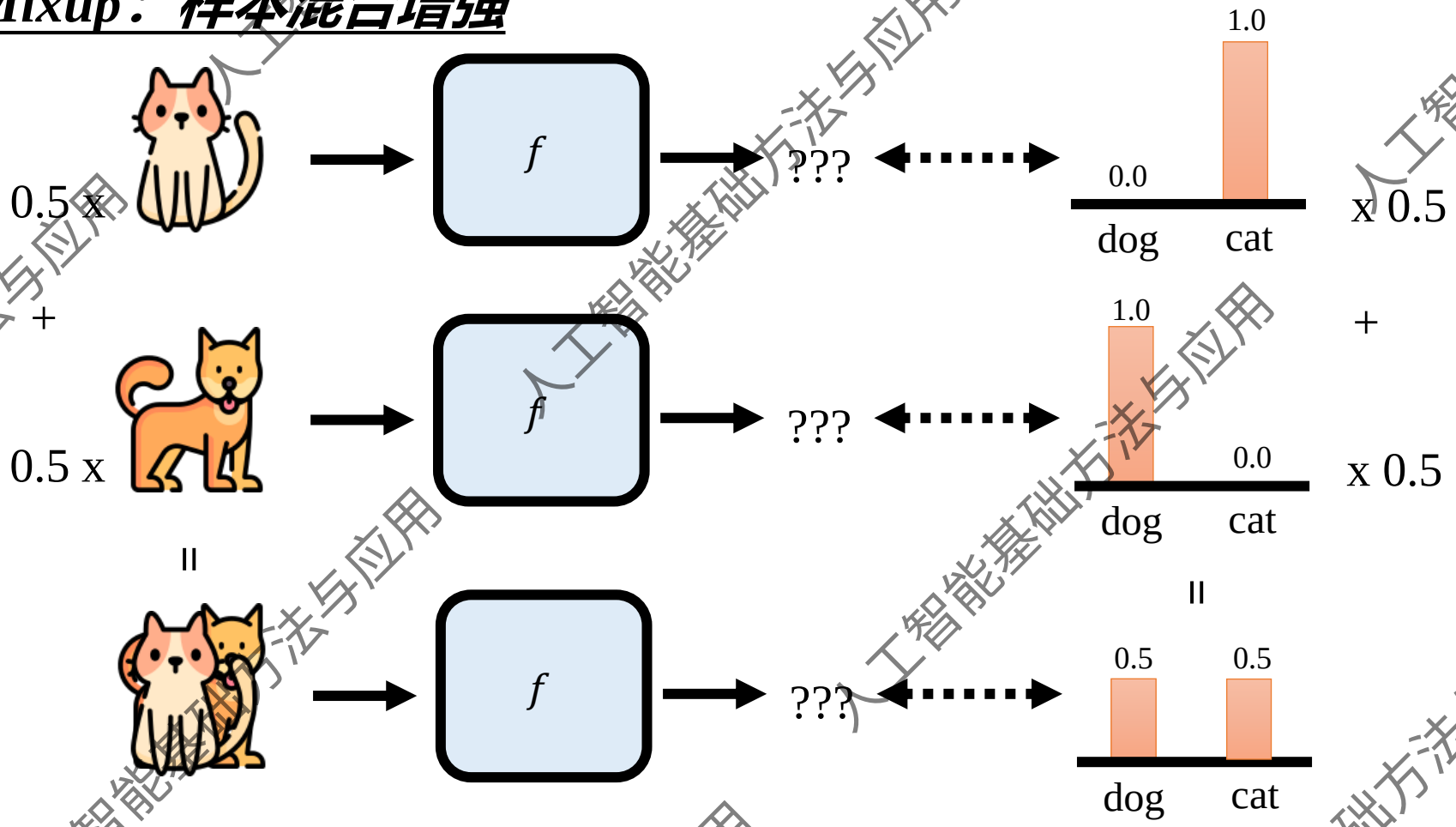
变声　　改变音量



Hello　　Hello　　Hello

~　　~　　~

Mixup：样本混合增强



人工智能基础方法与应用

数据增强

Overfitting

$$L = \sum_{x \sim Train} l(f(x), \hat{y}) \quad \cdots\cdots\blacktriangleright \quad L = \sum_{x \sim Test} l(f(x), \hat{y})$$

Collecting more training data

(be careful about the representative)

方法名	改哪一步	好在哪里
Data Augmentation (e.g. Mixup)	我要找什么	Better Generalization Not for Optimization

人工智能基础方法与应用

参数正则化

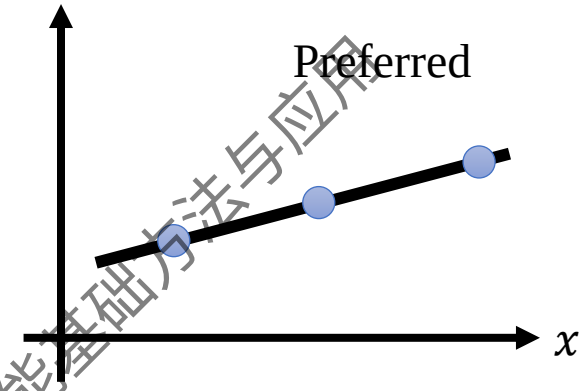
希望 Parameter 越接近 0 越好

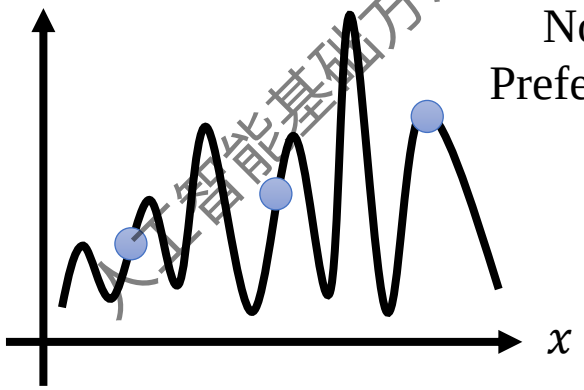
$$L'(\boldsymbol{\theta}) = L(\boldsymbol{\theta}) + \lambda R(\boldsymbol{\theta})$$

$$R(\boldsymbol{\theta}) = \sum_i (\theta_i)^2$$

跟资料有关

对于 Parameter 的偏好 (与资料无关)





参数正则化

希望 Parameter 越接近 0 越好

$$L'(\boldsymbol{\theta}) = L(\boldsymbol{\theta}) + \lambda R(\boldsymbol{\theta}) \qquad R(\boldsymbol{\theta}) = \sum_i (\theta_i)^2$$

跟资料有关　　对于 Parameter 的偏好 (与资料无关)

$$\boldsymbol{g} = \nabla L(\boldsymbol{\theta}) \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \mu \boldsymbol{g}$$

$$\boldsymbol{g}' = \nabla L'(\boldsymbol{\theta}) \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \mu \boldsymbol{g}' = \boldsymbol{\theta} - \mu(\boldsymbol{g} + 2\lambda\boldsymbol{\theta}) = \boldsymbol{\theta} - \mu\boldsymbol{g} - 2\mu\lambda\boldsymbol{\theta}$$

$$= \nabla L(\boldsymbol{\theta}) + \lambda \nabla R(\boldsymbol{\theta}) = \nabla L(\boldsymbol{\theta}) + 2\lambda\boldsymbol{\theta} \qquad = (1 - 2\mu\lambda)\boldsymbol{\theta} - \mu\boldsymbol{g}$$

$\boldsymbol{g}$

权重衰减

weight decay

人工智能基础方法与应用

参数正则化

希望 Parameter 越接近 0 越好

$$L'(\boldsymbol{\theta}) = L(\boldsymbol{\theta}) + \lambda R(\boldsymbol{\theta})$$

$$R(\boldsymbol{\theta}) = \sum_i (\theta_i)^2$$

跟资料有关

对于 Parameter 的偏好 (与资料无关)

方法名	改哪一步	好在哪里
Parameter Regularization	我要找什么	Better Generalization Not for Optimization
Weight Decay	找最优函数	Better Generalization Not for Optimization

人工智能基础方法与应用

AdamW

https://arxiv.org/abs/1711.05101

Momentum

For each dimension i: $m_i^t = \beta m_i^{t-1} + (1-\beta) g_i^t$

RMSProp

For each dimension i: $\sigma_i^t = \sqrt{\alpha \left(\sigma_i^{t-1}\right)^2 + (1-\alpha)\left(g_i^t\right)^2}$

Adam

$$\theta_i^{t+1} \leftarrow \theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t$$

AdamW

$$\theta_i^{t+1} \leftarrow (1 - 2\eta^t \lambda)\theta_i^t - \frac{\eta^t}{\sigma_i^t} m_i^t$$

方法名	改哪一步	好在哪里
AdamW	找最好的函数	Better Generalization (cf. Adam) Better Optimization (cf. Vanilla)

人工智能基础方法与应用

四、卷积神经网络

人工智能基础方法与应用

图像分类的基本流程



100 x 100

Model

$$\begin{bmatrix} \vdots \\ 0.2 \\ 0.7 \\ 0.1 \\ \vdots \end{bmatrix} \quad y'$$

$$\begin{matrix} \\ \text{dog} \\ \text{cat} \\ \text{tree} \\ \\ \end{matrix} \begin{bmatrix} \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \end{bmatrix} \quad \hat{y}$$

交叉熵

(所有待分类的图像尺寸相同)

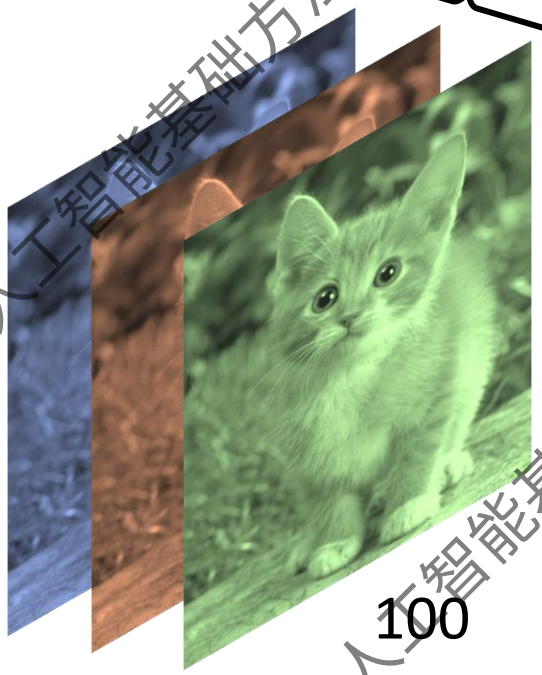
人工智能基础方法与应用

图像分类的基本流程



100 x 100

3-D tensor





每个数值表示对应位置、
对应颜色通道的强度

人工智能基础方法与应用

张量Tensor

- 向量（Vector）：一组按顺序排列的数，只有一个维度

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]$$

- 张量（Tensor）是对标量、向量、矩阵的统一推广，可以具有任意多个维度：

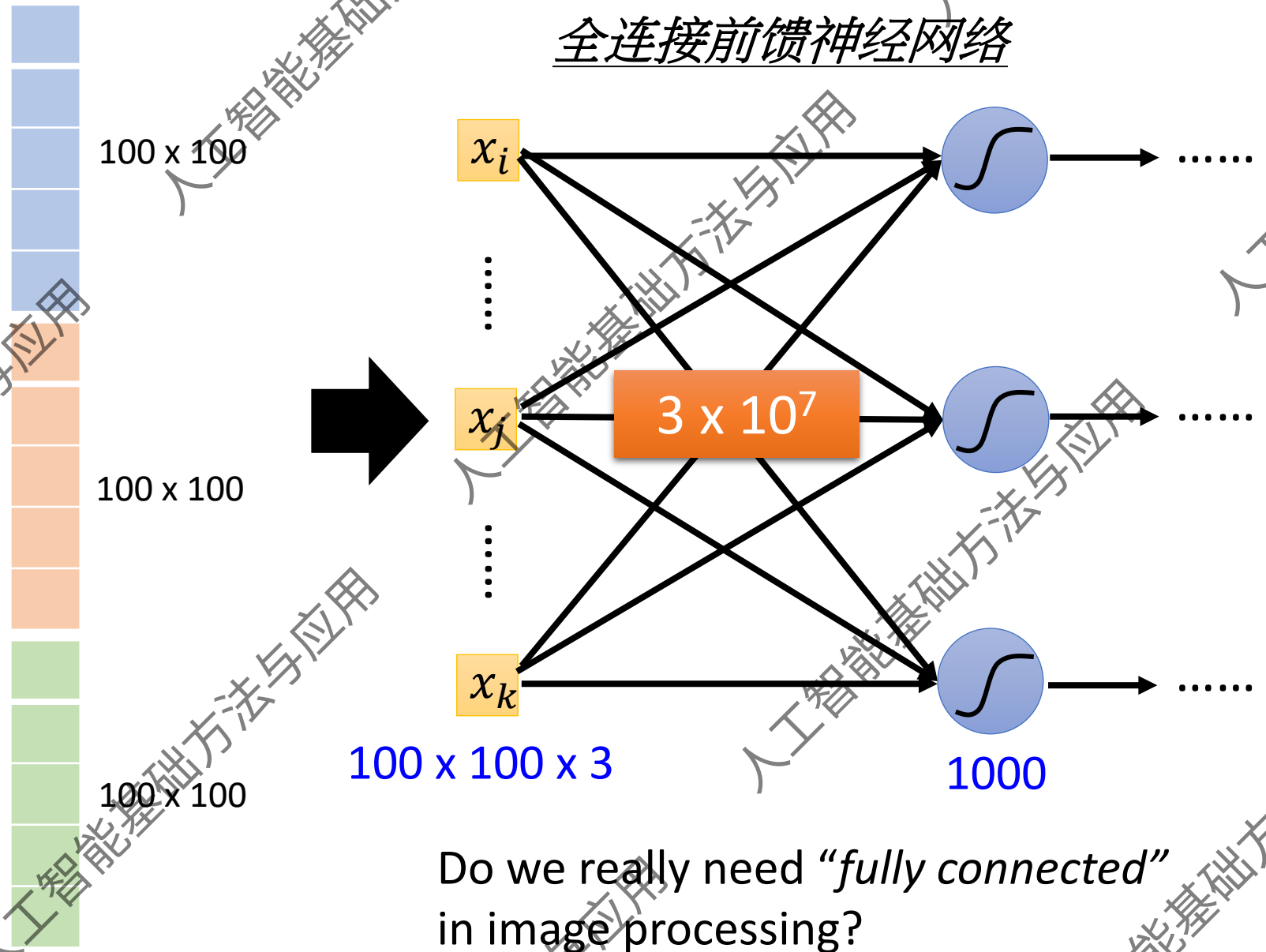
$$\begin{array}{lcl} \text{向量} & \rightarrow & \text{1-D Tensor} \\ \text{矩阵} & \rightarrow & \text{2-D Tensor} \\ \text{三维数据} & \rightarrow & \text{3-D Tensor} \end{array}$$

- 例如一张 RGB 图像：

$$X \in \mathbb{R}^{3\times 28\times 28}$$

人工智能基础方法与应用

全连接前馈神经网络

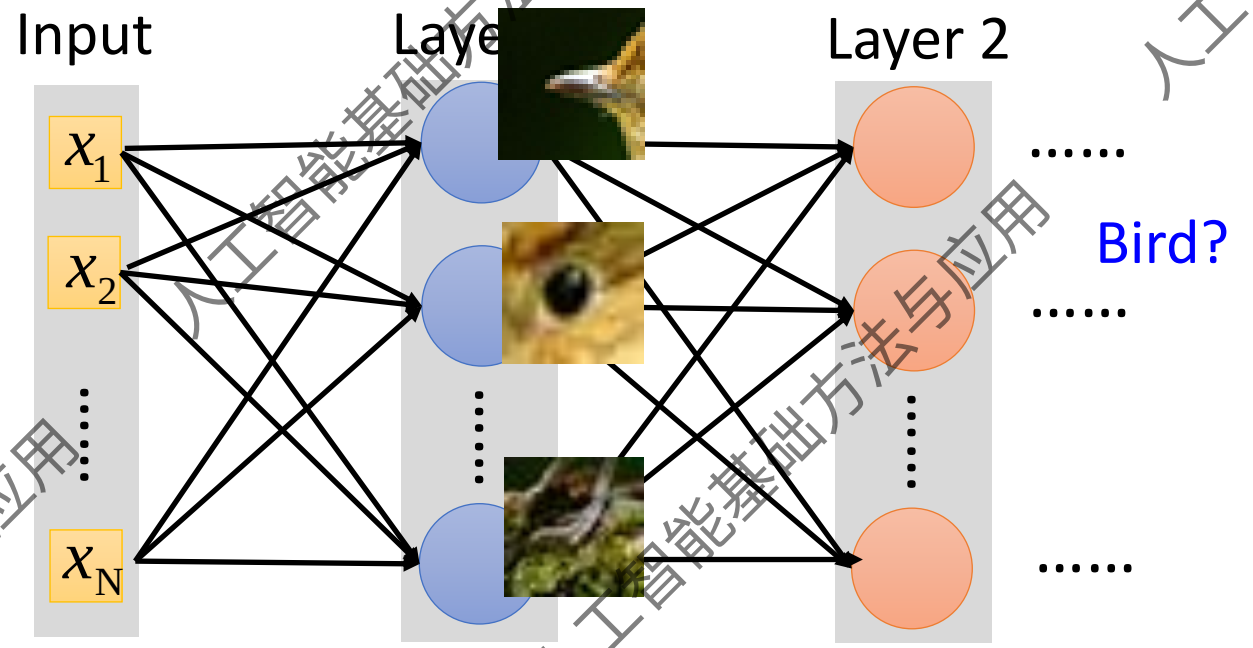


Do we really need *"fully connected"* in image processing?

从局部识别扩展到整体

识别一些关键局部特征





也许人类识别鸟类时，也会采用类似的方式… ☺

人工智能基础方法与应用

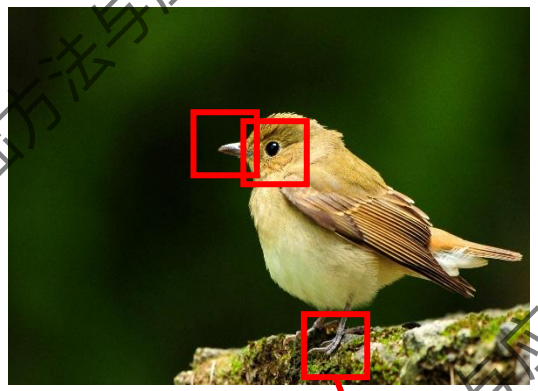


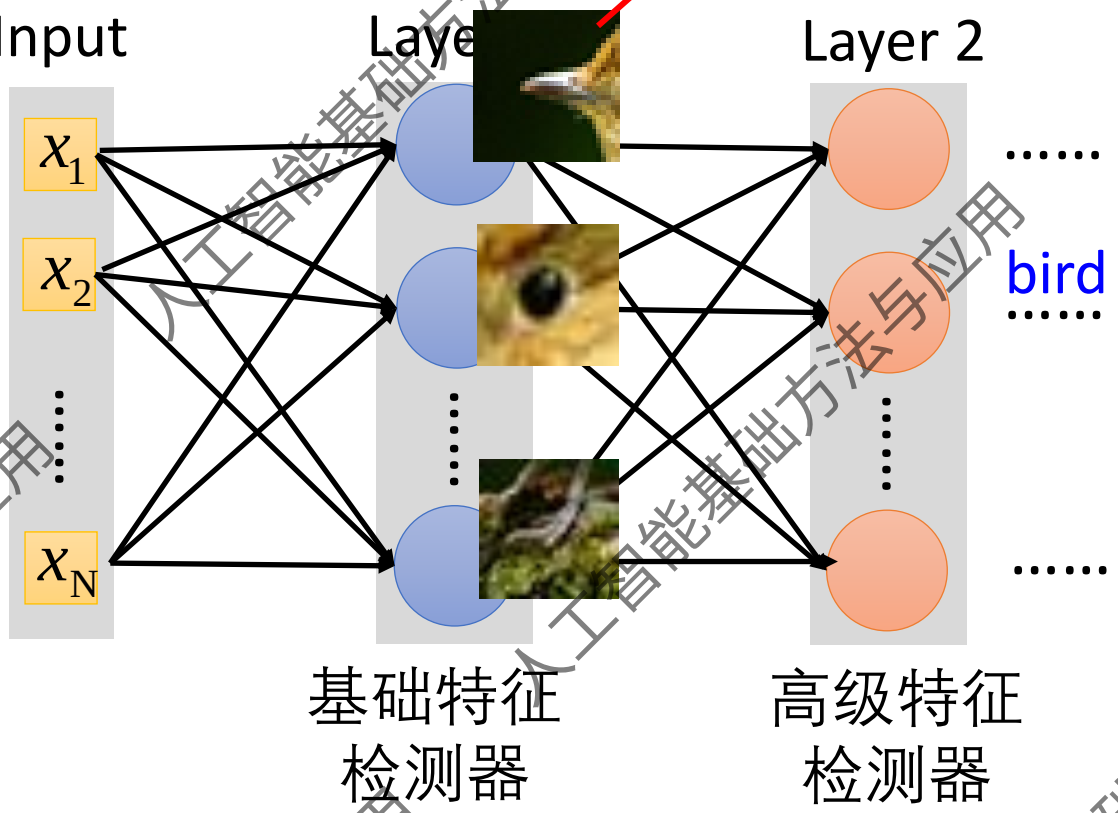
人工智能基础方法与应用

神经元只需关注局部区域

一个神经元不需要关注整张图像

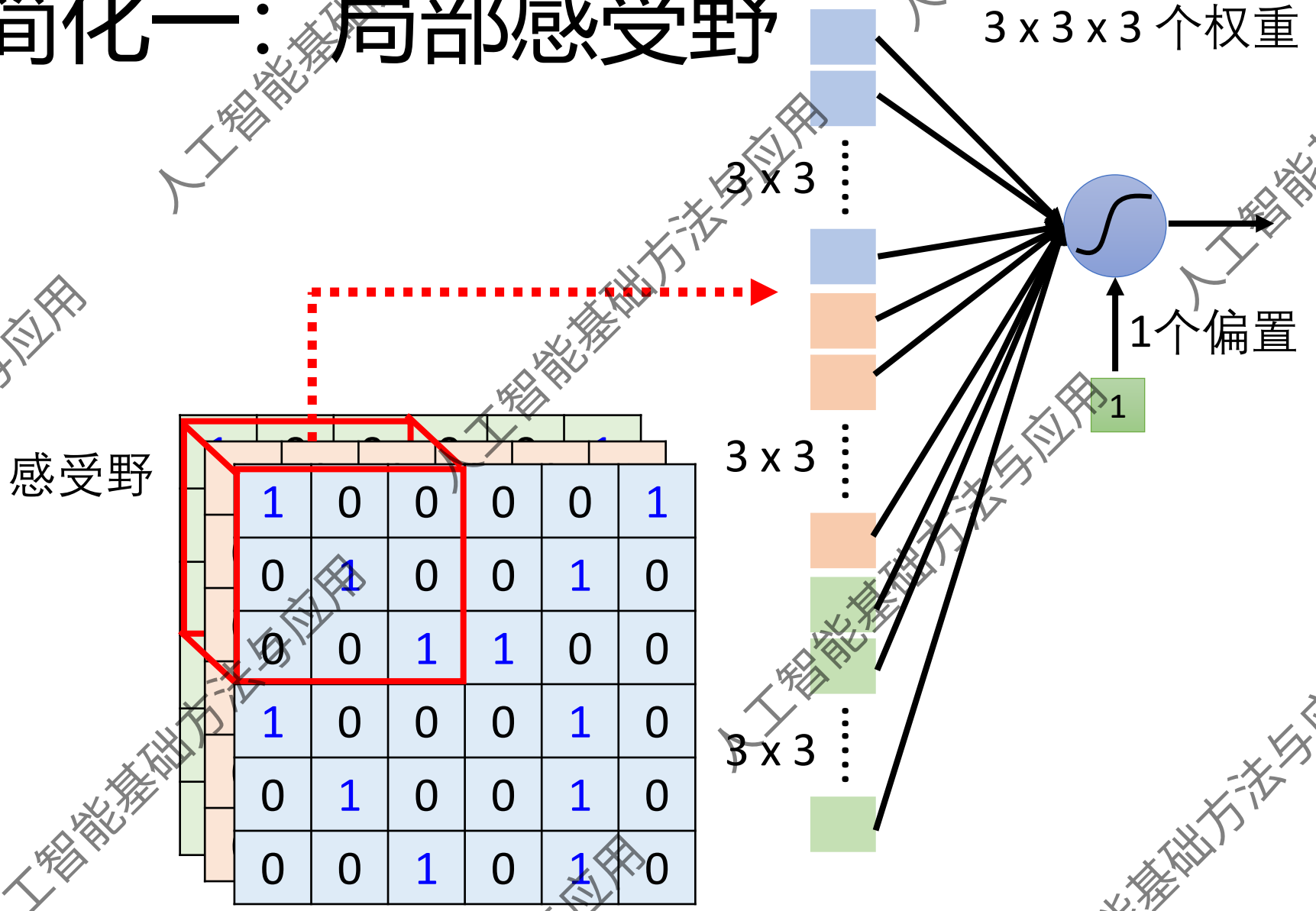
一定要看整张图像吗?





很多关键特征只存在于图像的局部区域

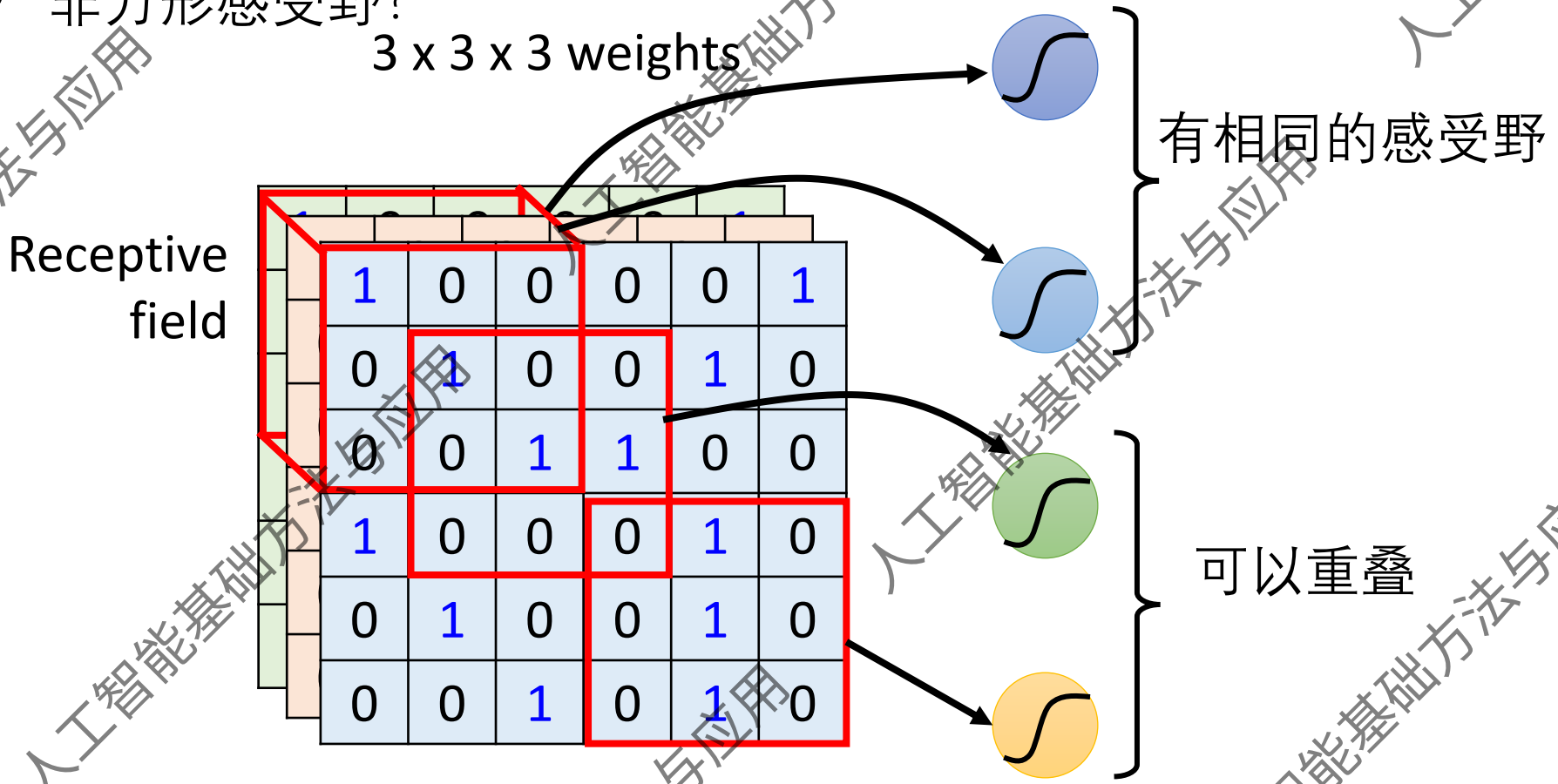
简化一：局部感受野



人工智能基础方法与应用

感受野的形状与排列

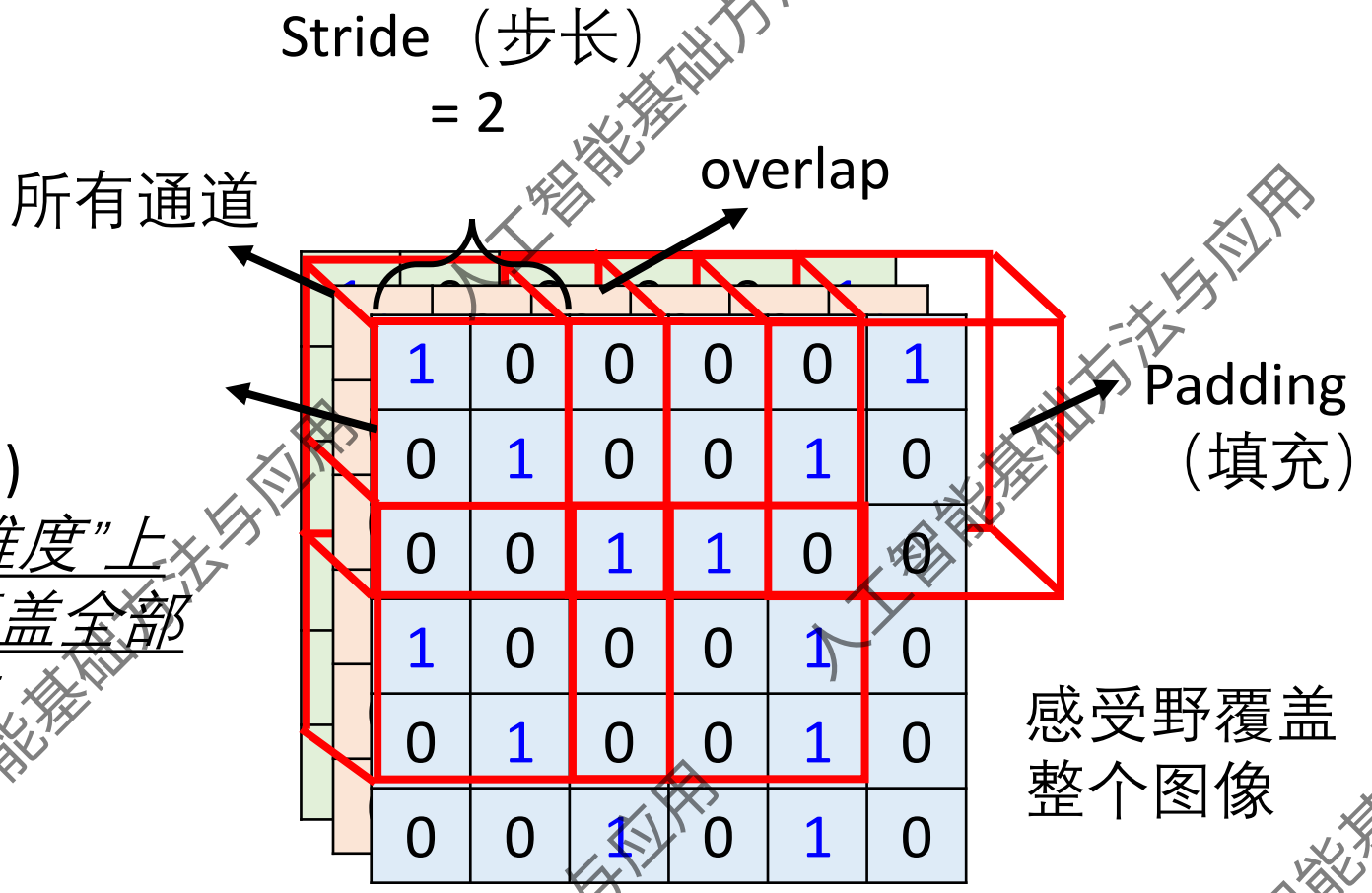
- 不同的神经元能否拥有不同大小的感受野?
- 可以只覆盖某些频道吗?
- 非方形感受野?



人工智能基础方法与应用

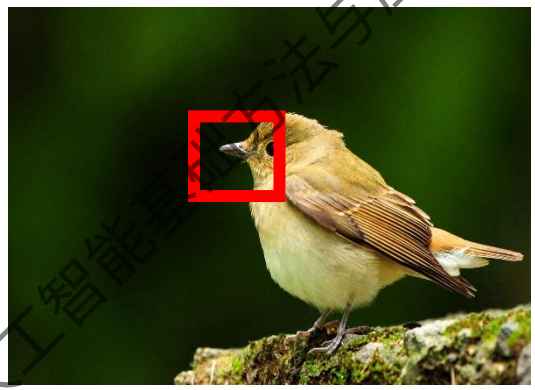
感受野的典型设定

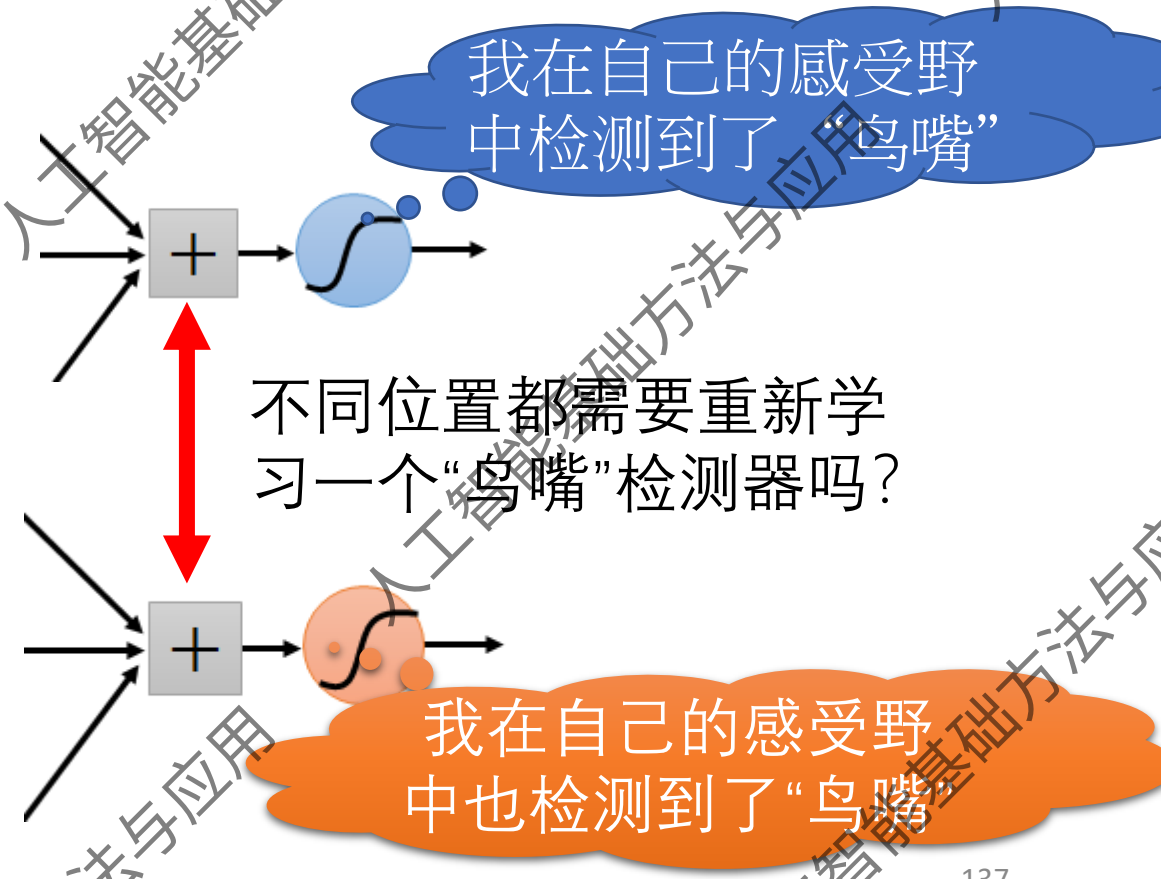
对输入图像中的同一个局部感受野，不是只有一个神经元去看它，而是可以有很多个神经元同时去看



同一模式可出现在不同位置

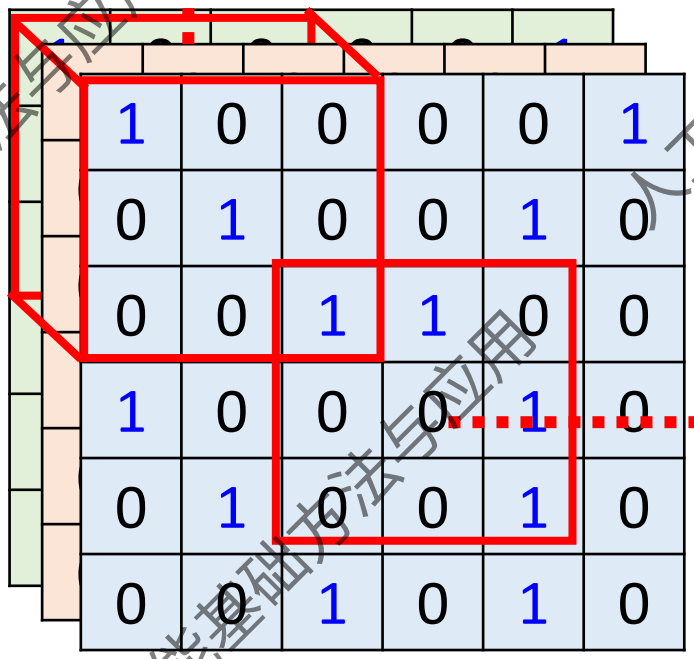


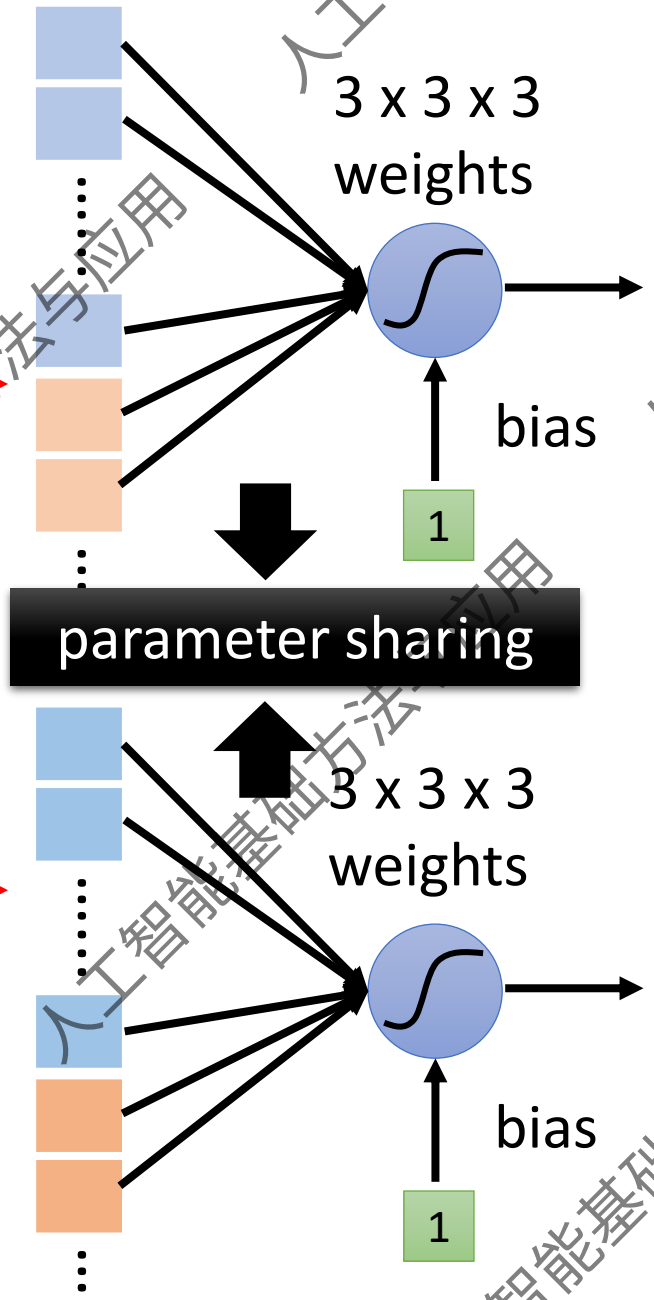




人工智能基础方法与应用

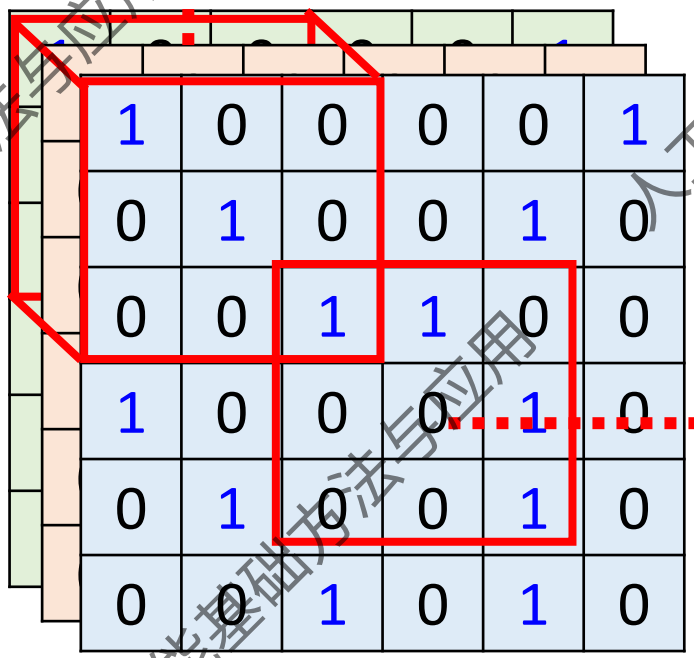
参数共享

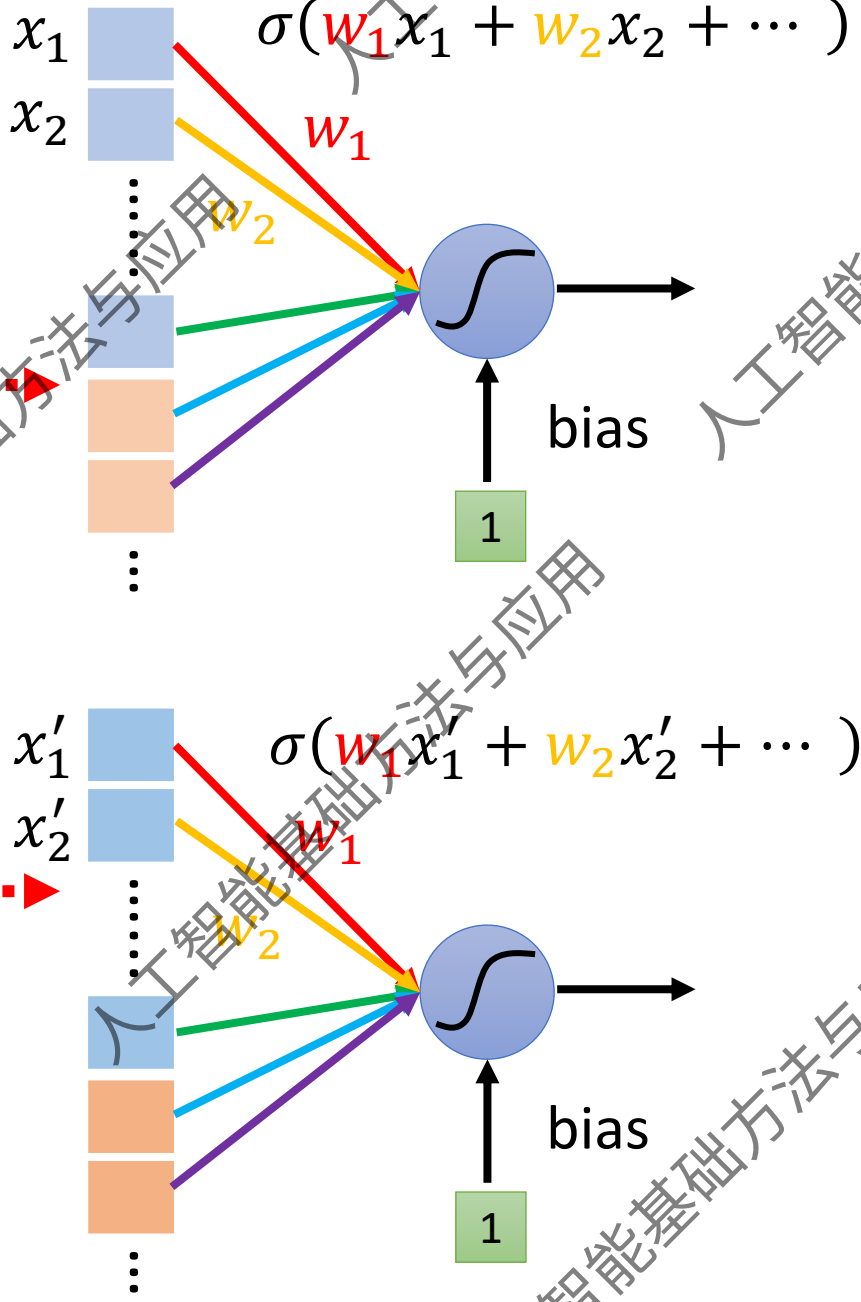




人工智能基础方法与应用

参数共享的范围



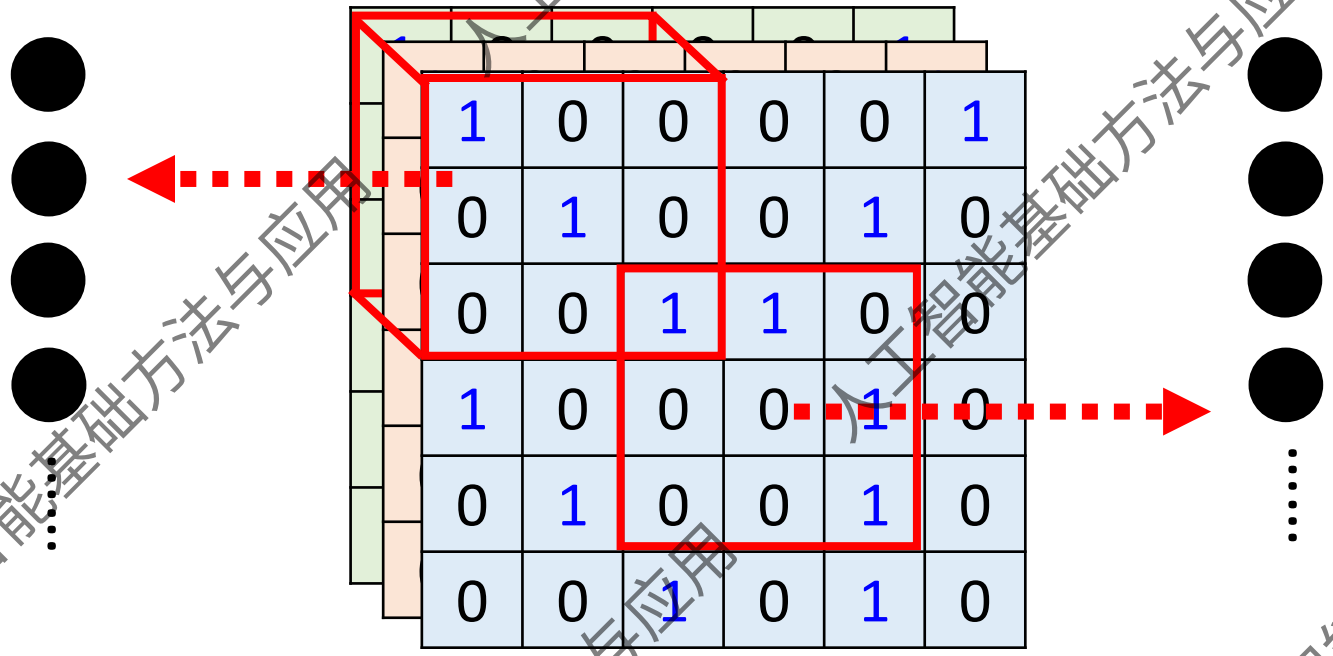


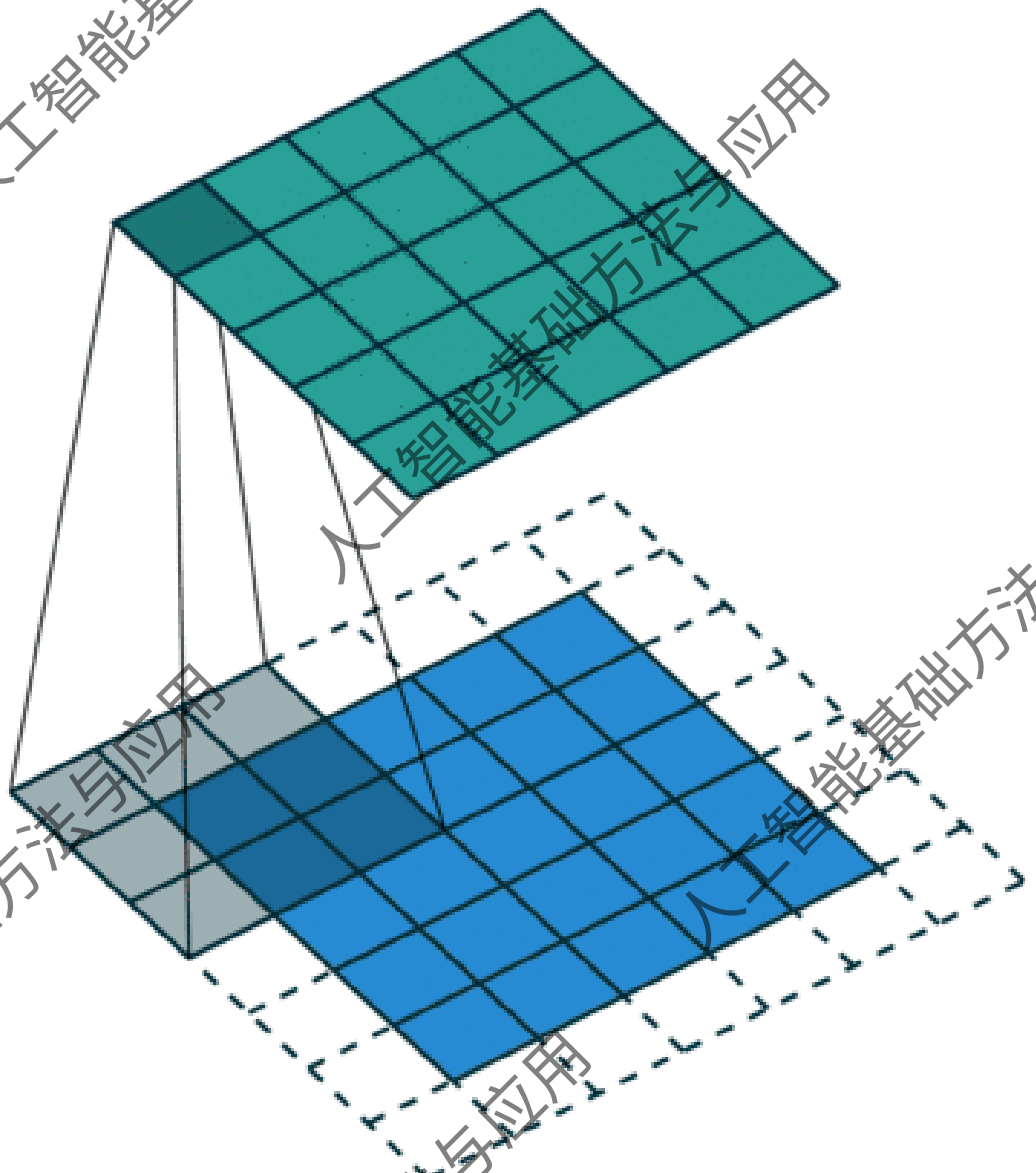
即使两个神经元的感受野相同，
它们的参数也可以彼此独立

人工智能基础方法与应用

每个位置都配置多个检测器

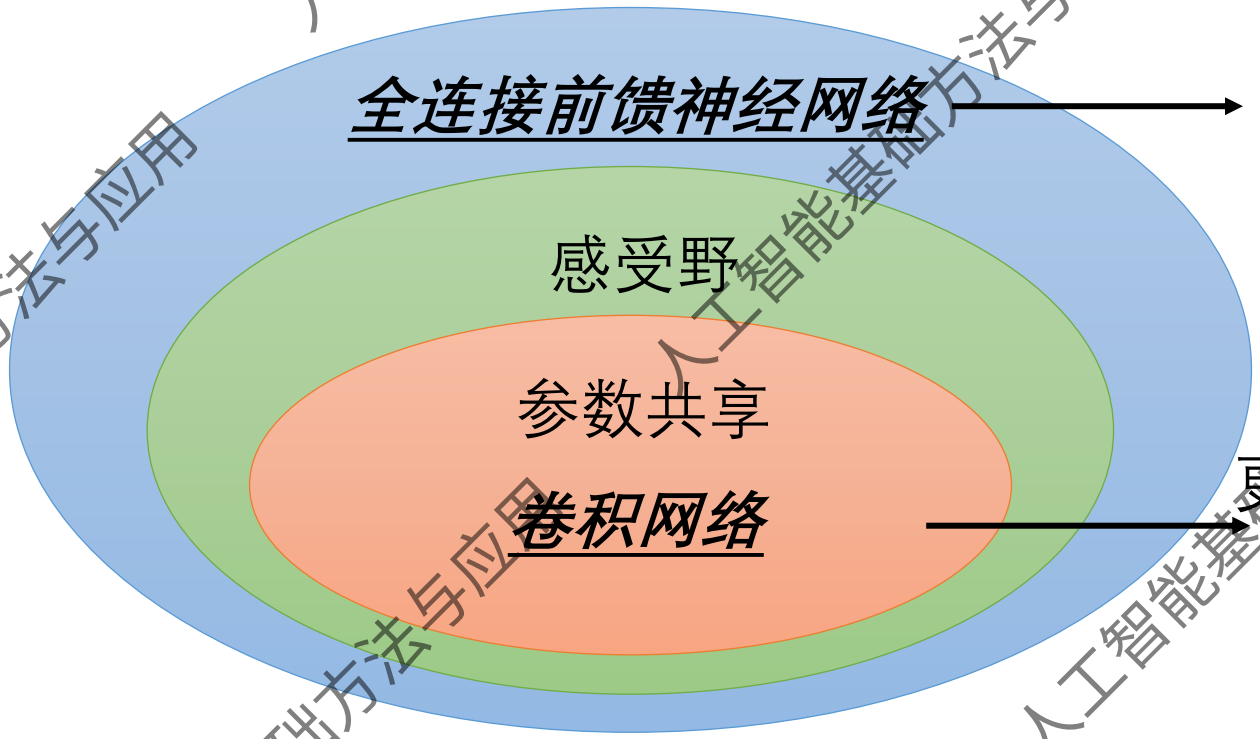
每个感受野都有一组神经元（例如，64个神经元）





人工智能基础方法与应用

卷积层的优势与归纳偏置



Jack of all trades, master of none

更强的图像任务先验 (for image)

- 很多有用的模式只存在于图像的局部区域
- 相同的局部模式可能出现在图像的不同位置

归纳偏置：模型在看数据之前，就已经带着的一些“先验假设”

人工智能基础方法与应用

适度下采样不改变对象身份

- 对图像像素进行适度下采样，不会改变物体的类别

bird



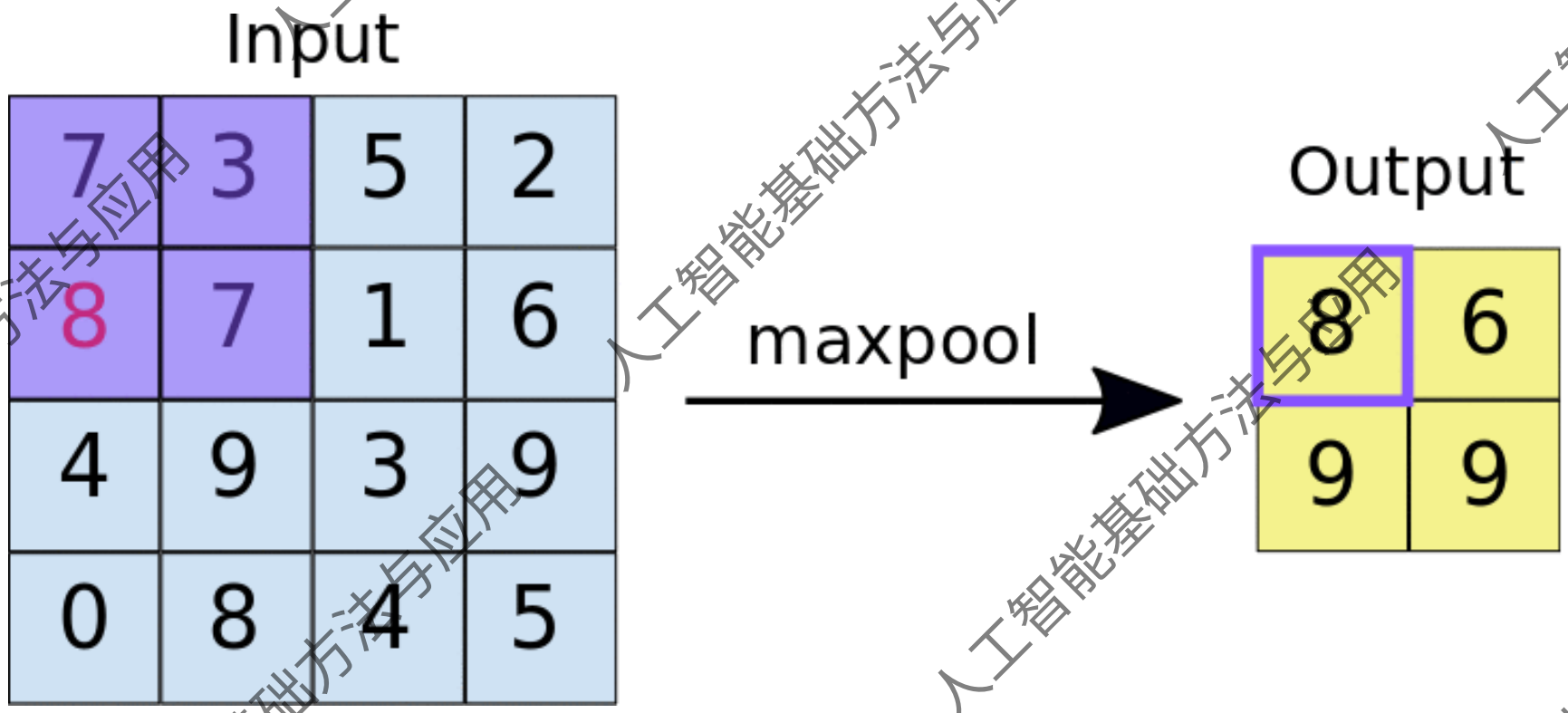
下采样

bird



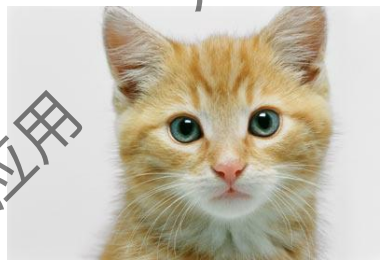
人工智能基础方法与应用

最大池化（Max Pooling）



人工智能基础方法与应用

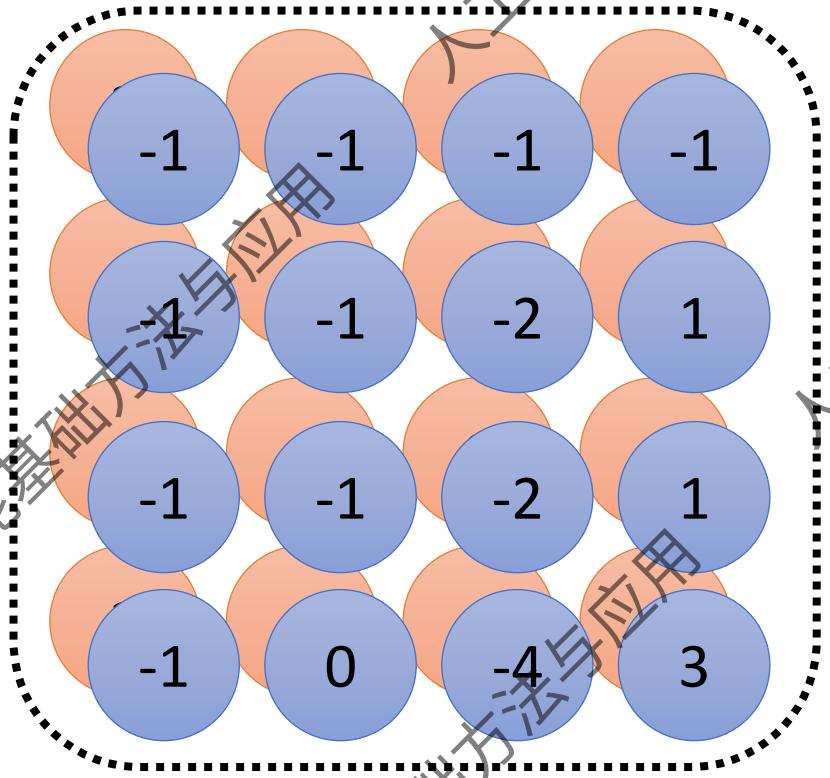
卷积层+ 池化层



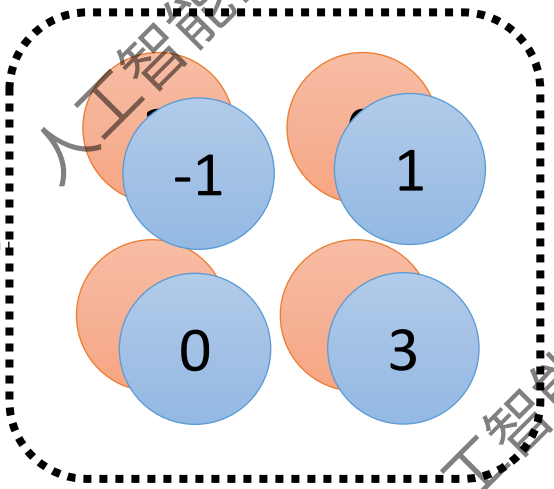
Repeat

Convolution

Pooling

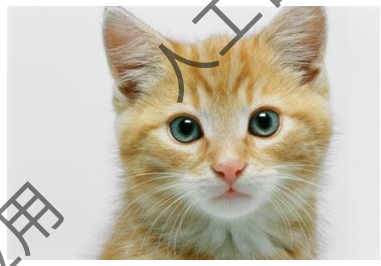


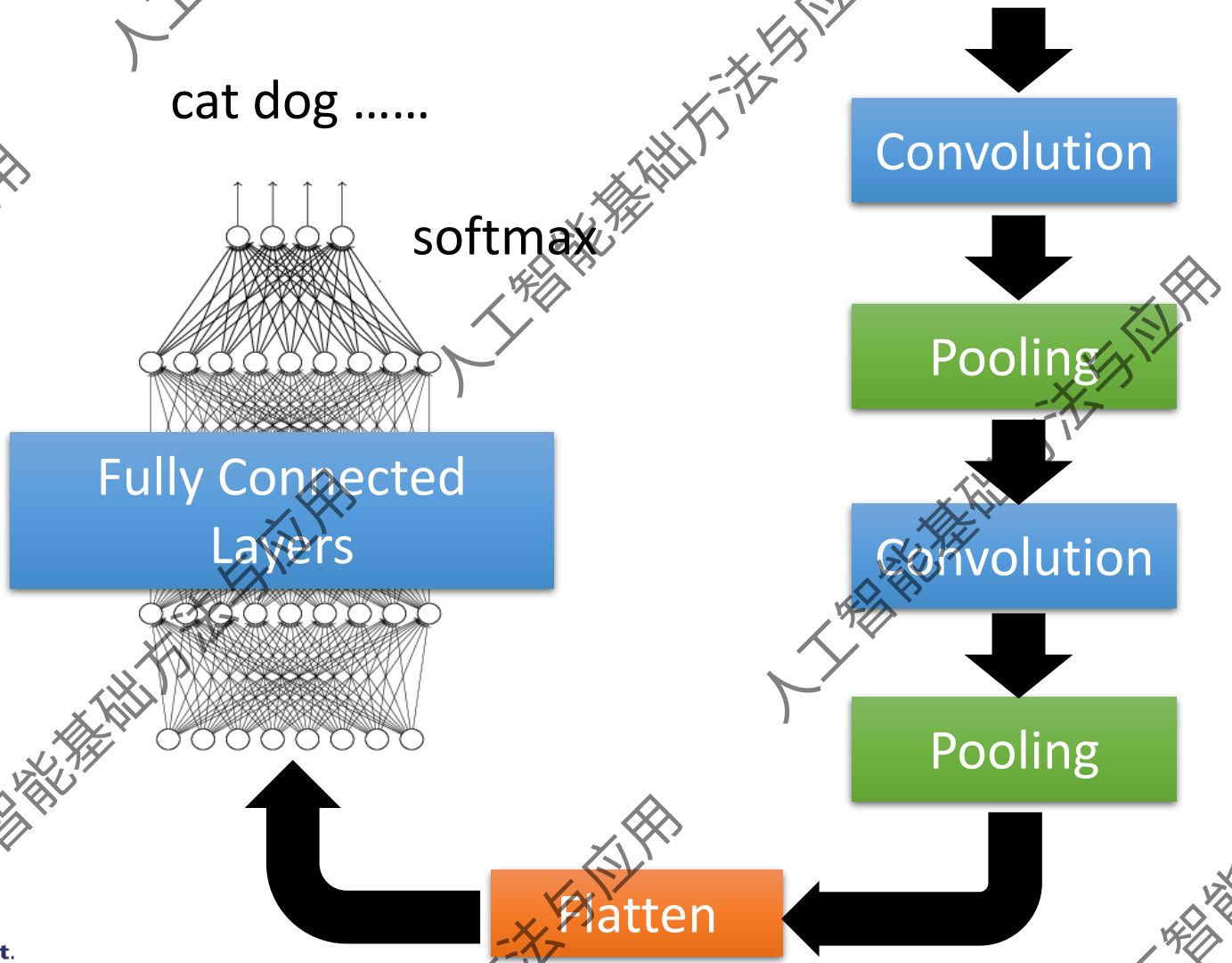
"Image" with 64 channels



人工智能基础方法与应用

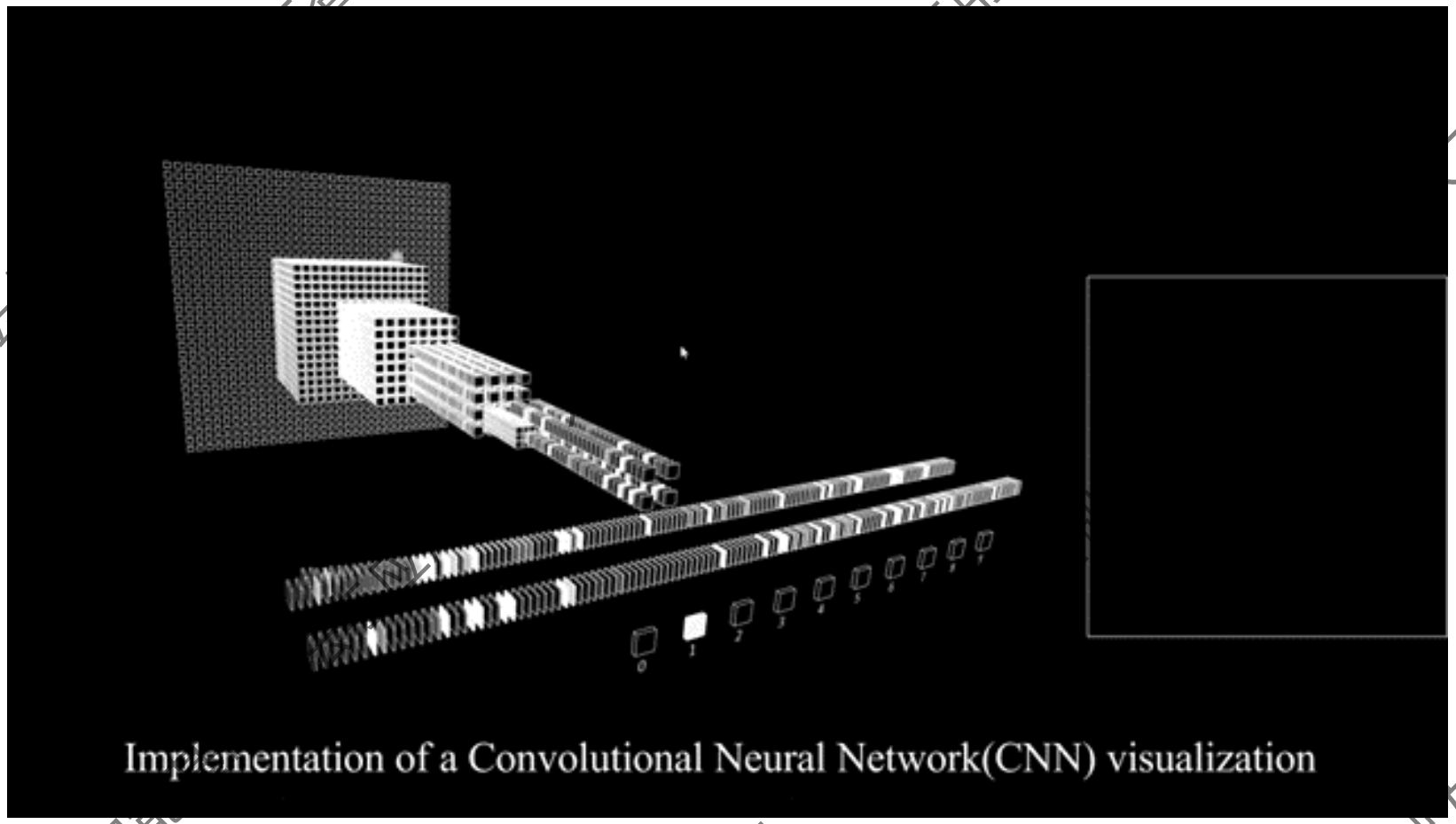
完整的卷积神经网络





人工智能基础方法与应用

卷积神经网络计算过程示例



卷积神经网络计算过程示例



人工智能基础方法与应用

应用案例：卷积网络下围棋



19 x 19 矩阵
(image)

Network

Next move
(19 x 19 positions)

19 x 19 classes

48 通道
in Alpha Go

Black: 1

white: -1

none: 0

Fully-connected network can be used

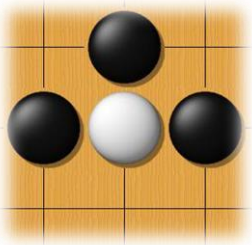
But CNN performs much better.

人工智能基础方法与应用

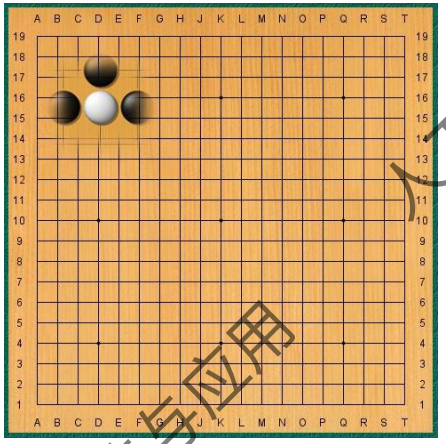
为什么围棋适合使用 CNN

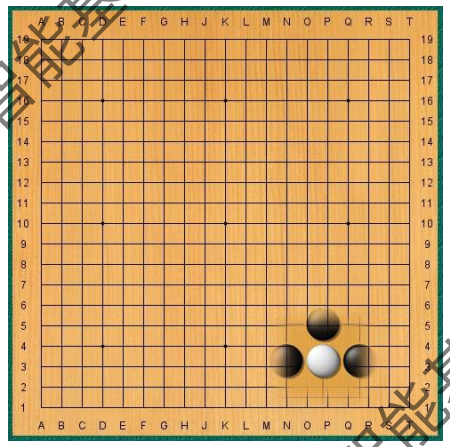
- 很多重要棋形只存在于棋盘的局部区域

Alpha Go使用5 x 5卷积核



- 相同的棋形可能出现在棋盘的不同位置

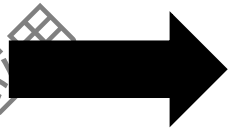




人工智能基础方法与应用

围棋网络为什么不做池化

- 对图像像素进行适度下采样，不会改变物体的类别



Pooling

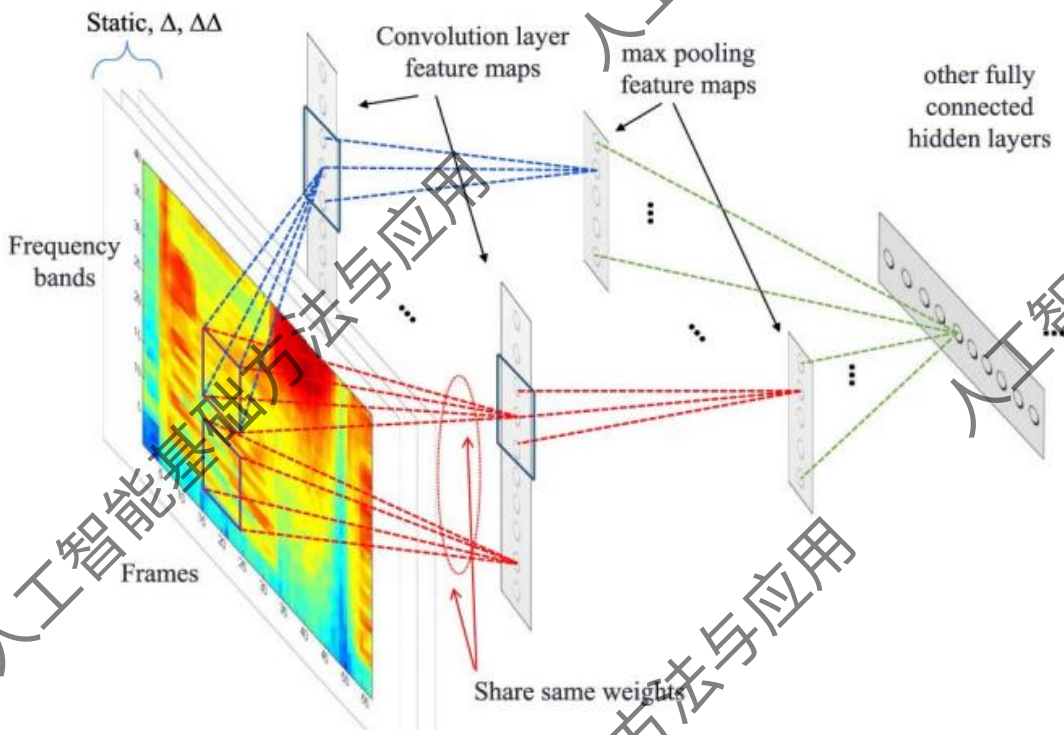
How to explain this???

Neural network architecture. The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Tabl[illegible] 256 and 384 filters

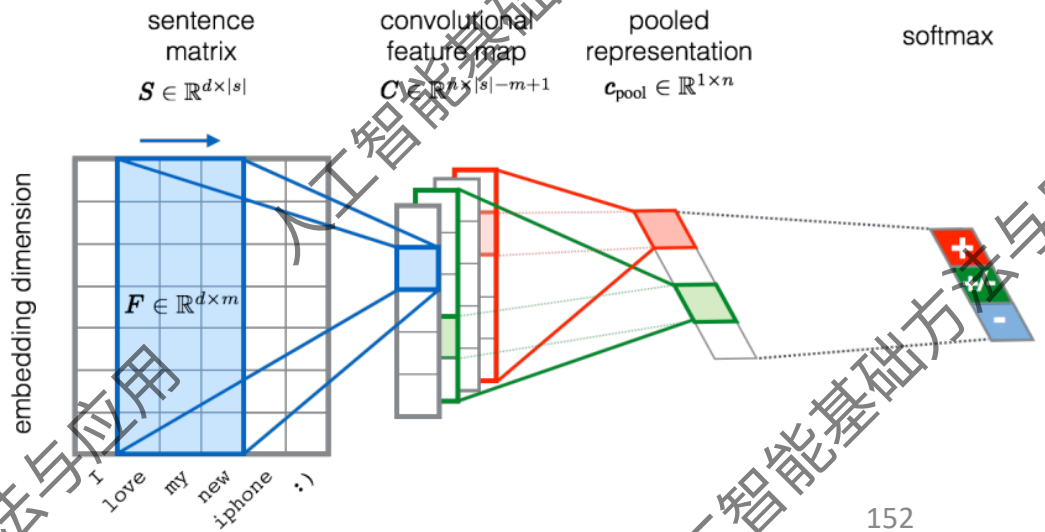
Alpha Go没有使用Pooling ……

更多的CNN应用

语音处理



自然语言处理

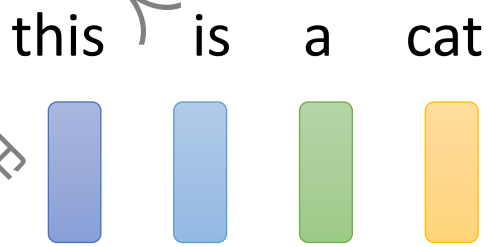


人工智能基础方法与应用

五、自注意力机制

人工智能基础方法与应用

文本输入



One-hot Encoding

apple = [1 0 0 0 0 .....]

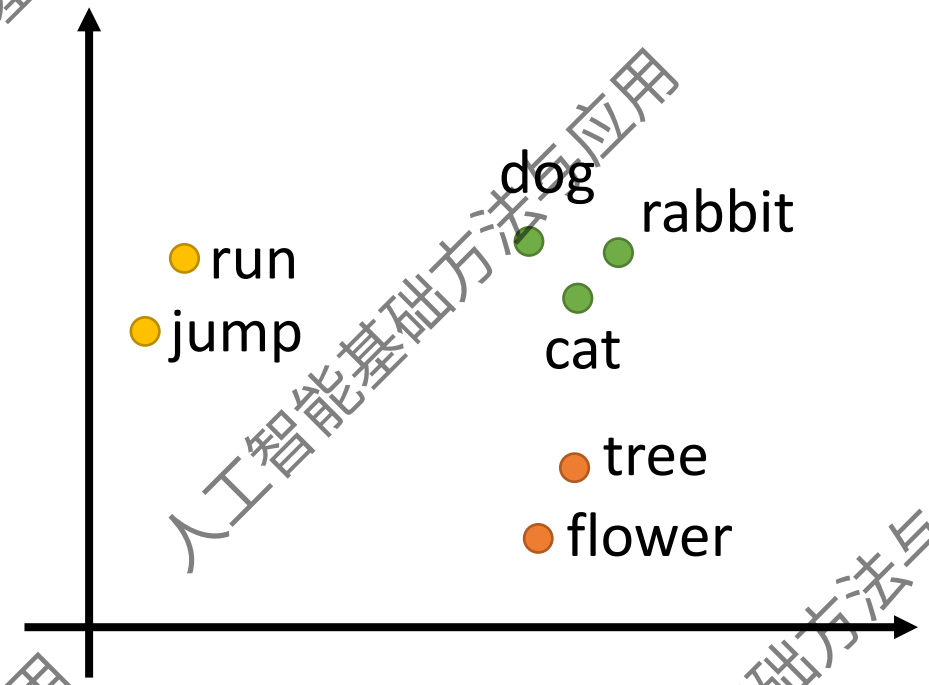
bag = [0 1 0 0 0 .....]

cat = [0 0 1 0 0 .....]

dog = [0 0 0 1 0 .....]

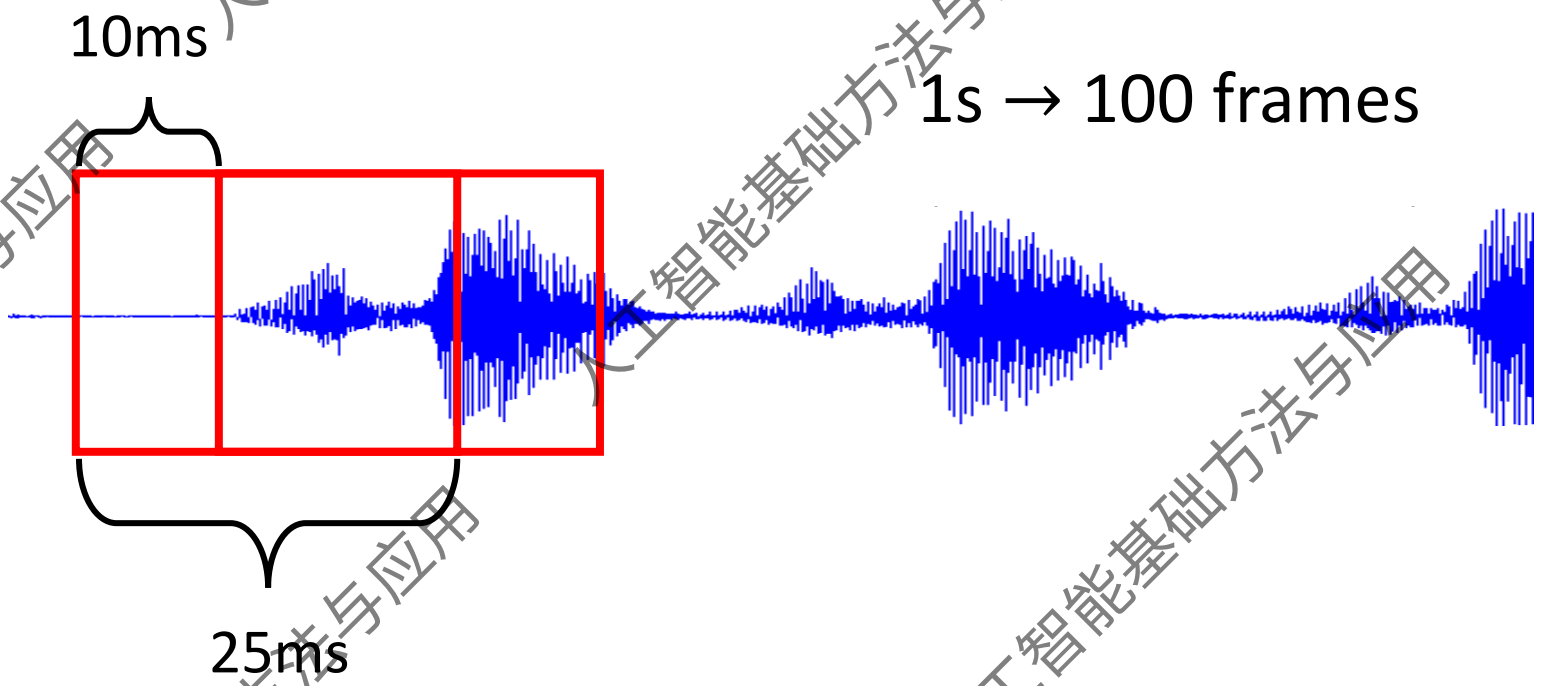
elephant = [0 0 0 0 1 .....]

Word Embedding



人工智能基础方法与应用

语音输入



frame

400 sample points (16KHz)

39-dim MFCC

80-dim filter bank output

人工智能基础方法与应用

图数据输入：社交网络

- 图也可以表示为张量（把每个节点看作一个向量）



Each profile
is a vector

人工智能基础方法与应用

图数据输入：分子结构

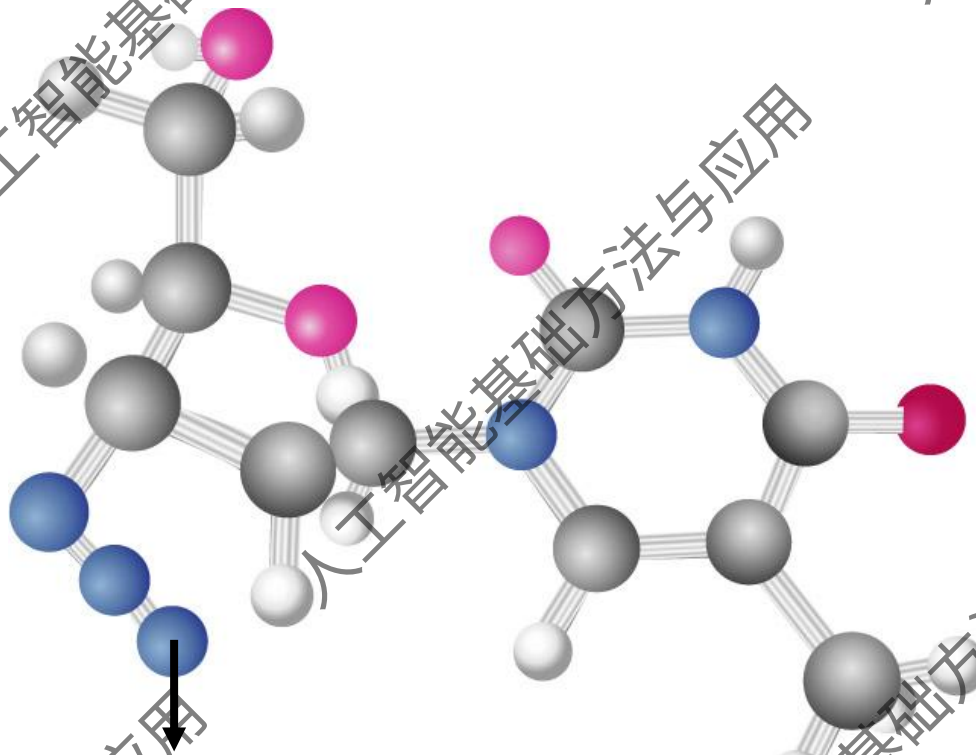
- 图也可以表示为张量（把每个节点看作一个向量）

H = [1 0 0 0 0 ……]

C = [0 1 0 0 0 ……]

O = [0 0 1 0 0 ……]

⋮



One-hot vector

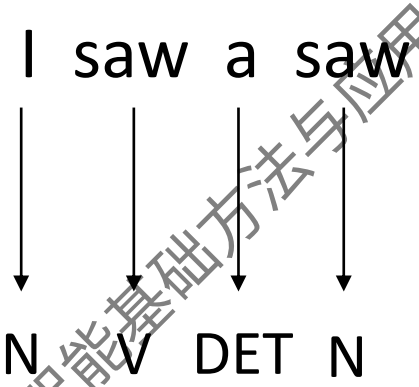
人工智能基础方法与应用

输出形式一：逐向量预测标签

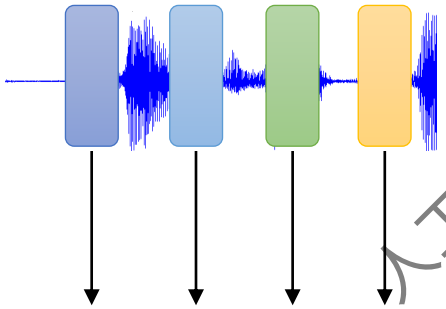
- 输入N个向量，模型对每个向量分别输出一个标签



典型应用



词性标注



语音帧标注

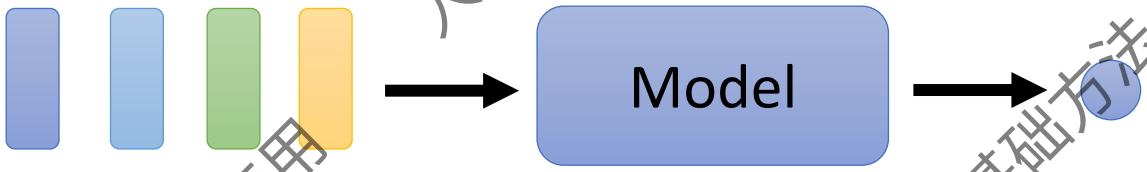


输出形式二：整个输入对应一个标签

- 逐向量预测标签



- 整个序列对应一个标签

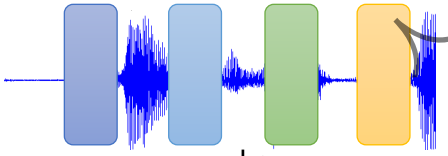


示例应用

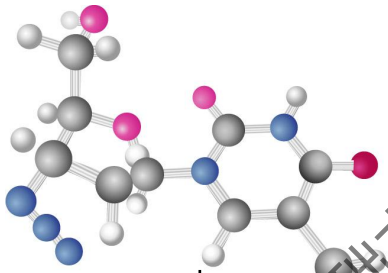
“这很好”

情感分析

正面



说话人身份



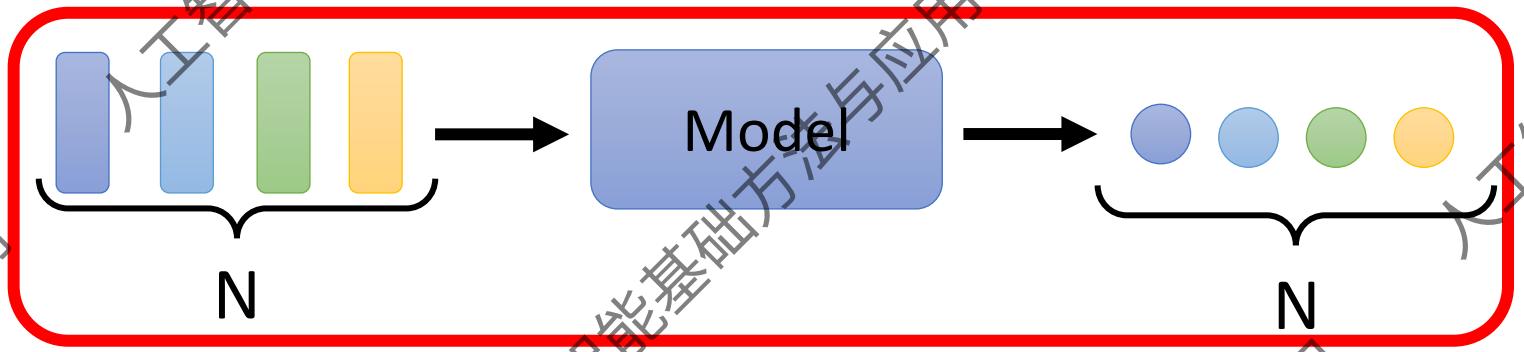
分子性质预测

亲水性

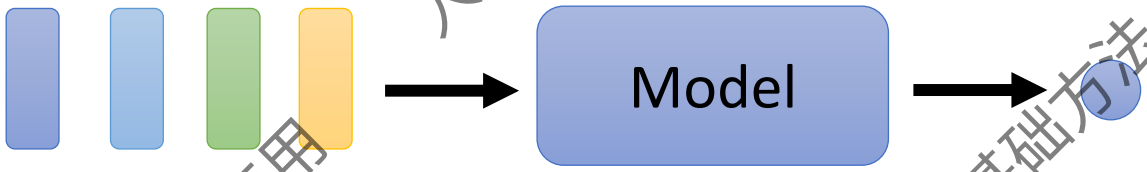
人工智能基础方法与应用

输出形式三：序列到序列

- 逐向量预测标签



- 整个序列对应一个标签



- 输出序列长度由模型决定 序列到序列（seq2seq）



人工智能基础方法与应用

序列标注：如何利用完整上下文

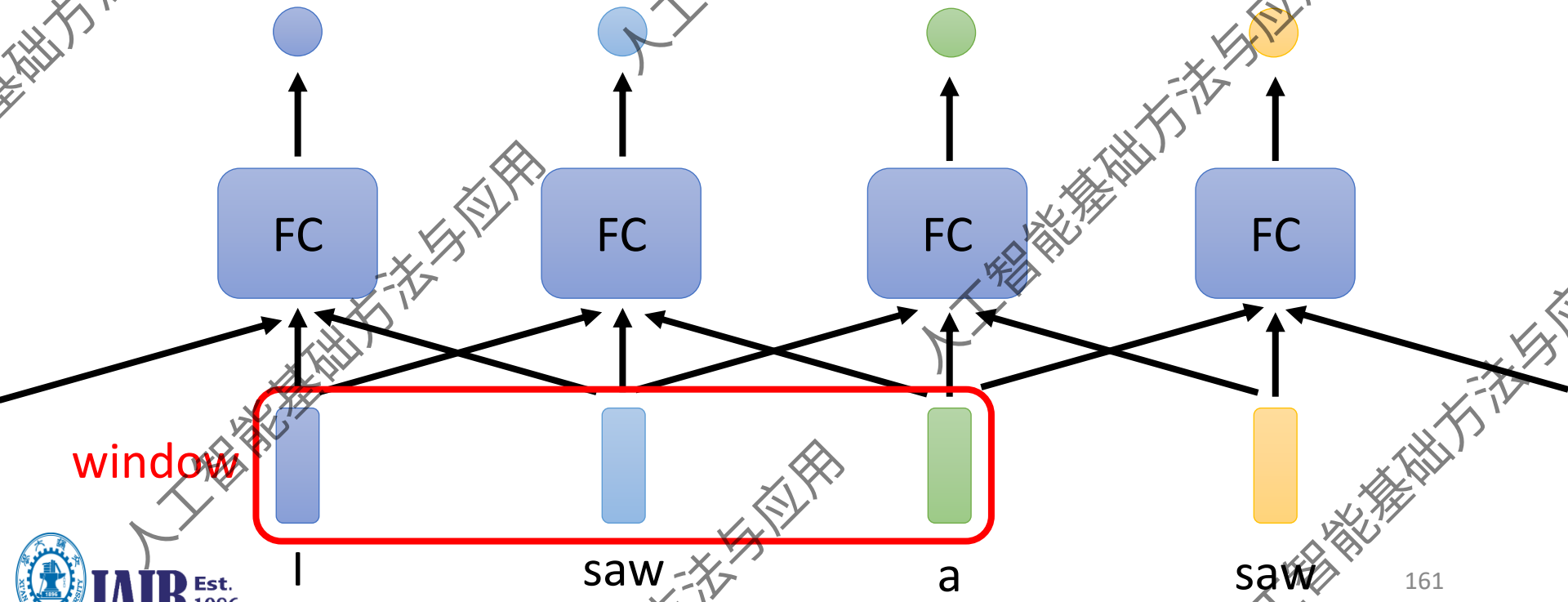
FC 全连接神经网络

能否考虑上下文信息？

全连接层可以考虑邻近位置的信息

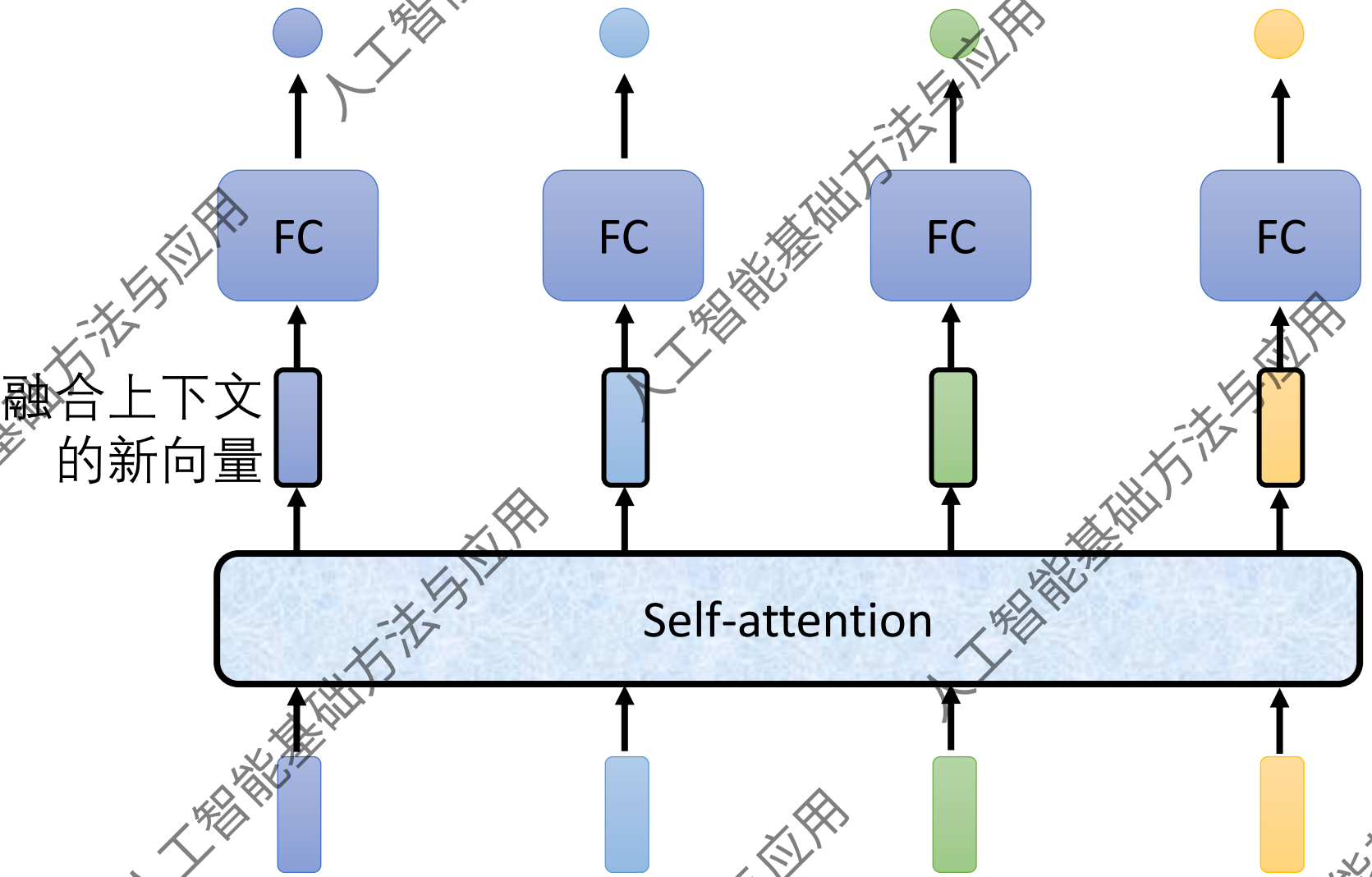
如何考虑整个序列的信息？

是否要让一个窗口覆盖整个序列？

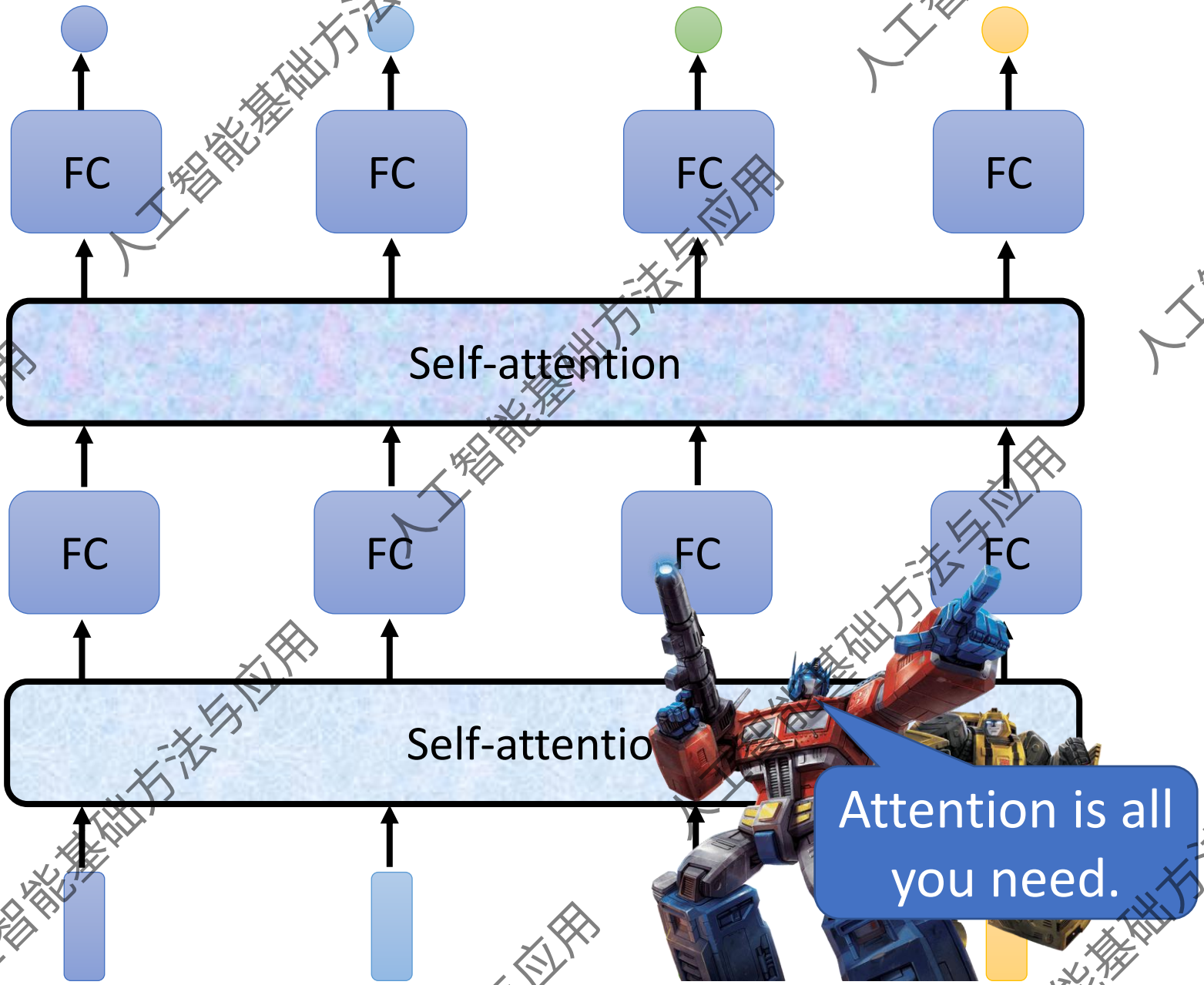


人工智能基础方法与应用

自注意力：为每个位置注入上下文

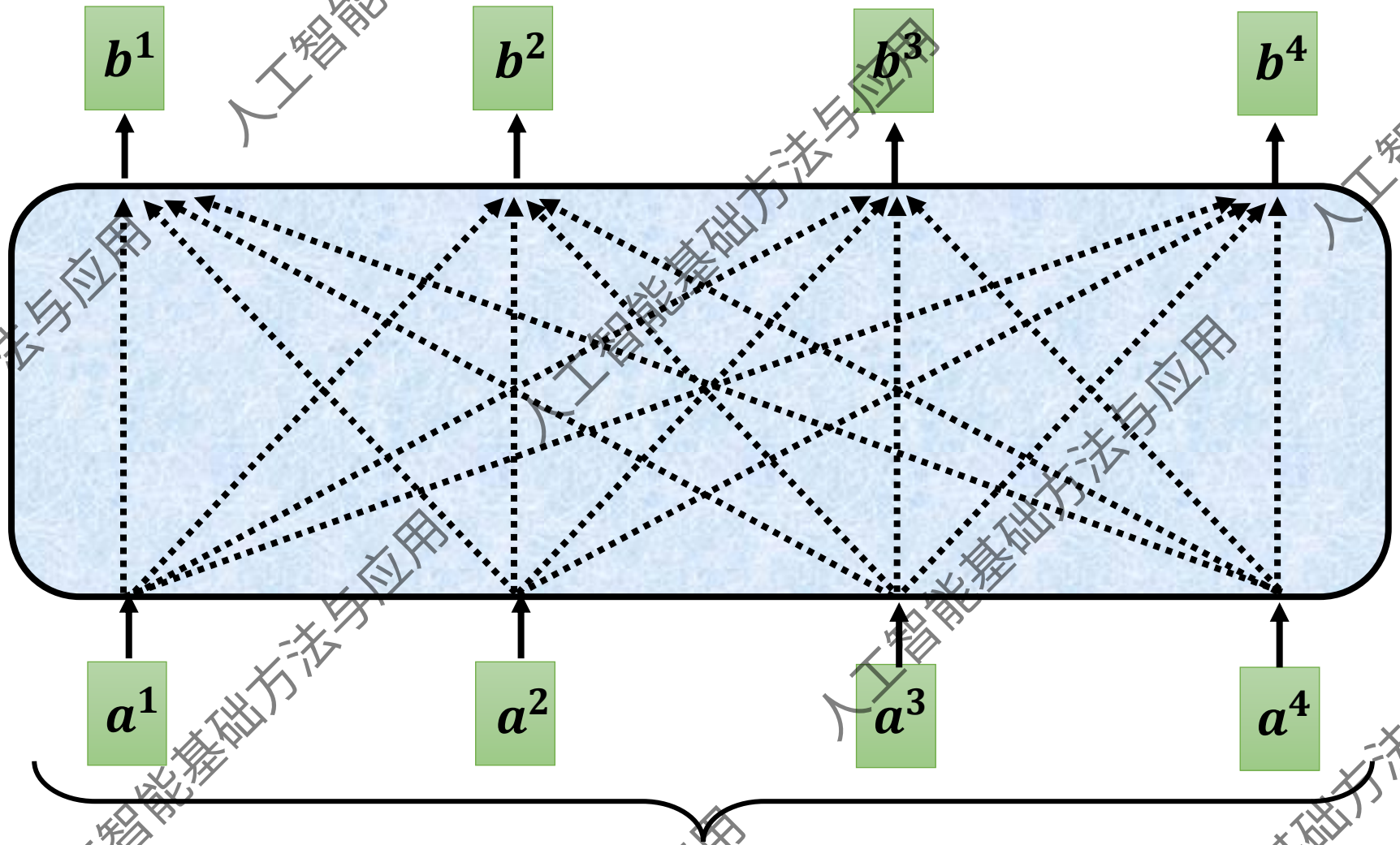


人工智能基础方法与应用



人工智能基础方法与应用

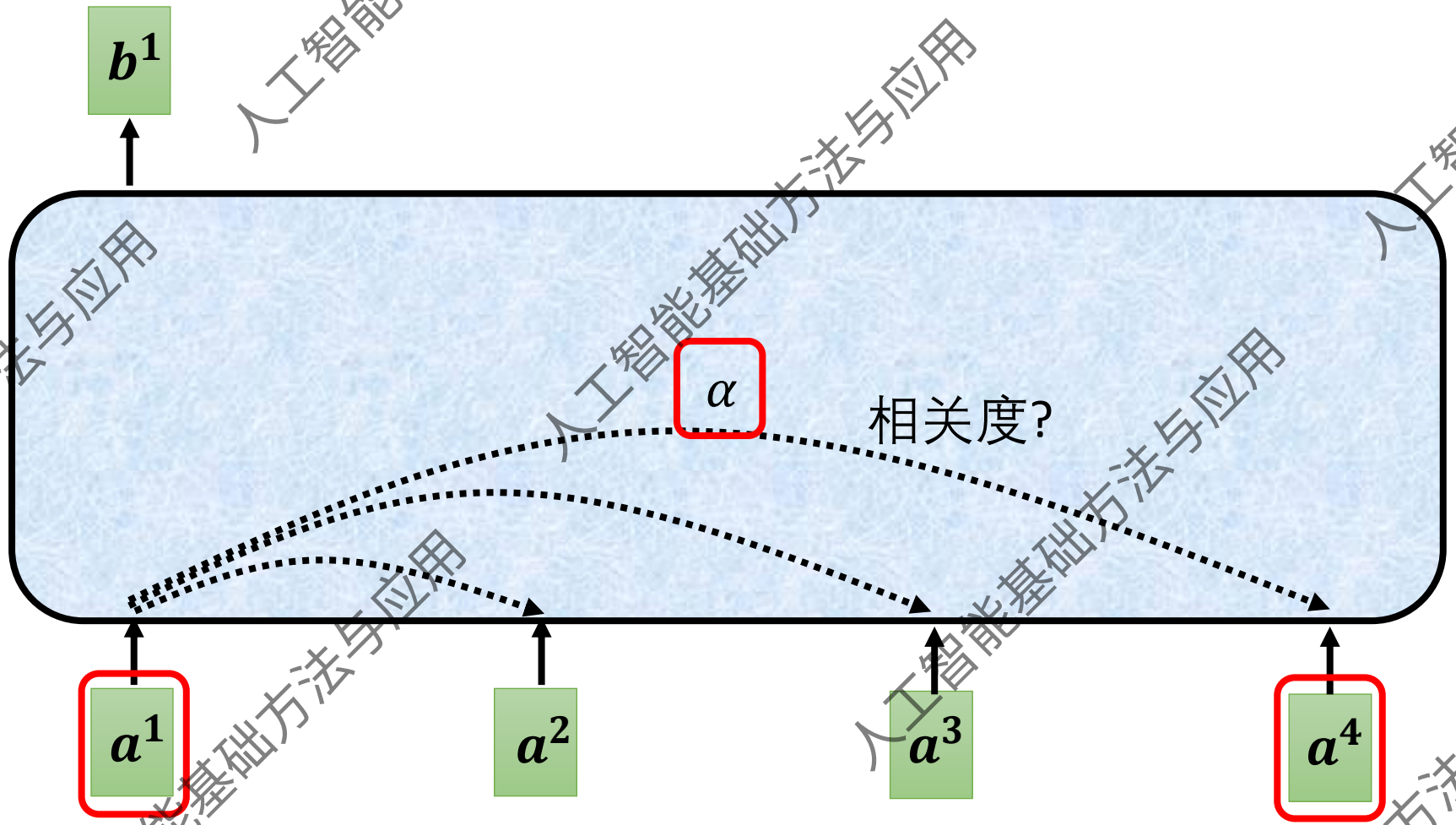
自注意力的输入与输出



既可以是输入层，也可以是隐藏层

人工智能基础方法与应用

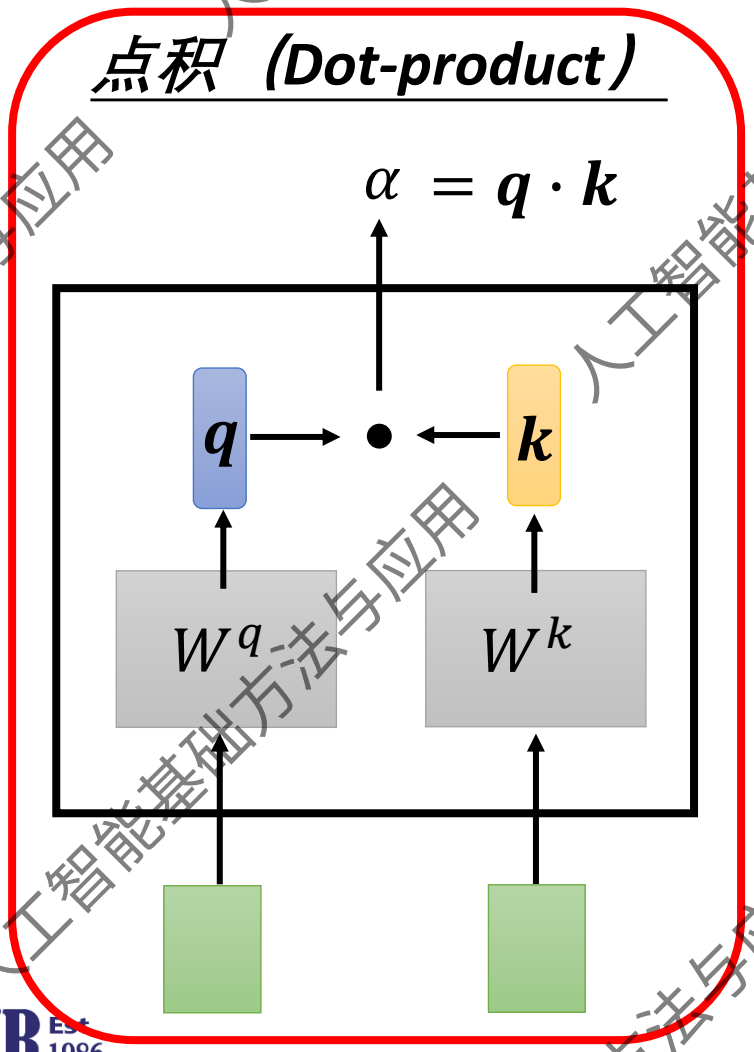
第一步：寻找与当前位置相关的向量

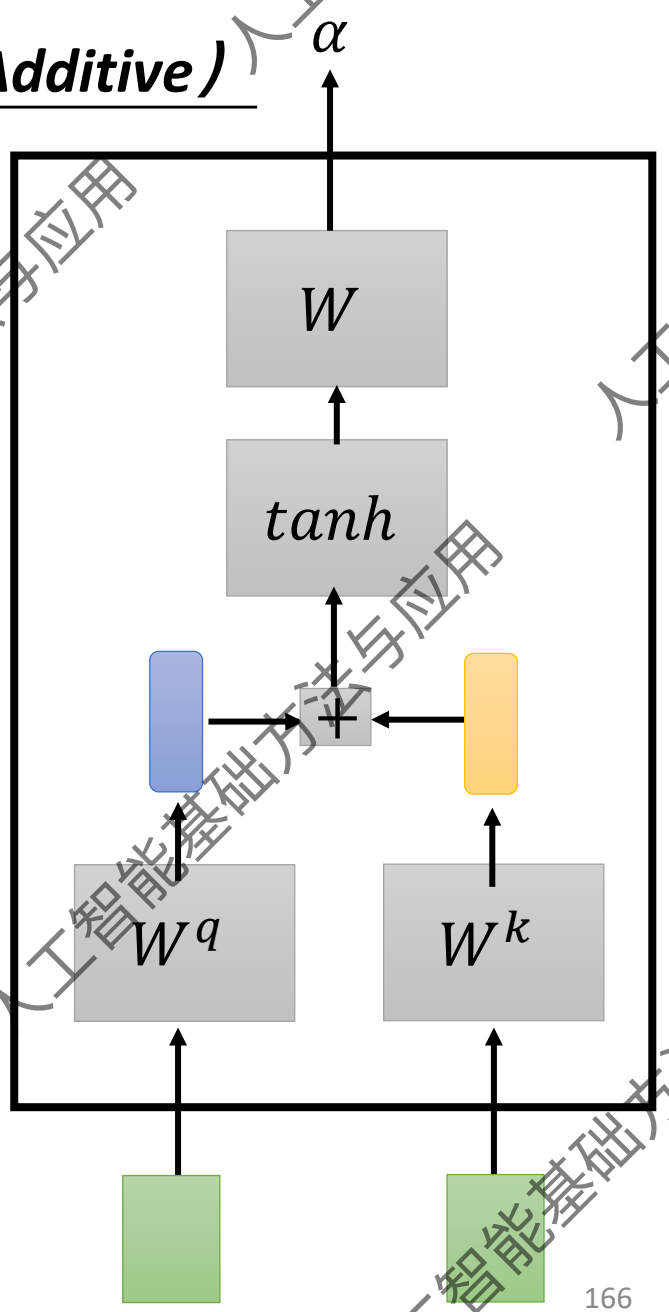


找出序列中哪些位置与当前信息相关

人工智能基础方法与应用

相关性分数



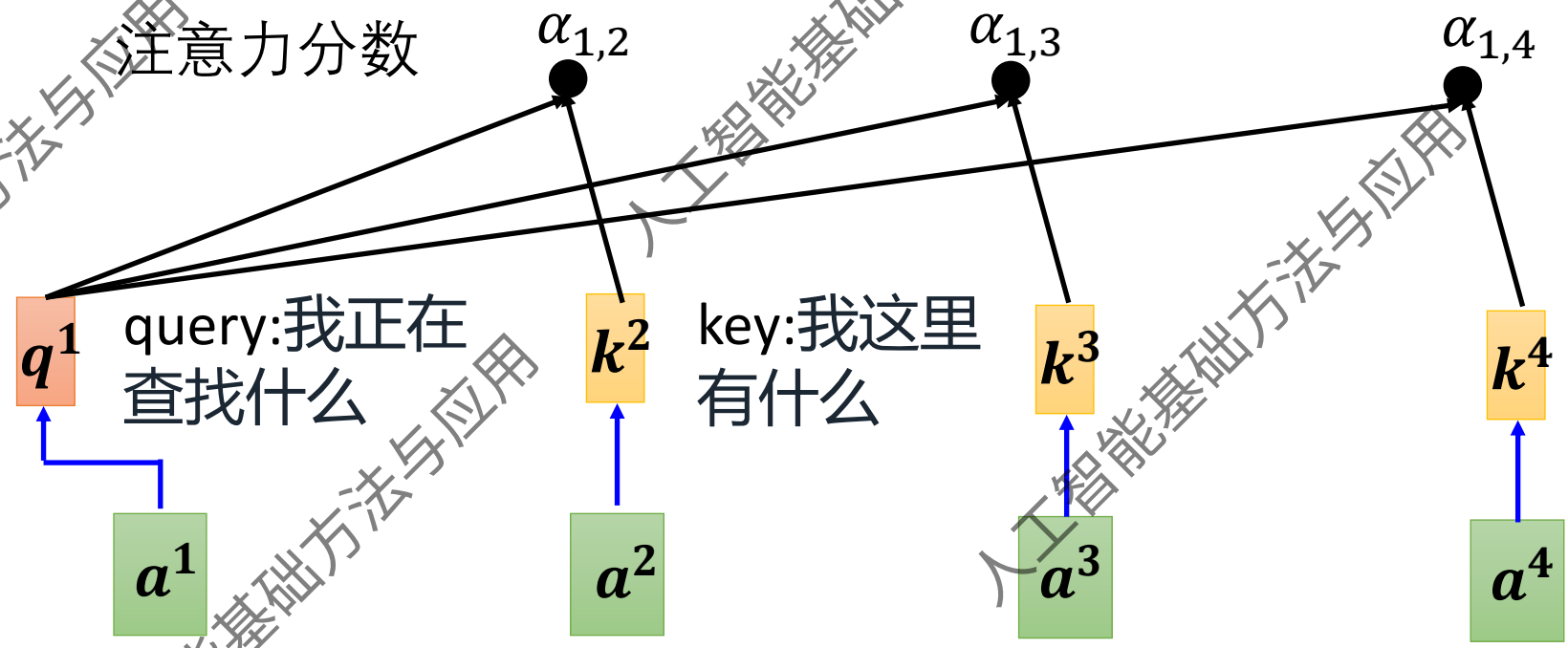


人工智能基础方法与应用

自注意力（Self-attention）

- “Self” 体现在哪里？Query 和 Key 都来自同一个输入序列。内部互相计算相关性，所以叫**自注意力**。

$$\alpha_{1,2} = \boldsymbol{q^1} \cdot \boldsymbol{k^2} \qquad \alpha_{1,3} = \boldsymbol{q^1} \cdot \boldsymbol{k^3} \qquad \alpha_{1,4} = \boldsymbol{q^1} \cdot \boldsymbol{k^4}$$

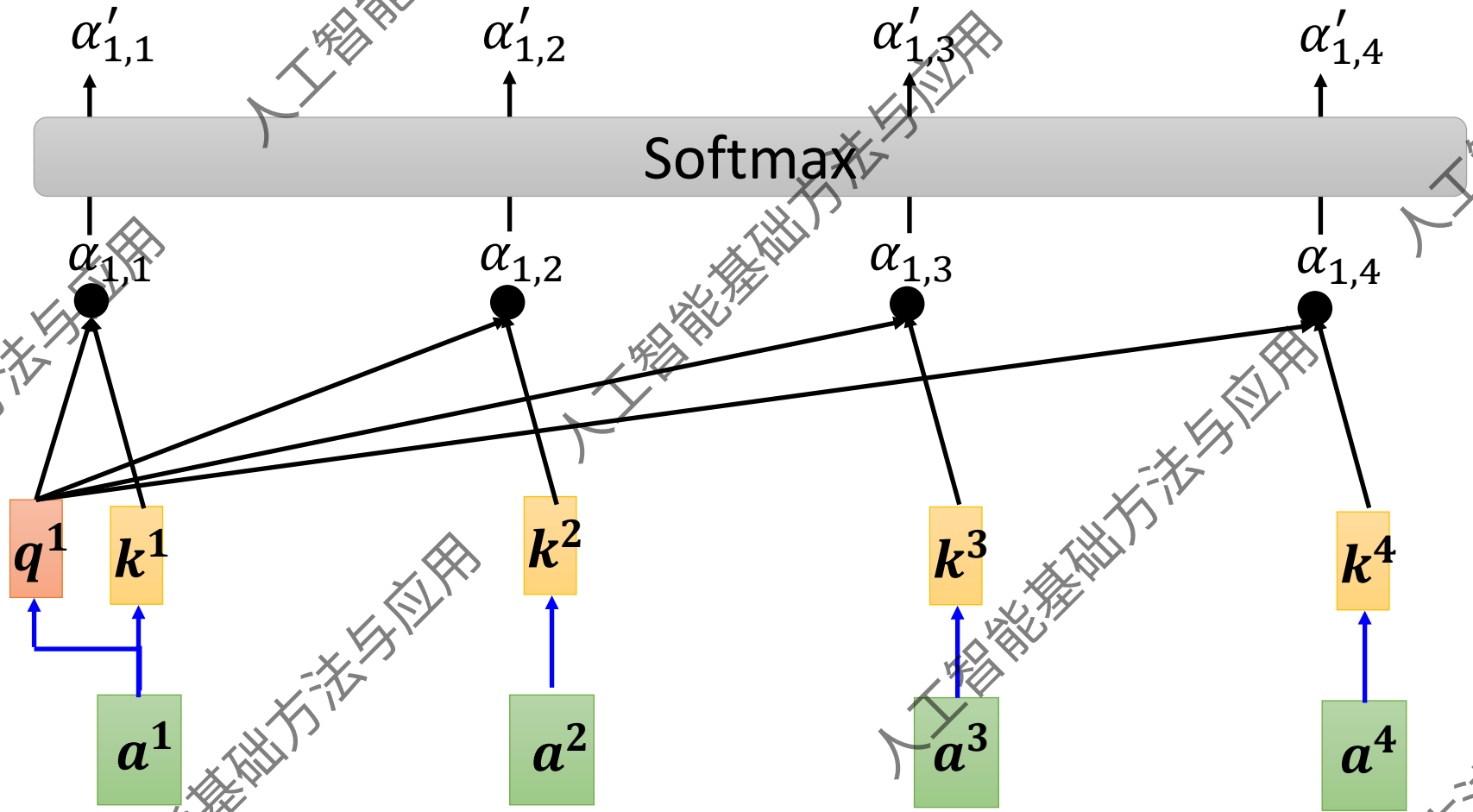


$$\boldsymbol{q^1} = W^q \boldsymbol{a^1} \qquad \boldsymbol{k^2} = W^k \boldsymbol{a^2} \qquad \boldsymbol{k^3} = W^k \boldsymbol{a^3} \qquad \boldsymbol{k^4} = W^k \boldsymbol{a^4}$$

人工智能基础方法与应用

用 softmax 归一化注意力权重

$$\alpha'_{1,i} = exp(\alpha_{1,i}) / \sum_j exp(\alpha_{1,j})$$

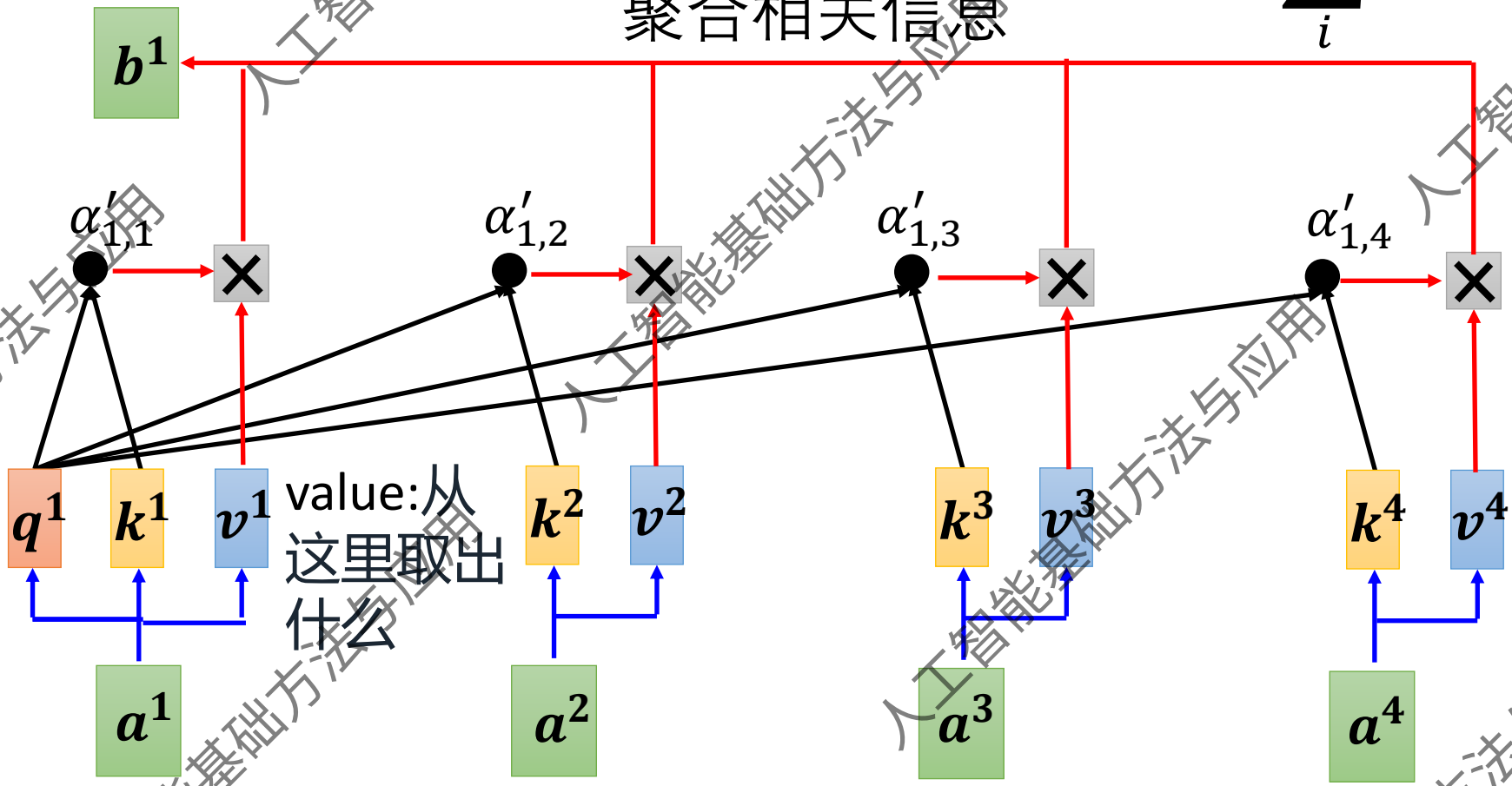


$q^1 = W^q a^1$
$k^1 = W^k a^1$
$k^2 = W^k a^2$
$k^3 = W^k a^3$
$k^4 = W^k a^4$

人工智能基础方法与应用

用v加权汇总信息

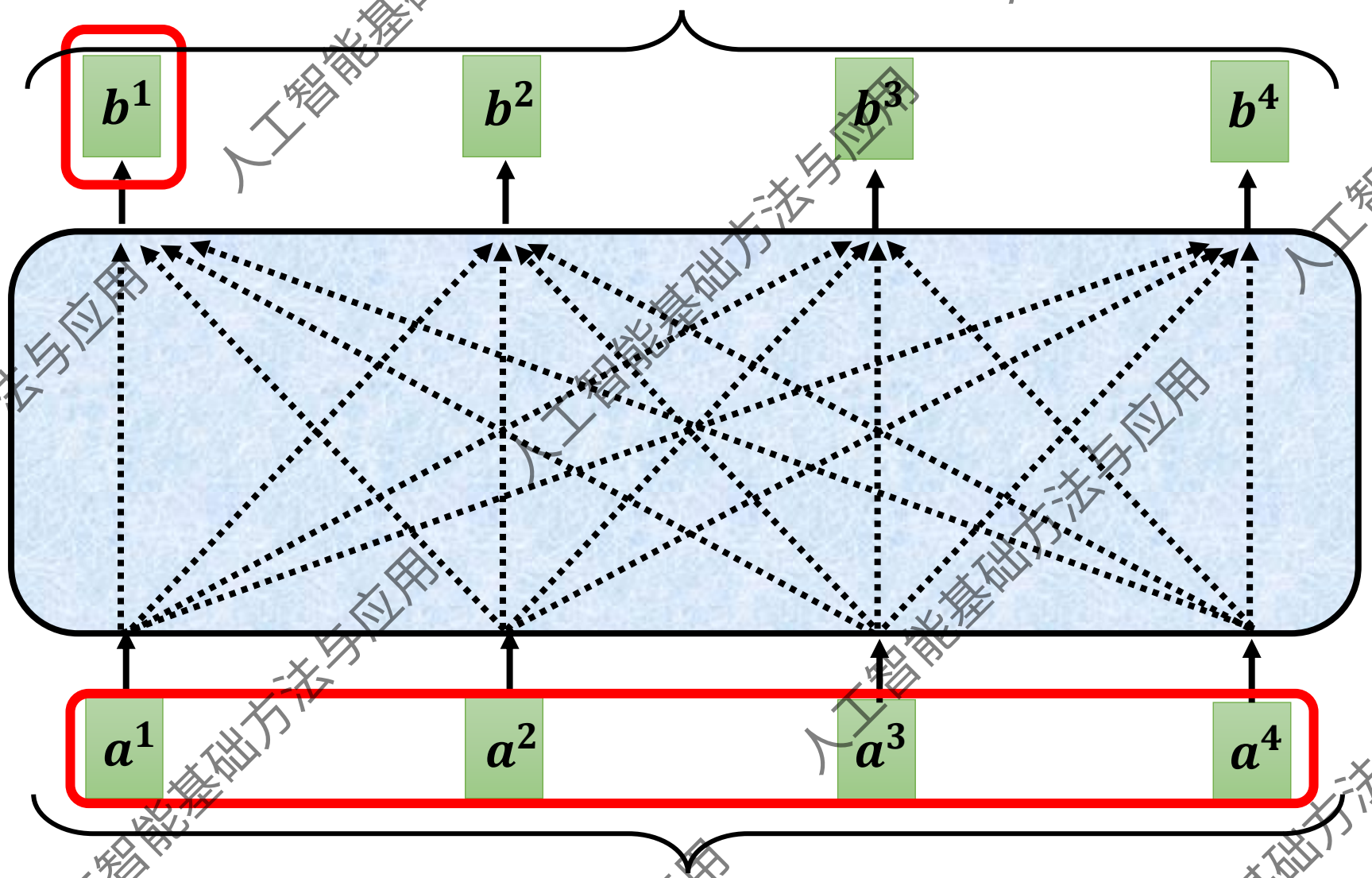
根据注意力权重聚合相关信息 $\boldsymbol{b^1} = \sum_i \alpha'_{1,i} \boldsymbol{v^i}$



$\boldsymbol{v^1} = W^v \boldsymbol{a^1}$ $\boldsymbol{v^2} = W^v \boldsymbol{a^2}$ $\boldsymbol{v^3} = W^v \boldsymbol{a^3}$ $\boldsymbol{v^4} = W^v \boldsymbol{a^4}$

人工智能基础方法与应用

Self-attention

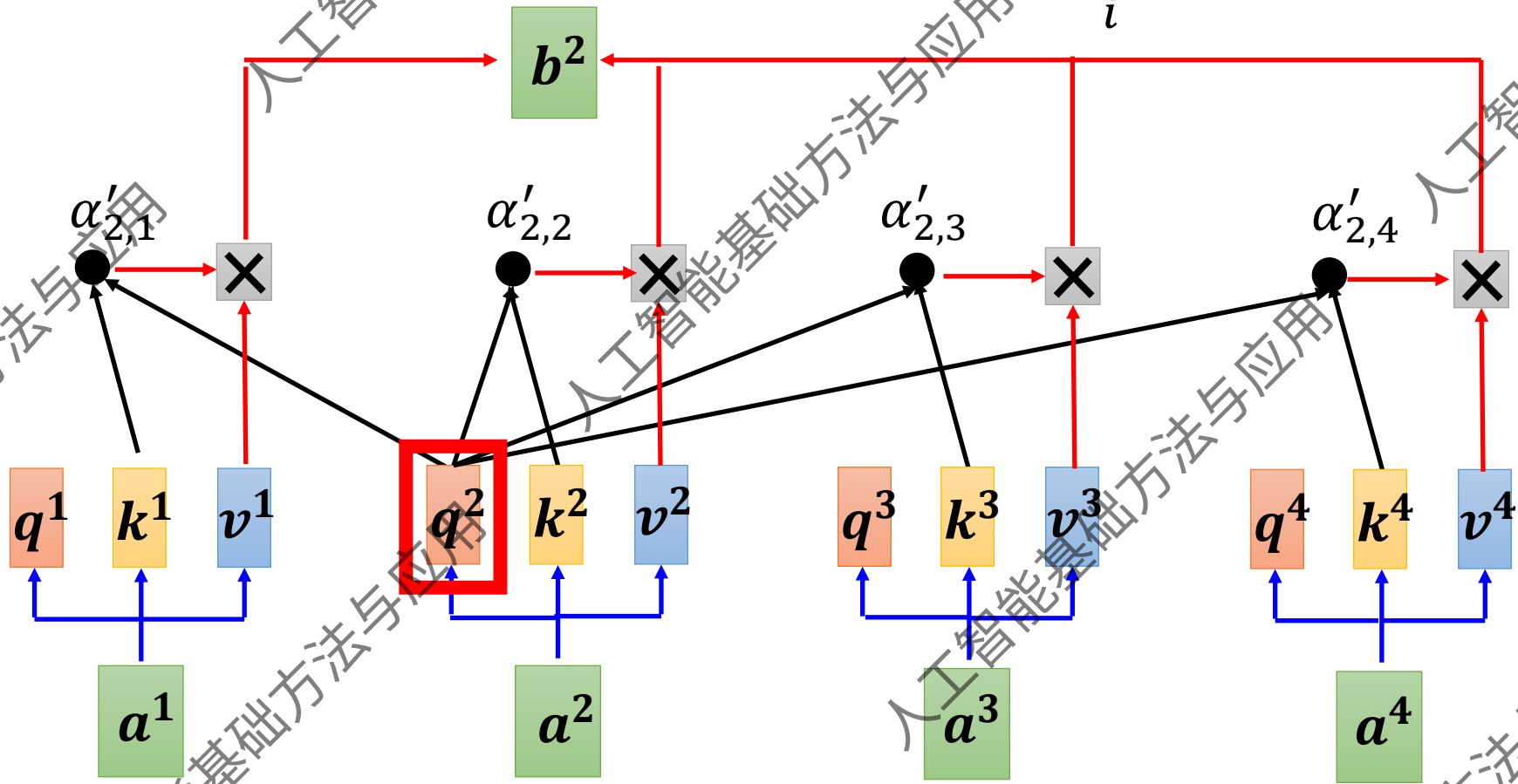


既可以是输入层，也可以是隐藏层

人工智能基础方法与应用

Self-attention

$$b^2 = \sum_i \alpha'_{2,i} v^i$$



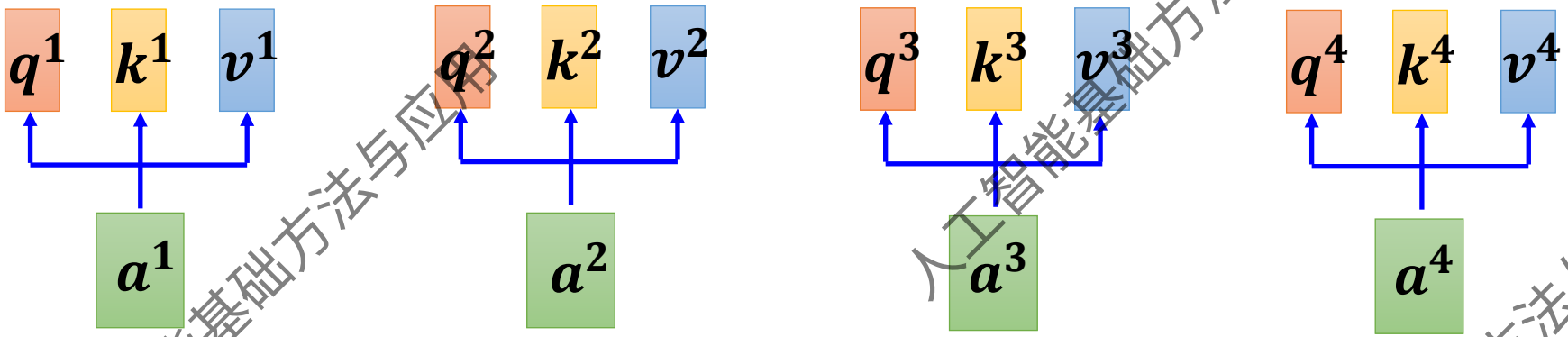
人工智能基础方法与应用

自注意力：Q、K、V

$$q^i = W^q a^i \qquad \underbrace{q^1\ q^2\ q^3\ q^4}_{Q} = W^q \underbrace{a^1\ a^2\ a^3\ a^4}_{I}$$

$$k^i = W^k a^i \qquad \underbrace{k^1\ k^2\ k^3\ k^4}_{K} = W^k \underbrace{a^1\ a^2\ a^3\ a^4}_{I}$$

$$v^i = W^v a^i \qquad \underbrace{v^1\ v^2\ v^3\ v^4}_{V} = W^v \underbrace{a^1\ a^2\ a^3\ a^4}_{I}$$



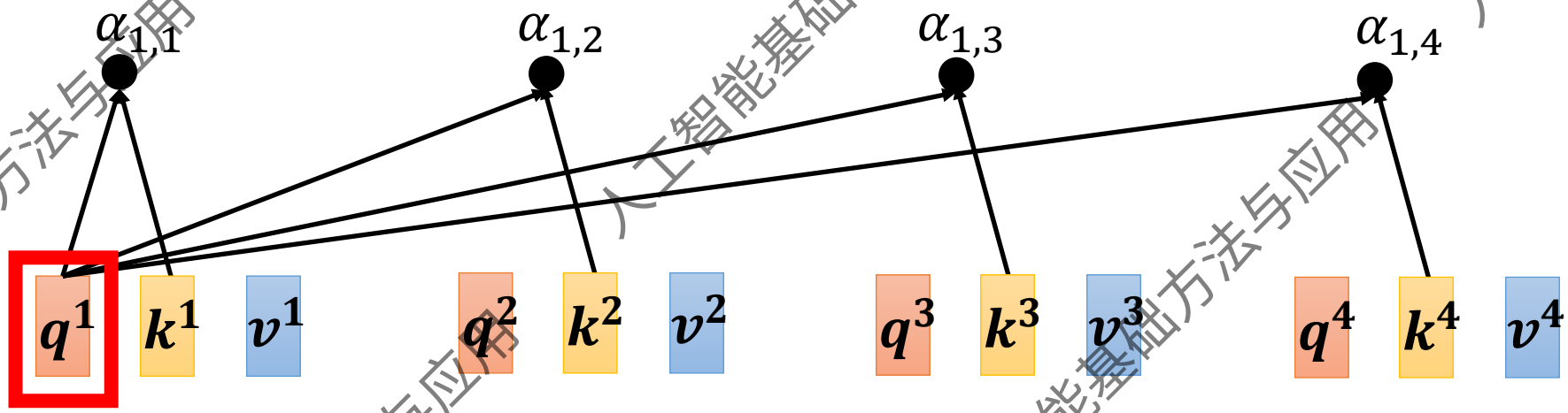
人工智能基础方法与应用

自注意力：计算查询的相关分数

$$\alpha_{1,1} = k^1 q^1 \quad \alpha_{1,2} = k^2 q^1$$

$$\alpha_{1,3} = k^3 q^1 \quad \alpha_{1,4} = k^4 q^1$$

$$\begin{matrix}\alpha_{1,1}\\ \alpha_{1,2}\\ \alpha_{1,3}\\ \alpha_{1,4}\end{matrix} = \begin{matrix}k^1\\ k^2\\ k^3\\ k^4\end{matrix} q^1$$



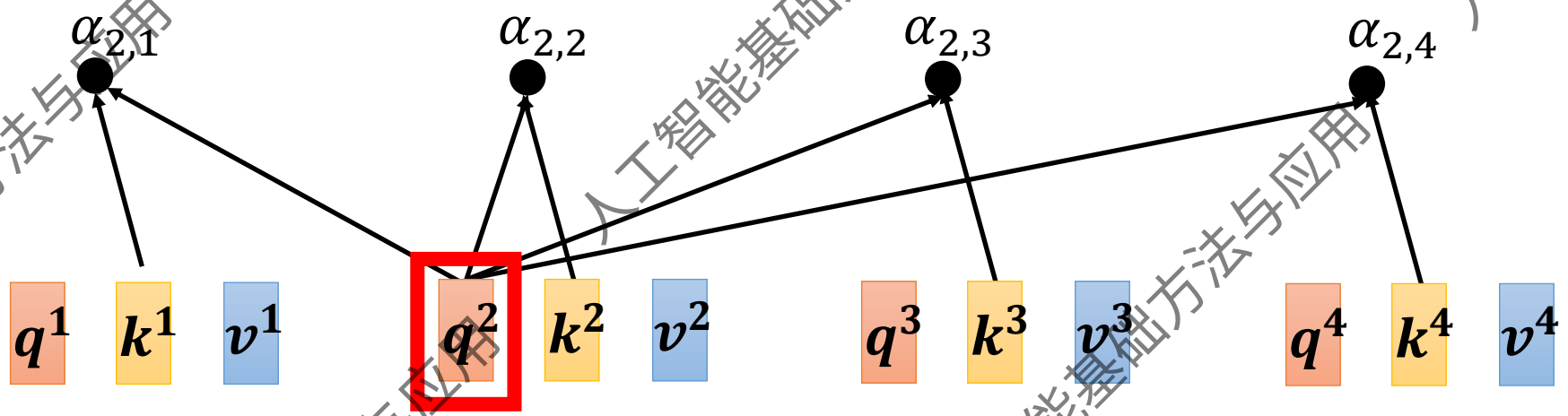
人工智能基础方法与应用

自注意力：形成注意力矩阵

$$\alpha_{1,1} = k^1 q^1 \quad \alpha_{1,2} = k^2 q^1$$

$$\alpha_{1,3} = k^3 q^1 \quad \alpha_{1,4} = k^4 q^1$$

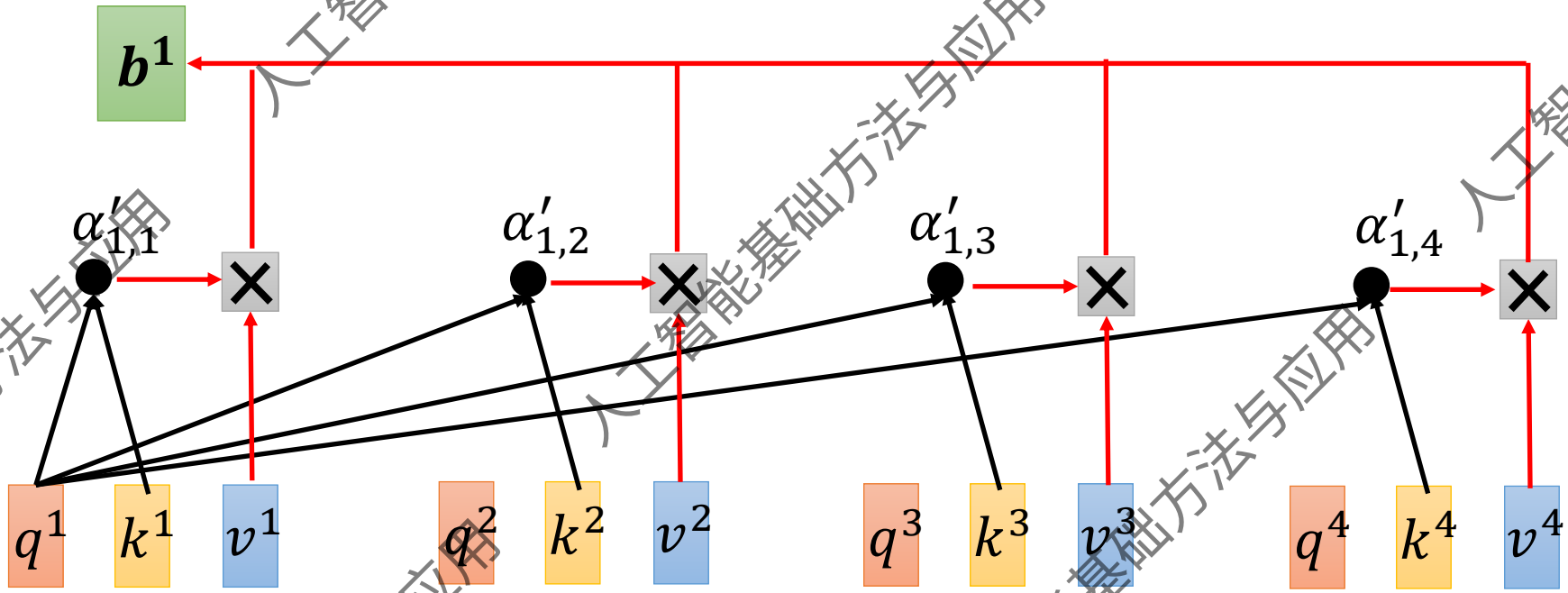
$$\begin{bmatrix} \alpha_{1,1} \\ \alpha_{1,2} \\ \alpha_{1,3} \\ \alpha_{1,4} \end{bmatrix} = \begin{bmatrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{bmatrix} q^1$$



$$\begin{bmatrix} \alpha'_{1,1} & \alpha'_{2,1} & \alpha'_{3,1} & \alpha'_{4,1} \\ \alpha'_{1,2} & \alpha'_{2,2} & \alpha'_{3,2} & \alpha'_{4,2} \\ \alpha'_{1,3} & \alpha'_{2,3} & \alpha'_{3,3} & \alpha'_{4,3} \\ \alpha'_{1,4} & \alpha'_{2,4} & \alpha'_{3,4} & \alpha'_{4,4} \end{bmatrix} A' \quad \text{softmax} \quad \begin{bmatrix} \alpha_{1,1} & \alpha_{2,1} & \alpha_{3,1} & \alpha_{4,1} \\ \alpha_{1,2} & \alpha_{2,2} & \alpha_{3,2} & \alpha_{4,2} \\ \alpha_{1,3} & \alpha_{2,3} & \alpha_{3,3} & \alpha_{4,3} \\ \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4} \end{bmatrix} A = \begin{bmatrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{bmatrix} K^T \begin{bmatrix} q^1 & q^2 & q^3 & q^4 \end{bmatrix} Q$$

人工智能基础方法与应用

自注意力：由权重得到输出矩阵



$$\begin{matrix} b^1 & b^2 & b^3 & b^4 \end{matrix} = \begin{matrix} v^1 & v^2 & v^3 & v^4 \end{matrix}$$

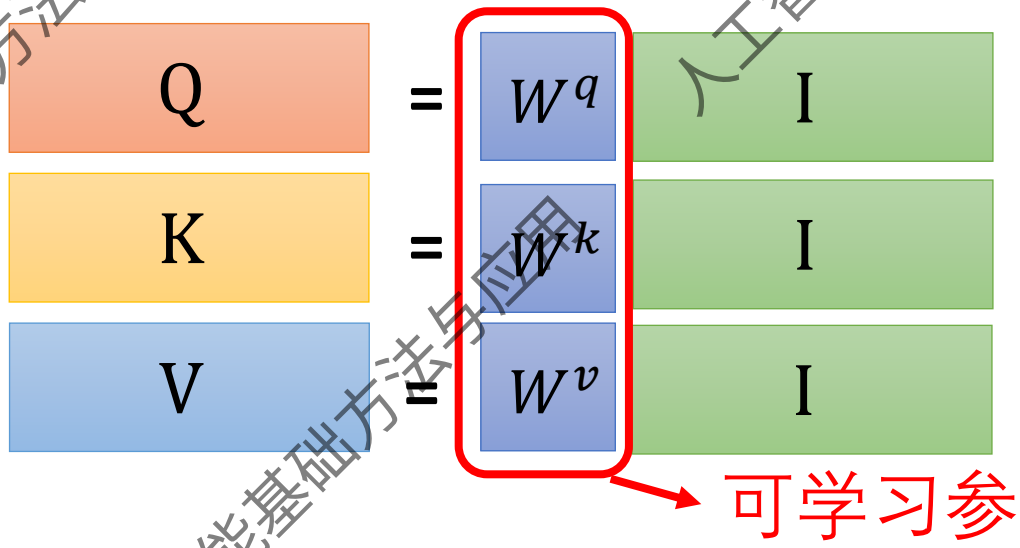
O V

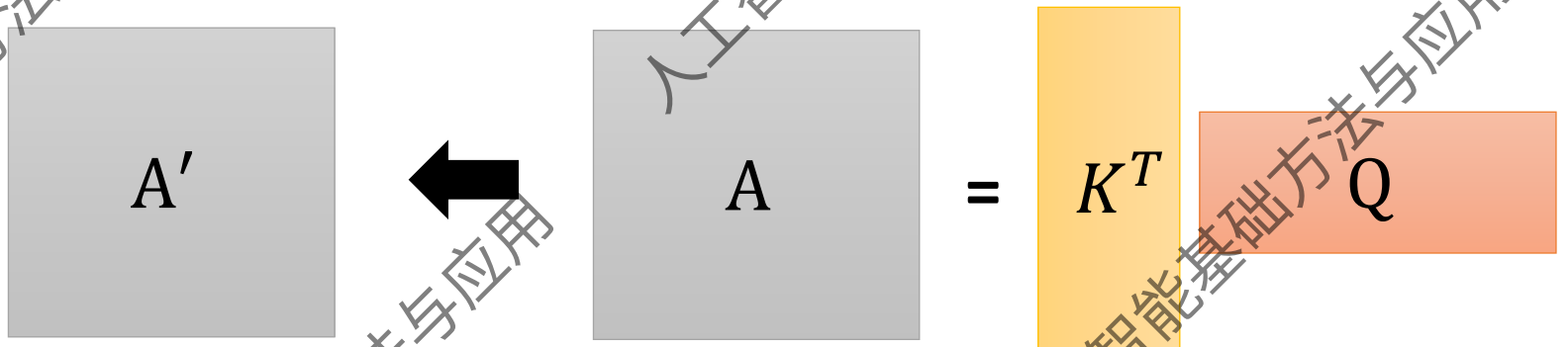
$$\begin{matrix} \alpha'_{1,1} & \alpha'_{2,1} & \alpha'_{3,1} & \alpha'_{4,1} \\ \alpha'_{1,2} & \alpha'_{2,2} & \alpha'_{3,2} & \alpha'_{4,2} \\ \alpha'_{1,3} & \alpha'_{2,3} & \alpha'_{3,3} & \alpha'_{4,3} \\ \alpha'_{1,4} & \alpha'_{2,4} & \alpha'_{3,4} & \alpha'_{4,4} \end{matrix}$$

A'

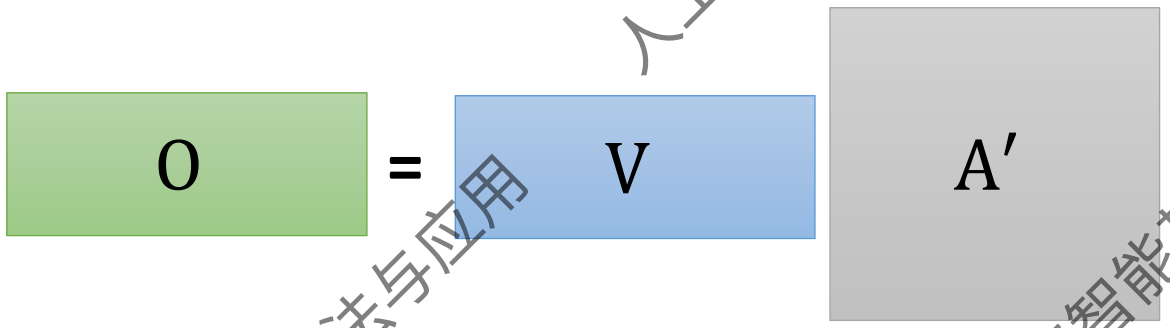
人工智能基础方法与应用

Self-attention





Attention Matrix



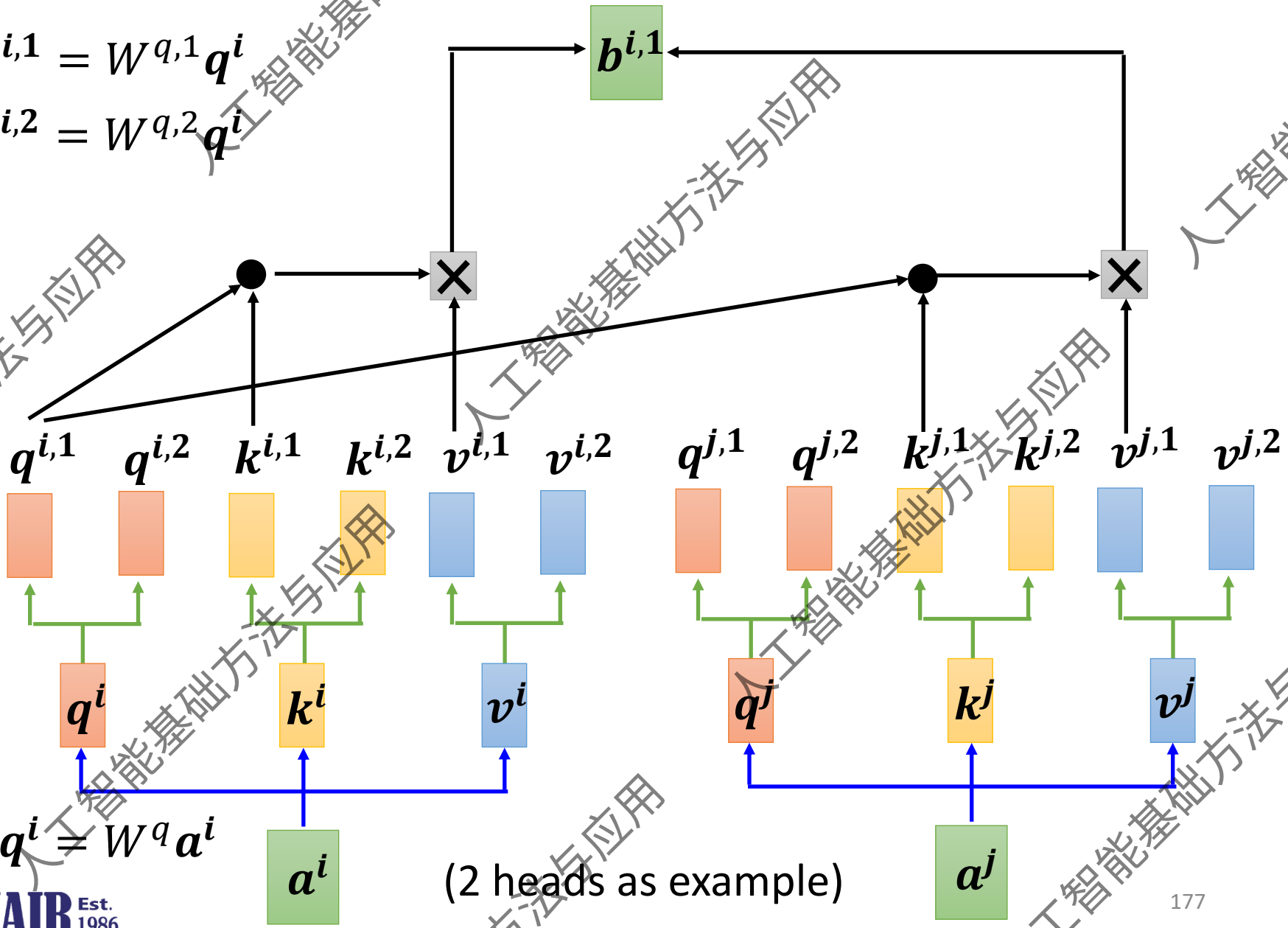
人工智能基础方法与应用

多头自注意力

Different types of relevance

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



(2 heads as example)

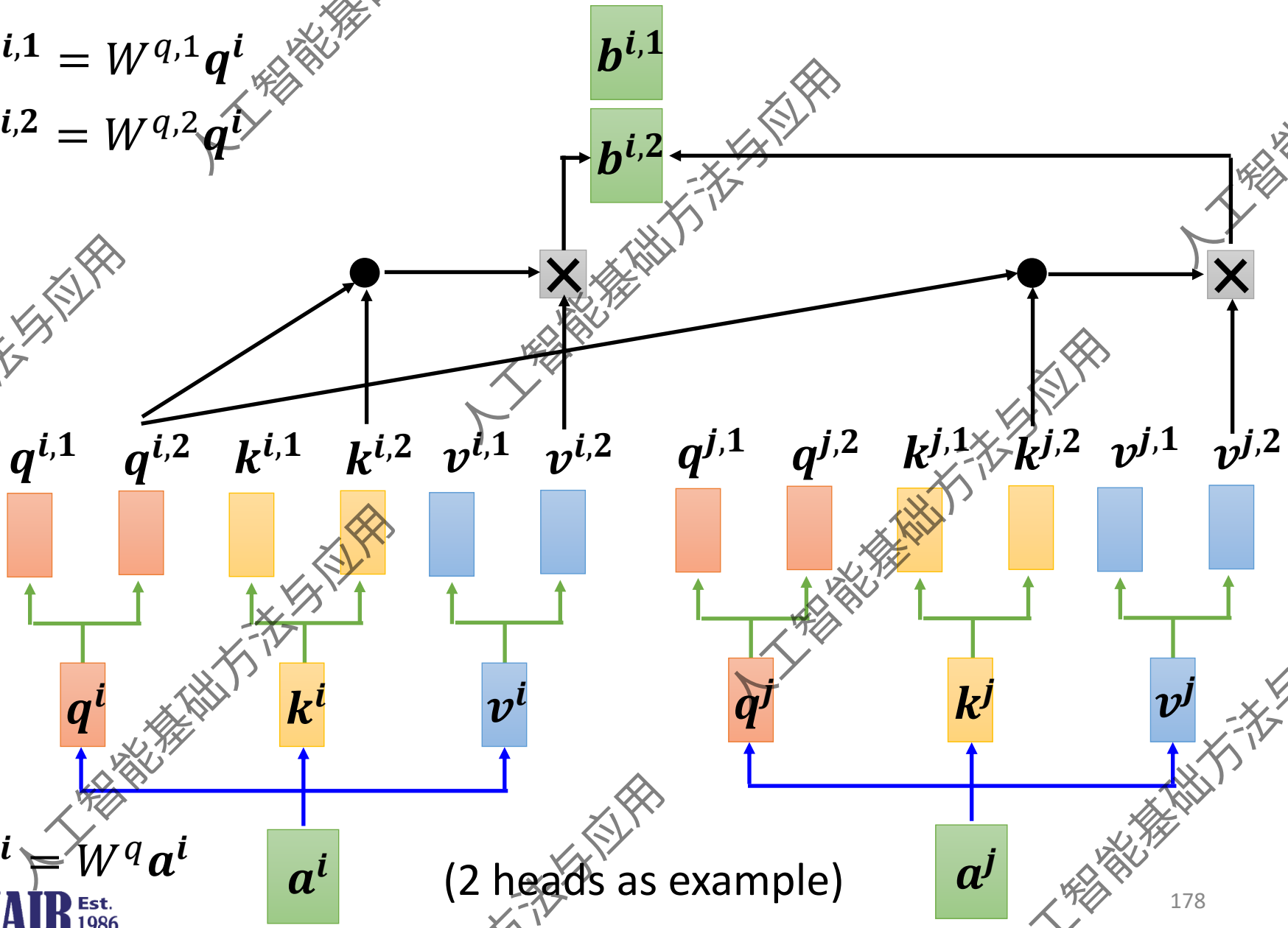
人工智能基础方法与应用

多头自注意力

Different types of relevance

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



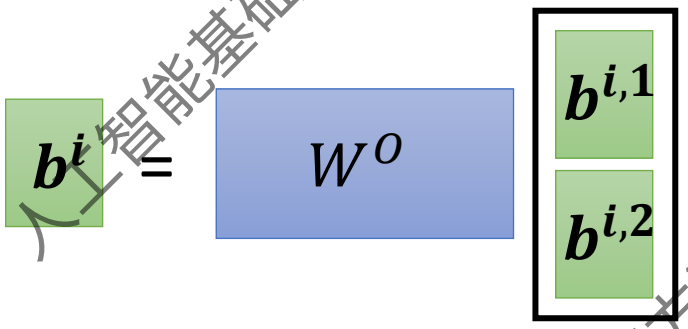
$$q^i = W^q a^i$$

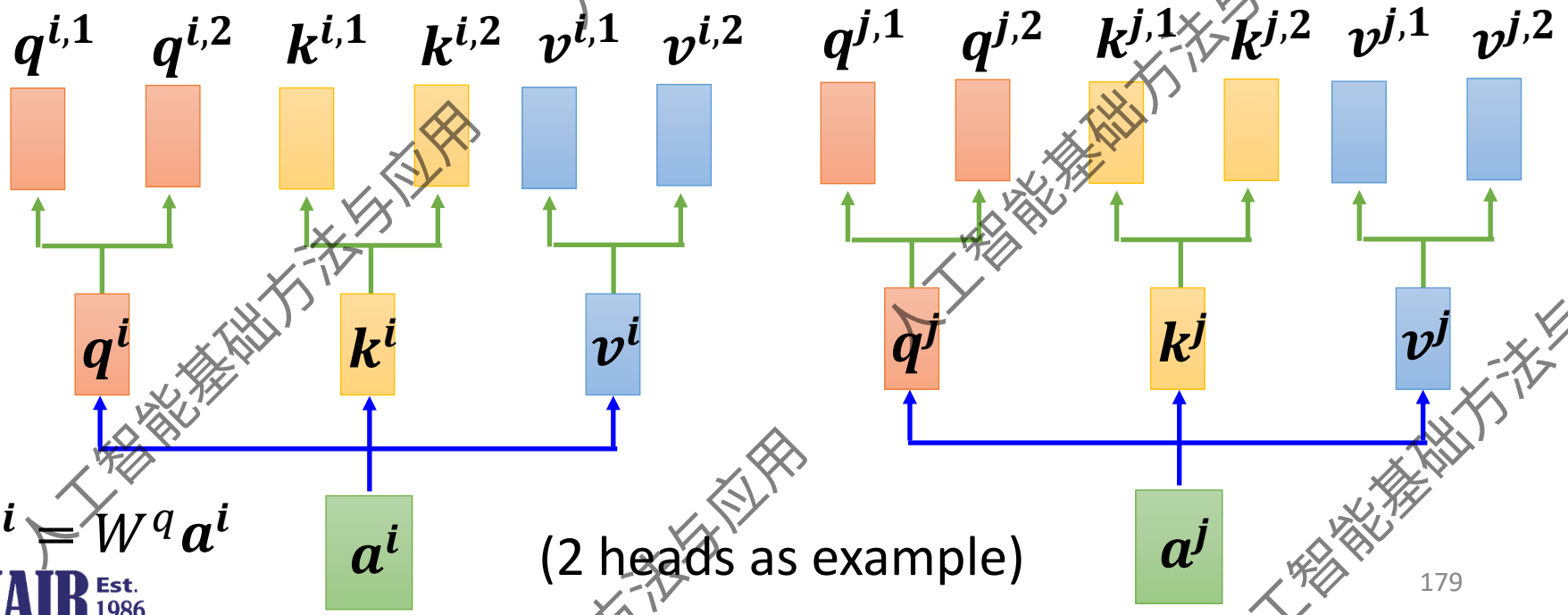
(2 heads as example)

人工智能基础方法与应用

多头自注意力

Different types of relevance





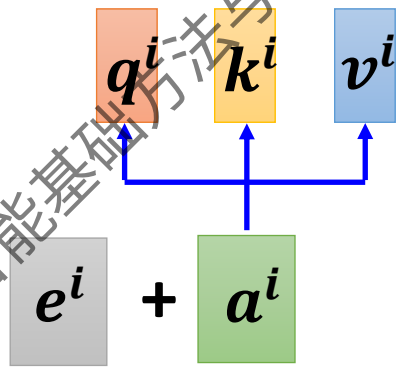
$q^i = W^q a^i$

(2 heads as example)

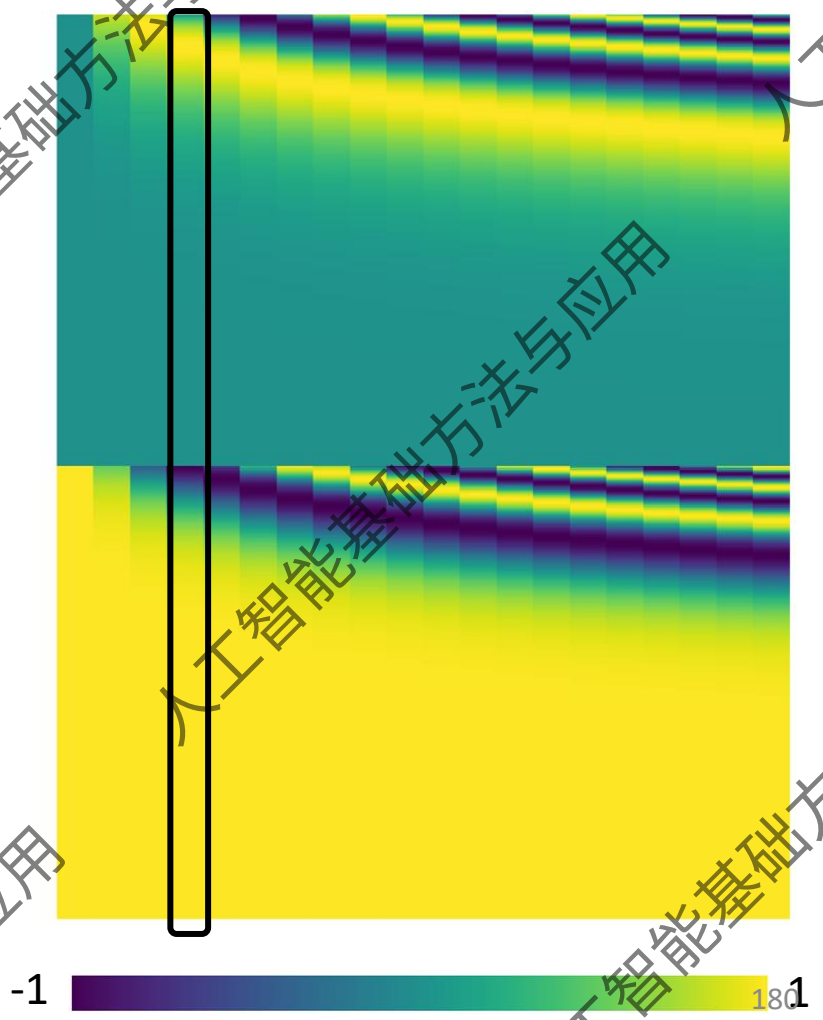
人工智能基础方法与应用

位置编码

- 自注意力本身不包含位置信息
- 序列中的每个位置都有一个独特的位置向量e^i
- **人工设计**
- **从数据中学习**



每一列表示一个位置向量e^i

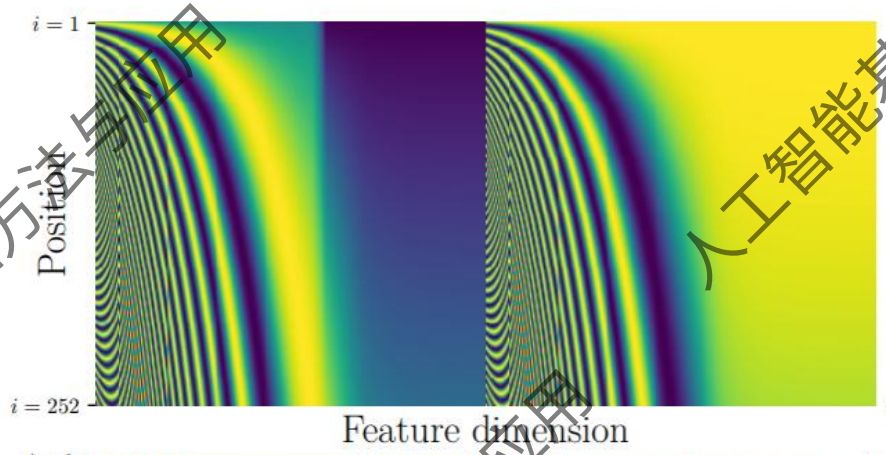


人工智能基础方法与应用

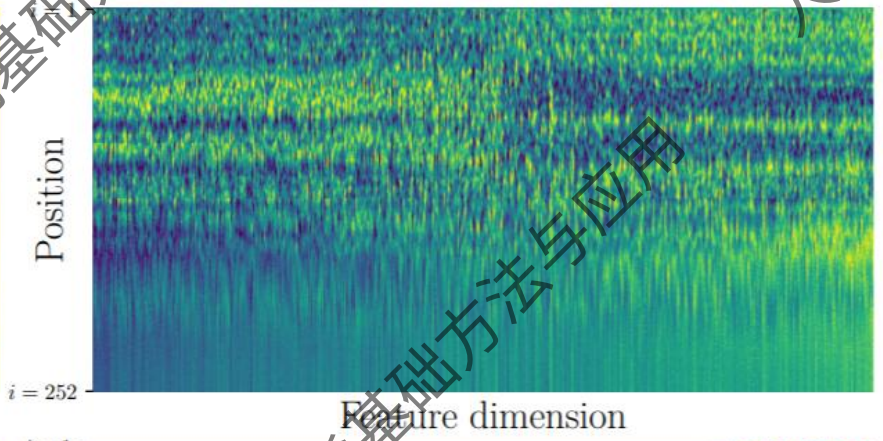
Table 1. Comparing position representation methods

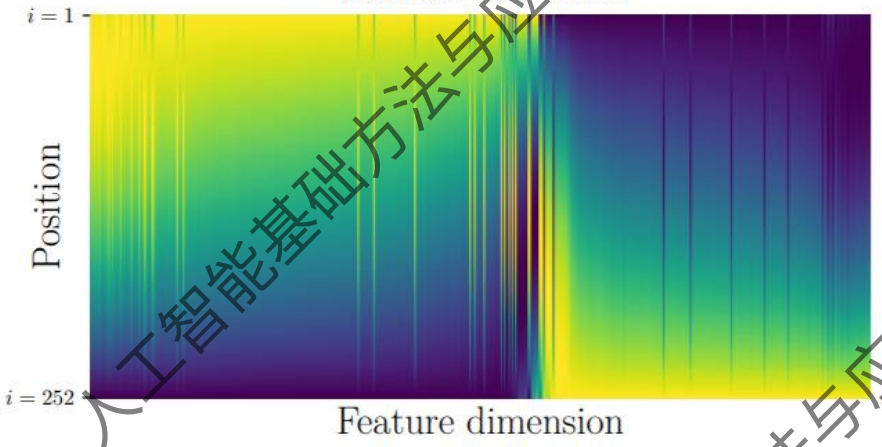
Methods	Inductive	Data-Driven	Parameter Efficient
Sinusoidal (Vaswani et al., 2017)	✓	✗	✓
Embedding (Devlin et al., 2018)	✗	✓	✗
Relative (Shaw et al., 2018)	✗	✓	✓
This paper	✓	✓	✓

(a) Sinusoidal

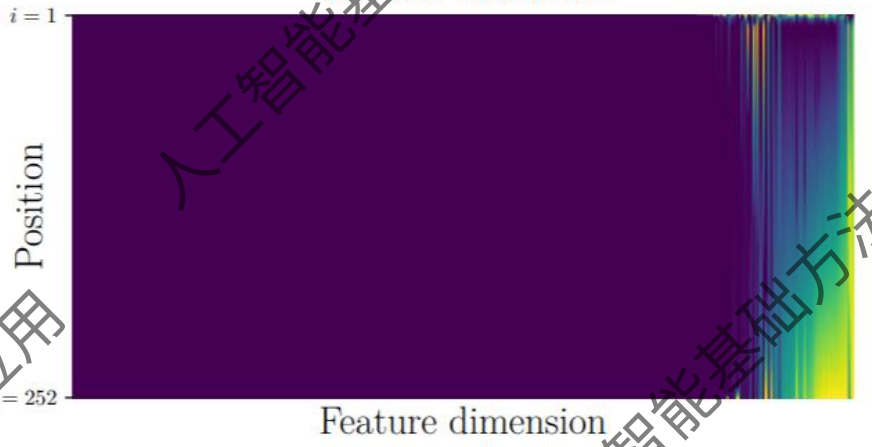


(b) Position embedding





(c) FLOATER



(d) RNN

人工智能基础方法与应用

自注意力的多种应用



Transformer



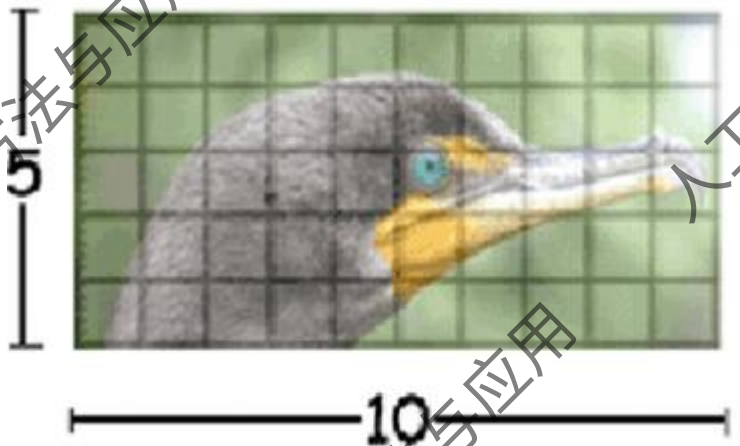
BERT

Widely used in Natural Langue Processing (NLP)!

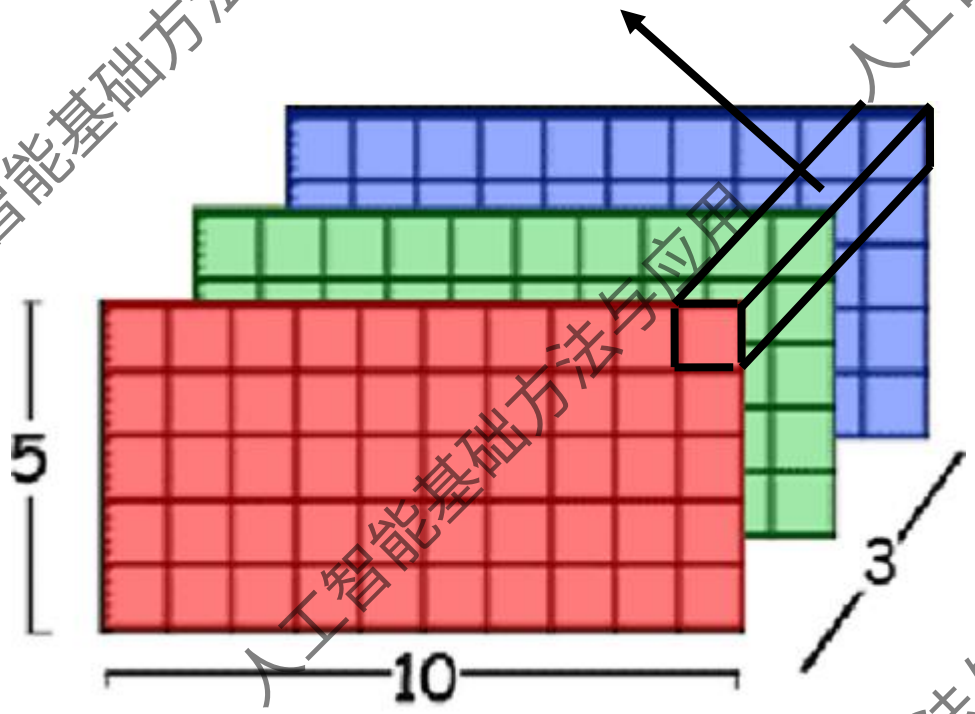
人工智能基础方法与应用

图像中的自注意力

An **image** can also be considered as a **vector set**.

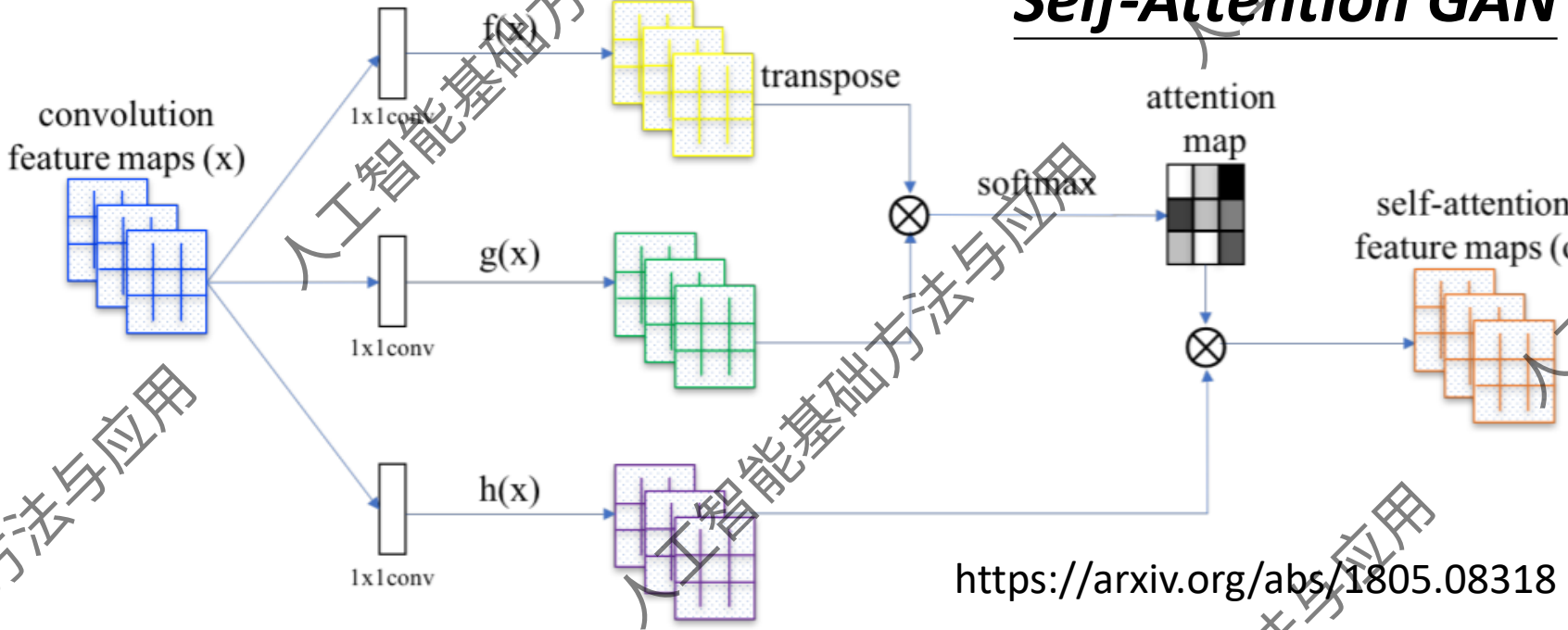


This is a vector.



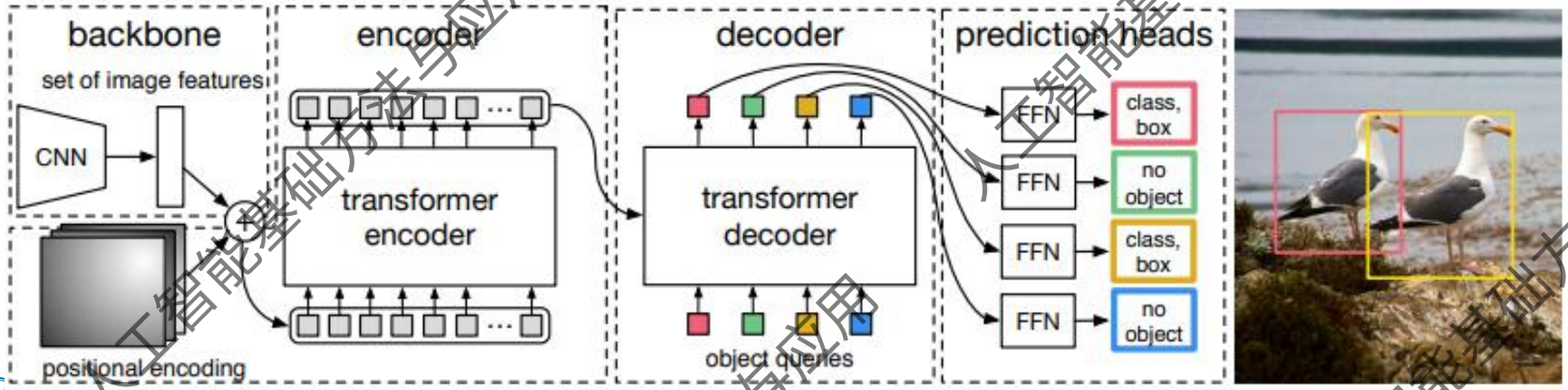
人工智能基础方法与应用

Self-Attention GAN



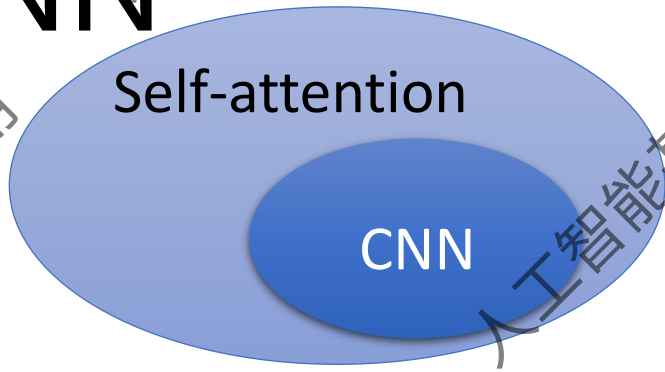
https://arxiv.org/abs/1805.08318

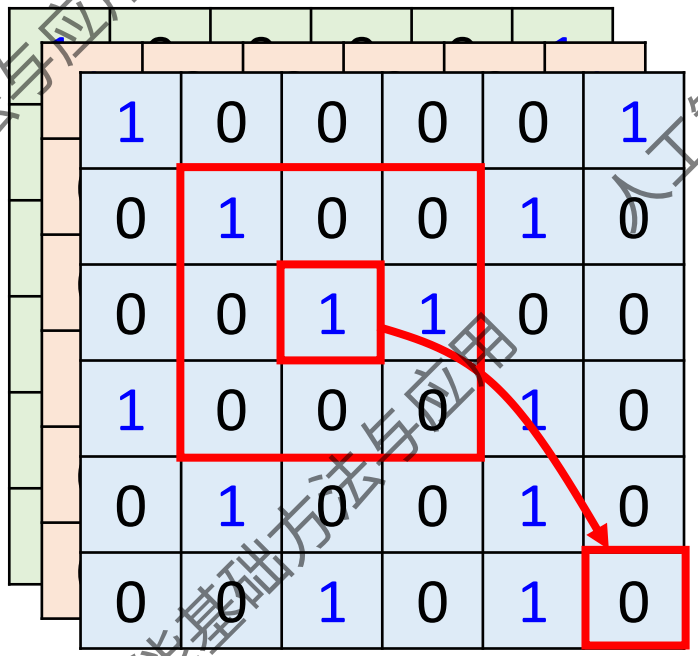
DEtection Transformer (DETR)



https://arxiv.org/abs/2005.12872

Self-attention v.s. CNN





CNN: 只能在局部感受野内进行注意

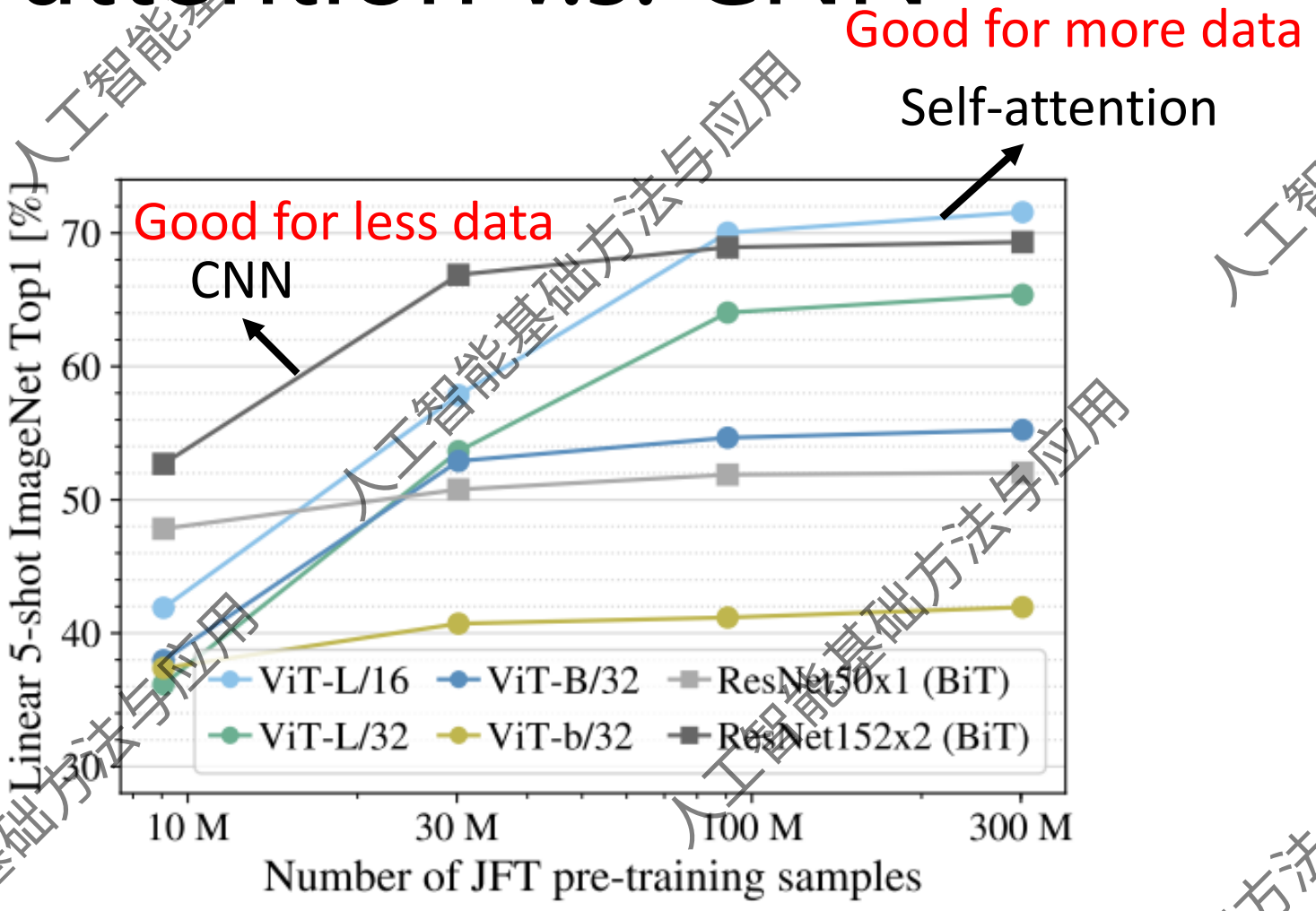
- CNN 可以看作一种受限的自注意力机制

自注意力：具有可学习“感受范围”的 CNN

- 自注意力可以看作更灵活、更复杂的 CNN

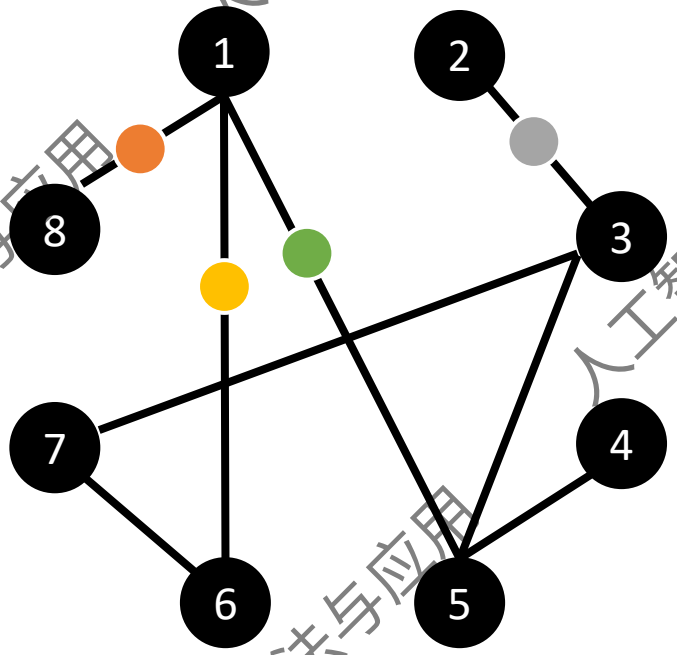
人工智能基础方法与应用

Self-attention v.s. CNN

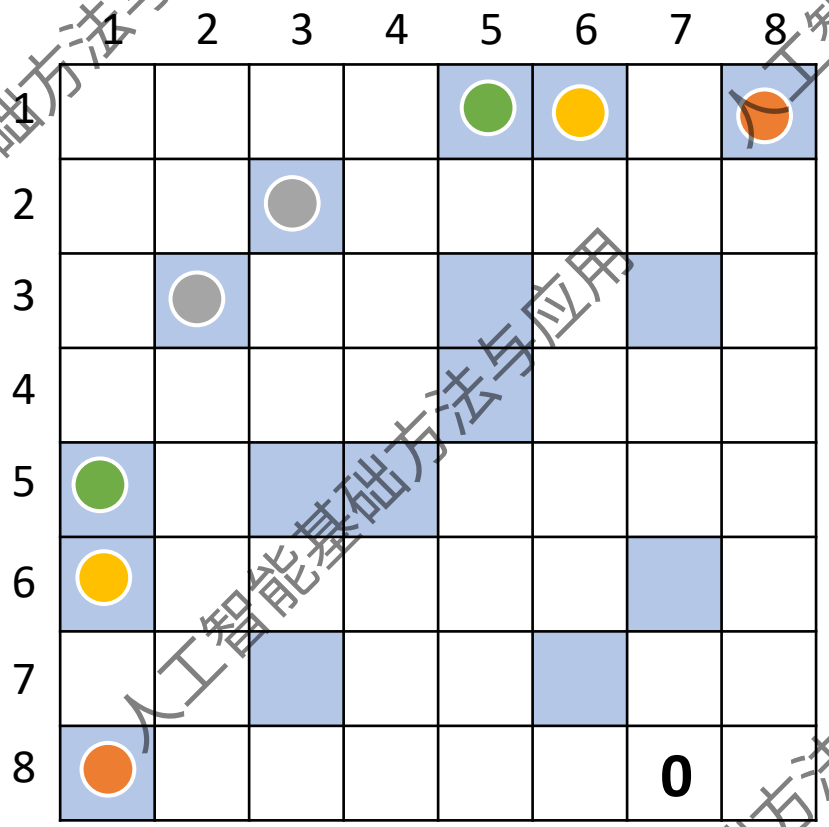


人工智能基础方法与应用

Self-attention for Graph



Attention Matrix



Consider **edge**: only attention to connected nodes

This is one type of **Graph Neural Network (GNN)**.

人工智能基础方法与应用

六、Transformer网络

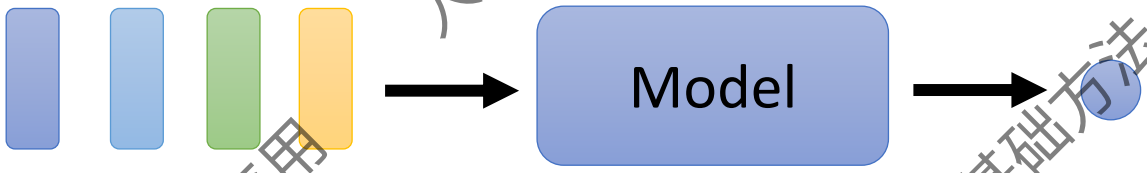
人工智能基础方法与应用

输出形式三：序列到序列

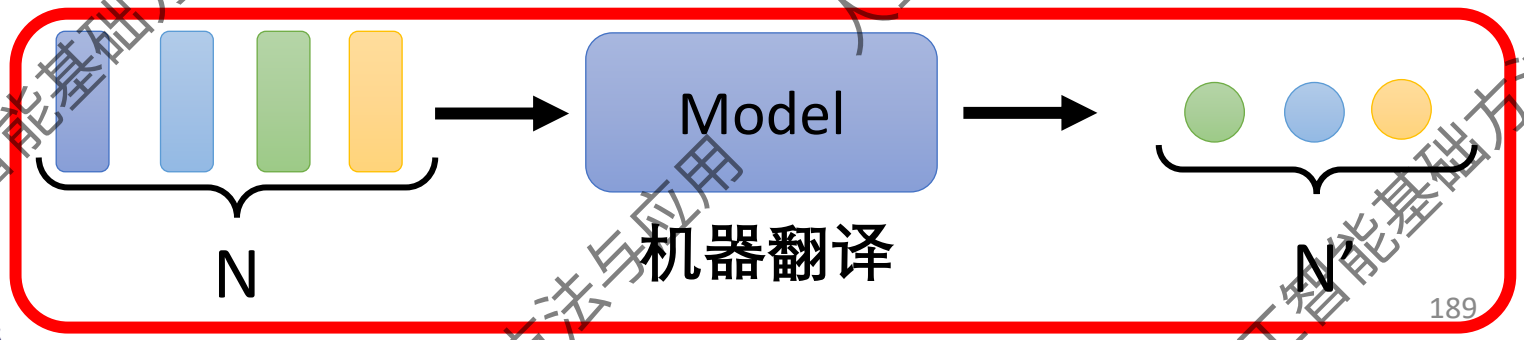
- 逐向量预测标签



- 整个序列对应一个标签



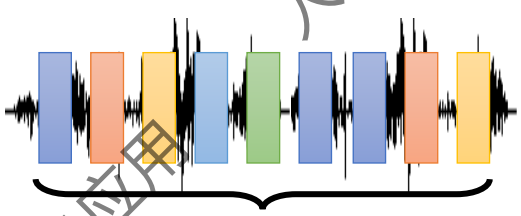
- 输出序列长度由模型决定 序列到序列（seq2seq）



什么是Sequence-to-sequence (Seq2seq)

输入一个序列，输出另一个序列

输出序列的长度由模型决定



T

语音识别

你 好 吗

N

机 器 学 习

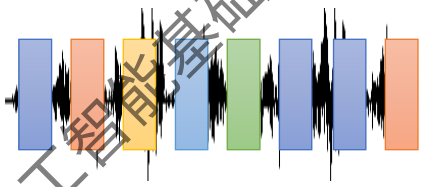
N

机器翻译

machine learning

N’

machine learning



语音翻译

机 器 学 习

无需经过文本的语言转换

人工智能基础方法与应用

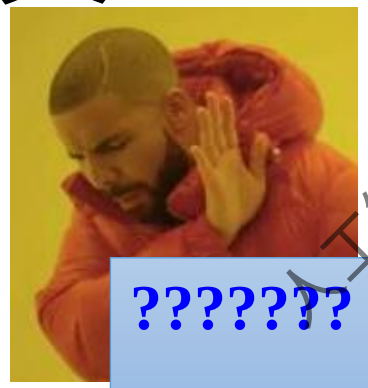
闽南语能不能直接翻成中文



(Speech)

语音识别

“母汤”

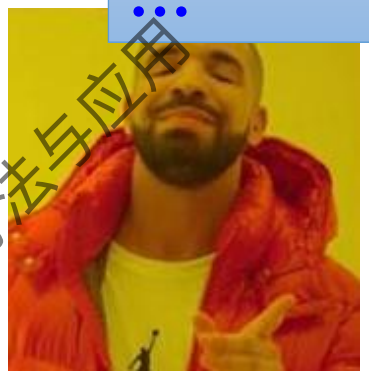




(Speech)

语音翻译

“不行”



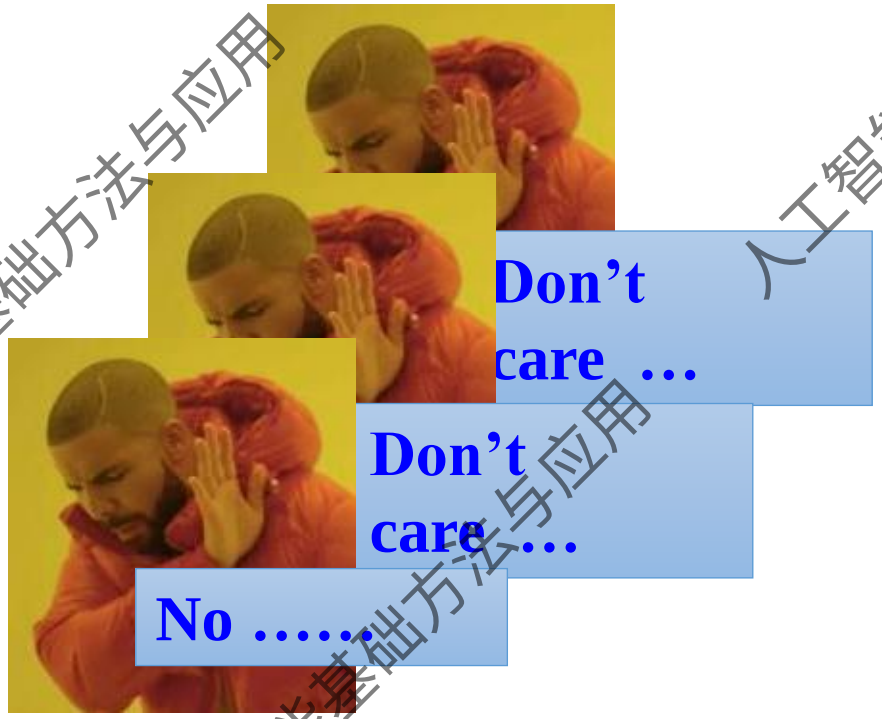
Using 1500 hours of data for training



人工智能基础方法与应用

从有噪声资料中端到端学习

- 背景音&噪声?
- 转录文本可能存在错误?
- 是否可以直接使用音素?



“硬train”
端到端学习

人工智能基础方法与应用

闽南语语音识别



你的身体撑不住



没事为什么你要请假



要生了吗　　Answer:不会腻吗



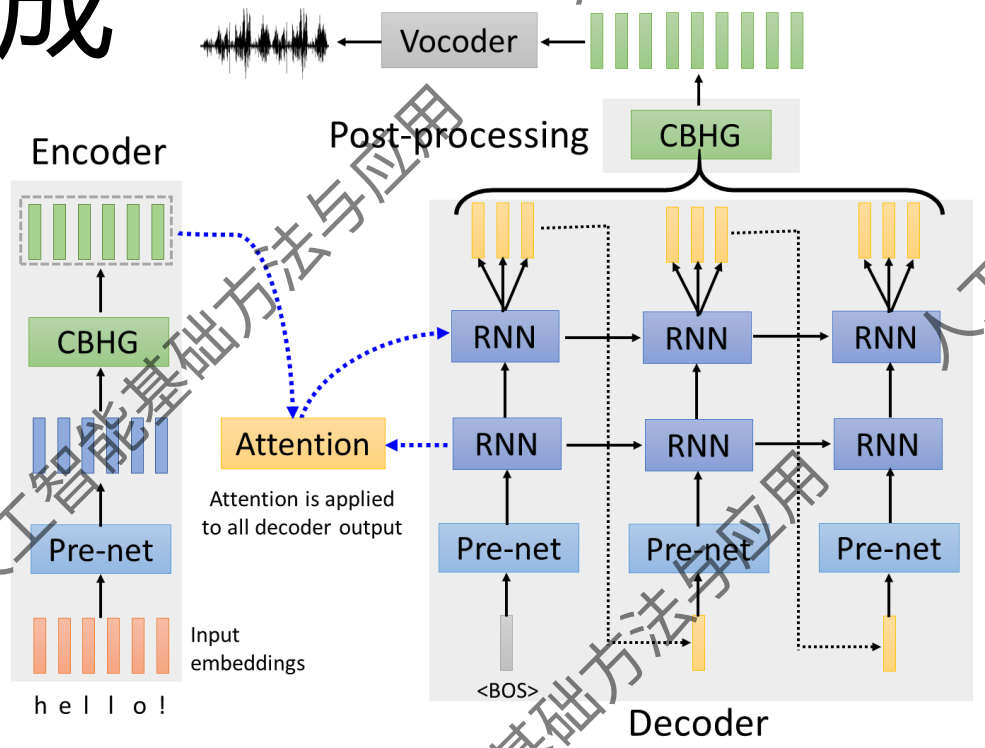
我有帮厂长拜托

Answer: 我拜托厂长了

To learn more: https://sites.google.com/speech.ntut.edu.tw/fsw/home/challenge-2020

人工智能基础方法与应用

文本转语音合成



最近肺炎真严重，要记得戴口罩、勤洗手，有病就要看医生

人工智能基础方法与应用

用Seq2seq训练聊天机器人



Training data:

[PERSON 1:] Hi
[PERSON 2:] Hello ! How are you today ?
[PERSON 1:] I am good thank you , how are you.
[PERSON 2:] Great, thanks ! My children and I were just about to watch Game of Thrones.
[PERSON 1:] Nice ! How old are your children?
[PERSON 2:] I have four that range in age from 10 to 21. You?
[PERSON 1:] I do not have children at the moment.
[PERSON 2:] That just means you get to keep all the popcorn for yourself.
[PERSON 1:] And Cheetos at the moment!
[PERSON 2:] Good choice. Do you watch Game of Thrones?
[PERSON 1:] No, I do not have much time for TV.
[PERSON 2:] I usually spend my time painting: but, I love the show.

人工智能基础方法与应用

把 NLP 任务统一成问答

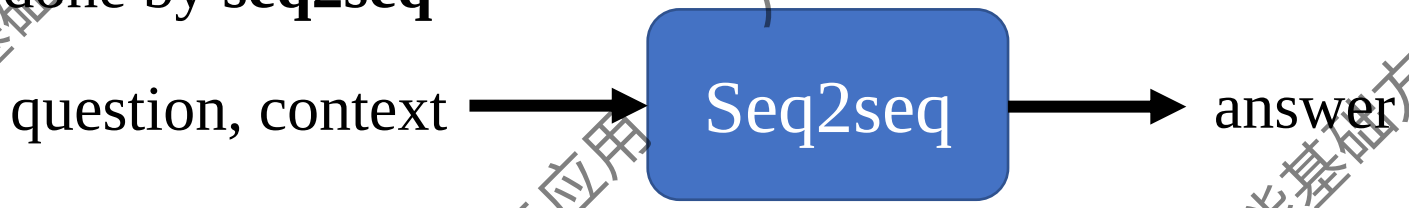
Question Answering (QA)

Question	Context	Answer
What is a major importance of Southern California in relation to California and the US?	...Southern California is a major economic center for the state of California and the US....	major economic center
What is the translation from English to German?	Most of the planet is ocean water.	Der Großteil der Erde ist Meerwasser
What is the summary?	Harry Potter star Daniel Radcliffe gains access to a reported £320 million fortune...	Harry Potter star Daniel Radcliffe gets £320M fortune...
Hypothesis: Product and geography are what make cream skimming work. Entailment, neutral, or contradiction?	Premise: Conceptually cream skimming has two basic dimensions – product and geography.	Entailment
Is this sentence positive or negative?	A stirring, funny and finally transporting re-imagining of Beauty and the Beast and 1930s horror film.	positive

(sentiment analysis)

decaNLP

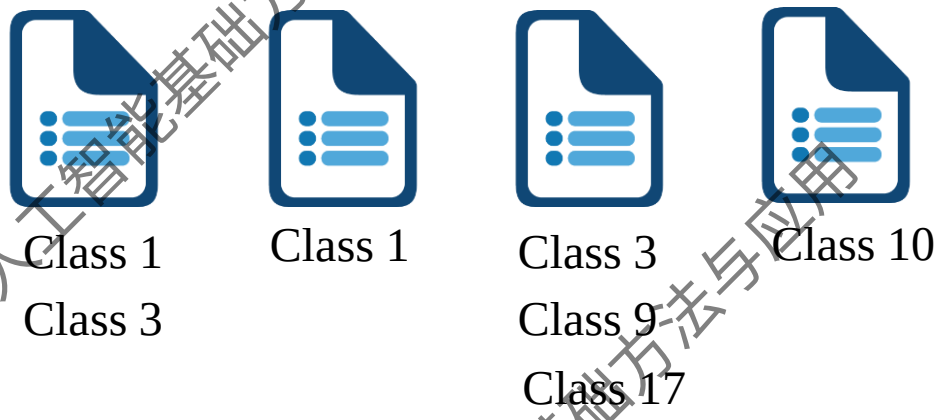
QA can be done by **seq2seq**

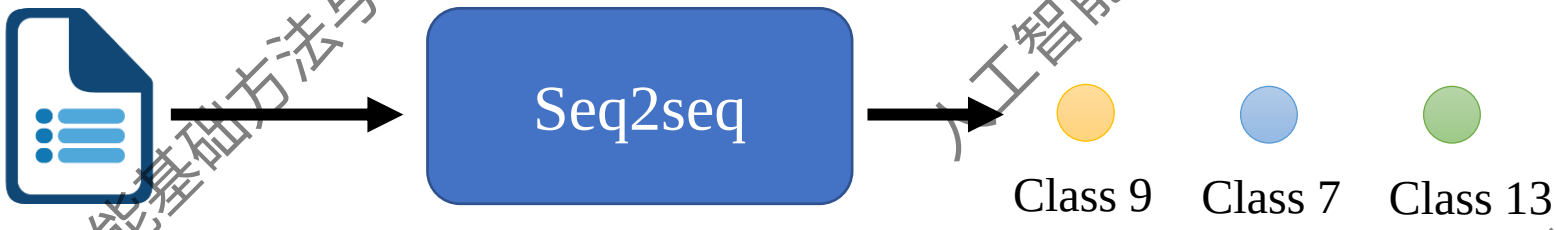


用 Seq2seq 做多标签分类

c.f. Multi-class Classification

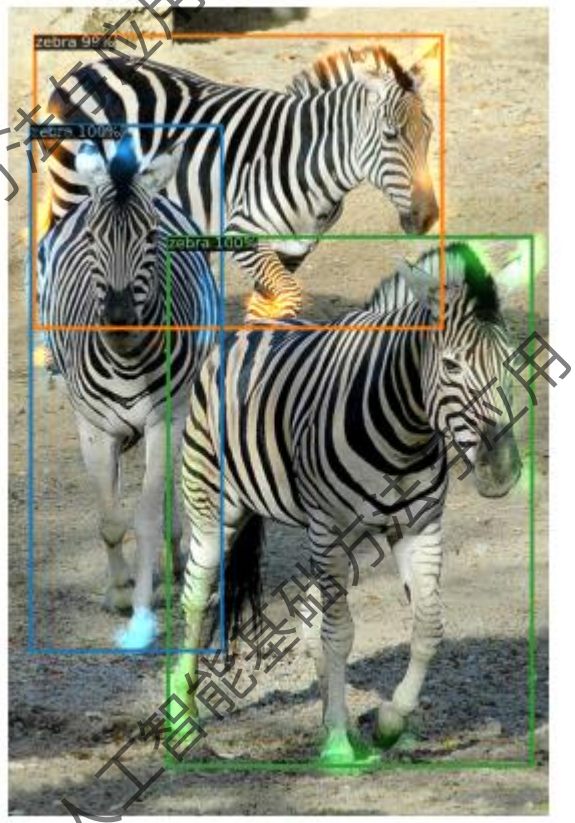
An object can belong to multiple classes.

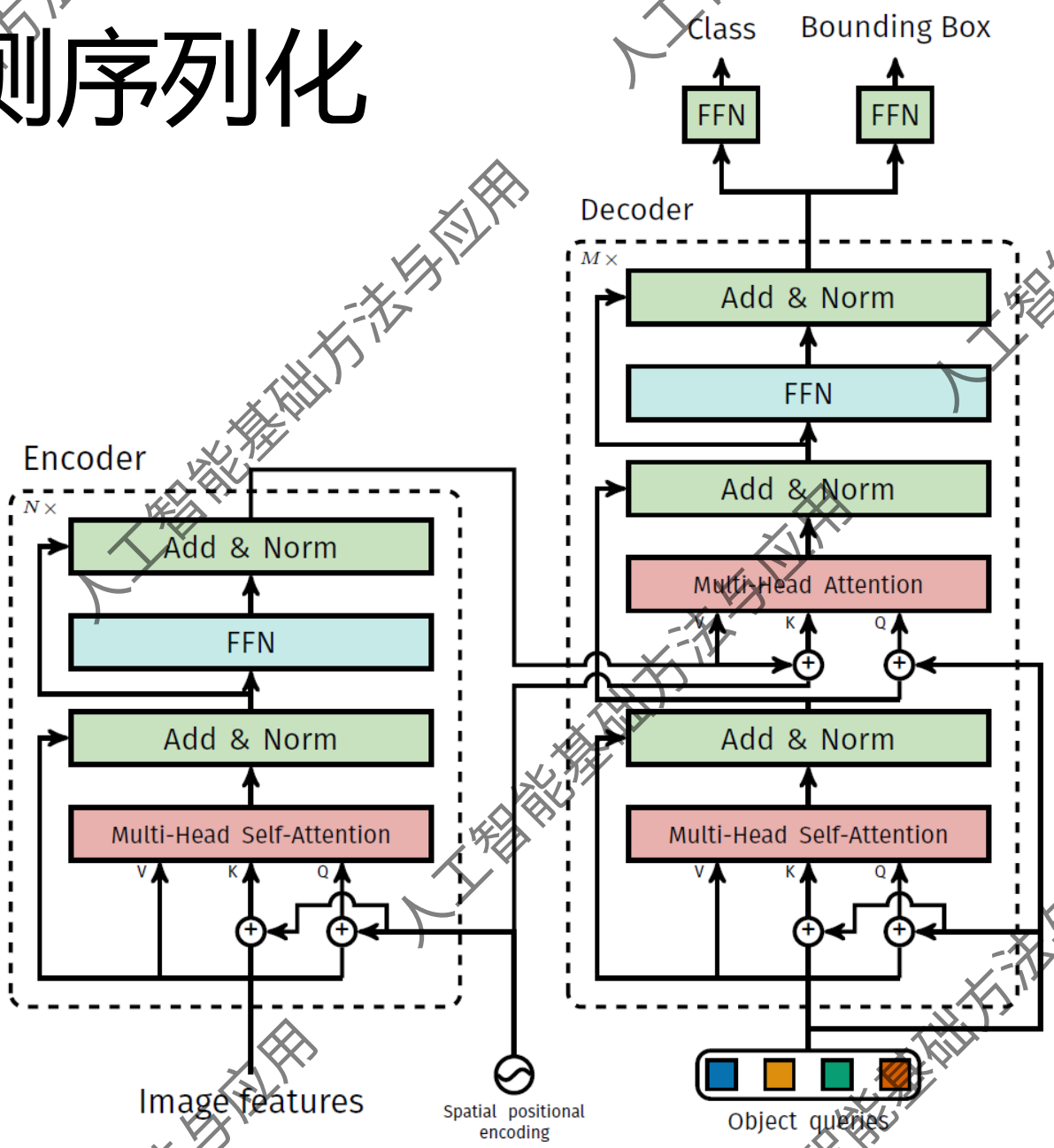




人工智能基础方法与应用

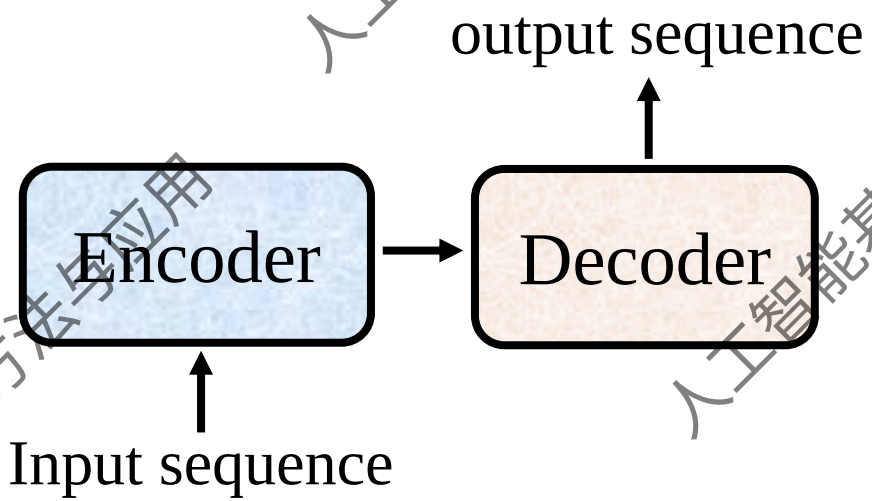
把目标检测序列化

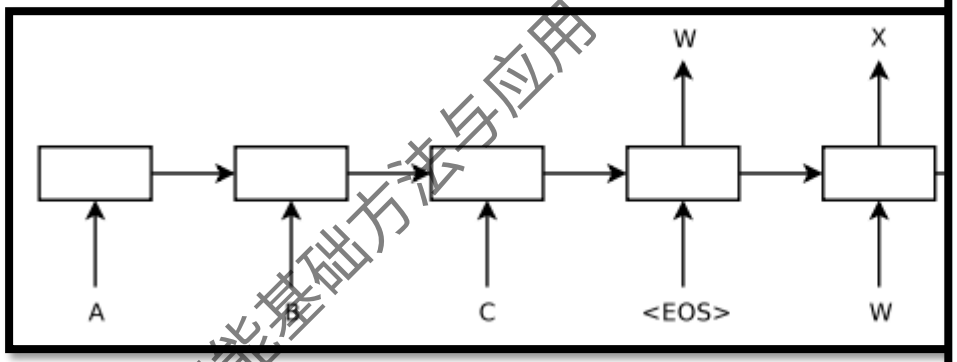




人工智能基础方法与应用

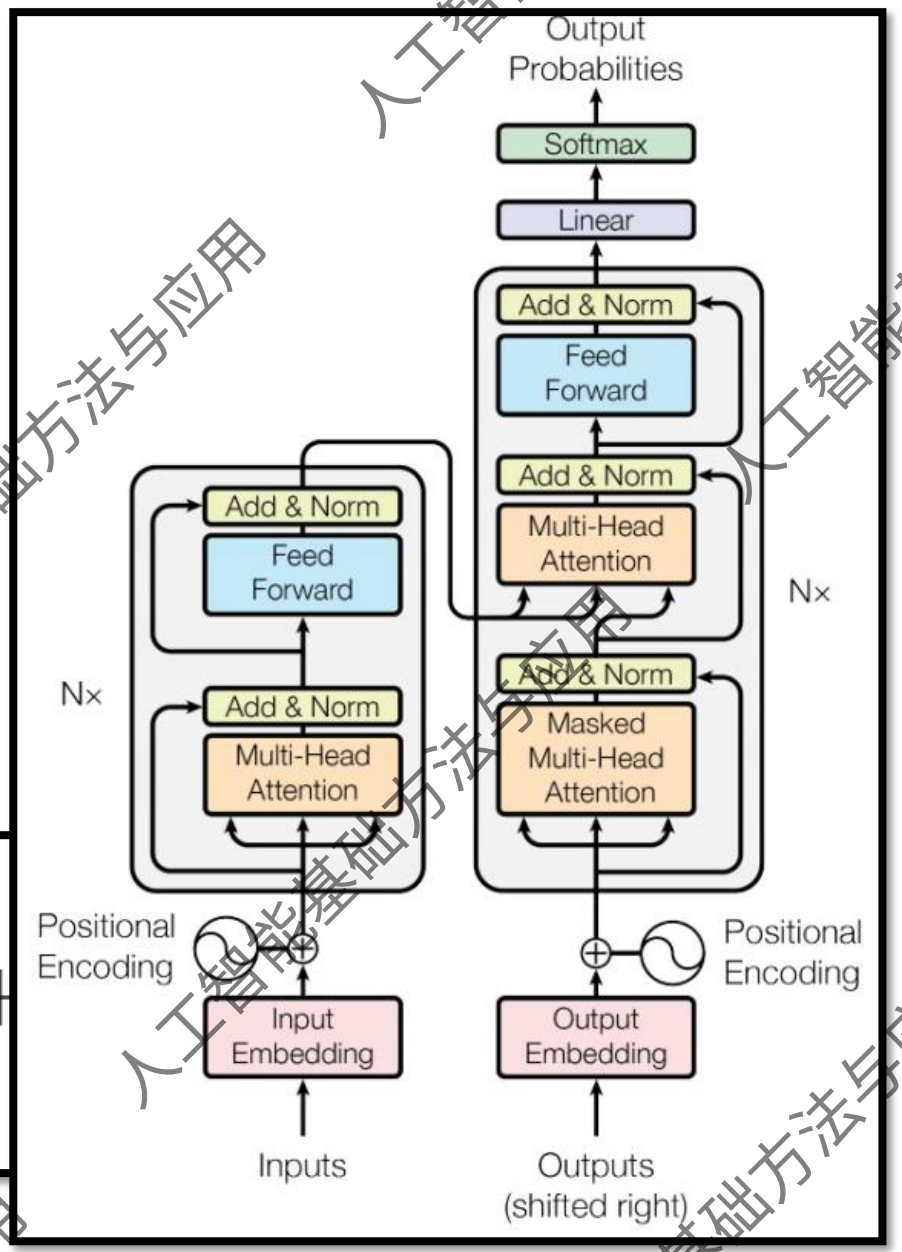
Seq2seq





Sequence to Sequence Learning with Neural Networks

https://arxiv.org/abs/1409.3215



Transformer

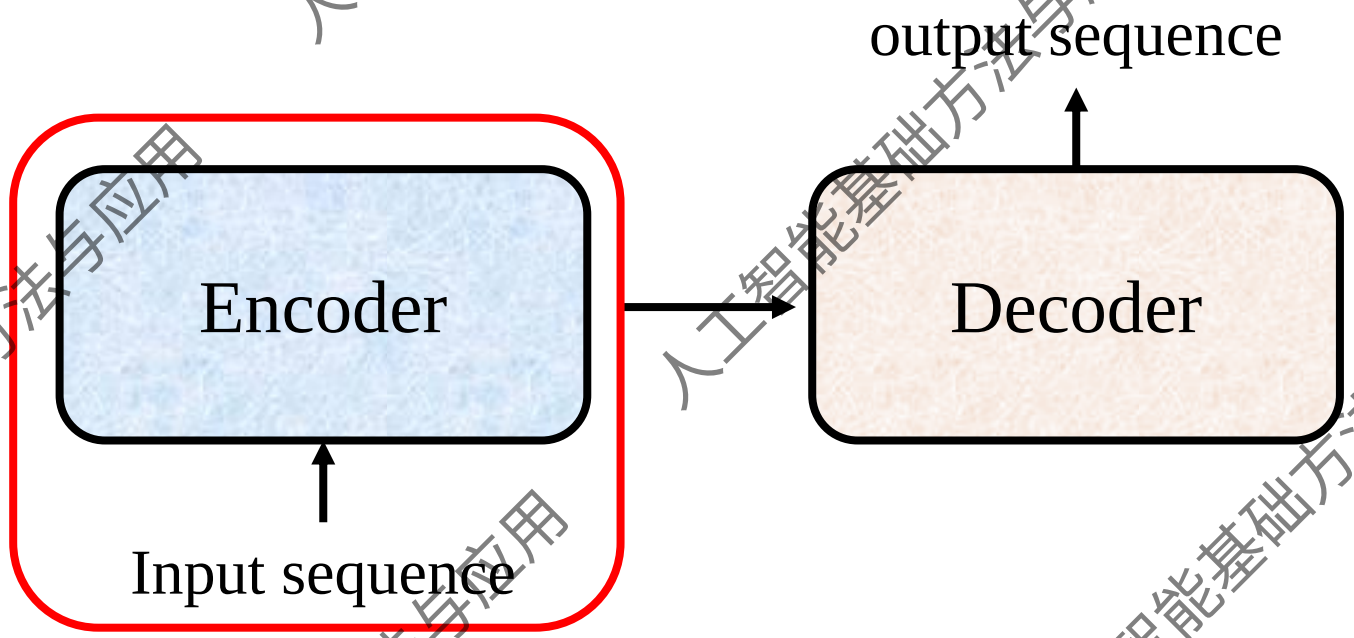
https://arxiv.org/abs/1706.03762

6.1 Transformer Encoder

人工智能基础方法与应用

Encoder(编码器)

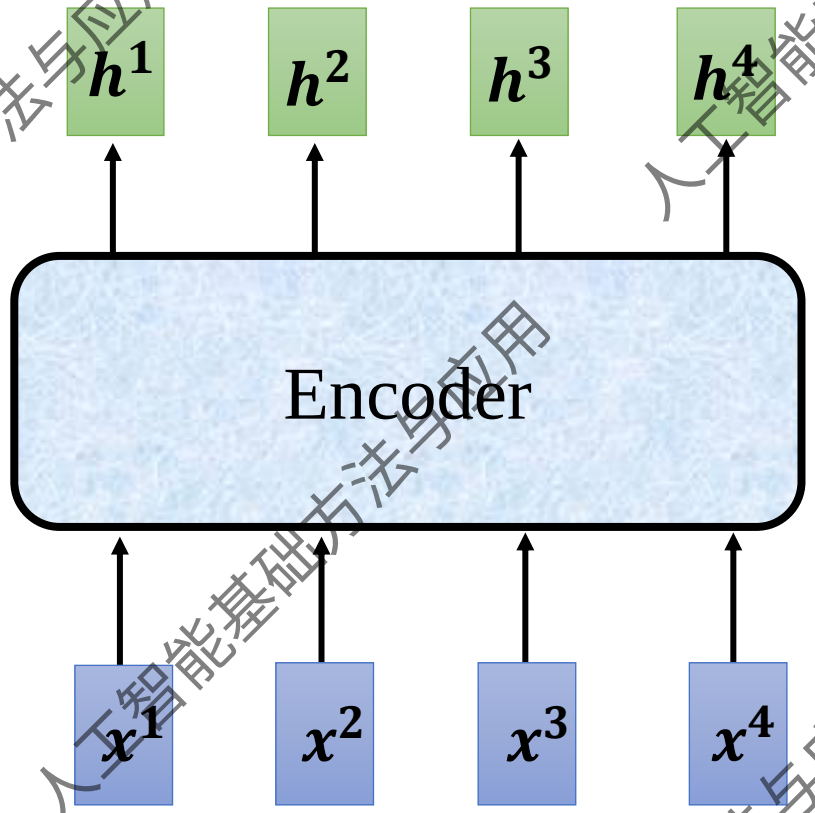
- 编码器负责读取输入序列，并将原始输入逐步转换为更高层次的特征表示



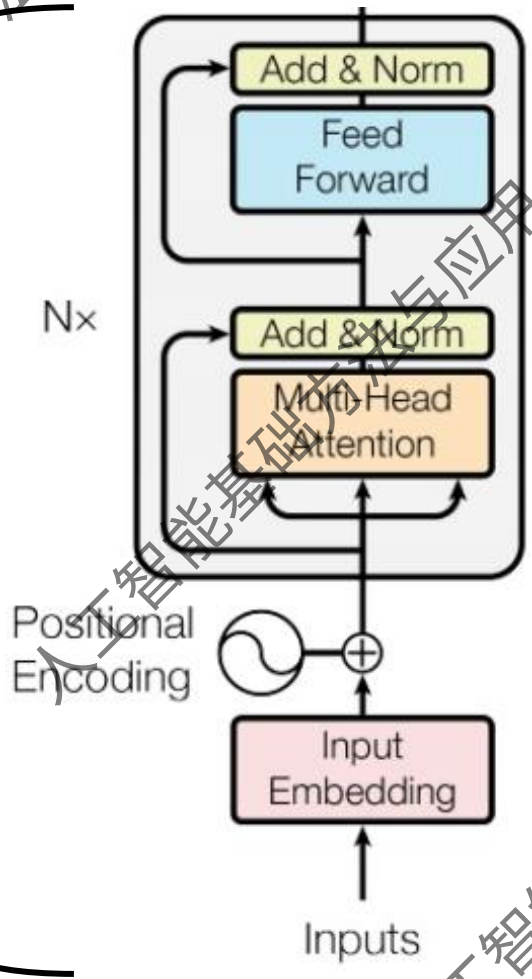
人工智能基础方法与应用

Encoder的序列输入与输出

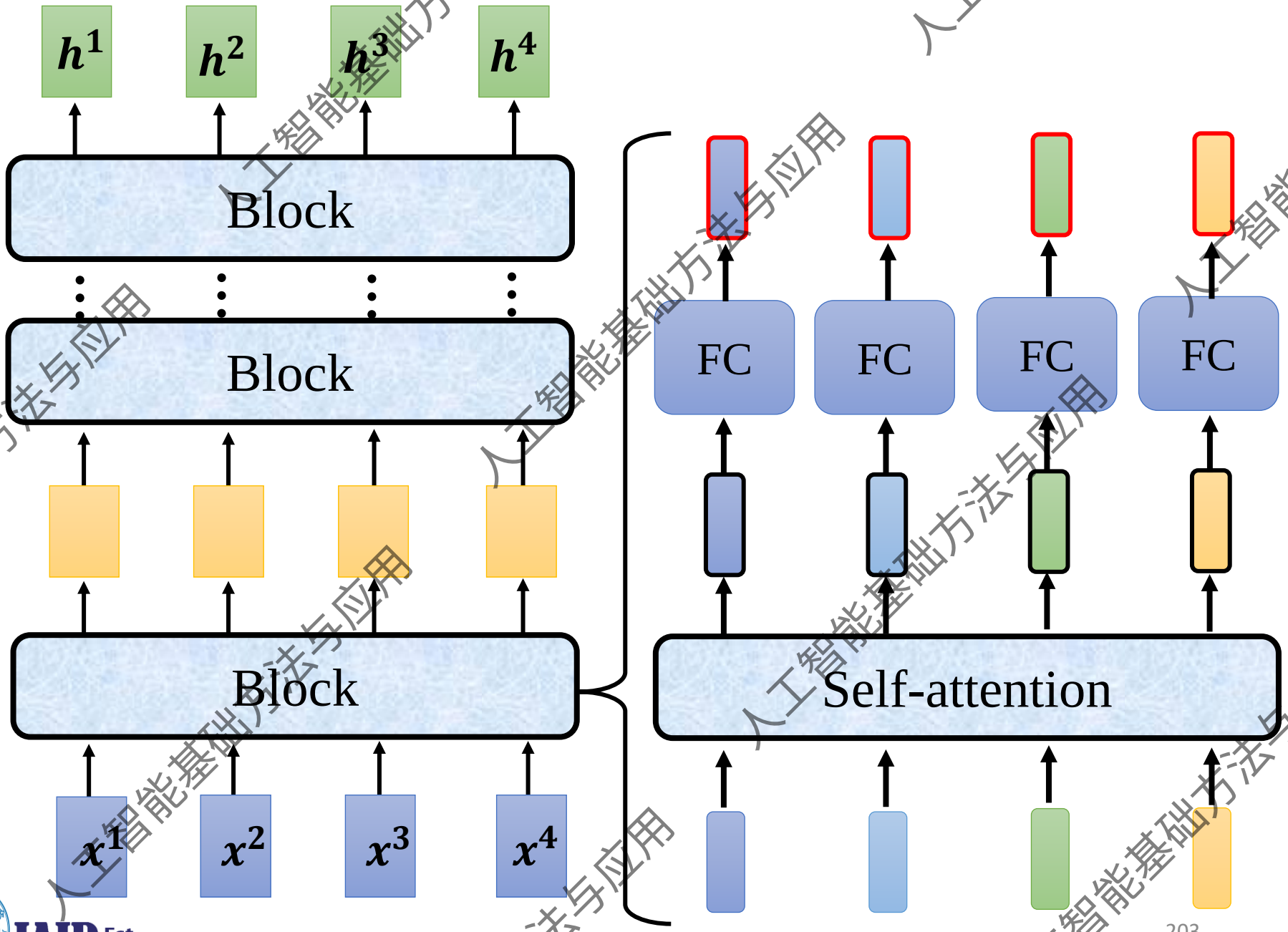
CNN也可以完成



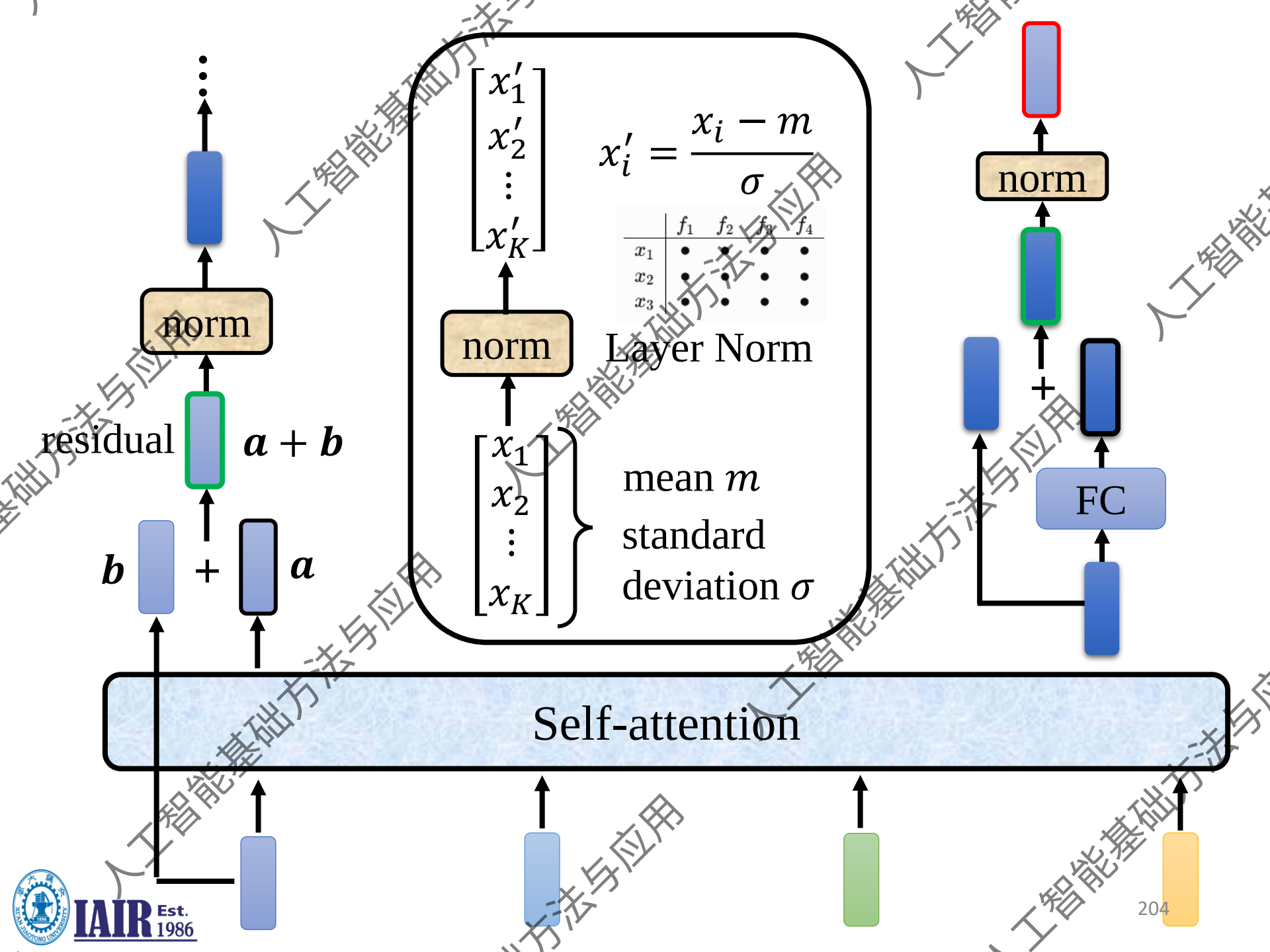
Transformer's Encoder



人工智能基础方法与应用



人工智能基础方法与应用

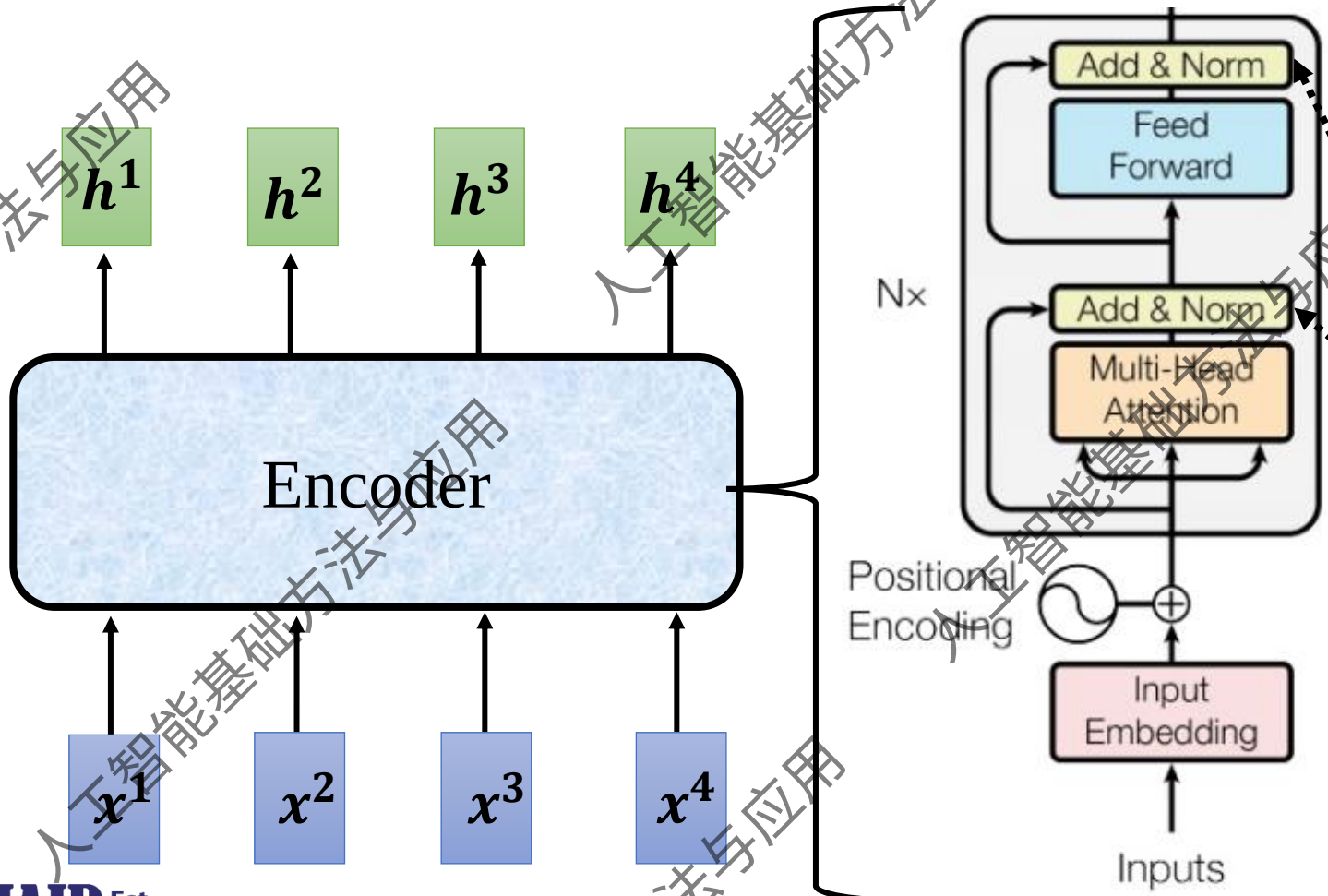


人工智能基础方法与应用

BERT



I use the **same** network architecture as **transformer encoder**.



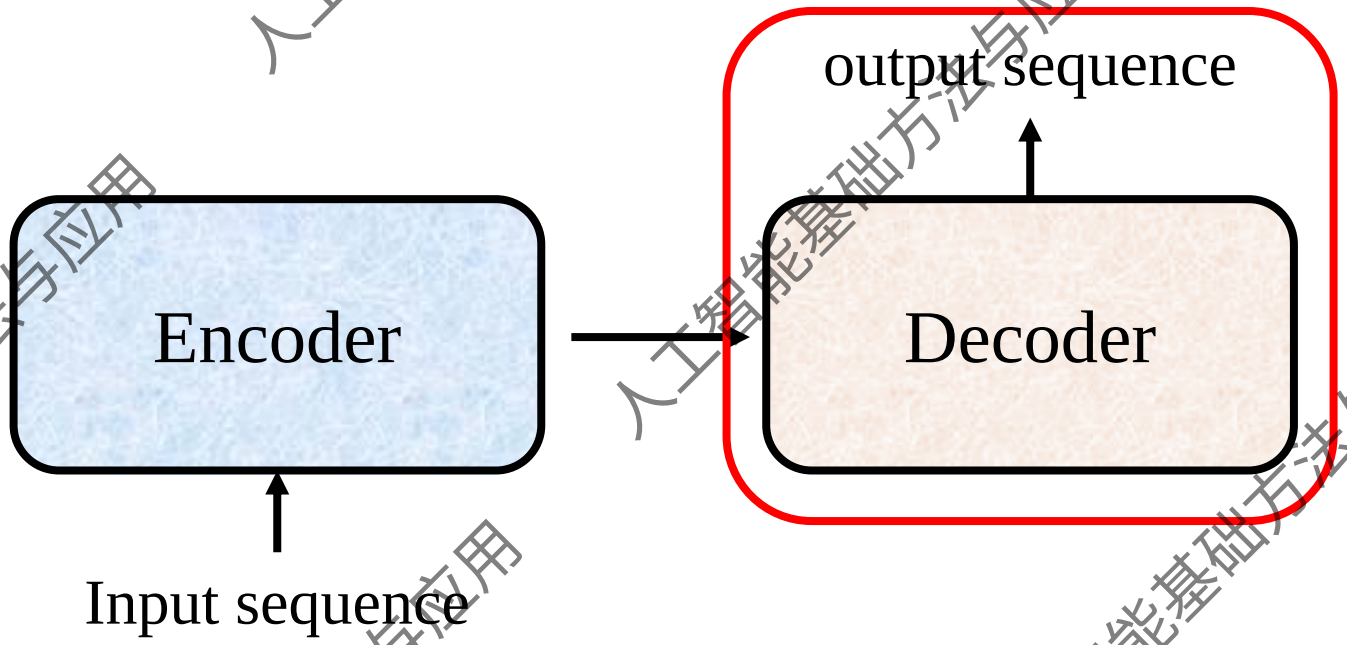
Residual + Layer norm

6.2 自回归解码器

人工智能基础方法与应用

Decoder（解码器）

- 解码器负责根据编码器提供的内部表示，逐步生成最终输出序列
- 编码器负责“读懂输入”，解码器负责“把理解结果生成出来”



人工智能基础方法与应用

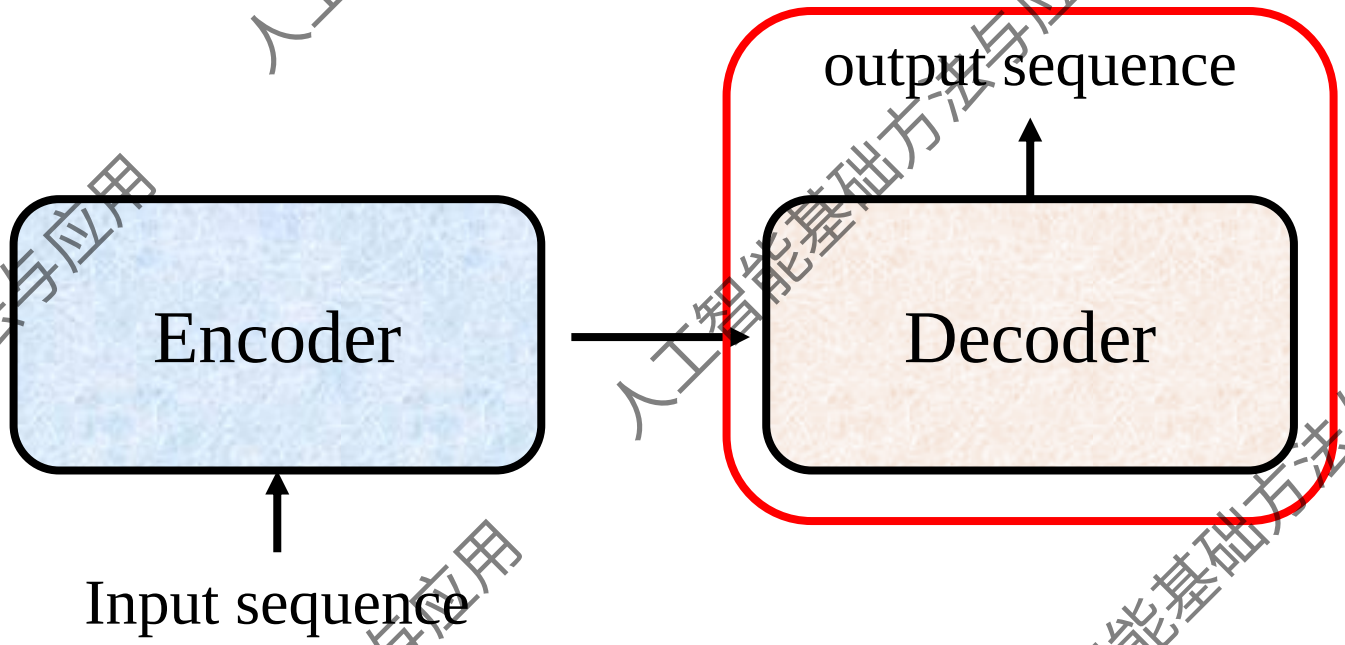
Decoder–自回归解码器

- 自回归解码器（Autoregressive，AT）

  ①从起始符开始生成第一个 token

  ② 将已经生成的 token 作为下一步的输入

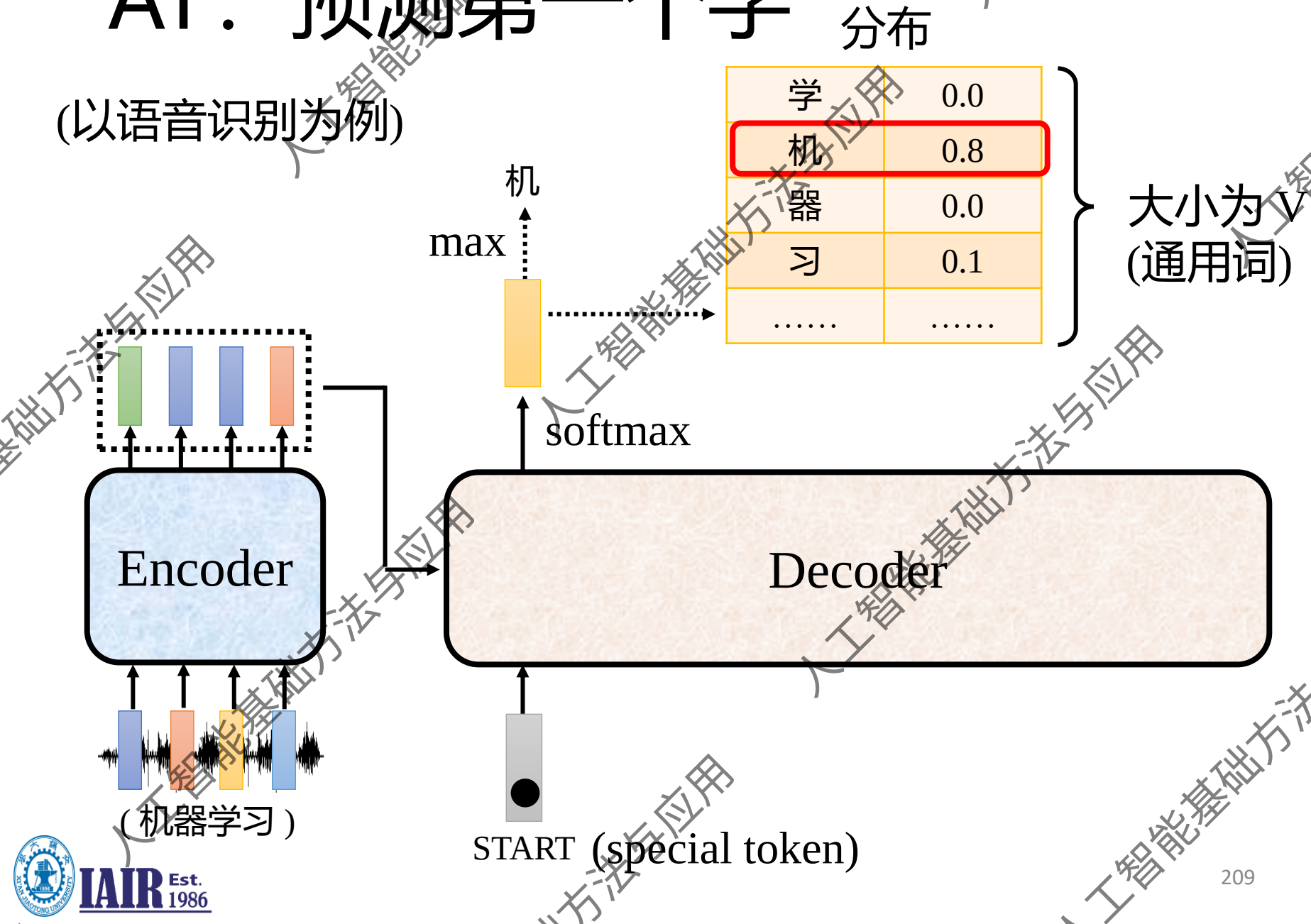
  ③ 一次生成一个 token，直到生成结束符



人工智能基础方法与应用

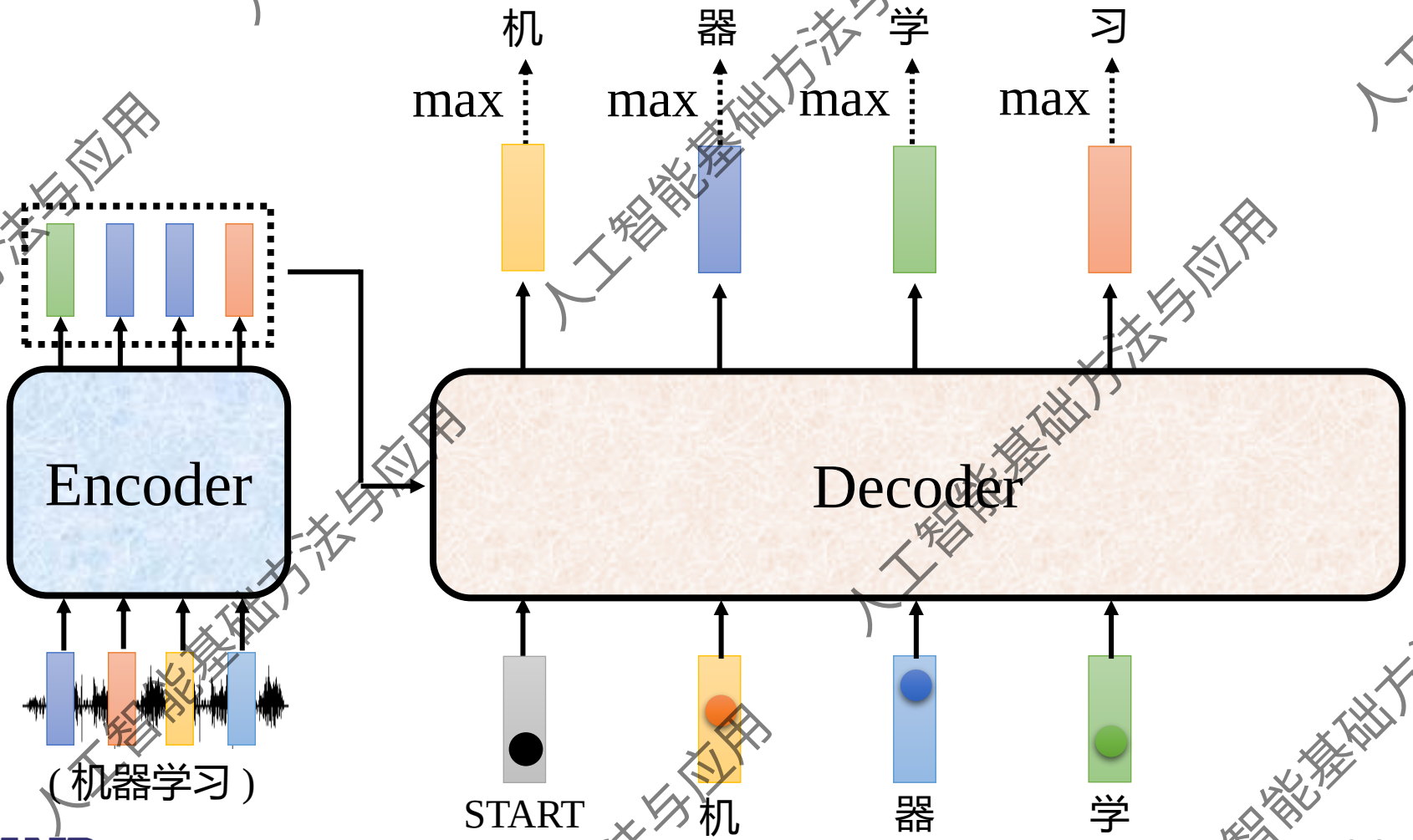
AT：预测第一个字

(以语音识别为例)



人工智能基础方法与应用

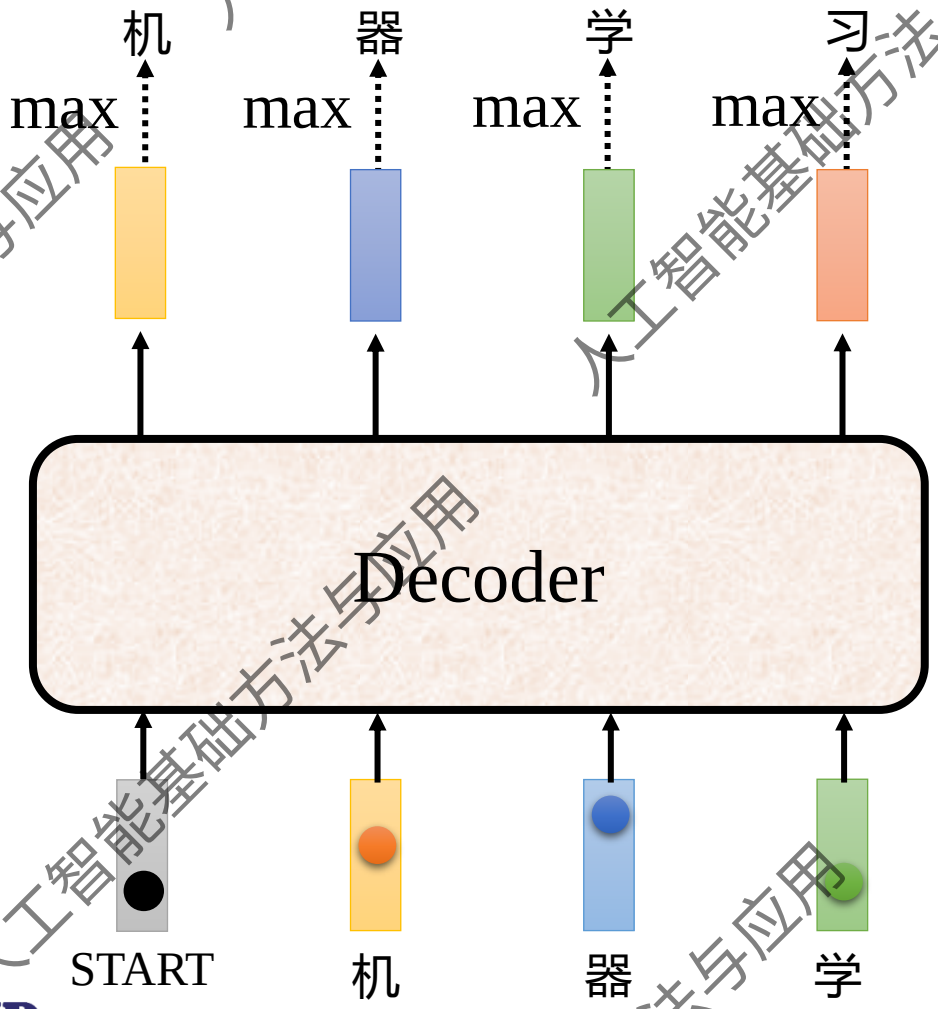
AT：逐步反馈输出

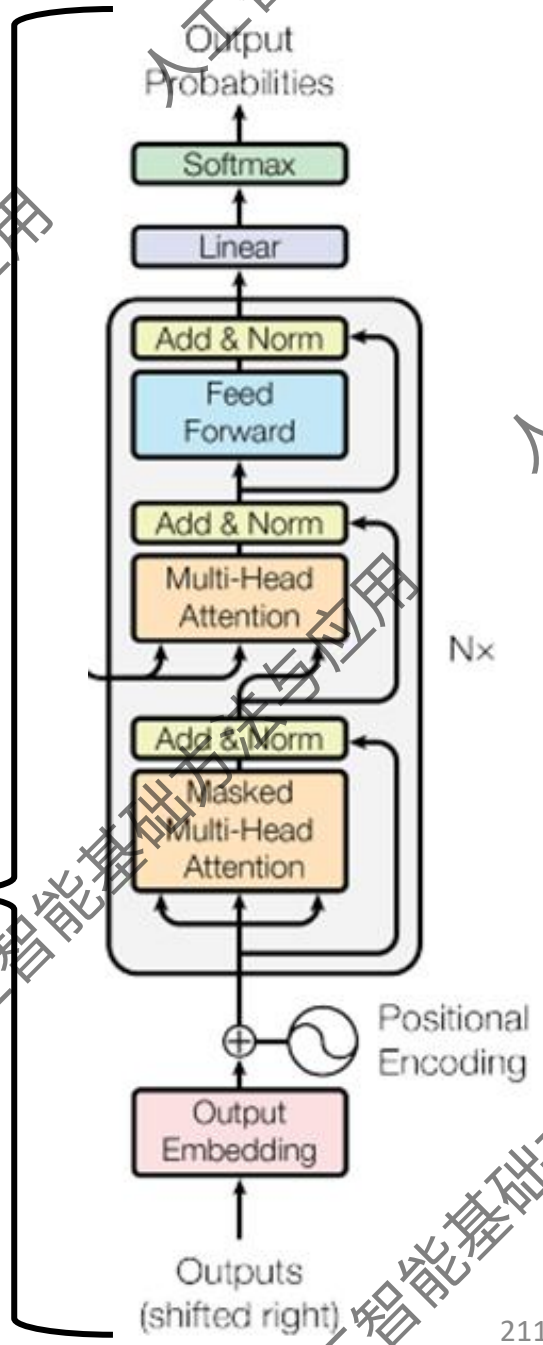


人工智能基础方法与应用

解码器内部

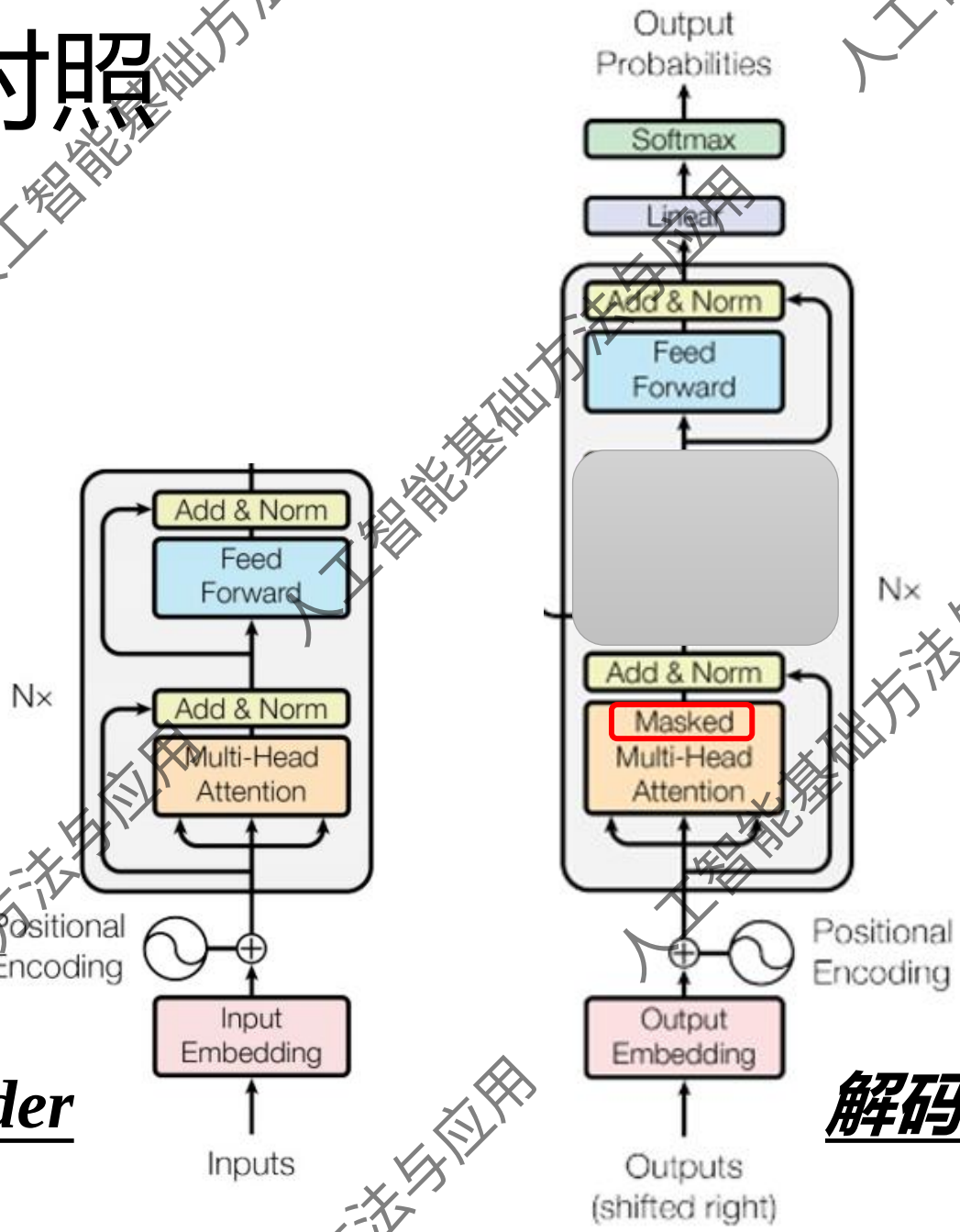
ignore the input from the encoder here





人工智能基础方法与应用

结构对照

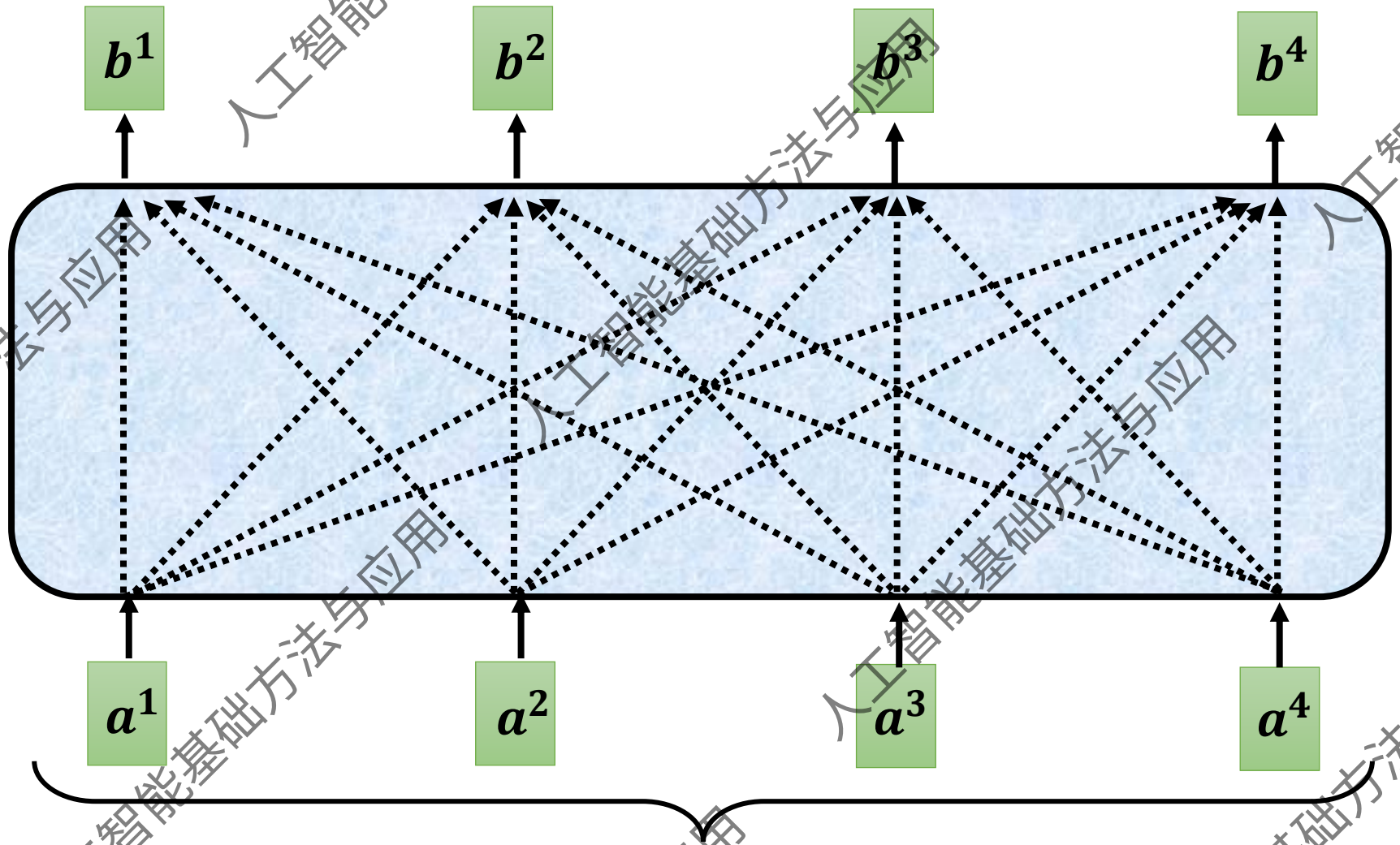


编码器 Encoder

解码器 Decoder

人工智能基础方法与应用

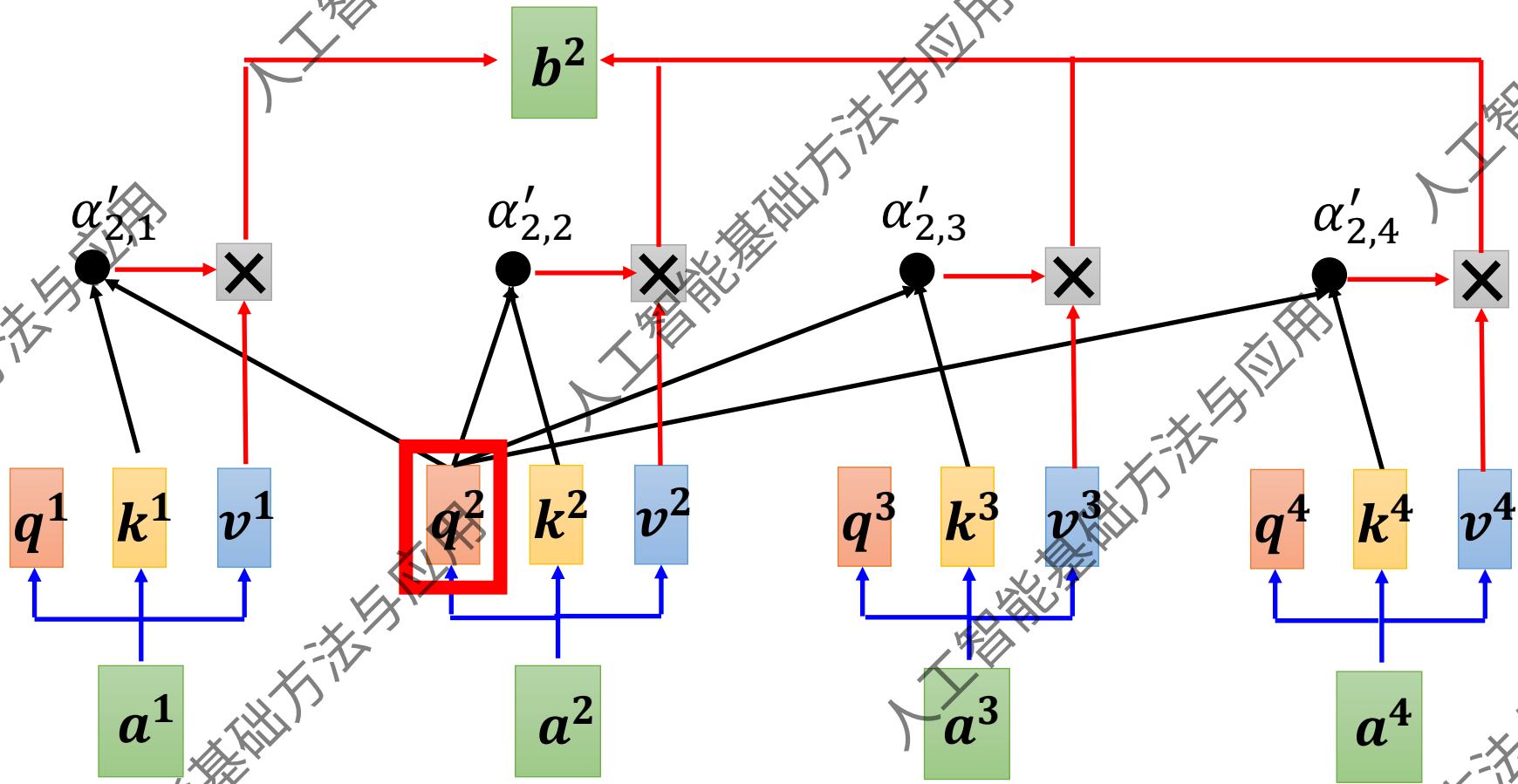
Self-attention → *Masked Self-attention*



Can be either **input** or **a hidden layer**

人工智能基础方法与应用

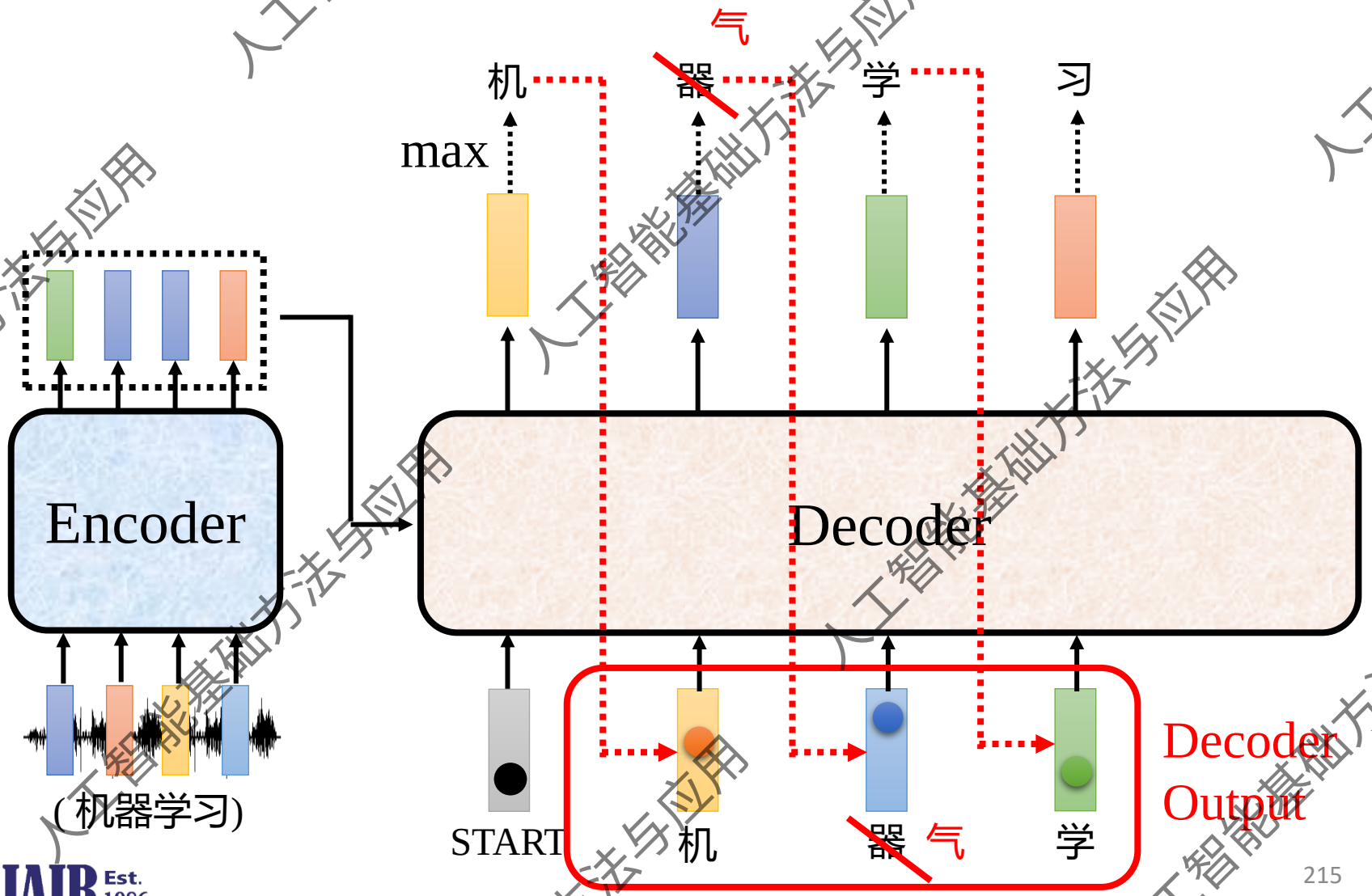
掩码如何作用于注意力分数



Why masked? Consider how does decoder work

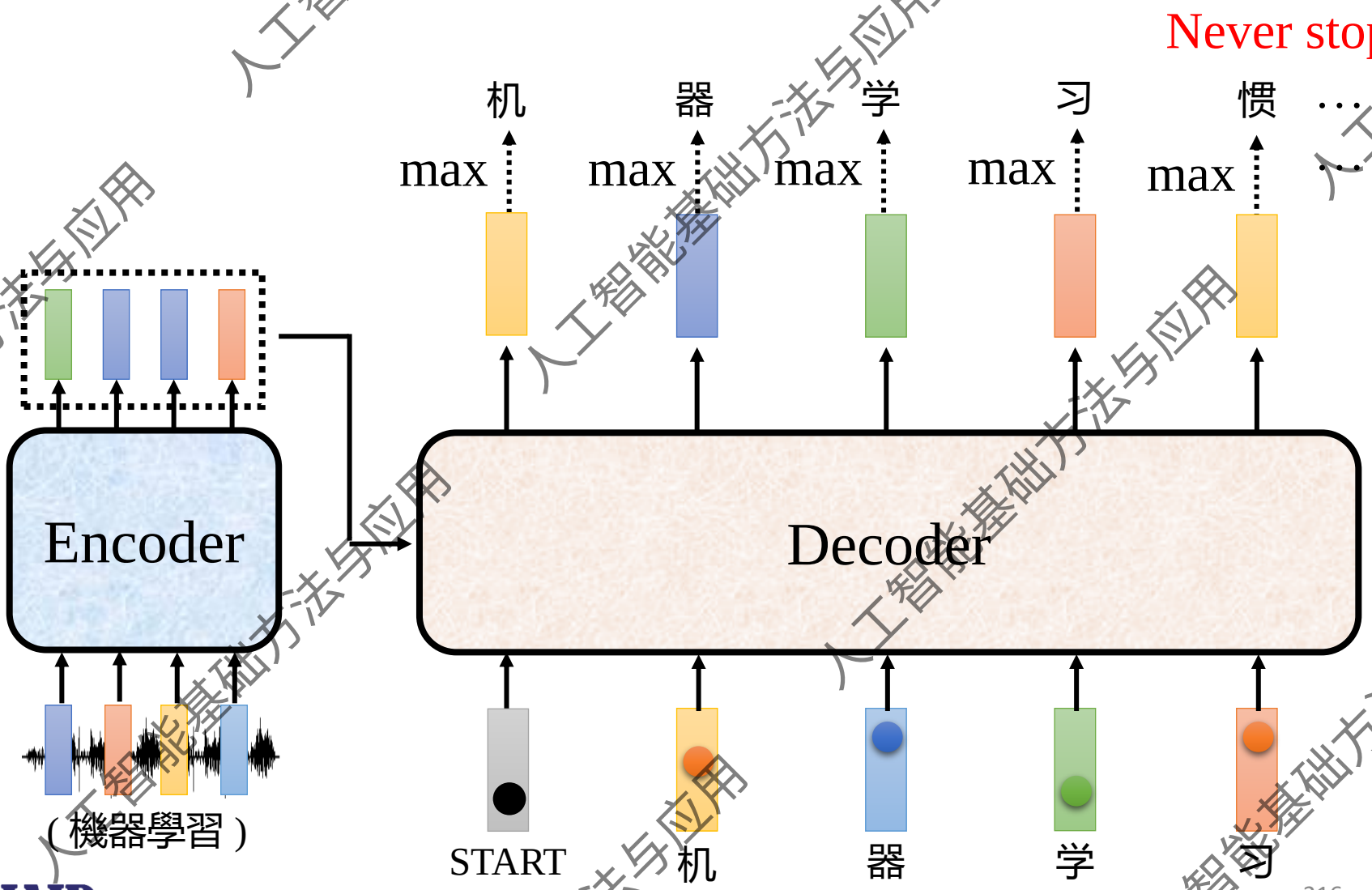
人工智能基础方法与应用

为什么需要掩码：因果与误差传播



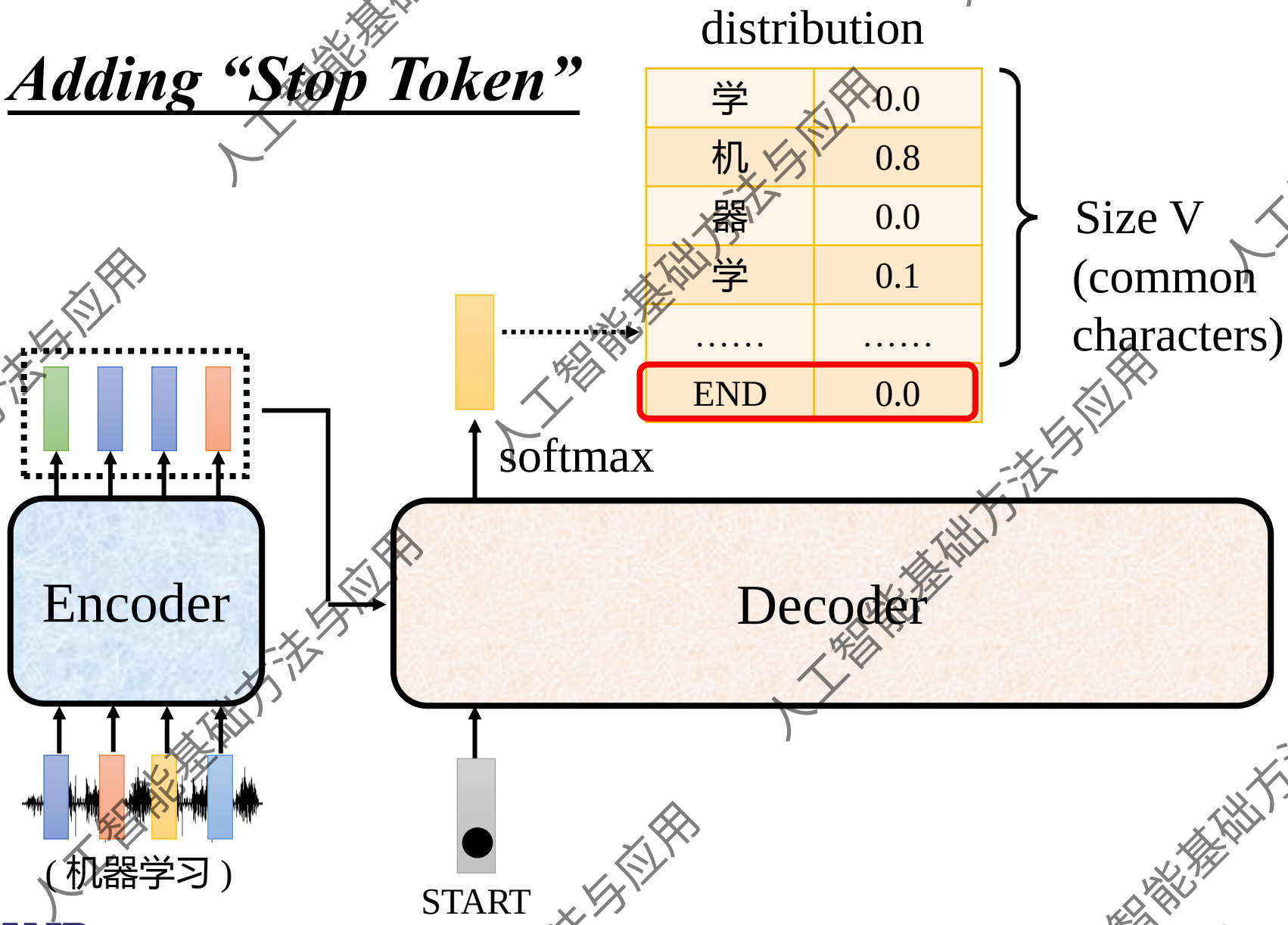
人工智能基础方法与应用

解码器不知道何时停止



人工智能基础方法与应用

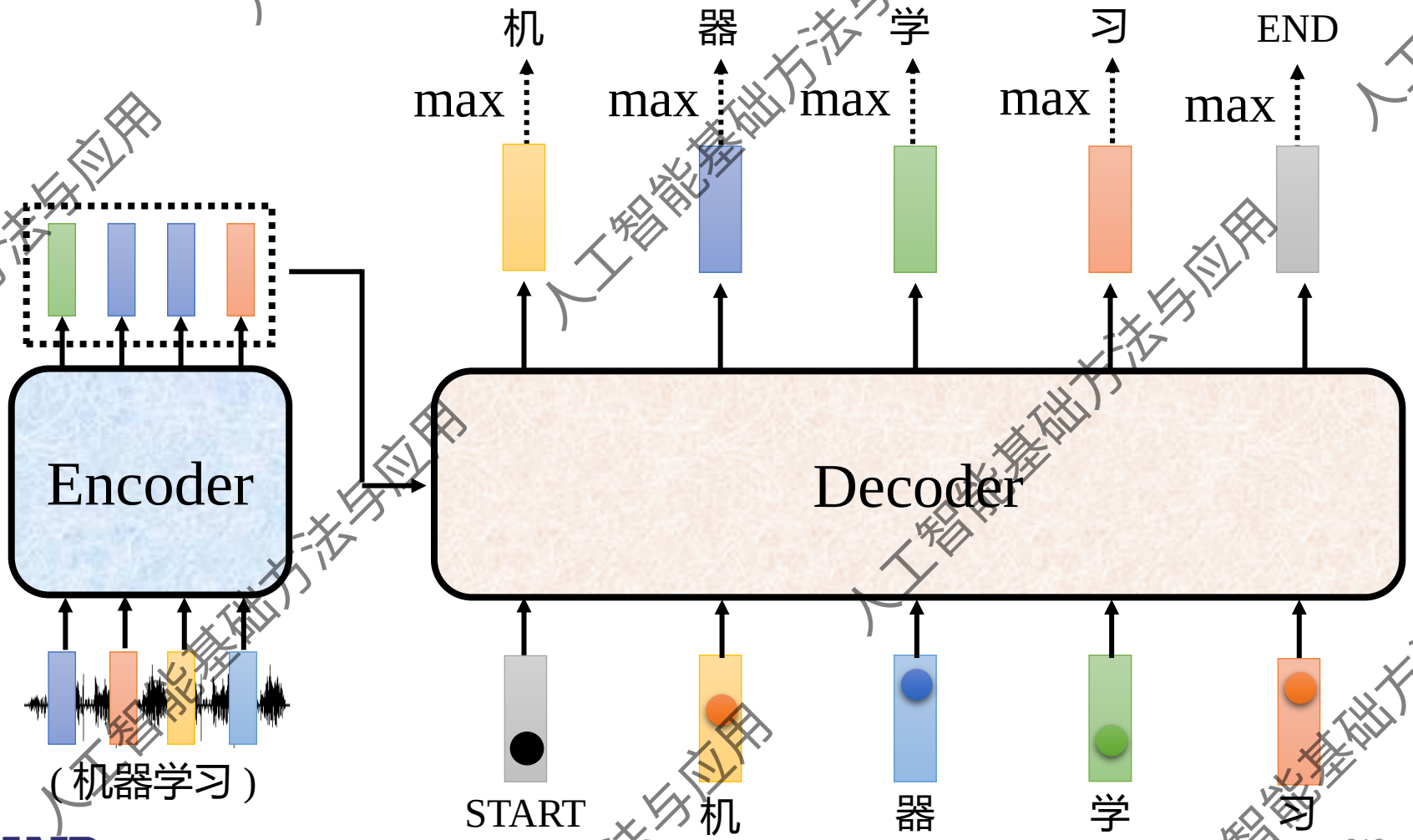
Adding "Stop Token"



人工智能基础方法与应用

Autoregressive

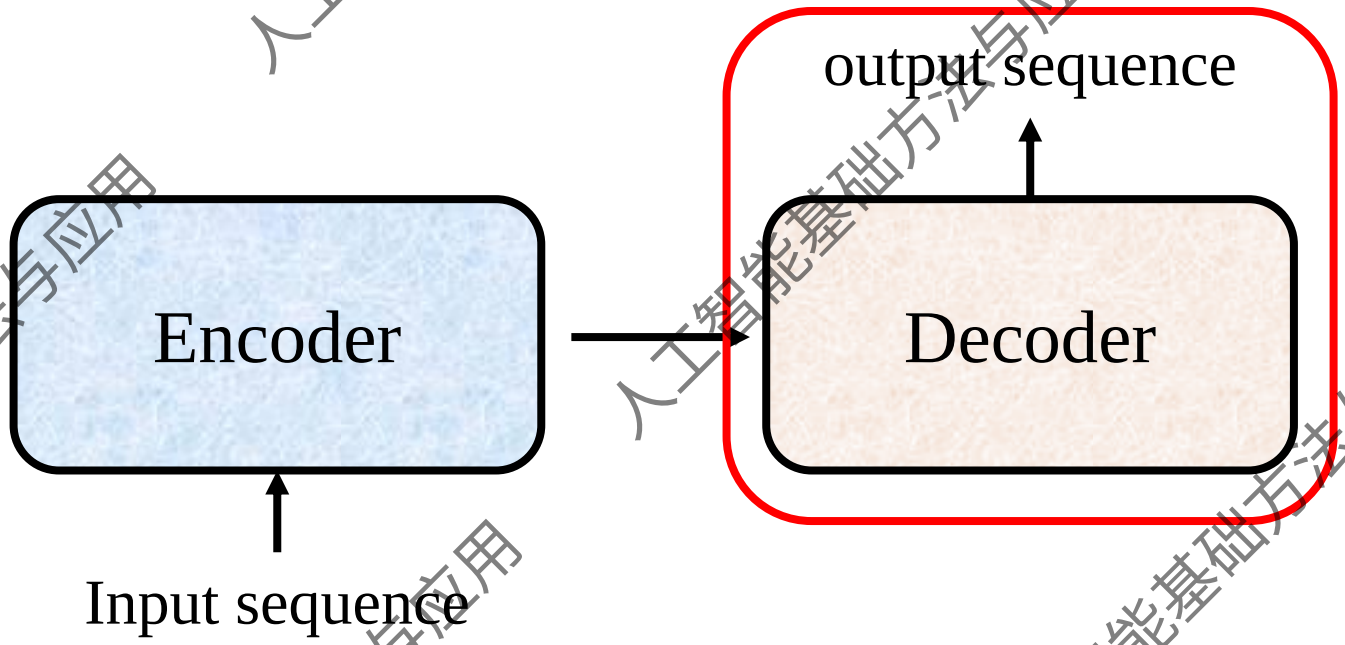
Stop at here!



人工智能基础方法与应用

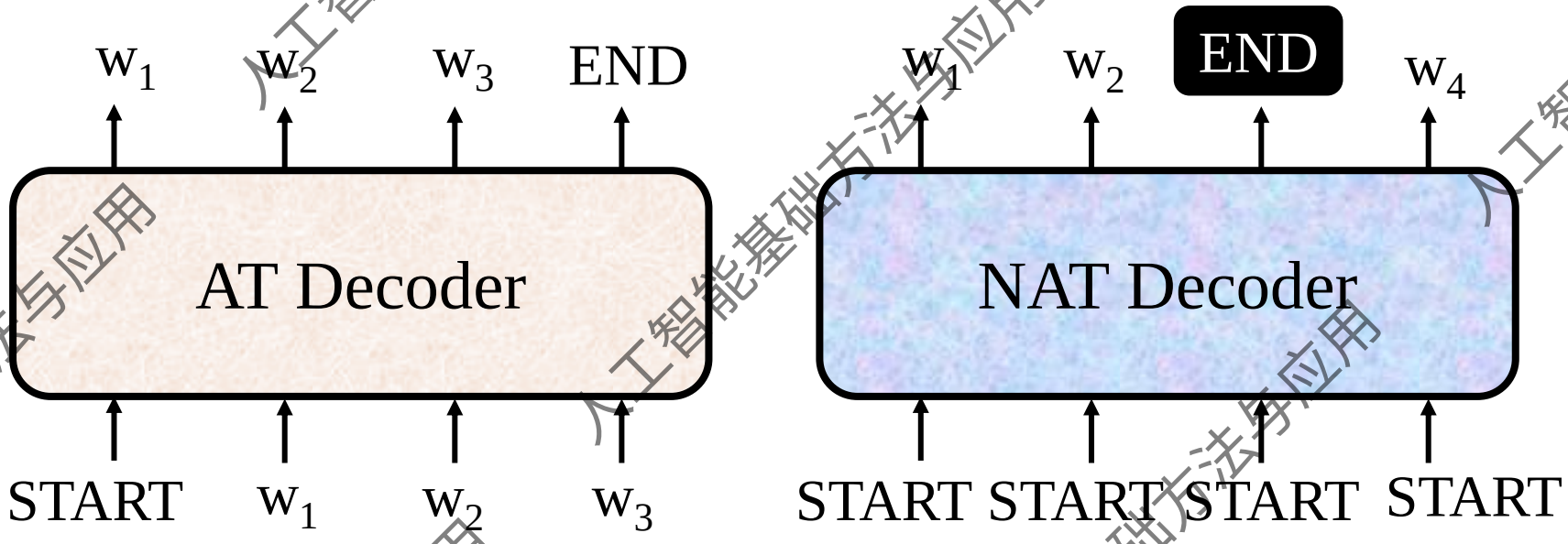
非自回归解码器

- 非自回归解码器（Non-autoregressive，NAT）
  - 不依赖前面已经生成的 token
  - 多个位置可以同时预测
  - 推理速度更快，适合低延迟场景
  - 不同输出位置之间的依赖建模更困难，生成质量有时会受到影响



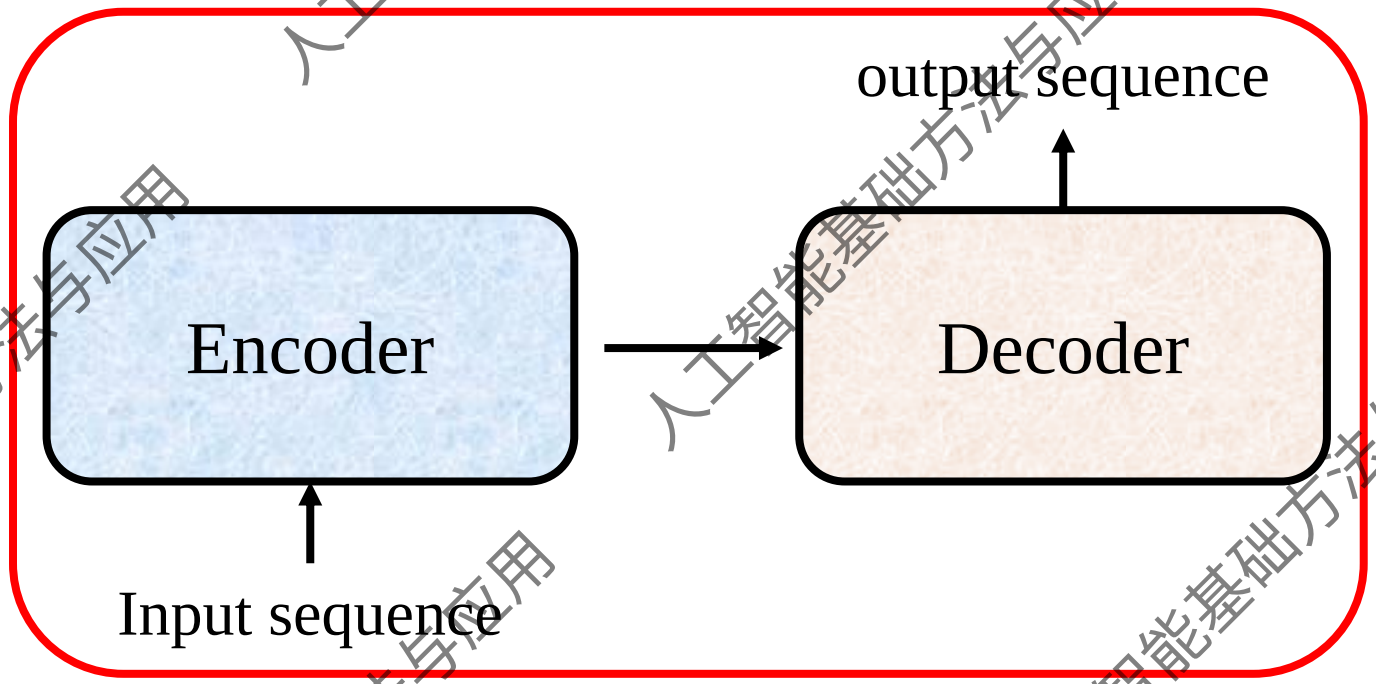
人工智能基础方法与应用

AT v.s. NAT



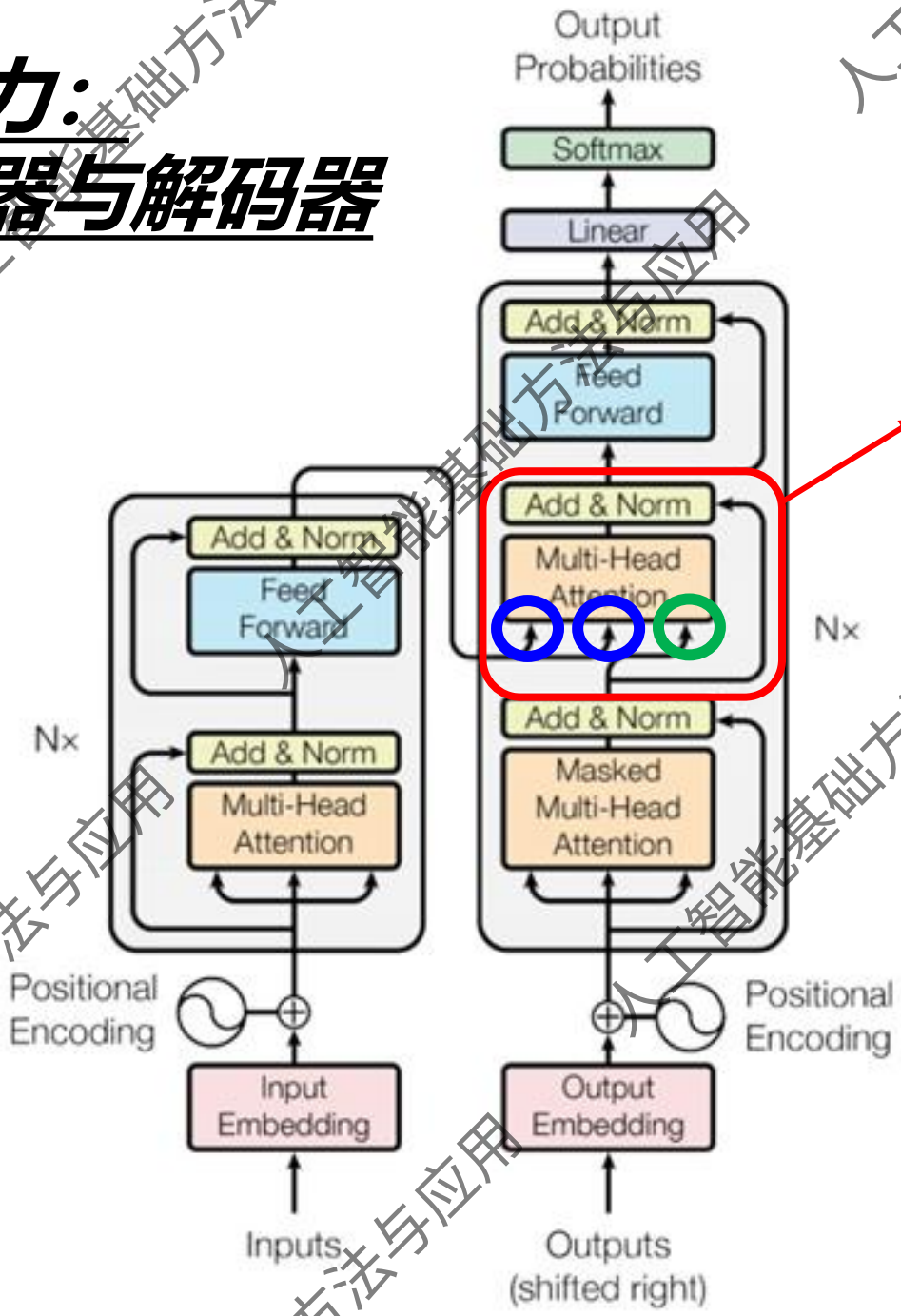
- NAT 解码器如何确定输出序列长度?
  - 额外使用一个模型预测输出长度
  - 先生成一个较长序列，遇到 END 后忽略后续 token
- Advantage: 并行, 更稳定(e.g., TTS)
- NAT生成质量通常弱于AT (why? **Multi-modality**)

Encoder-Decoder

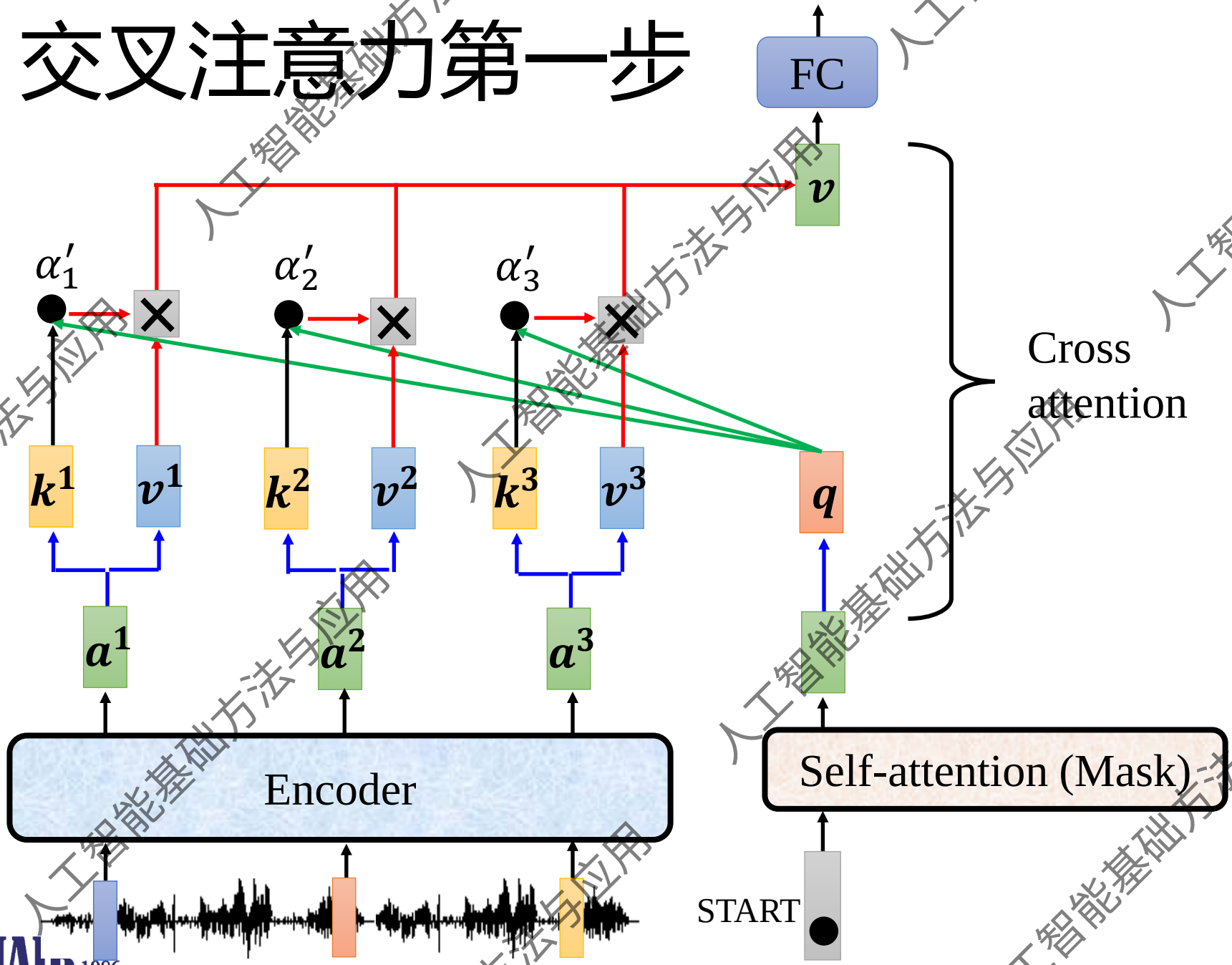


人工智能基础方法与应用

交叉注意力：连接编码器与解码器

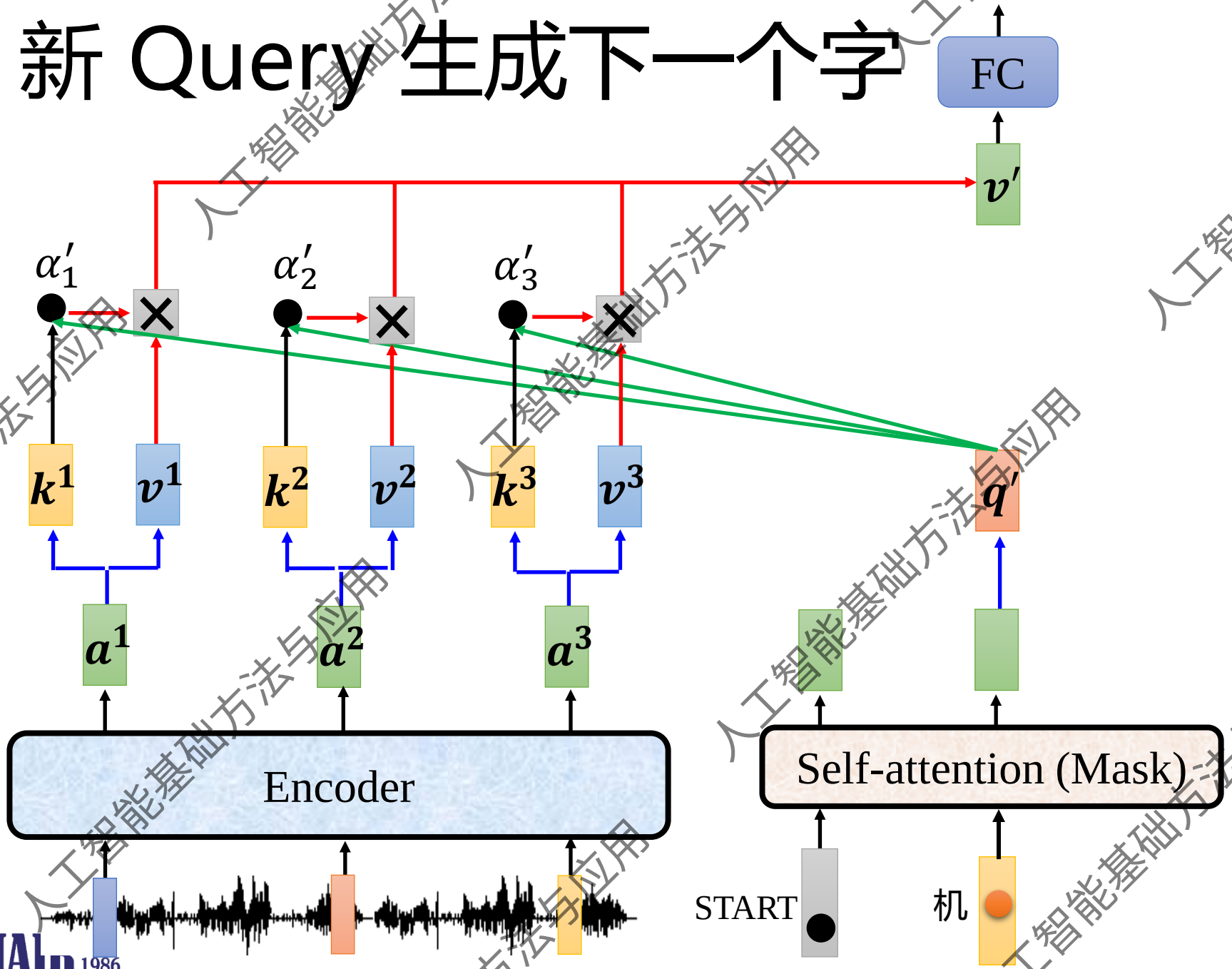


交叉注意力第一步



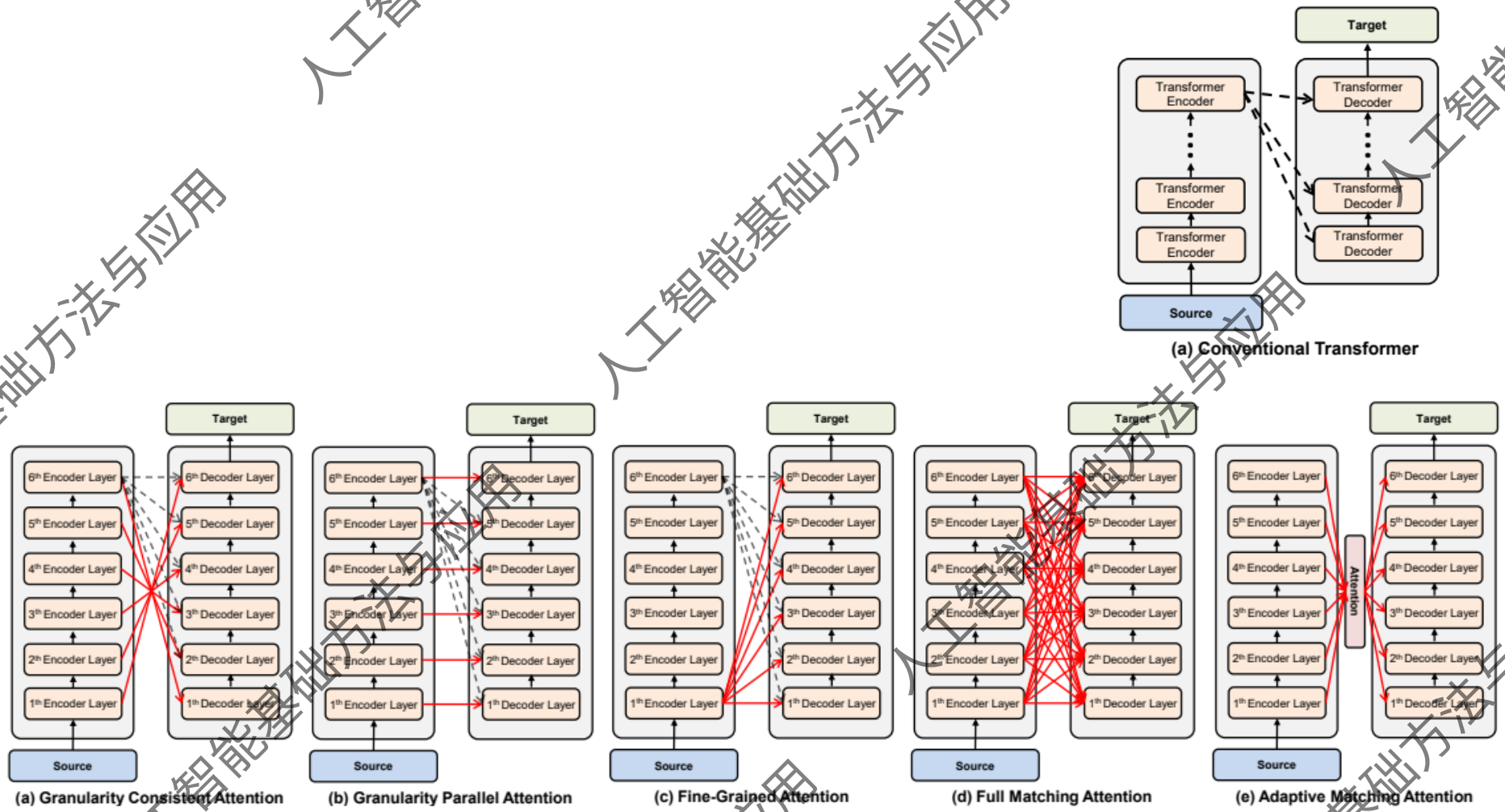
人工智能基础方法与应用

新 Query生成下一个字



人工智能基础方法与应用

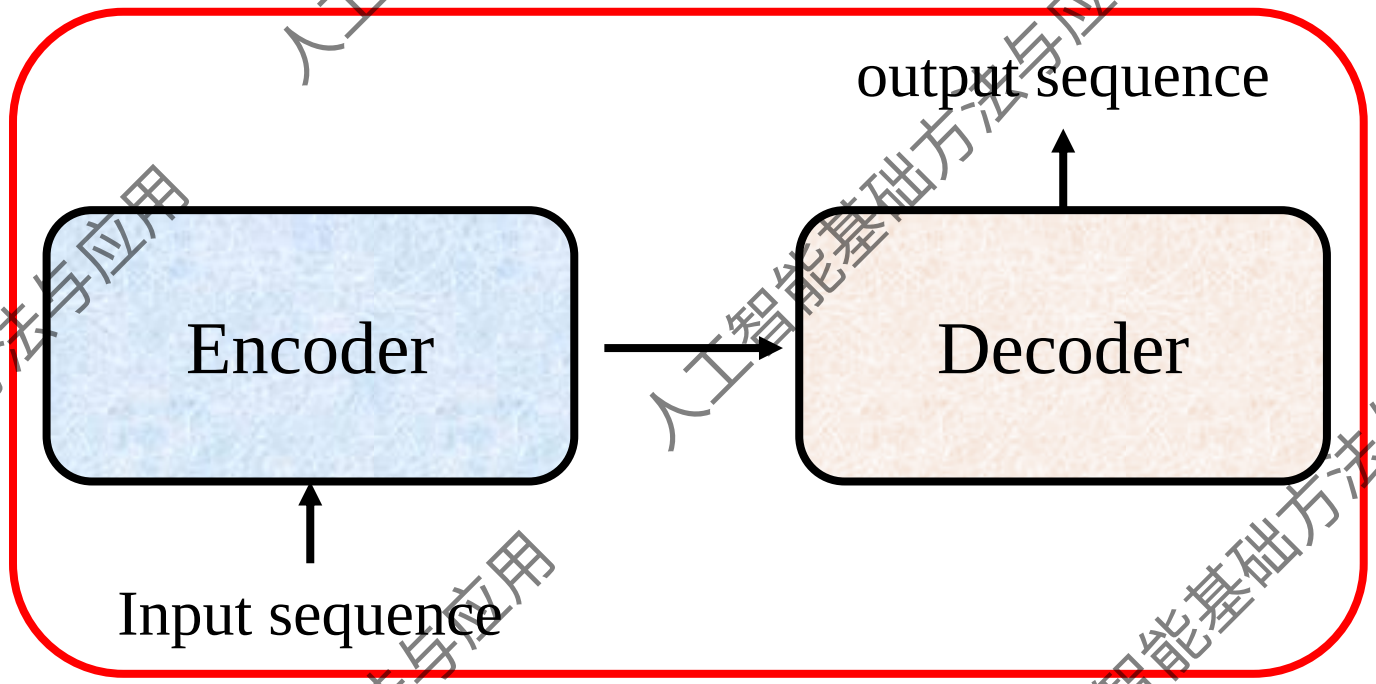
Cross-attention 的层级连接方式



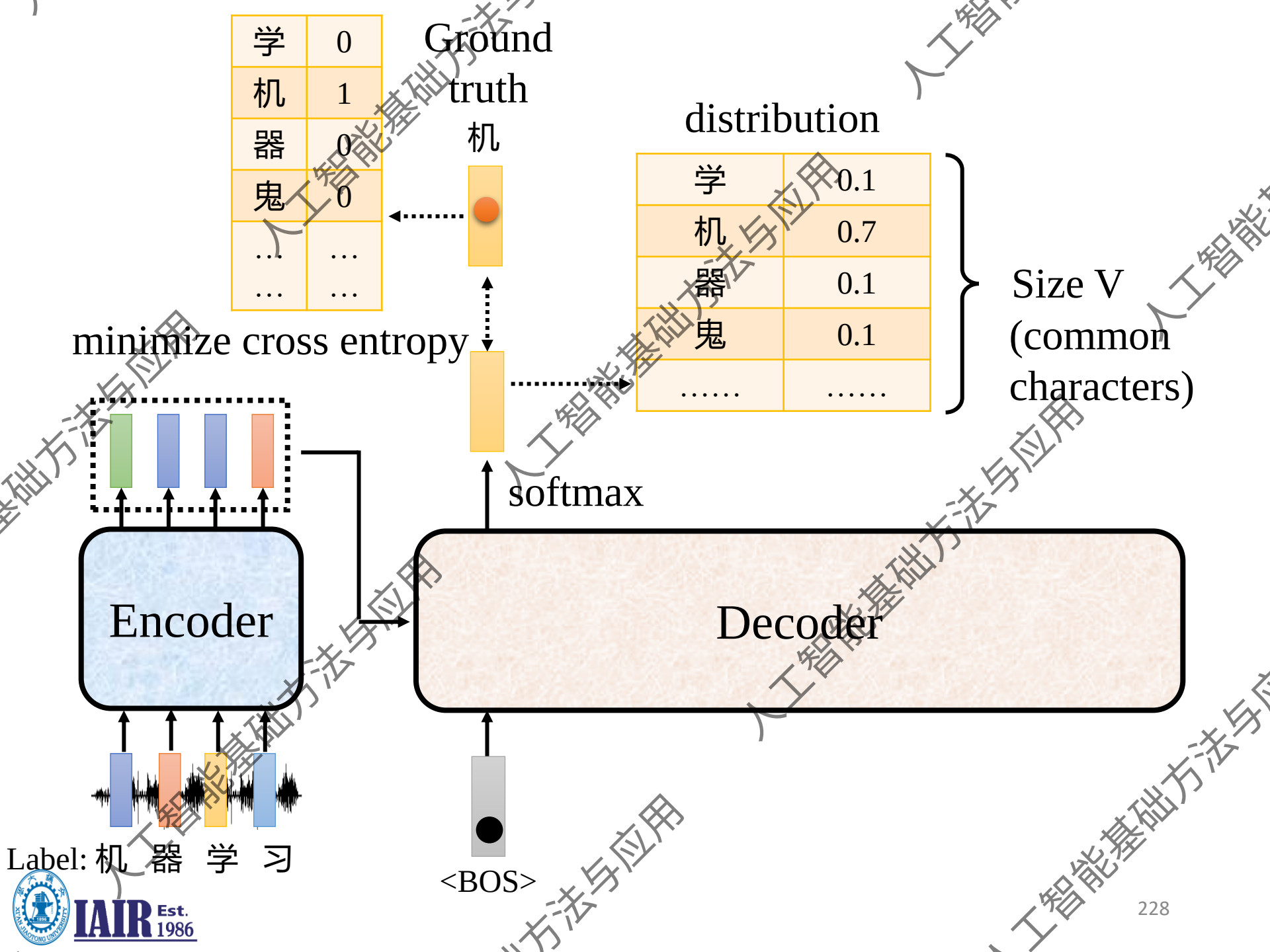
6.4 训练方法与实用技巧

人工智能基础方法与应用

Training

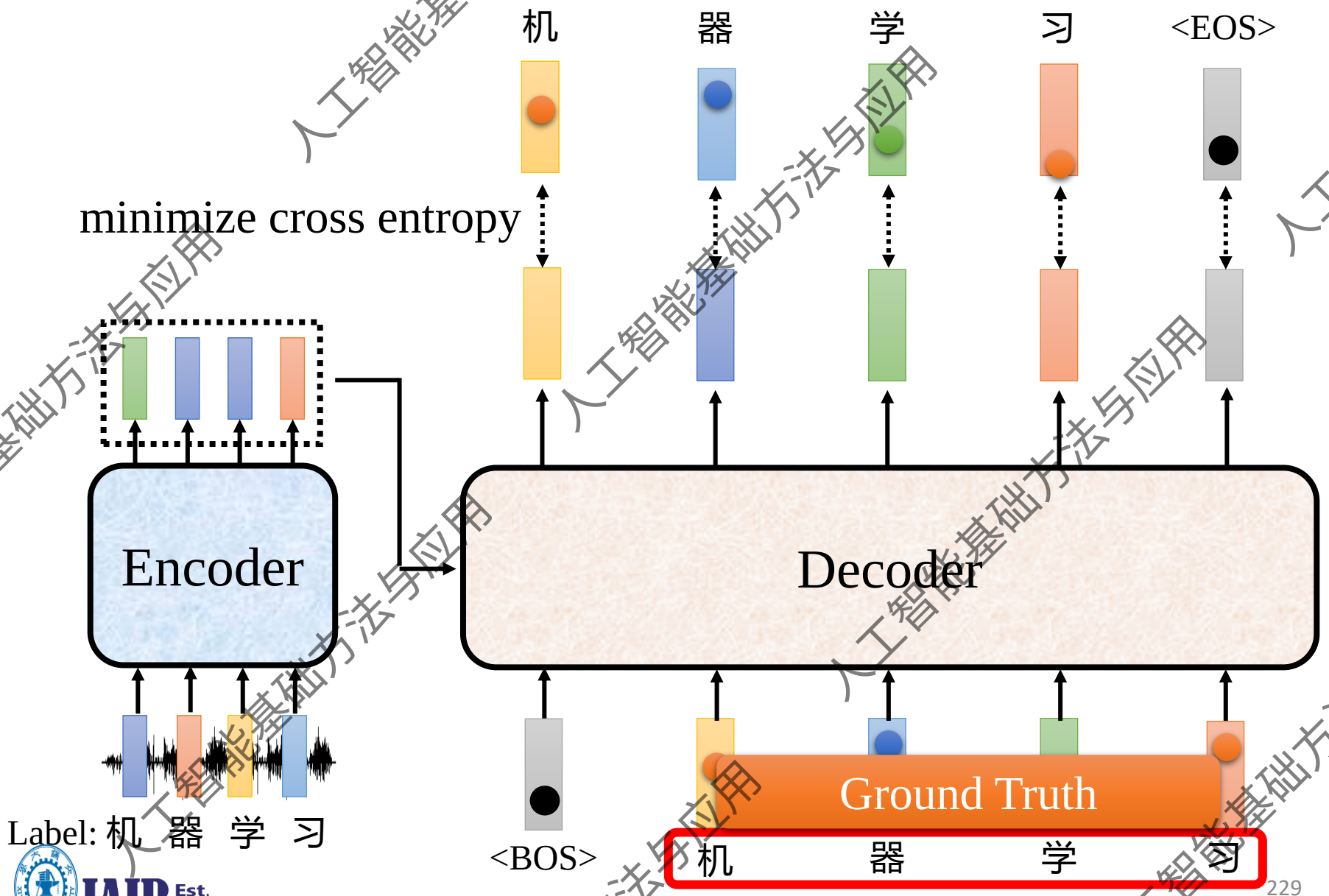


人工智能基础方法与应用



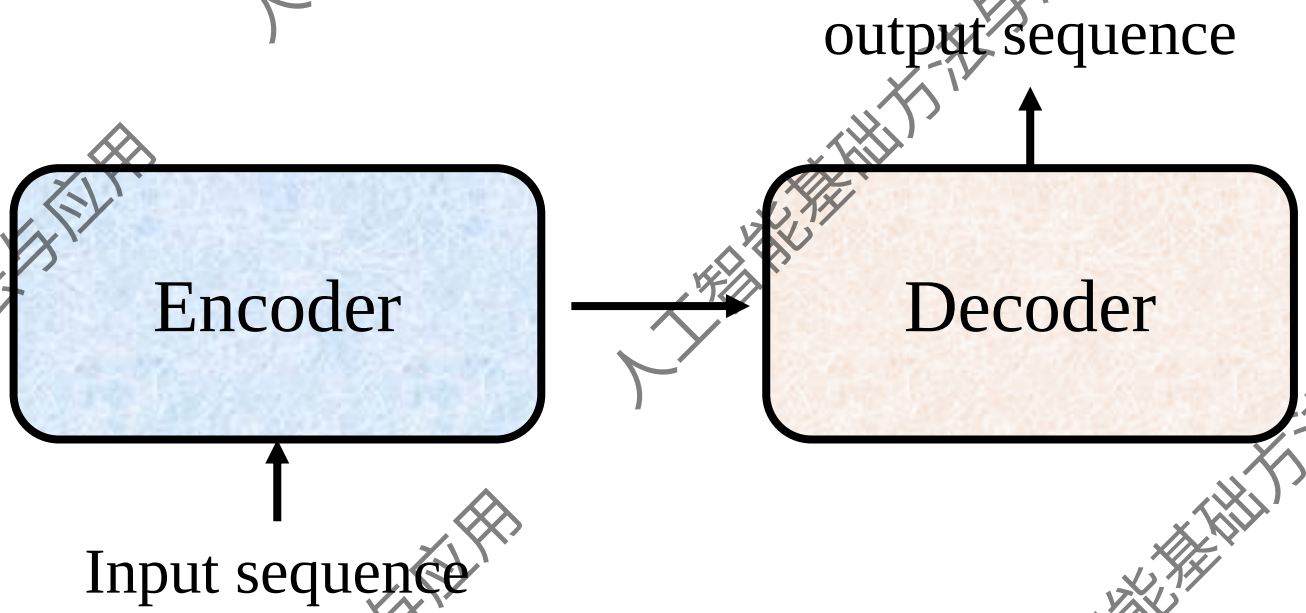
人工智能基础方法与应用

教师力：训练时给正确答案



人工智能基础方法与应用

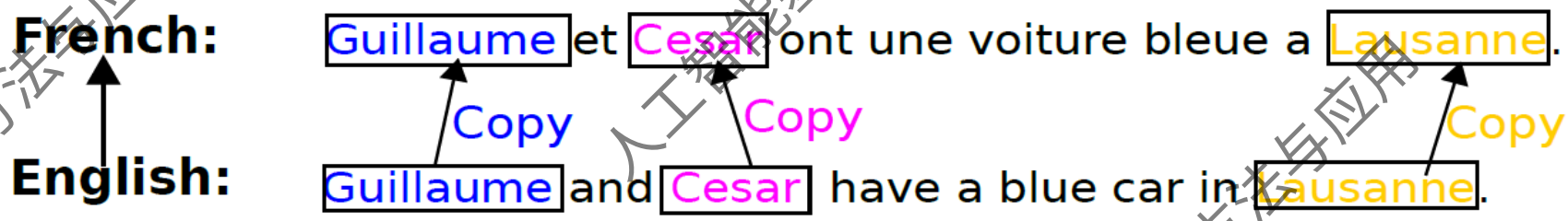
序列模型的训练技巧



人工智能基础方法与应用

Copy Mechanism

Machine Translation



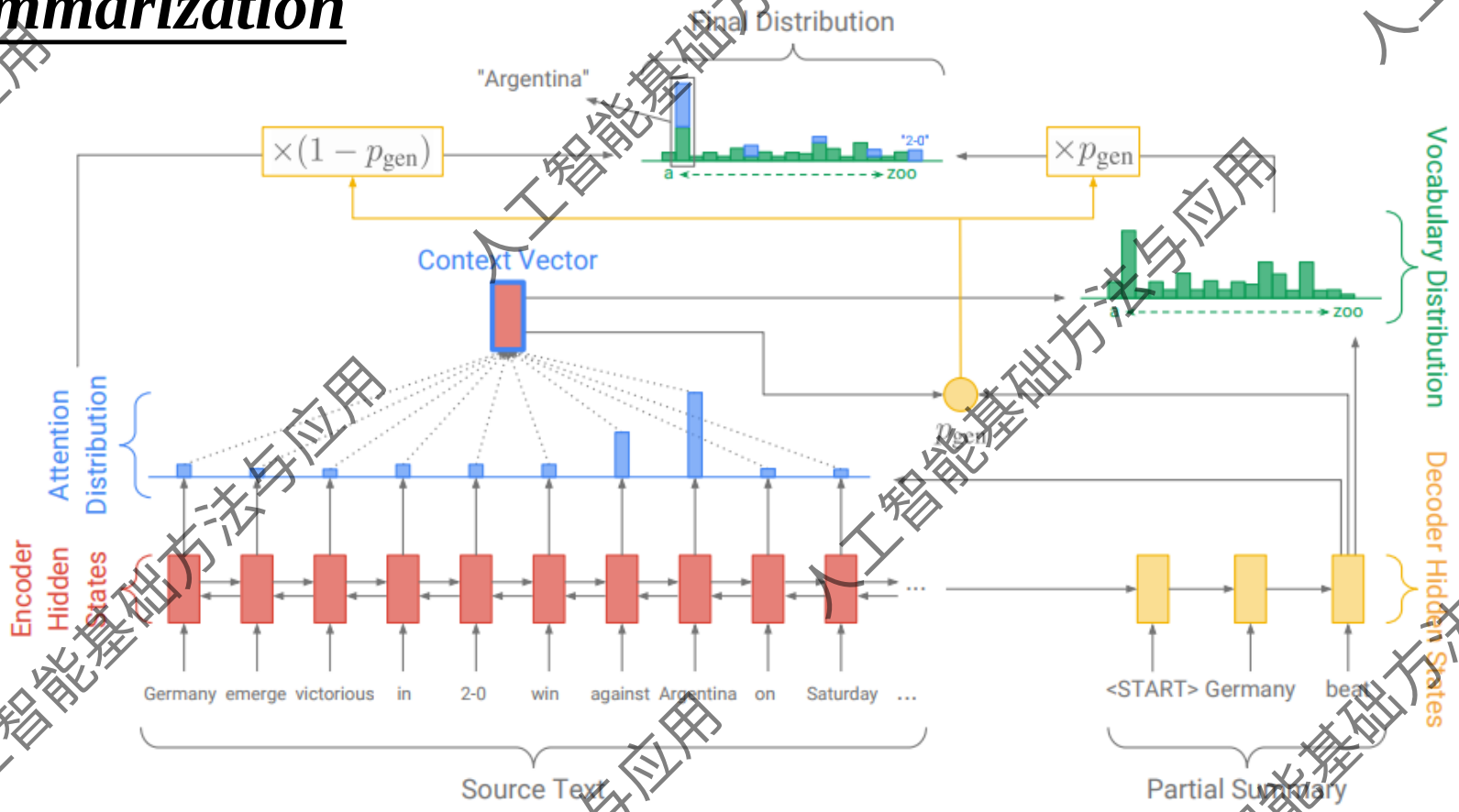
Chat-bot

User: 你好，我是库洛洛

Machine: 库洛洛你好，很高兴认识你

摘要任务：重要信息常常来自原文复制

Summarization



人工智能基础方法与应用

Guided Attention

- 发财发财发财发财
- 发财发财发财
- 发财发财
- 发财 (Missing an input character!)

人工智能基础方法与应用

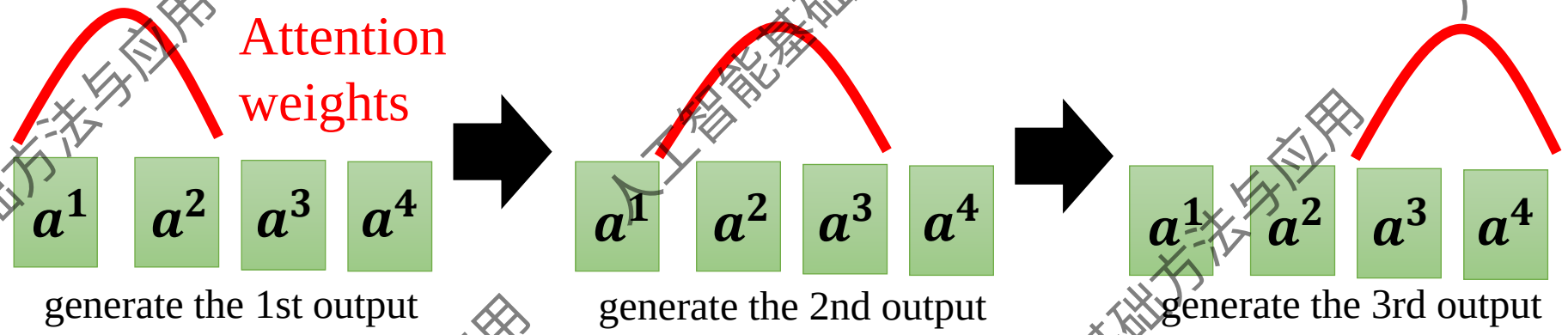
Guided Attention

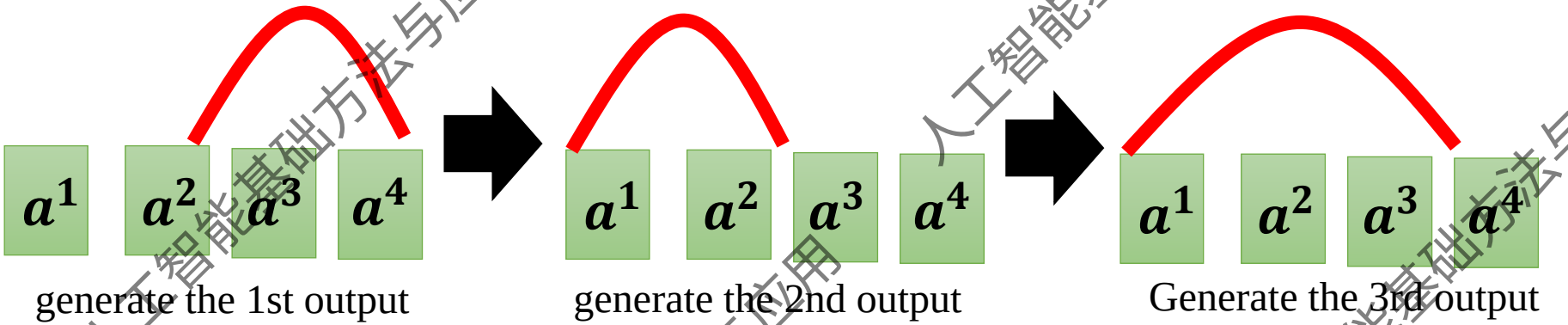
Monotonic Attention

Location-aware attention

In some tasks, input and output are monotonically aligned.

For example, speech recognition, TTS, etc.





Something wrong!

人工智能基础方法与应用

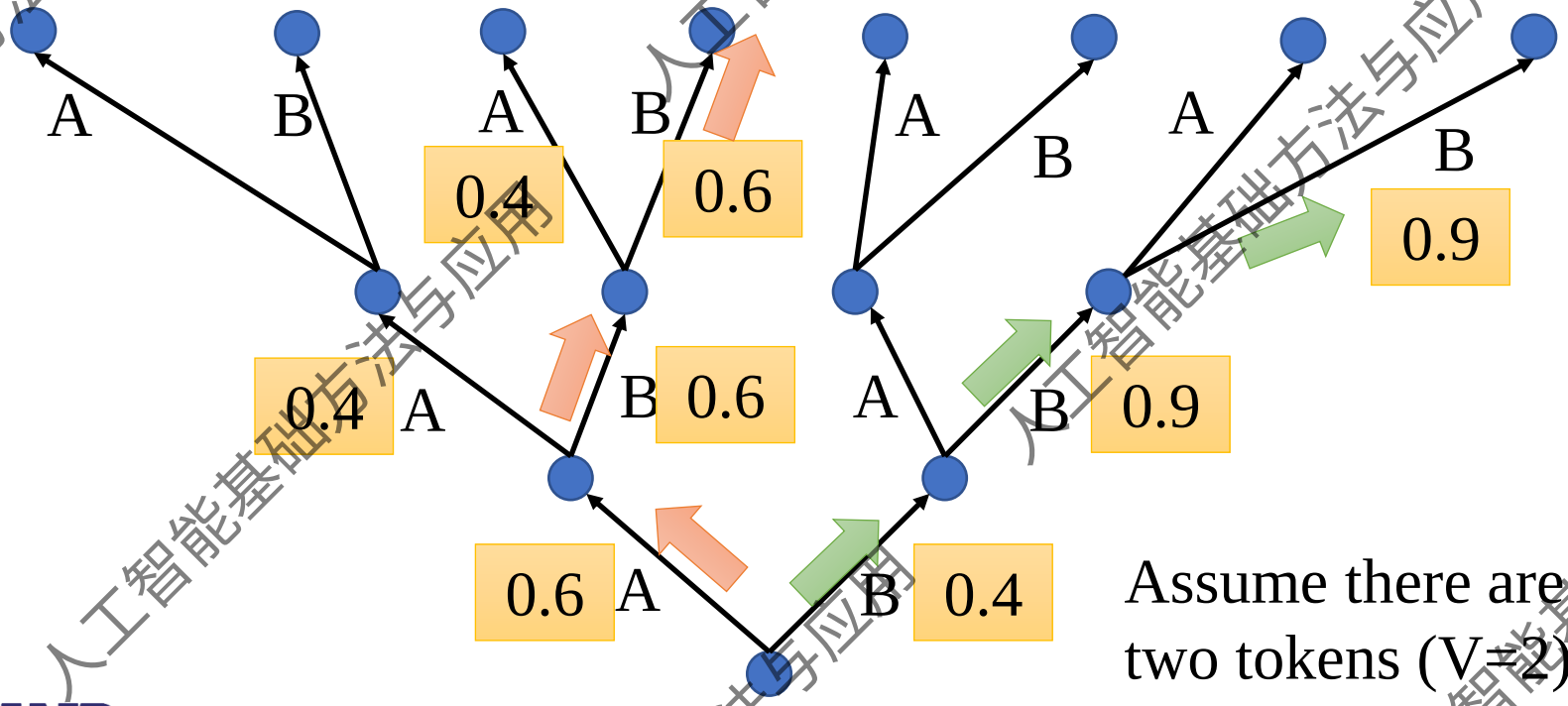
IAIR Est. 1986

Beam Search

The red path is ***Greedy Decoding***.

The green path is the best one.

Not possible to check all the paths … → Beam Search



Assume there are only two tokens (V=2).

人工智能基础方法与应用

IAIR Est. 1986

从 Cross Entropy 到 BLEU：用强化学习优化序列奖励

The Curious Case of Neural Text Degeneration

https://arxiv.org/abs/1904.09751

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Beam Search, *b*=32:
"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

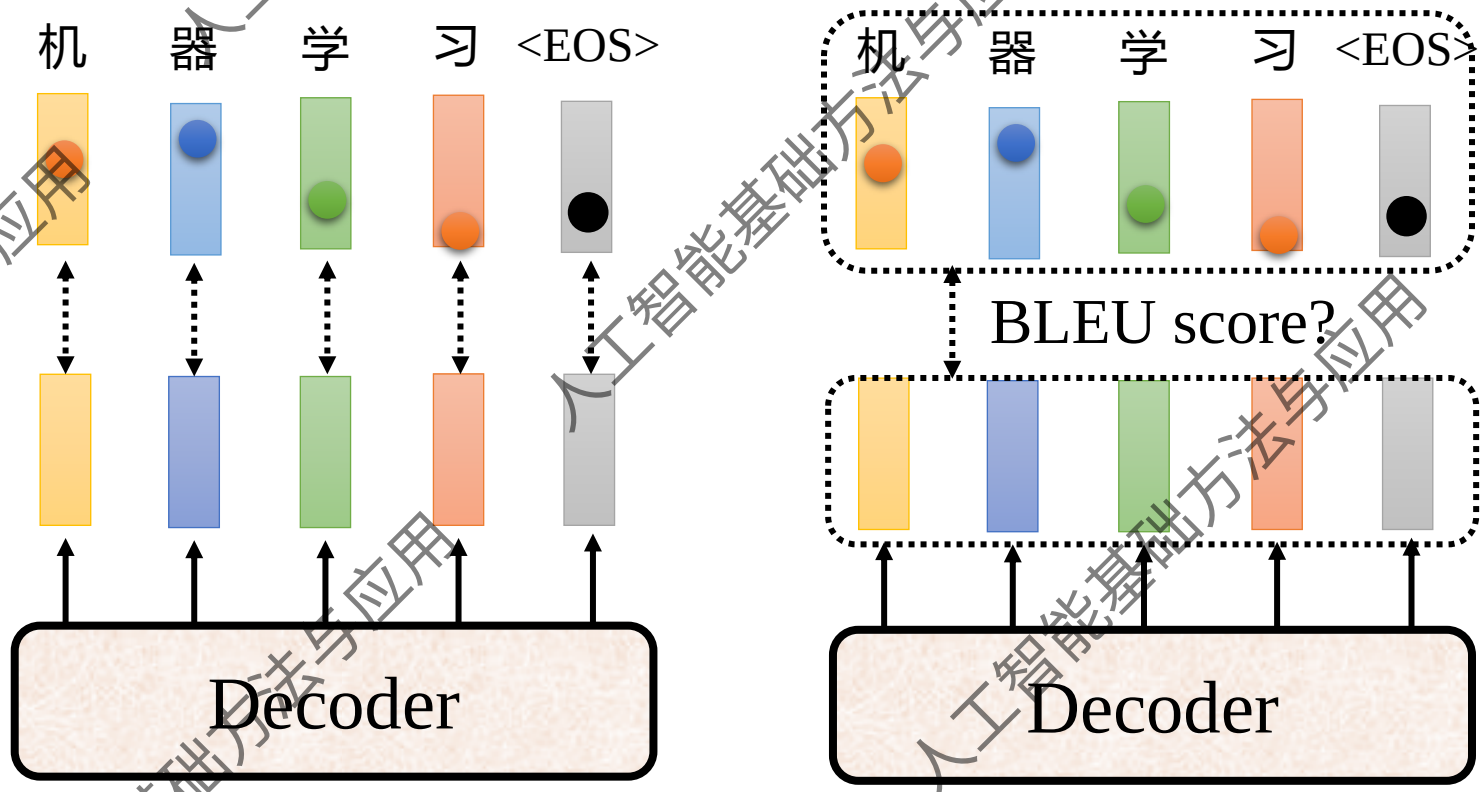
Pure Sampling:
They were cattle called Bolivian Cavalleros; they live in a remote desert uninterrupted by town, and they speak huge, beautiful, paradisiacal Bolivian linguistic thing. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros."

Randomness is needed for decoder when generating sequence in some tasks.

Accept that nothing is perfect. True beauty lies in the cracks of imperfection.

人工智能基础方法与应用

Optimizing Evaluation Metrics?



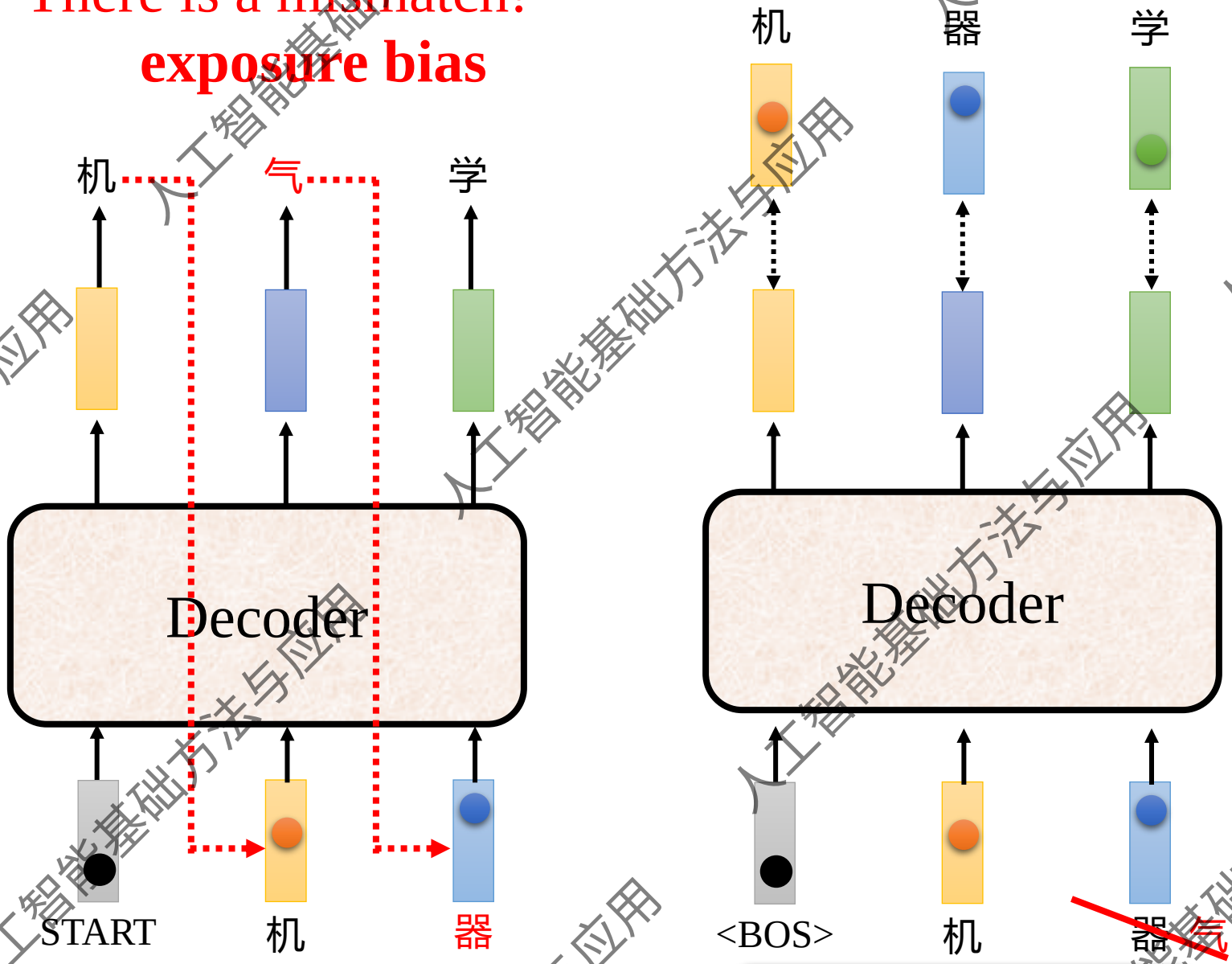
How to do the optimization?

When you don't know how to optimize, just use reinforcement learning (RL)!

人工智能基础方法与应用

There is a mismatch!

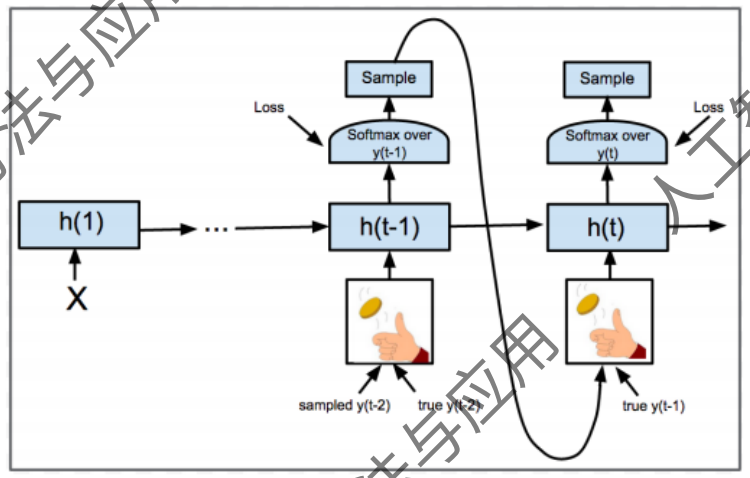
exposure bias

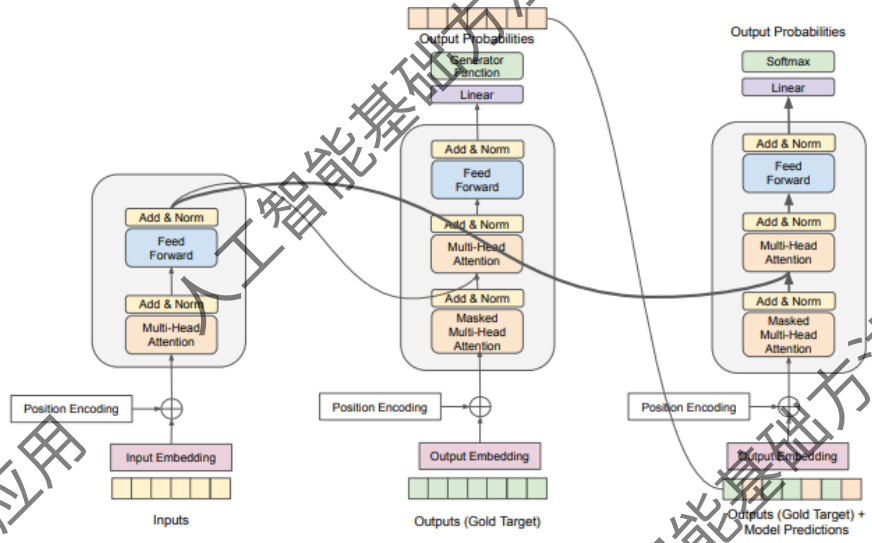


人工智能基础方法与应用

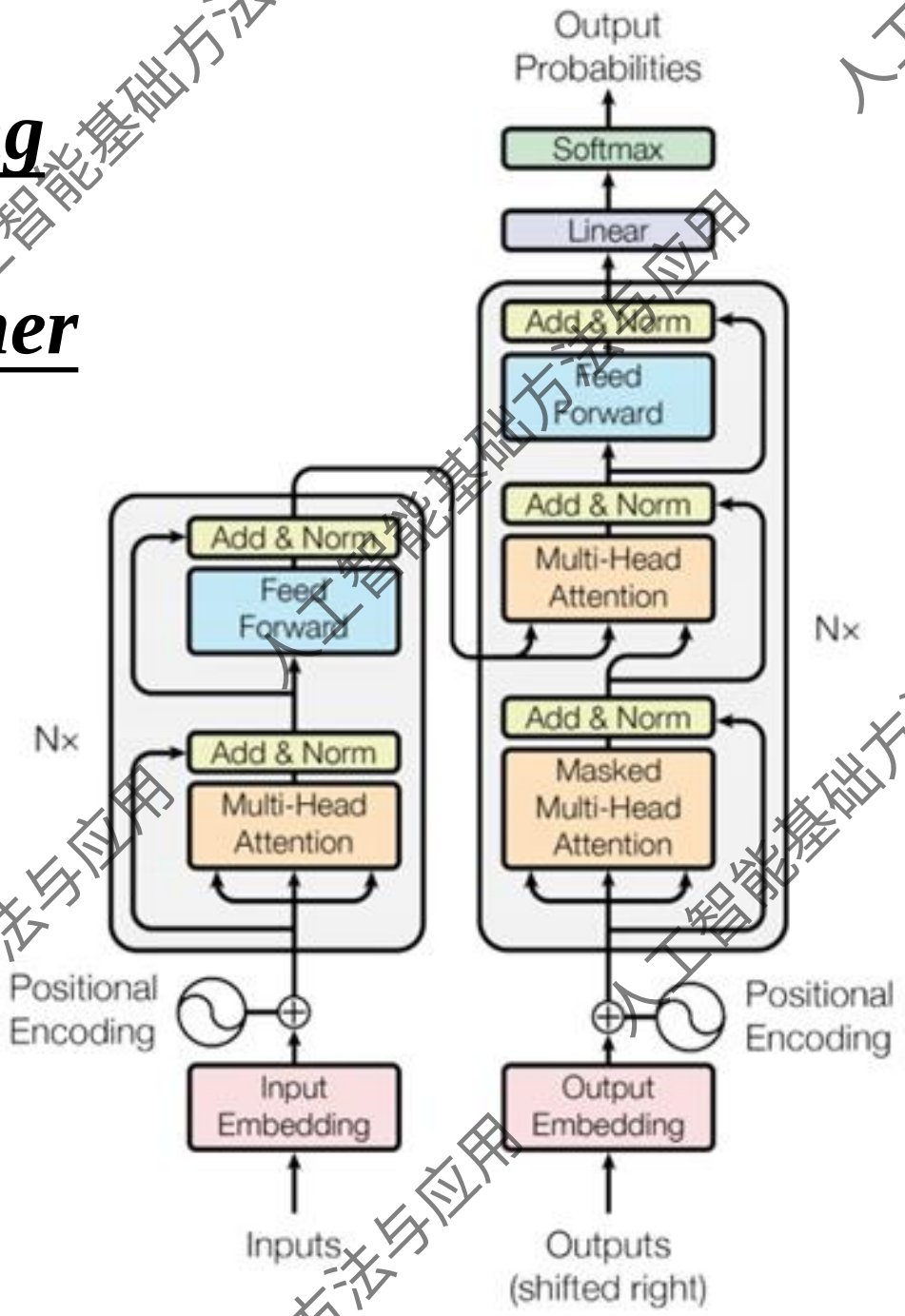
Scheduled Sampling

- Original Scheduled Sampling
- Scheduled Sampling for Transformer
- Parallel Scheduled Sampling





Concluding Remarks: Transformer



人工智能基础方法与应用